Universidade Federal do ABC

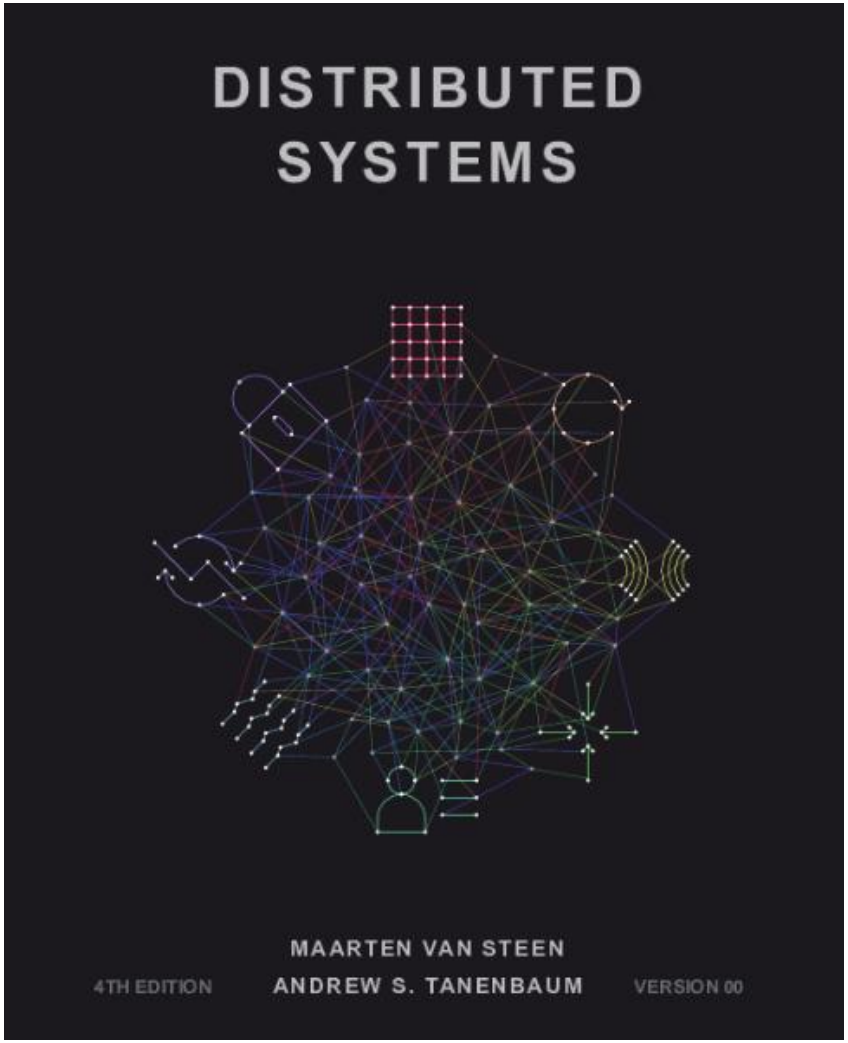
Sistemas Distribuídos

Apresentação da Disciplina

Professor Vladimir Rocha
vladimir.rocha@ufabc.edu.br

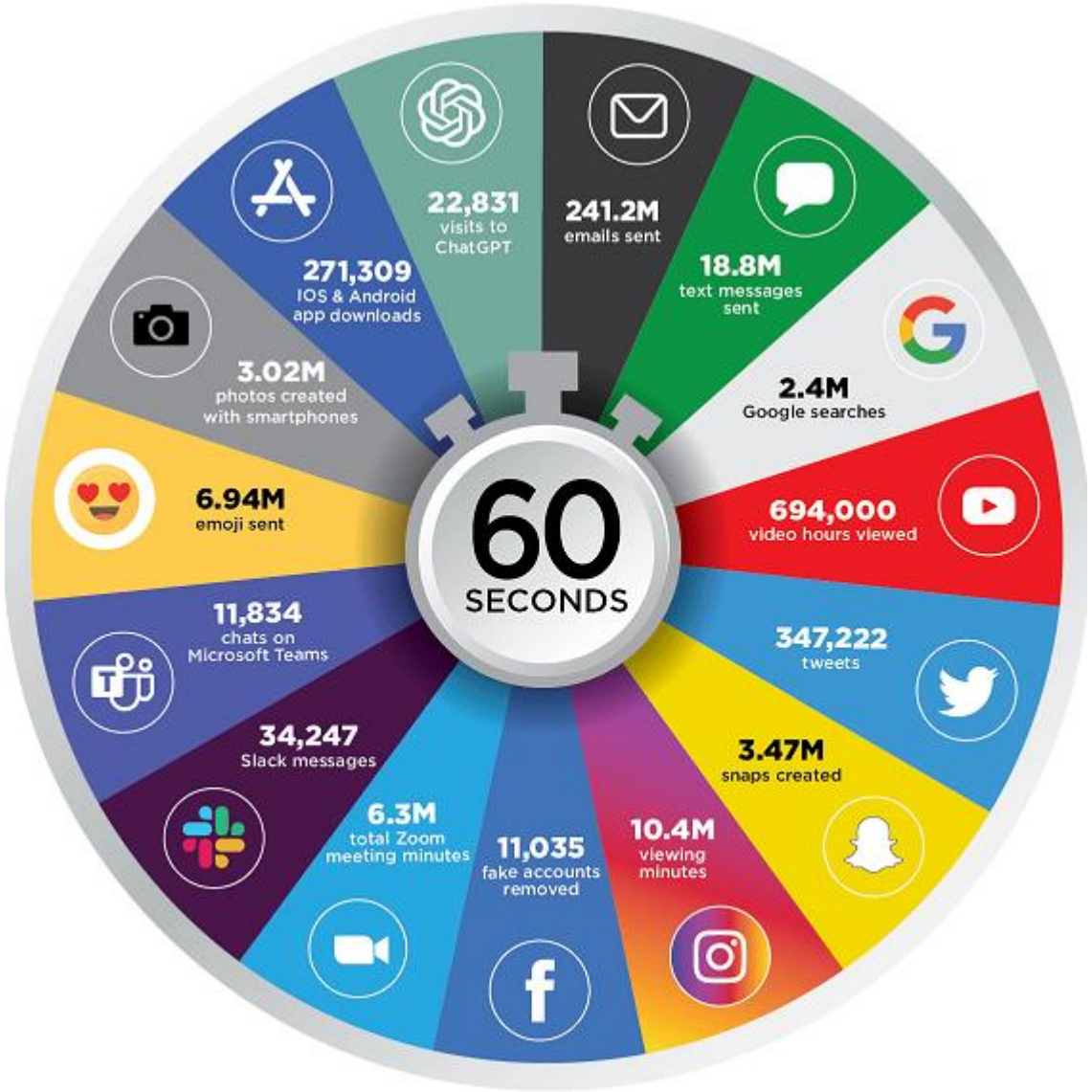
Distributed Systems 4th edition (free)

https://www.distributed-systems.net/index.php/books/ds4/



4.03 (2025)

Introdução



https://ediscoverytoday.com/2023/04/20/2023-internet-minute-infographic-by-ediscovery-today-and-ltmg-ediscovery-trends/ (2023)

Introdução



http://techbakbak.com/wp-content/uploads/2014/08/smartphone-apps.jpg

Introdução





encripted-tbn0.gstatic.com/images?q=tbn:ANd9GcQwr_n6SyMN4IHJtumBOzCIDuwEQjk3vgyNTjV3x1JOTHc11cIiHw

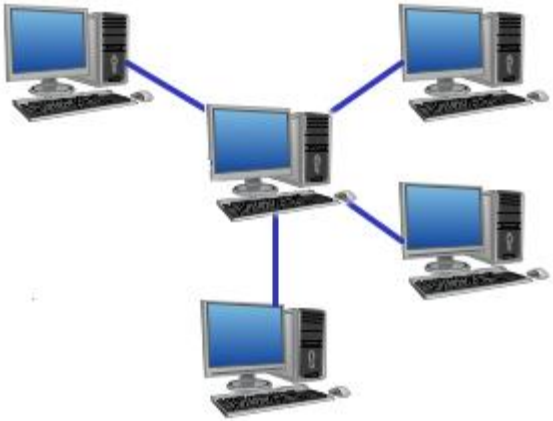
Introdução

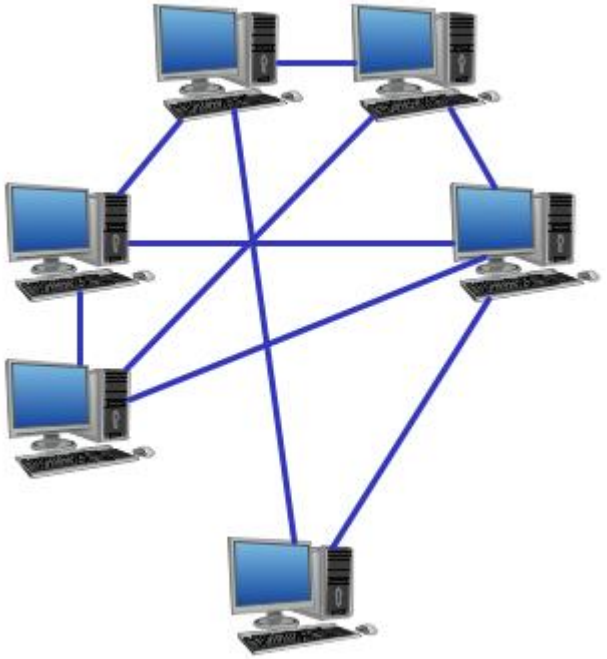


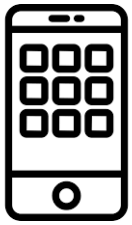


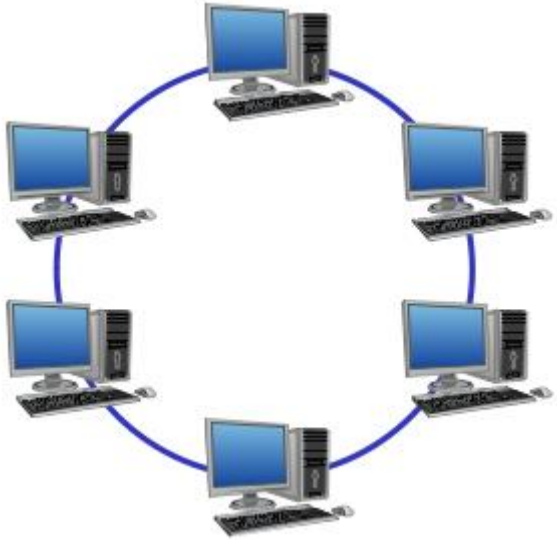
encripted-tbn0.gstatic.com/images?q=tbn:ANd9GcQwr_n6SyMN4IHJtumBOzCIDuwEQjk3vgyNTjV3x1JOTHc11cIiHw

Introdução





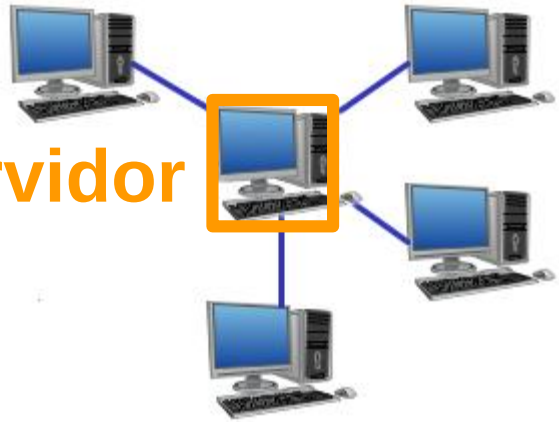




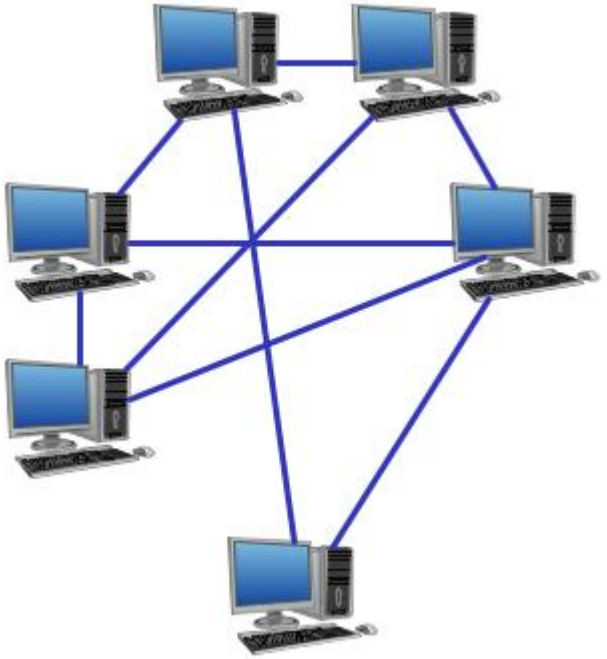
Arquiteturas: capítulo 2

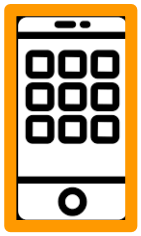
http://www.conceptdraw.com/How-To-Guide/picture/Common-network

Introdução

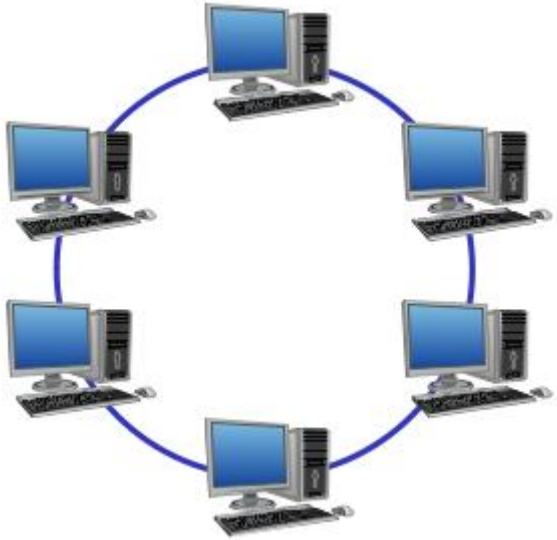


Servidor





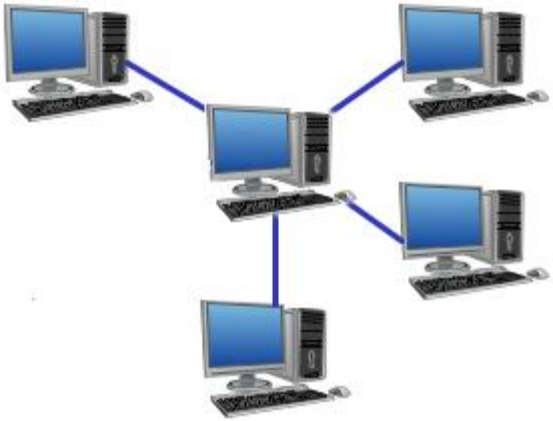
Cliente

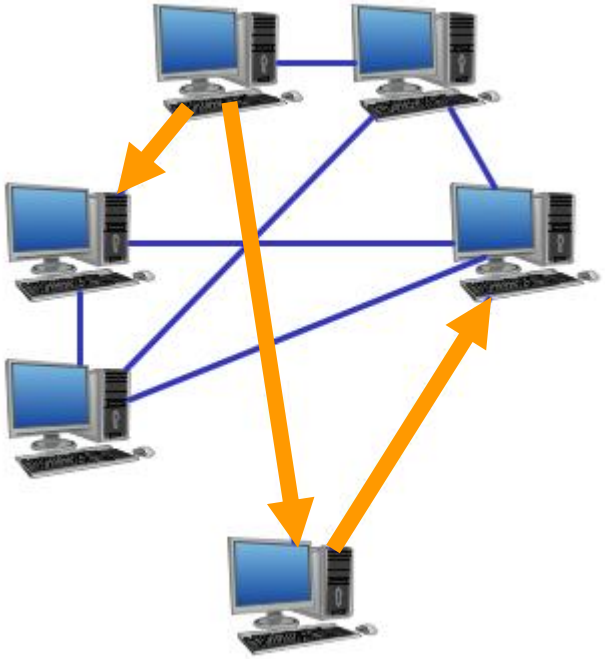


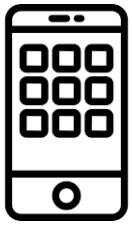
Clientes e Servidores: capítulo 3

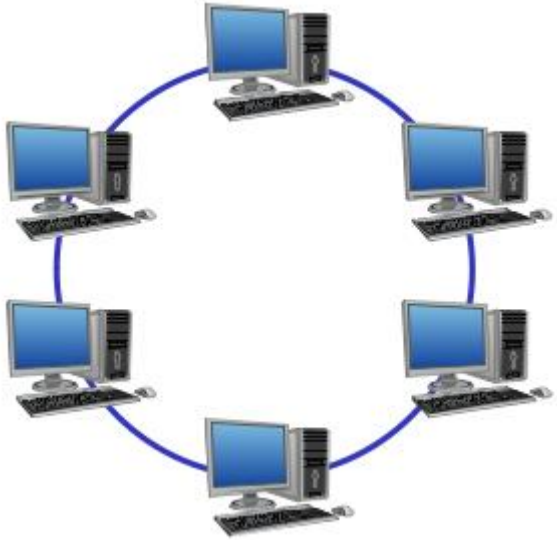
http://www.conceptdraw.com/How-To-Guide/picture/Common-network

Introdução





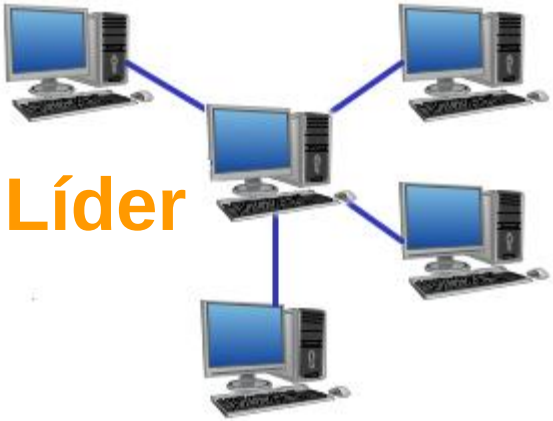


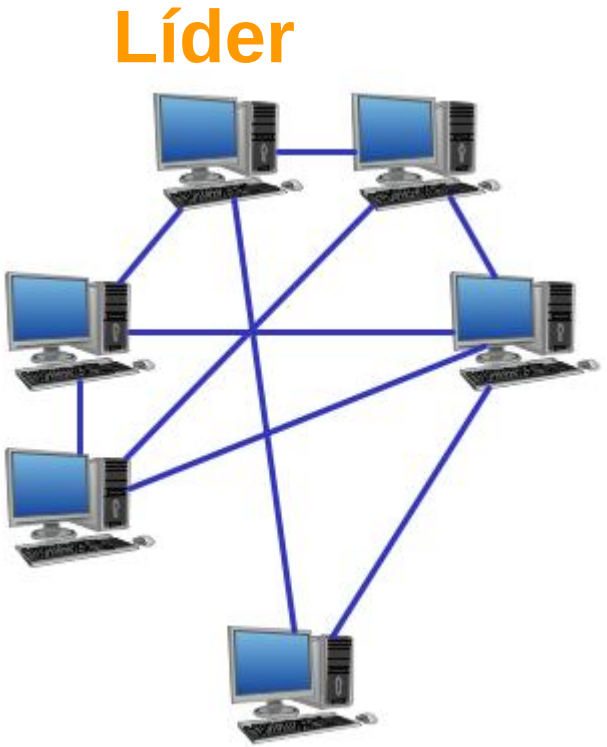


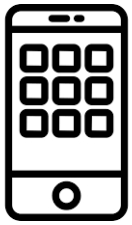
Comunicação: capítulo 4

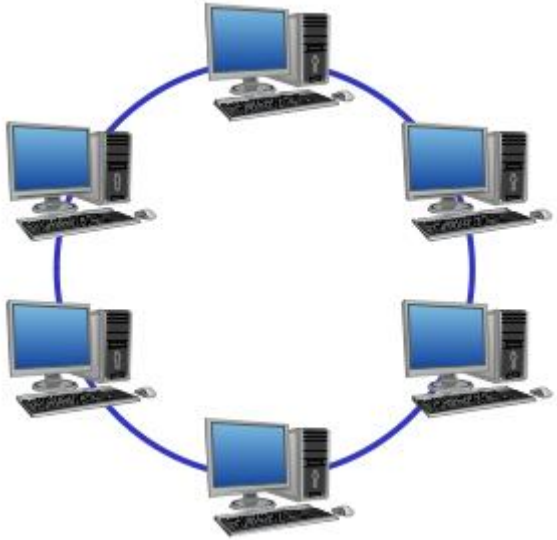
http://www.conceptdraw.com/How-To-Guide/picture/Common-network

Introdução





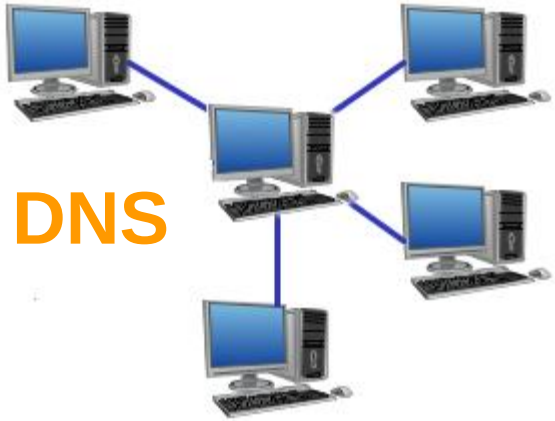


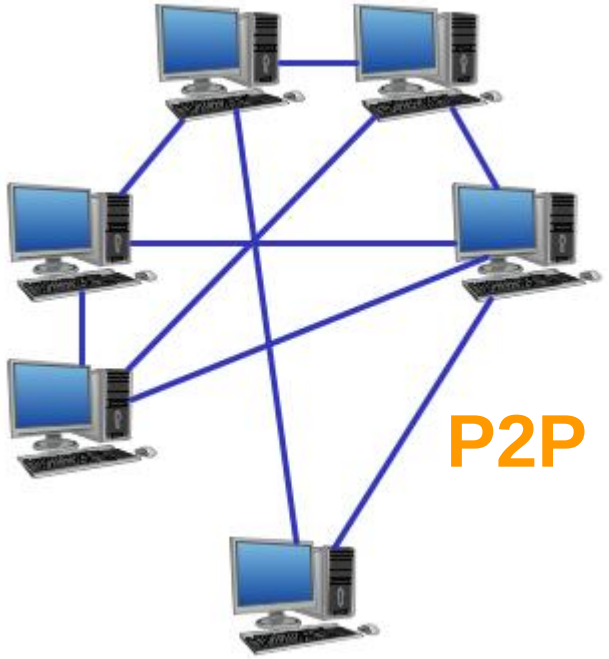


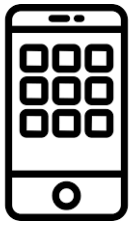
Sincronização: capítulo 5

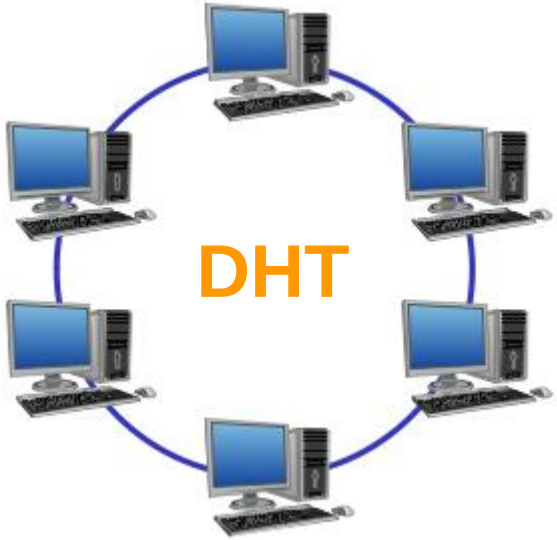
http://www.conceptdraw.com/How-To-Guide/picture/Common-network

Introdução





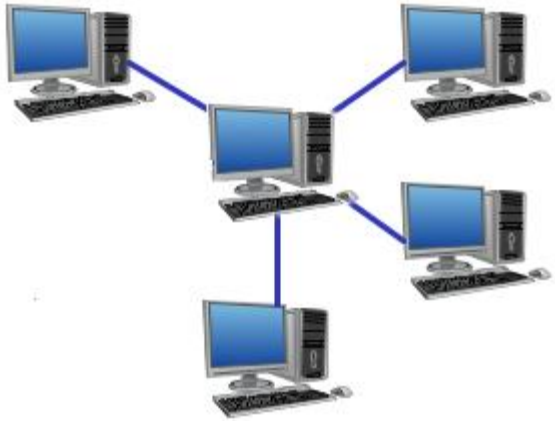


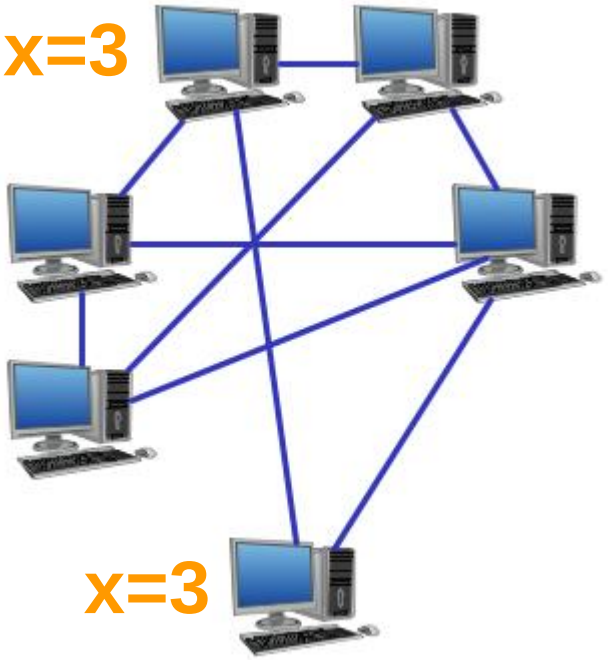


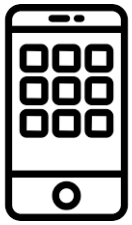
Nomeação: capítulo 6

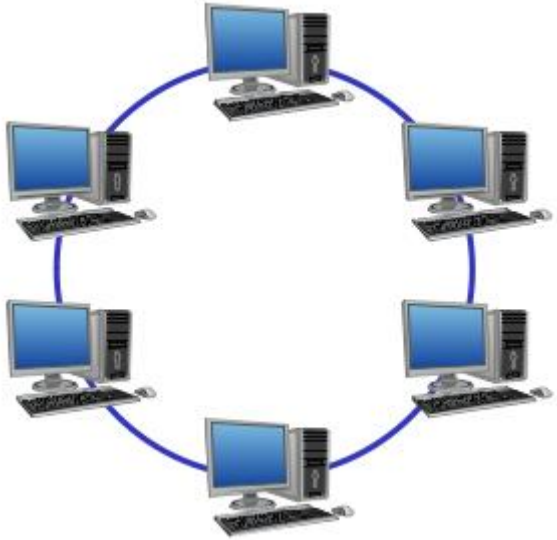
http://www.conceptdraw.com/How-To-Guide/picture/Common-network

Introdução





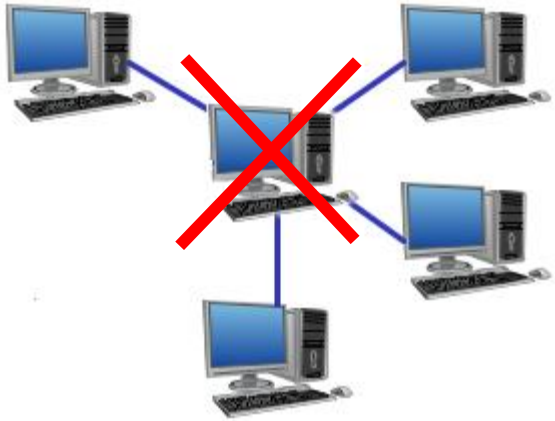


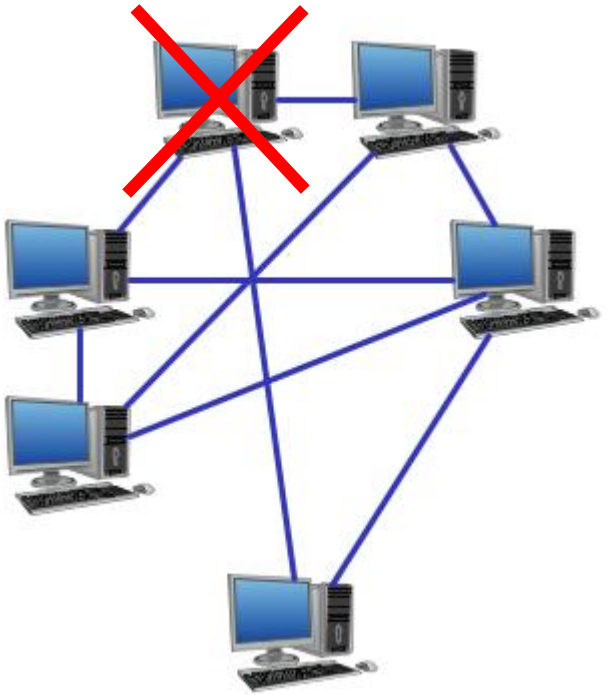


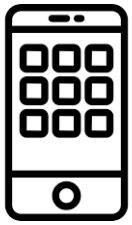
Consistência: capítulo 7

http://www.conceptdraw.com/How-To-Guide/picture/Common-network

Introdução

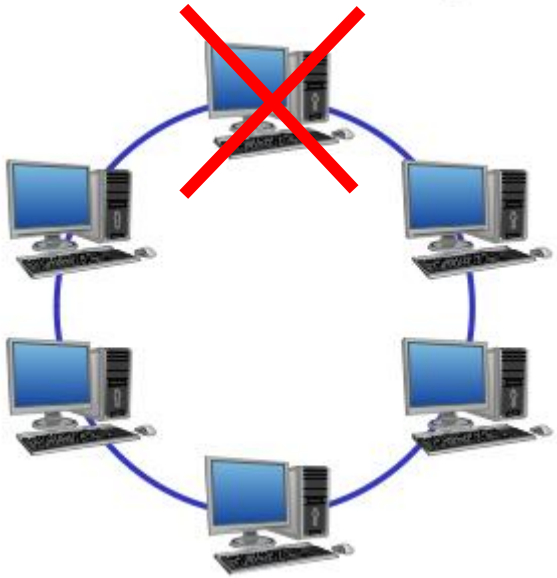






Ficou fora aí? Nuvem do Google falha e Spotify sofre instabilidade

ESTADÃO *conteúdo*
São Paulo
13/06/2025 08h30

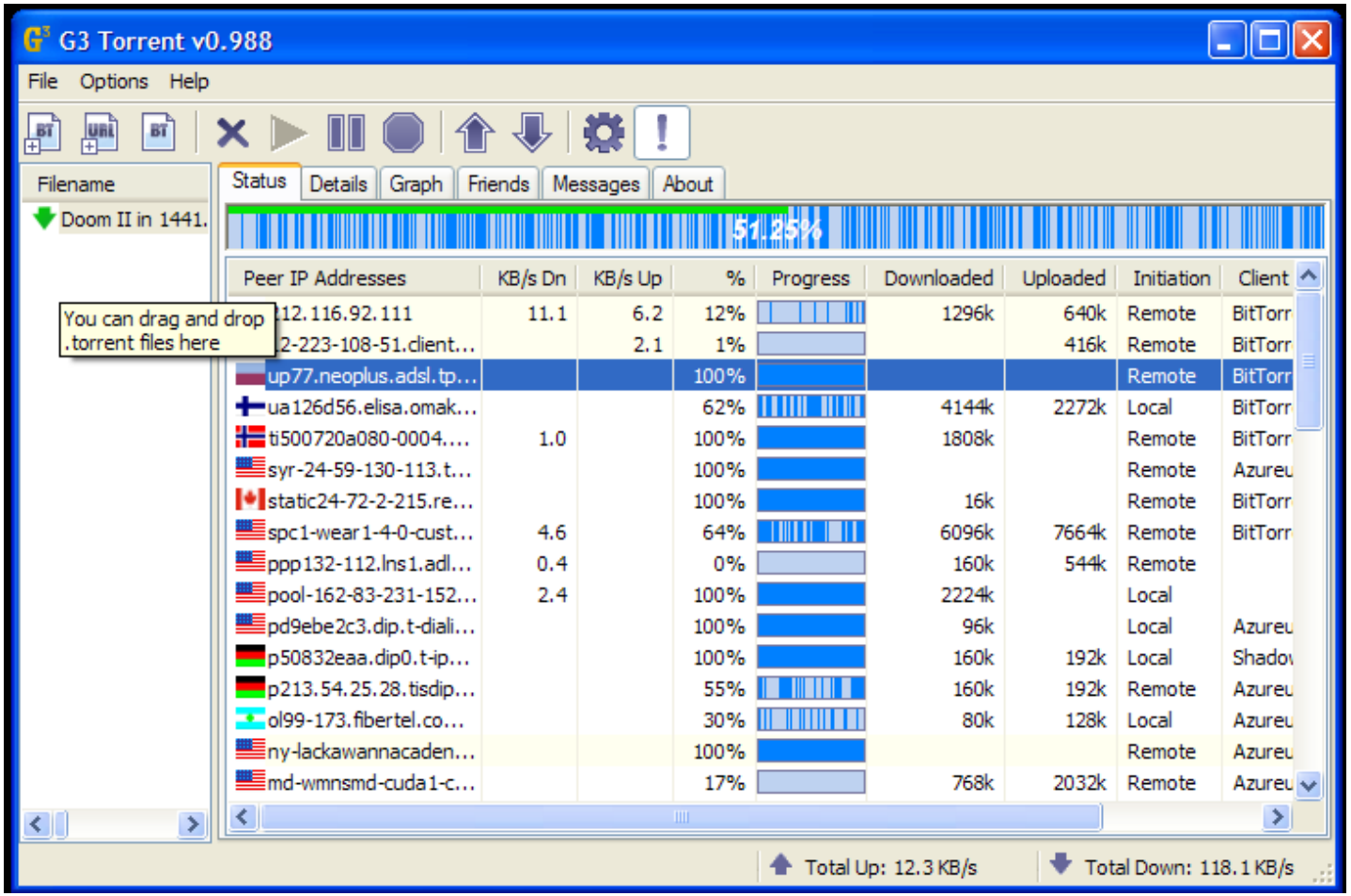


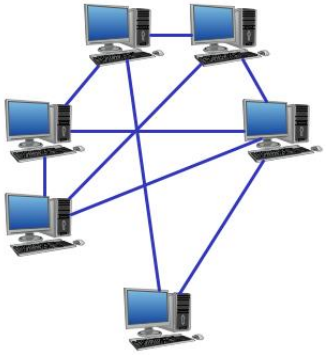
Falhas: capítulo 8

http://www.conceptdraw.com/How-To-Guide/picture/Common-network

Introdução

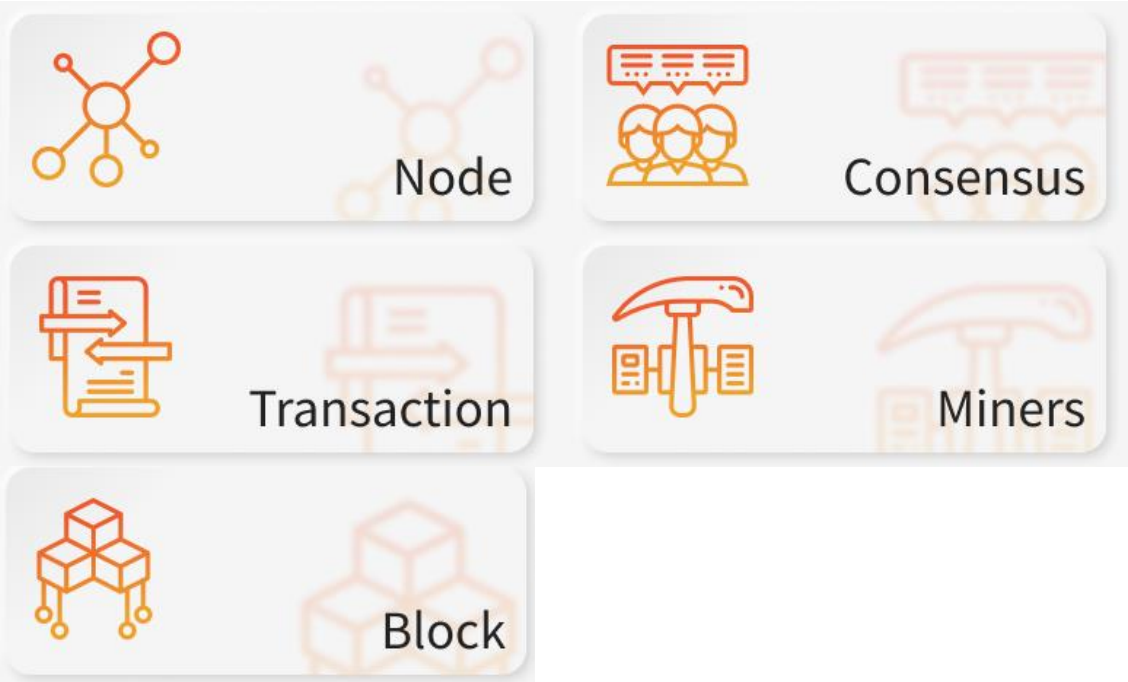
BitTorrent™





Aplicações: BitTorrent

Introdução



https://www.allerin.com/blog/breaking-down-the-blockchain-architecture

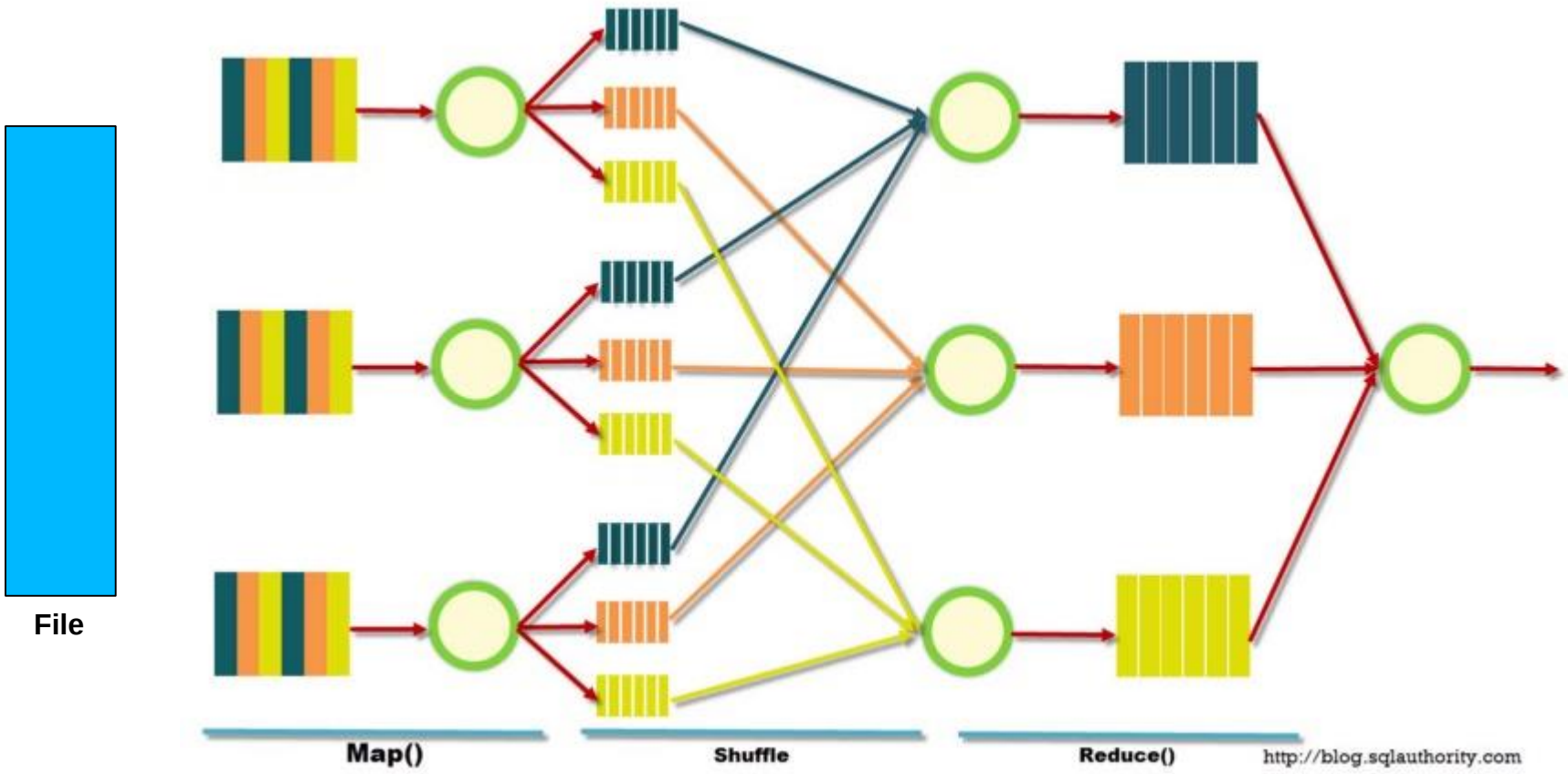
bitcoin

ethereum

NFT

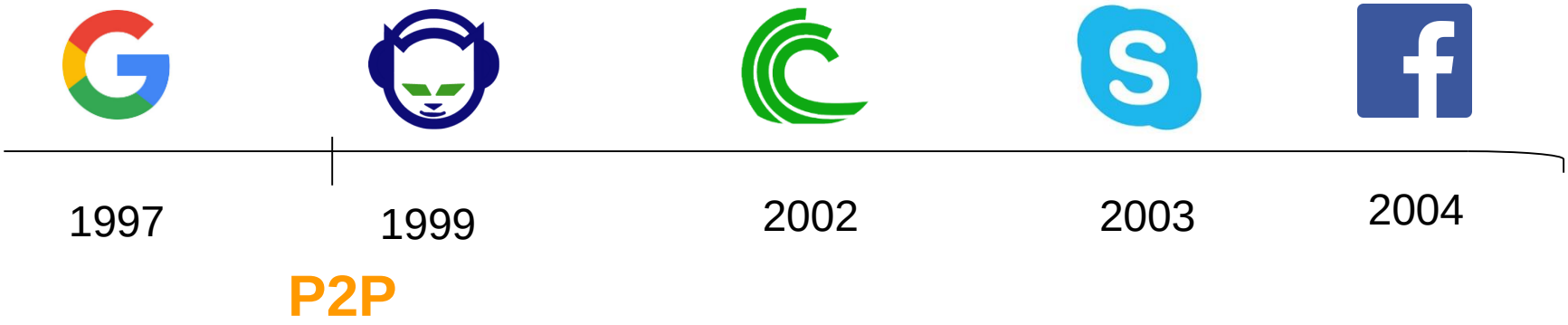
Aplicações: Blockchain

Introdução

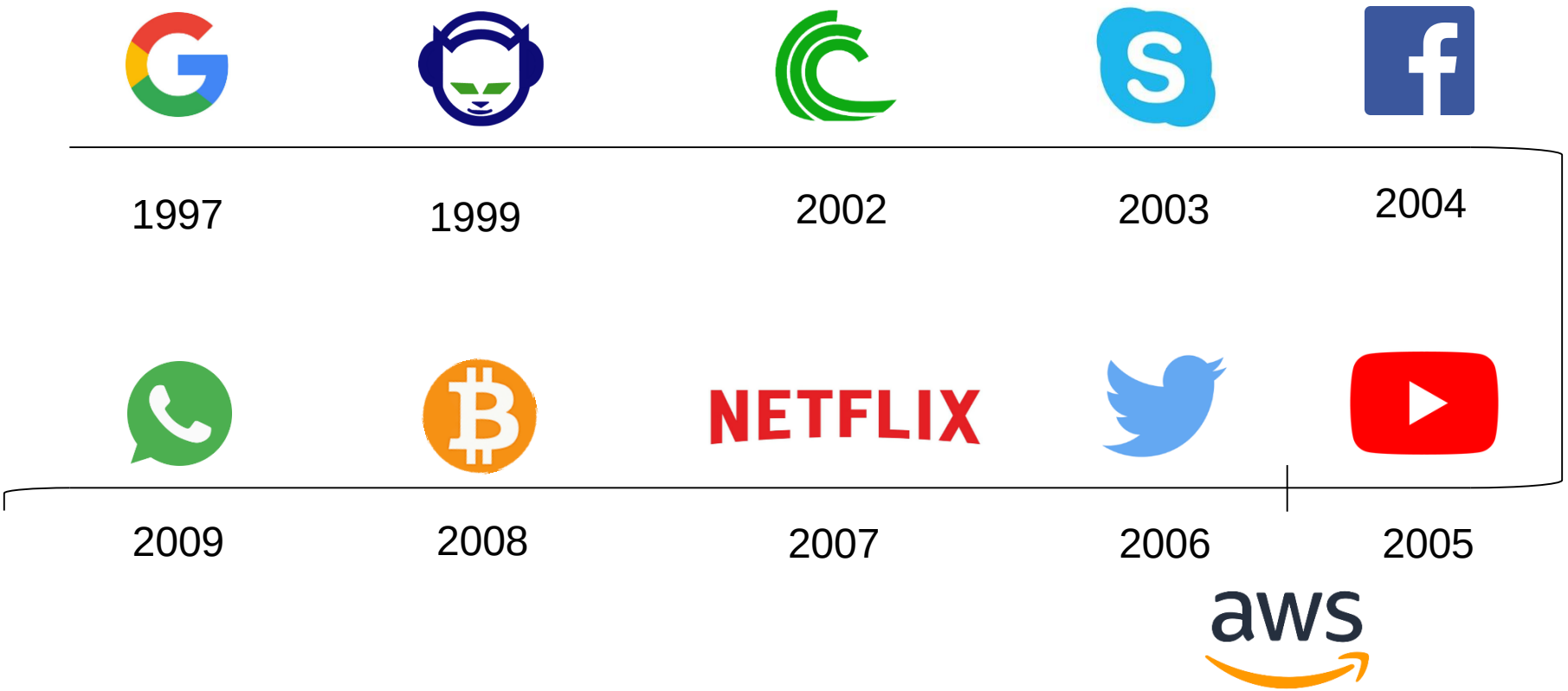


Aplicações: Hadoop Map-Reduce

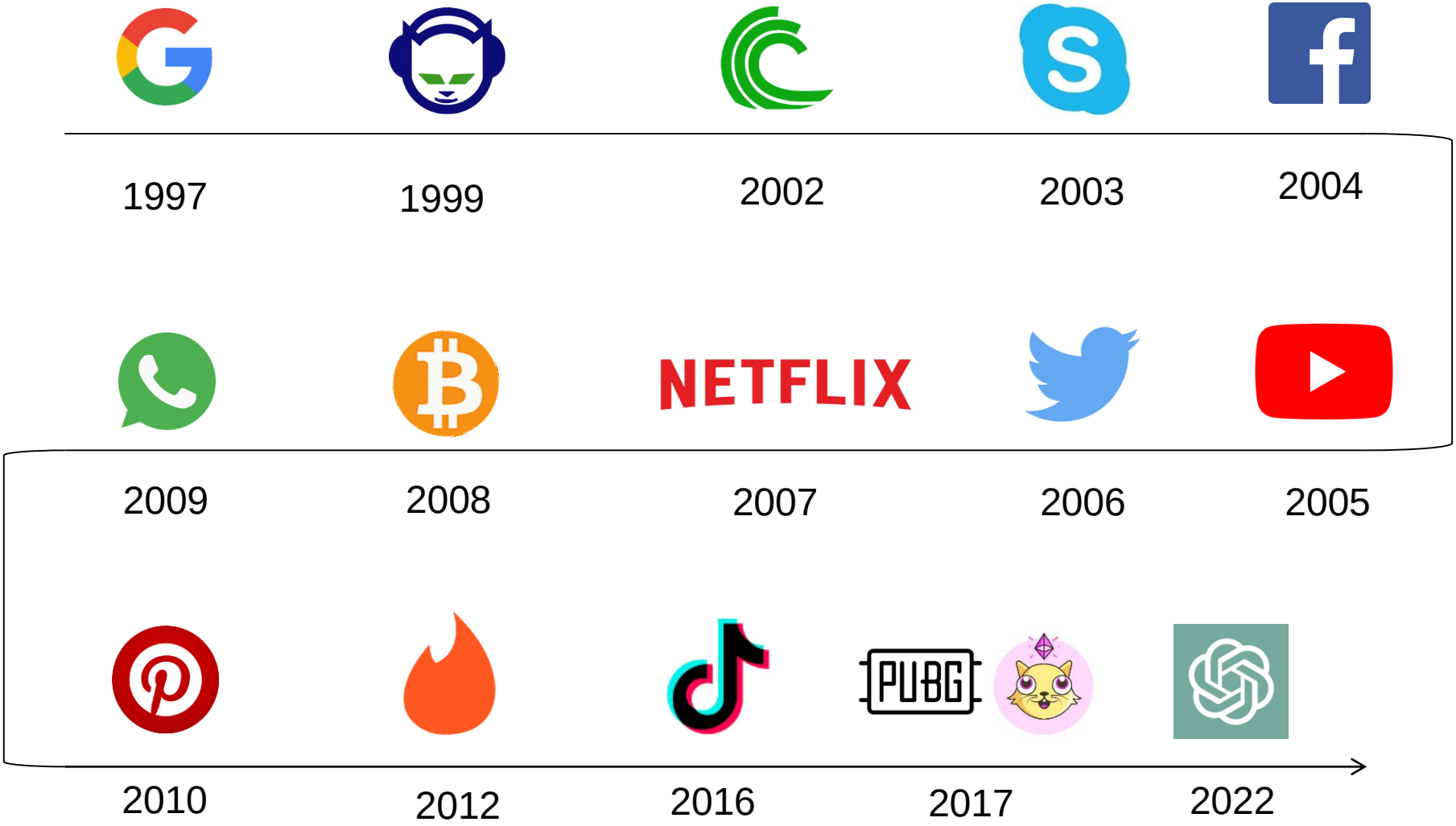
Relevância



Relevância


1997
1999
2002
2003
2004
2009
2008
NETFLIX
2007
2006
2005
aws

Relevância


1997
1999
2002
2003
2004
2009
2008
2007
2006
2005
2010
2012
2016
2017
2022
NETFLIX
PUBG

Recomendações da Disciplina

- Ter cursado Redes de Computadores
  - Comunicação TCP, UDP, IP (programação com sockets)
- Ter conhecimento de estruturas de dados, algoritmos para busca, ordenação e grafos
  - Listas, Filas, Tabelas de Hash, Árvores, etc.
  - Buscas O(1) O(log n) O(n). X-Sort, etc.
  - DFS, BFS, Dijkstra, etc.
- Ter conhecimento nas técnicas de concorrência de Sistemas Operacionais
  - Semáforo, mutex, variáveis de condição, etc.
- Ter conhecimento de probabilidades e estatística

Recomendações da Disciplina

- Leitura das seções do livro da bibliografia base
  - Servirá para compreender melhor o conteúdo dos slides.
  - Complementar os conhecimentos.
- Baixe o livro de forma gratuita no link

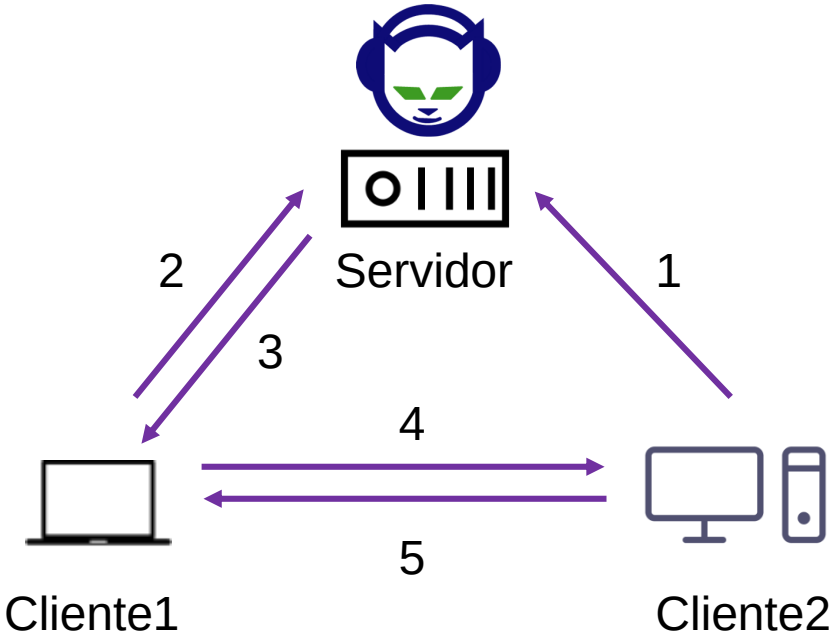
  https://www.distributed-systems.net/index.php/books/ds4/

Avaliações

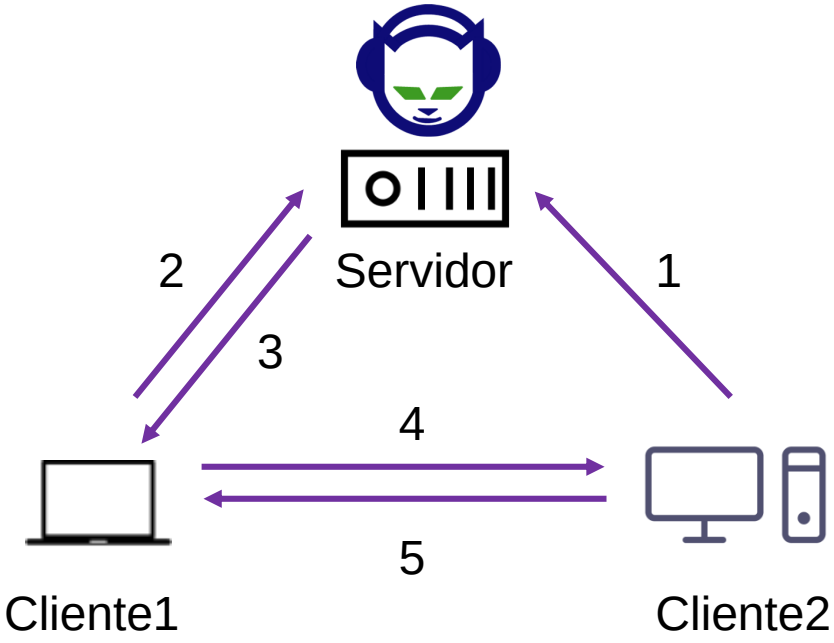
Tipo	Percentual	Realização	Data
Avaliação 1	30%	Presencial síncrona	15/10
EP1 - Projeto Programação	20%	Entrega pelo SIGAA	Até 5/11 às 23.00
Avaliação 2	50%	Presencial síncrona	23/11

Exercícios Programáticos

- Porquê é necessário programar



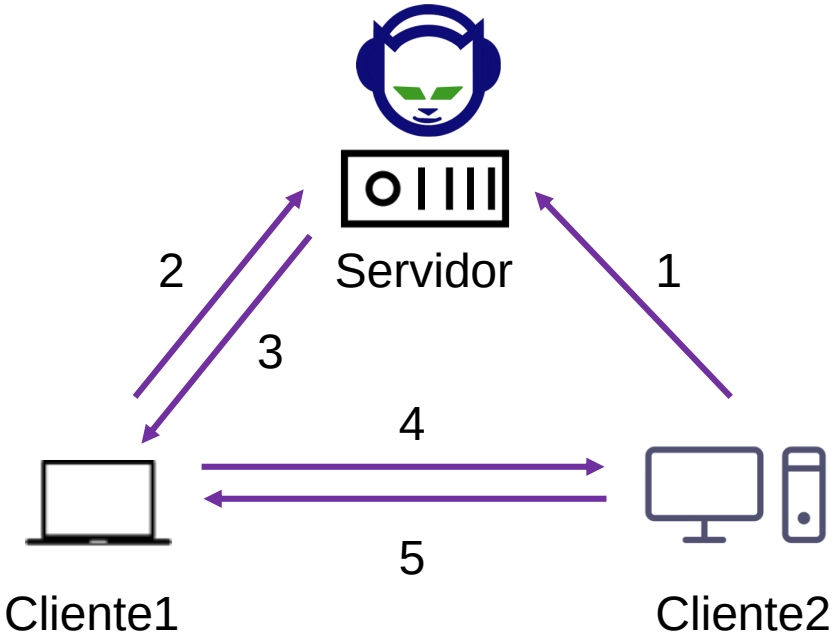
Exercícios Programáticos



Do ponto de vista do Cliente1, que acontece se:

- Servidor não responde
- Arquivo é muito grande
- Cliente2 não pode enviar nesse momento
- Cliente2 apagou o arquivo
- Cliente2 saiu no meio da transferência

Exercícios Programáticos



Do ponto de vista do Servidor:

- Quantos clientes posso atender?
- Que informações devo armazenar?
- Cliente2 sai (como ficam os arquivos)
- Cliente2 morre (não avisou a saída)
  - Como estimar o tempo?
  - Eliminar arquivos? E se volta?

Exercícios Programáticos

- As aulas práticas de laboratório servirão para que você possa desenvolver o projeto e tirar dúvidas.

- Também, pode assistir/implementar os seguintes vídeos:

  - https://youtu.be/nysfXweTI7o (programação com sockets)
  - https://youtu.be/knCXDo_NhGA (comunicação via RMI)
  - https://youtu.be/MVzWi14rLC8 (eleição em anel)
  - https://youtu.be/bCLspkNzFXU (hadoop em máquina virtual)

- **Não será cobrada presença.**

O que sim será abordado

- Fundamentos, técnicas, algoritmos, trade-offs.

O que não será abordado

- Instalação, implantação e detalhes de configuração de uma tecnologia específica.

Sistemas Distribuídos

Introdução aos Sistemas Distribuídos

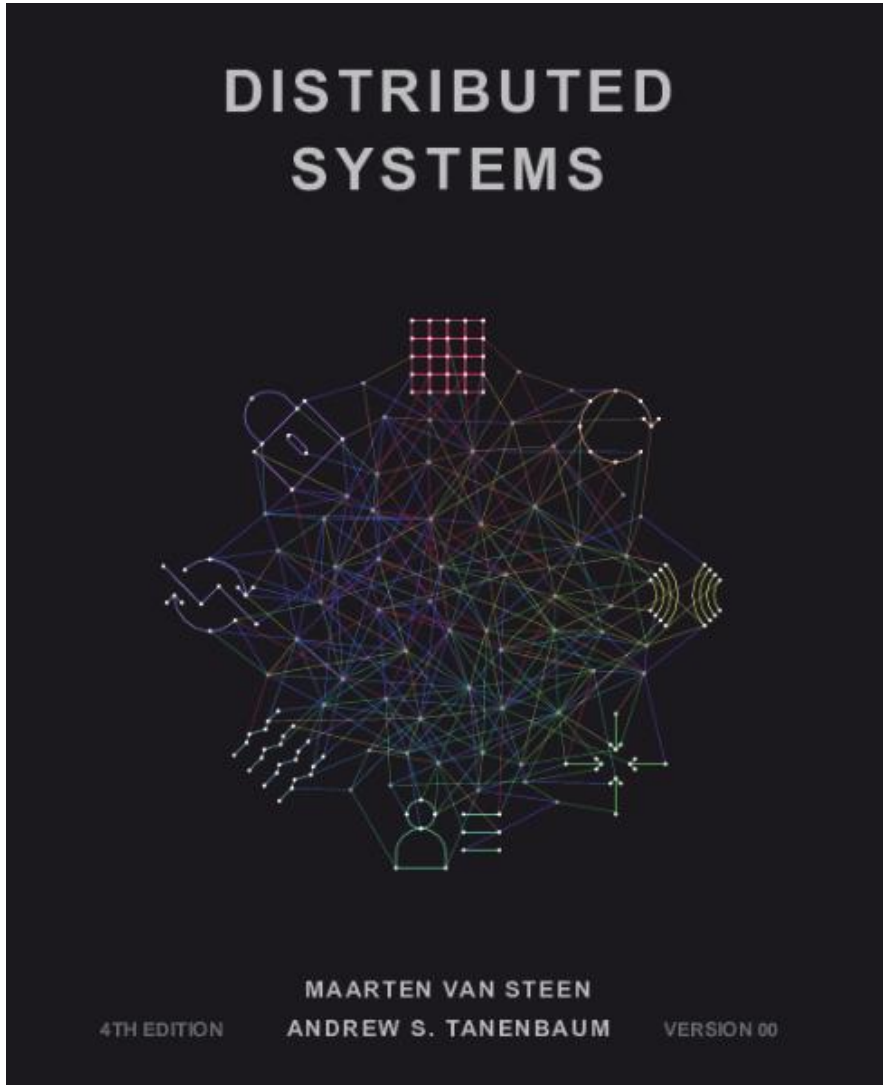
Introduction

Vladimir Rocha (Vladi)

CMCC - Universidade Federal do ABC

Distributed Systems 4th edition (free)

https://www.distributed-systems.net/index.php/books/ds4/



4.03 (2025)

Disclaimer

- Estes slides foram baseados nos do professor **Emilio Francesquini** para o curso de Sistemas Distribuídos na UFABC.
- Este material pode ser usado livremente desde que sejam mantidos, além deste aviso, os créditos aos autores e instituições.
- Estes slides foram adaptados daqueles originalmente preparados (e gentilmente cedidos) pelo professor **Daniel Cordeiro, da EACH-USP** que por sua vez foram baseados naqueles disponibilizados online pelos autores do livro "Distributed Systems", 3ª Edição em: `https://www.distributed-systems.net`.

Agenda

- Definição de Sistemas Distribuídos
- Organização, Coerência, Middleware
- Objetivos
- Modelos de sistema
- Armadilhas

Agenda

- Definição de Sistemas Distribuídos
- Organização, Coerência, Middleware
- Objetivos
- Armadilhas

Sistemas Distribuídos: definição

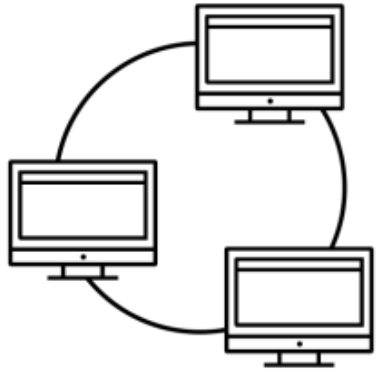
Um sistema distribuído é um sistema de software que garante: *uma coleção de elementos de computação autônomos que são vistos pelos usuários como um sistema único e coerente*

Características importantes

- Elementos de computação autônomos, também denominados *nós* (ou *nodos*), sejam eles dispositivos de hardware ou processos de software
- Sistema único e coerente: usuários ou aplicações veem um único sistema ⇒ nós precisam colaborar entre si



Google



Coleção de nós autônomos

Comportamento independente

Cada nó é autônomo e, portanto, tem sua própria percepção de tempo: não há um relógio global. Leva a problemas fundamentais de sincronização e de coordenação.

Coleção de nós

- Como gerenciar *associações em grupos*
- Como saber se você realmente está se comunicando com um *(não-)membro autorizado do grupo*

Agenda

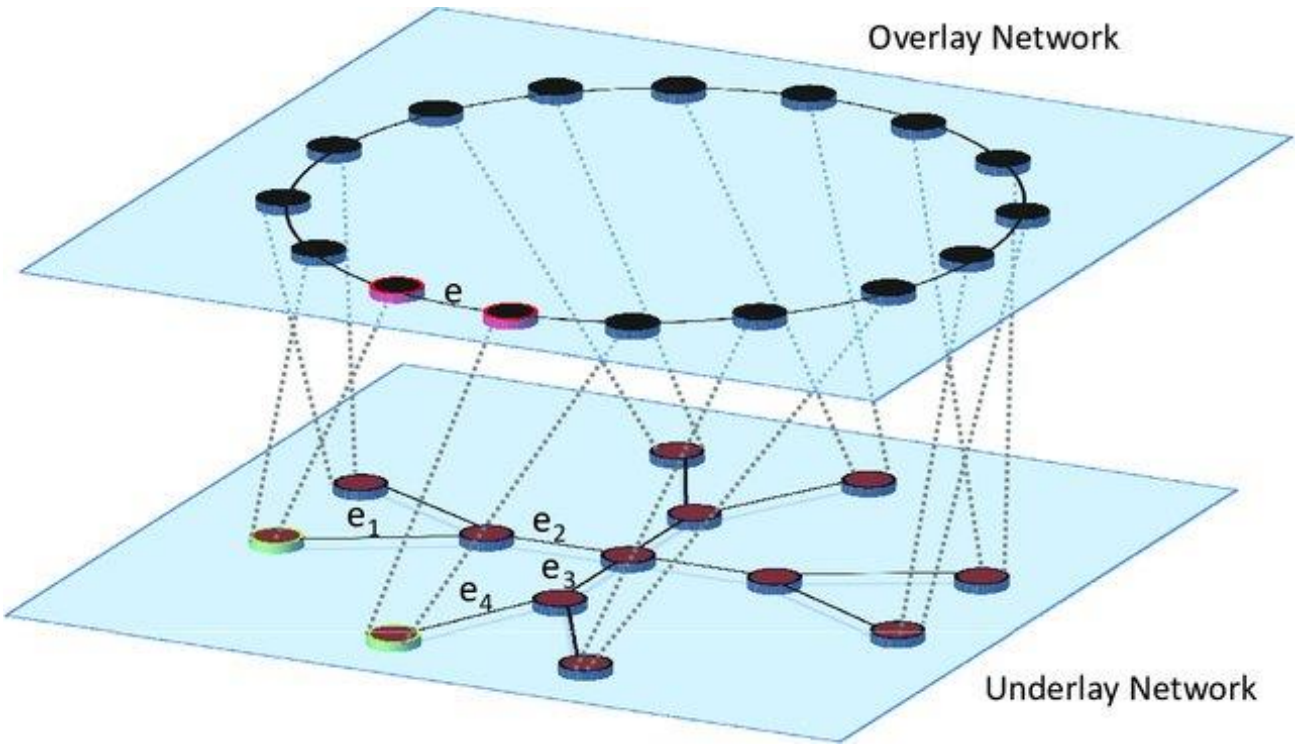
- Definição de Sistemas Distribuídos
- Organização, Coerência, Middleware
- Objetivos
- Armadilhas

Organização

Redes de overlay

Cada nó na coleção se comunica apenas com nós no sistema, seus vizinhos. O conjunto de vizinhos pode ser dinâmico, ou pode ser descoberto de forma implícita (ex: pode ser necessário procurá-lo)

Não necessariamente estão associados à estrutura física.



https://www.researchgate.net/publication/230774628_Harnessing_Complex_Structures_and_Collective_Dynamics_in_Large_Networked_Computing_Systems

Organização

Tipos de overlay: estruturada e não estruturada

Um exemplo bem conhecido de redes de overlay: *sistemas peer-to-peer*

A falha de um peer não compromete o funcionamento do sistema.

Organização

Tipos de overlay: estruturada e não estruturada

Um exemplo bem conhecido de redes de overlay: *sistemas peer-to-peer*

Estruturada cada nó tem um *conjunto bem definido de vizinhos* com os quais pode comunicar (árvore, anel)



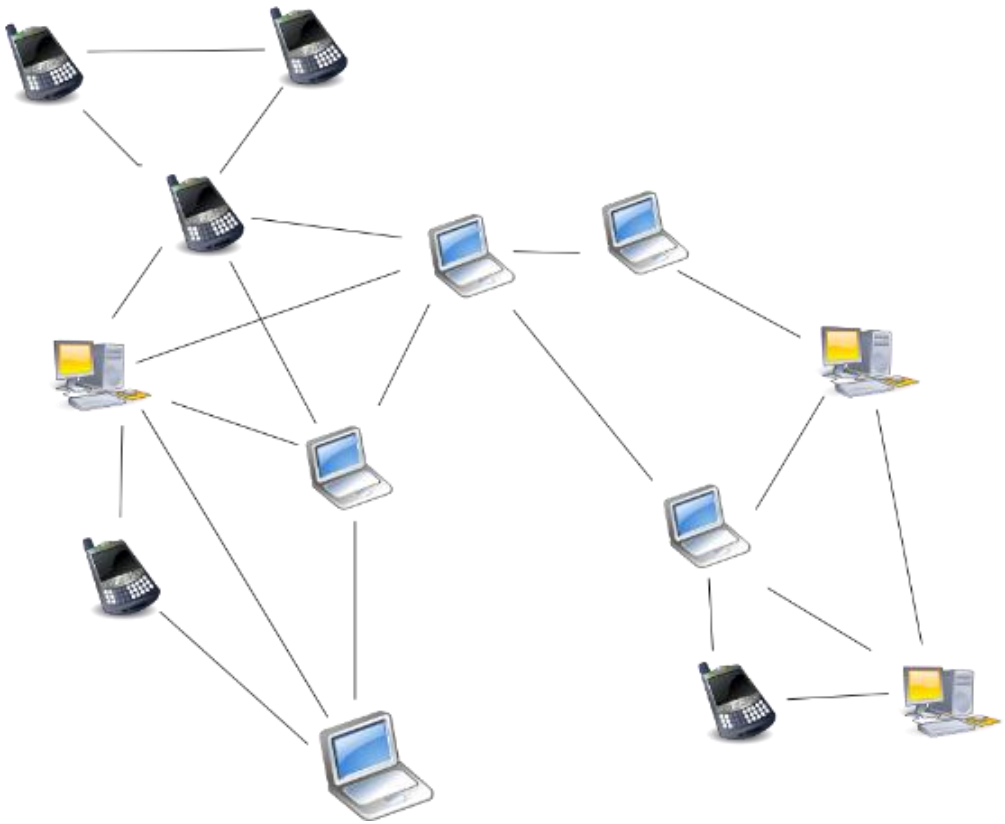
amazon cloudfront

Organização

Tipos de overlay: estruturada e não estruturada

Um exemplo bem conhecido de redes de overlay: *sistemas peer-to-peer*

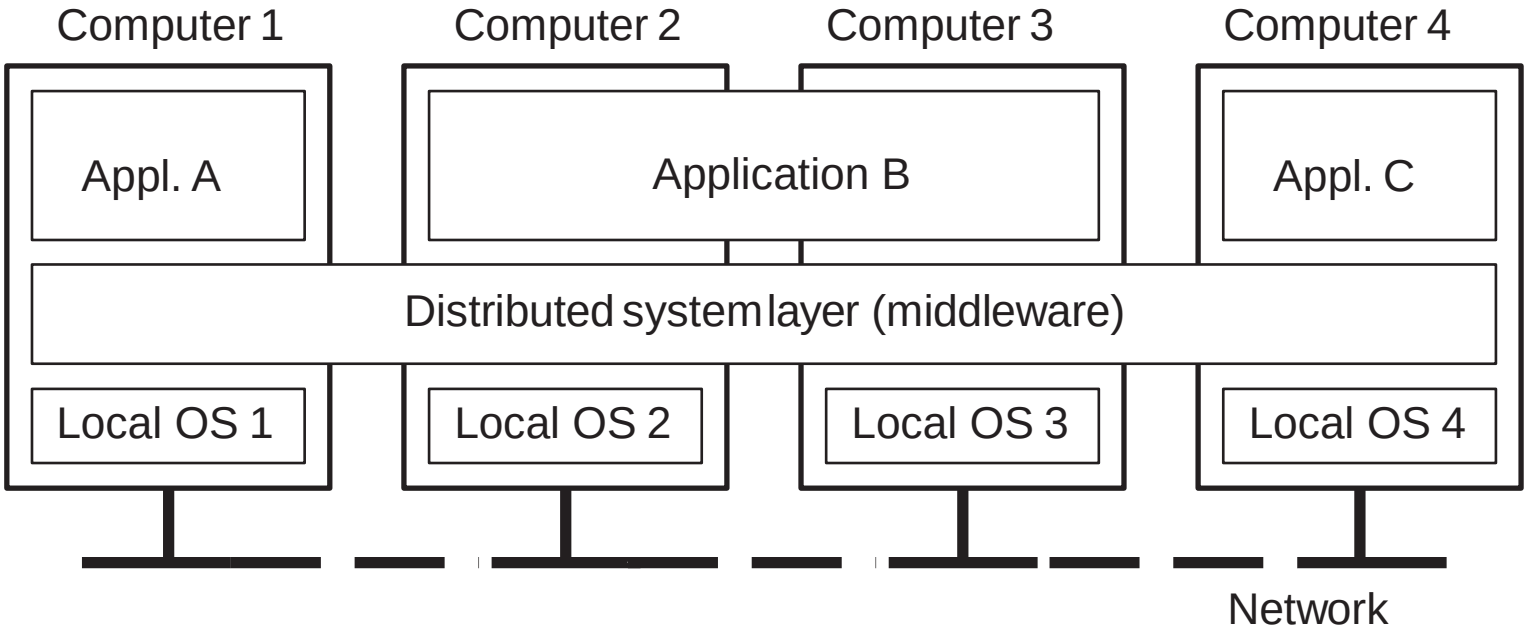
Não estruturada cada nó tem referências a *um conjunto aleatoriamente selecionado de outros nós* do sistema





https://en.wikipedia.org/wiki/Peer-to-peer

Middleware: o SO dos sistemas distribuídos



O que tem em um middleware?

Grosso modo, um conjunto de funções e componentes que não precisam ser reimplementados por cada aplicação separadamente.

Exemplos: middleware de replicação, de segurança, comunicação broadcast

Agenda

- Definição de Sistemas Distribuídos
- Organização, Coerência, Middleware
- Objetivos
- Armadilhas

Objetivos de sistemas distribuídos:

1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade

Objetivos de sistemas distribuídos:

1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade

Compartilhamento de recursos

Exemplos clássicos

- Compartilhamento de dados e arquivos na nuvem
- *Streaming* multimídia 
- Serviços de mensagens compartilhadas 
- Serviços de hospedagem web compartilhados (à lá redes de distribuição de conteúdo)
- Serviços de busca e processamento 

A ponto de pensarmos que:

"A rede é o computador"

John Gage, à época na Sun Microsystems

Objetivos de sistemas distribuídos:

1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade

Transparência de distribuição

Tipos:

Transparência	Descrição
Acesso	Esconder diferenças entre as representações de dados e mecanismos de invocação

Transparência de distribuição

Tipos:

Transparência	Descrição
Acesso	Esconder diferenças entre as representações de dados e mecanismos de invocação
Localização	Esconder onde o objeto está localizado
Relocalização	Esconder que um objeto pode ser movido para outra localidade *enquanto está sendo utilizado*
Migração	Esconder que um objeto pode ser movido para outra localidade
Replicação	Esconder que um objeto está sendo replicado
Concorrência	Esconder que um objeto pode ser compartilhado entre diferentes usuários independentes
Falhas	Esconder falhas e a possível recuperação de um objeto

Graus de transparência

Expor a distribuição pode ser bom:

- Fazer uso de serviços baseados na localização (encontrando seus amigos mais próximos)
- Trabalhar com usuários em diferentes zonas horárias
- Quando é fácil para um usuário entender o que está acontecendo (e.g., falha no servidor pela conexão lenta).

Conclusão

- A transparência na distribuição é um bonito objetivo, mas alcançá-lo é uma questão bem diferente.

Objetivos de sistemas distribuídos:

1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade

Abertura de sistemas distribuídos

Sistemas distribuídos abertos

São capazes de interagir com outros sistemas abertos:

- devem respeitar interfaces bem definidas OK
- devem permitir a portabilidade de aplicações OK
- devem ser fáceis de estender OK
- devem ser facilmente interoperáveis NÃO



Objetivos de sistemas distribuídos:

1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade
    1. Tamanho, Geográfica, Administrativa
    2. Técnicas
    3. Problemas

Objetivos de sistemas distribuídos:

1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade
    1. Tamanho, Geográfica, Administrativa [Neuman 1994]
    2. Técnicas
    3. Problemas

Escalabilidade em Sistemas Distribuídos

Observação:

Muitos desenvolvedores de sistemas distribuídos modernos usam o adjetivo "escalável" sem deixar claro o porquê deles escalarem.

Escalabilidade em Sistemas Distribuídos

Observação:

Muitos desenvolvedores de sistemas distribuídos modernos usam o adjetivo "escalável" sem deixar claro o porquê deles escalarem.

Escalabilidade se refere a pelo menos três componentes:

- Número de usuários e/ou processos – escalabilidade de tamanho
- Distância entre nós – escalabilidade geográfica
- Número de domínios administrativos – escalabilidade administrativa

Escalabilidade em Sistemas Distribuídos

Observação:

Muitos desenvolvedores de sistemas distribuídos modernos usam o adjetivo "escalável" sem deixar claro o porquê deles escalarem.

Escalabilidade se refere a pelo menos três componentes:

- Número de usuários e/ou processos – escalabilidade de tamanho
- Distância entre nós – escalabilidade geográfica
- Número de domínios administrativos – escalabilidade administrativa

Observação:

A maior parte dos sistemas escalam apenas (e até certo ponto) em tamanho. Como?

Usando servidores poderosos – *scale-in*.

Hoje em dia, o desafio é conseguir escalabilidade geográfica e administrativa.

Problemas para obtenção de escalabilidade

Um sistema completamente **descentralizado** tem as seguintes características:

- Nenhuma máquina tem informação completa sobre o estado do sistema (composto por várias máquinas)
- Máquinas tomam decisões baseadas apenas em informação local
- Falhas em uma máquina não devem arruinar a execução do algoritmo
- Não é possível assumir a existência de um relógio global

Problemas na escalabilidade de tamanho

Número de usuários e/ou processos – escalabilidade de tamanho

- Capacidade computacional, limitada pelas CPUs
- Capacidade de armazenamento e de transferência entre CPU e HD
- Capacidade de largura de banda entre o usuário e o serviço.

Resumo da notícia

- Amazon Prime Vídeo decepciona fãs ao não conseguir transmitir os jogos da NBA; fornecedor externo teria causado o problema
- Transmissões ao vivo são o calcanhar de Aquiles das plataformas de streaming, mas se tornaram essenciais para atrair assinantes com esportes

https://www.uol.com.br/splash/colunas/guilherme-ravache/2022/10/23/caos-da-nba-na-amazon-expoe-riscos-da-gigante-que-busca-dominar-esportes.htm

Como estimar a capacidade?

Problemas na escalabilidade de tamanho

Número de usuários e/ou processos – escalabilidade de tamanho

- *Back-of-the-Envelope estimation*

Serve para estimar a capacidade e desempenho do sistema para entender quais serão os requisitos (e.g., vazão e armazenamento) a serem atendidos.

Exemplo do Twitter:

400 milhões de usuários ativos em 2023.
50% dos usuários usam o Twitter diariamente.
Cada usuário envia na média 2 tweets por dia.
10% dos tweets têm arquivos de mídia de 1 MB.
Os dados devem ser armazenados por 5 anos.

Tweets por seg. TPS?

Total HD da mídia?

Problemas na escalabilidade de tamanho

Exemplo do Twitter:

400 milhões de usuários ativos em 2023.
50% dos usuários usam o Twitter diariamente.
Cada usuário envia na média 2 tweets por dia.
10% dos tweets têm arquivos de mídia de 1 MB.
Os dados devem ser armazenados por 5 anos.

Vazão por segundo:

- Usuários ativos por dia: 400 milhões * 0.5 = 200 milhões.
- TPS: 200 milhões * 2 tweets / (24 horas * 3600 seg) = ~ 4.629,63.
- **Picos TPS: 2 * TPS** = ~ 9.259,26.

Picos de 2x é suficiente?

Armazenamento (só da mídia):

- Por dia: 200 milhões * 2 tweets * 0.1 * 1 MB = 38,147 TB.
- Para 5 anos: 38,147 TB * 365 dias * 5 anos = 67,98 PB.

Problemas na escalabilidade de tamanho

Exemplo do Ticketmaster em novembro de 2022 (10x)

Media outlets described the demand for the Eras Tour's tickets as "astronomical",[1] with 3.5 million people registering for the Ticketmaster's Verified Fan pre-sale program in the U.S. When the sale went online on November 15, 2022, the website crashed in an hour, with users logged out or in a frozen queue …

https://en.wikipedia.org/wiki/Taylor_Swift%E2%80%93Ticketmaster_controversy

Problemas na escalabilidade geográfica

Distância máxima entre nós – escalabilidade geográfica

- Não é possível só ir da LAN para WAN: muitos dos sistemas distribuídos assumem interações cliente-servidor com latências pequenas.
- Links das WAN são pouco confiáveis: e.g.mover dados de streaming de vídeo da LAN para WAN falhará.
- O broadcast da camada de rede não pode ser aplicado.

Problemas na escalabilidade administrativa

Número de domínios administrativos – escalabilidade administrativa

Essência

Políticas conflitantes a respeito do uso (e pagamento), gerenciamento e segurança.

Exemplos

- Compartilhamento de recursos caros entre diferentes domínios.
- Processamento de informações em diferentes lugares de forma confidencial.
- Onde armazenar e para quem disponibilizar os dados mantendo políticas como LGPD.

Objetivos de sistemas distribuídos:

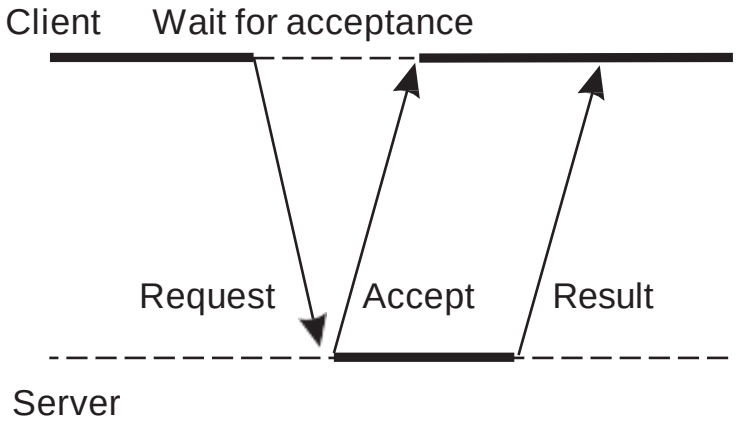
1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade
    1. Tamanho, Geográfica, Administrativa
    2. Técnicas
    3. Problemas

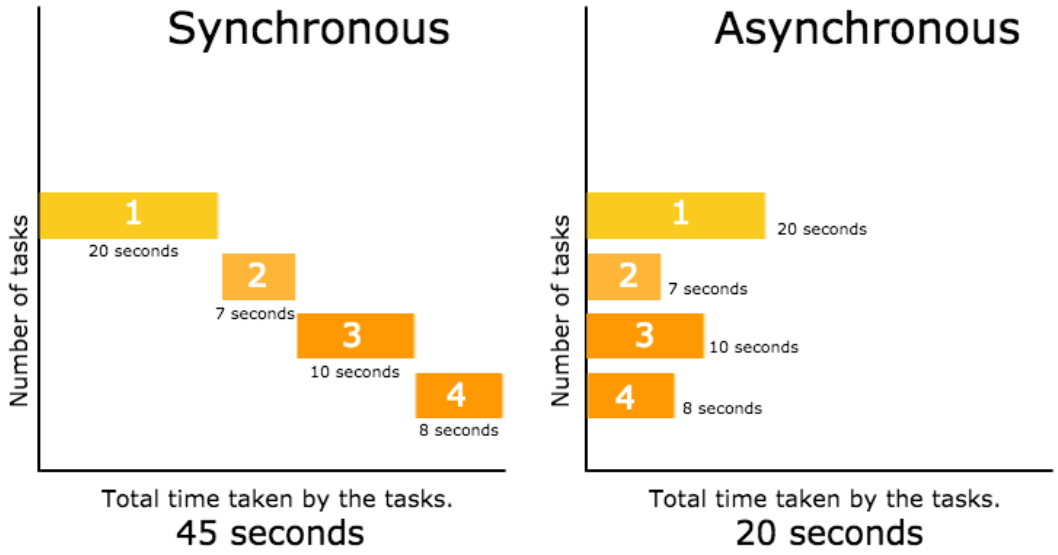
Técnicas de escalabilidade

Ideia geral: esconder latência de comunicação

1. Não fique esperando por respostas; faça outra coisa

- Use comunicação assíncrona → não confundir com modelo assíncrono
- Use diferentes *handlers* para tratamento de mensagens (multithread)
- Problema: nem toda aplicação se encaixa nesse modelo





Técnicas de escalabilidade

2. Particionamento de dados e computação em muitas máquinas

- Mova a computação para os clientes (ex: Javascript, Spark, etc.)
- Serviços de nomes decentralizados (DNS)
- Sistemas de informação decentralizados (WWW)
- Microsserviços
- Sharding (particionamento) de tabelas de bancos de dados ou de objetos

Escalabilidade Vertical/Horizontal (Scale-In/Scale-Out)

Técnicas de escalabilidade

3. Replicação/caching

Faça cópias dos dados e disponibilize-as em diferentes máquinas:

- Bancos de dados e sistemas de arquivos replicados
- Sites web "espelhados"
- Caches web (nos navegadores)
- Cache de arquivos (no servidor e nos clientes)
- Cache de dados (nos servidores – memcached)

Objetivos de sistemas distribuídos:

1. Disponibilização de recursos compartilhados
2. Transparência de distribuição
3. Abertura
4. Escalabilidade
    1. Tamanho, Geográfica, Administrativa
    2. Técnicas
    3. Problemas

Escalabilidade – O Problema

Observação

Aplicar técnicas para obtenção de escalabilidade é fácil, exceto por:

- Manter múltiplas cópias (em cache ou replicadas) leva a inconsistências: a modificação em uma cópia a torna diferente das demais
- Manter as cópias consistentes requer sincronização global em cada modificação (mas não há um relógio global)
- Sincronização global impossibilita soluções escaláveis

Escalabilidade – O Problema

Observação

Aplicar técnicas para obtenção de escalabilidade é fácil, exceto por:

- Manter múltiplas cópias (em cache ou replicadas) leva a inconsistências: a modificação em uma cópia a torna diferente das demais
- Manter as cópias consistentes requer sincronização global em cada modificação (mas não há um relógio global)
- Sincronização global impossibilita soluções escaláveis

Observação:

Se pudermos tolerar inconsistências, poderíamos reduzir a dependência de sincronização globais, mas tolerar inconsistências é algo que depende da aplicação.

Agenda

- Definição de Sistemas Distribuídos
- Organização, Coerência, Middleware
- Objetivos
- Modelos de sistema
- Armadilhas

Modelos de sistema [tanenbaum17]

- Síncrono
- Assíncrono
- Parcialmente Síncrono

Não confundir com comunicação síncrona e assíncrona entre cliente-servidor

Modelos de sistema [tanenbaum17]

Síncrono

- Os limites de tempo para transferir uma mensagem são conhecidos
- Os limites de tempo para processar uma ação são conhecidos

  LAN/Datacenters

Assíncrono

- Sem limites de tempo para transferir uma mensagem
- Sem limites de tempo para processar uma ação

  Internet

Parcialmente síncrono

- Inicialmente o comportamento do sistema é assíncrono
- Eventualmente (sim ou sim) o comportamento será síncrono

  Internet

Agenda

- Definição de Sistemas Distribuídos
- Organização, Coerência, Middleware
- Objetivos
- Modelos baseados no tempo
- Armadilhas

Armadilhas no desenvolvimento de sistemas distribuídos

Observação:

Muitos sistemas distribuídos se tornam desnecessariamente complexos por causa de "consertos" ao longo do tempo. Em geral, há muitas hipóteses falsas:

- A rede é confiável
- A rede é segura
- A rede é homogênea
- A topologia da rede não muda
- A latência é zero
- **Largura de banda é infinita**
- O custo de transporte é zero
- A rede possui um administrador

Conceitos adquiridos

- Definição de sistemas distribuídos.
- Não há relógio global.
- Overlay e Middleware.
- Transparência
- Escalabilidade.
- Cálculo *back-of-the-envelope*.
- Replicação e inconsistência.
- Comunicação:
    - Síncrona, Assíncrona
- Modelos de sistema:
    - Síncrono, Assíncrono, Parcialmente síncrono.

Sistemas Distribuídos

Tipos de Sistemas Distribuídos

Vladimir Rocha (Vladi)

CMCC - Universidade Federal do ABC

Disclaimer

- Estes slides foram baseados nos do professor **Emilio Francesquini** para o curso de Sistemas Distribuídos na UFABC.
- Este material pode ser usado livremente desde que sejam mantidos, além deste aviso, os créditos aos autores e instituições.
- Estes slides foram adaptados daqueles originalmente preparados (e gentilmente cedidos) pelo professor **Daniel Cordeiro, da EACH-USP** que por sua vez foram baseados naqueles disponibilizados online pelos autores do livro "Distributed Systems", 3ª Edição em: https://www.distributed-systems.net.

Agenda

Tipos de Sistemas Distribuídos

- Sistemas para computação distribuída de alto desempenho
- Sistemas de informação distribuídos
- Sistemas distribuídos para computação pervasiva

Agenda

Tipos de Sistemas Distribuídos

- Sistemas para computação distribuída de alto desempenho
  - Cluster
  - Grade
  - Nuvem

Aglomerados de computação (cluster computing)

Essencialmente um grupo de computadores de boa qualidade conectados via LAN

- Homogêneo: mesmo SO, hardware quase idêntico
- Um único nó gerenciador



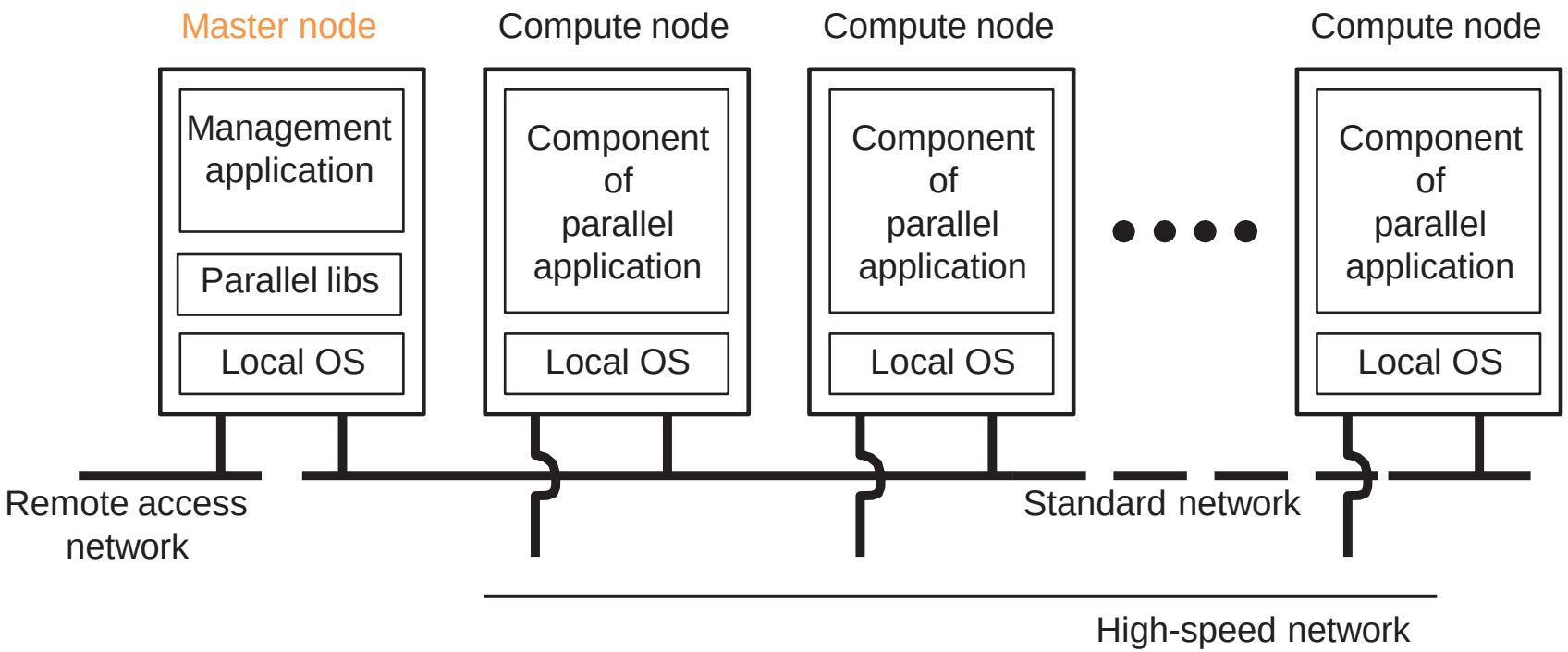


https://en.wikipedia.org/wiki/Computer_cluster

Aglomerados de computação (cluster computing)

Essencialmente um grupo de computadores de boa qualidade conectados via LAN

- Homogêneo: mesmo SO, hardware quase idêntico
- Um único nó gerenciador



Computação em Grade

O próximo passo: vários nós vindos de todos os cantos [Foster 2001]

- Heterogêneos
- Espalhados entre diversas organizações ou pessoas
- Normalmente formam uma rede de longa distância (*wide-area network*)

Nota:

Para permitir colaborações, grades normalmente usam *organizações virtuais*. Essencialmente, isso significa que os usuários (ou melhor, seus IDs) são organizados em grupos que possuem autorização para usar alguns recursos.

Computação em Grade



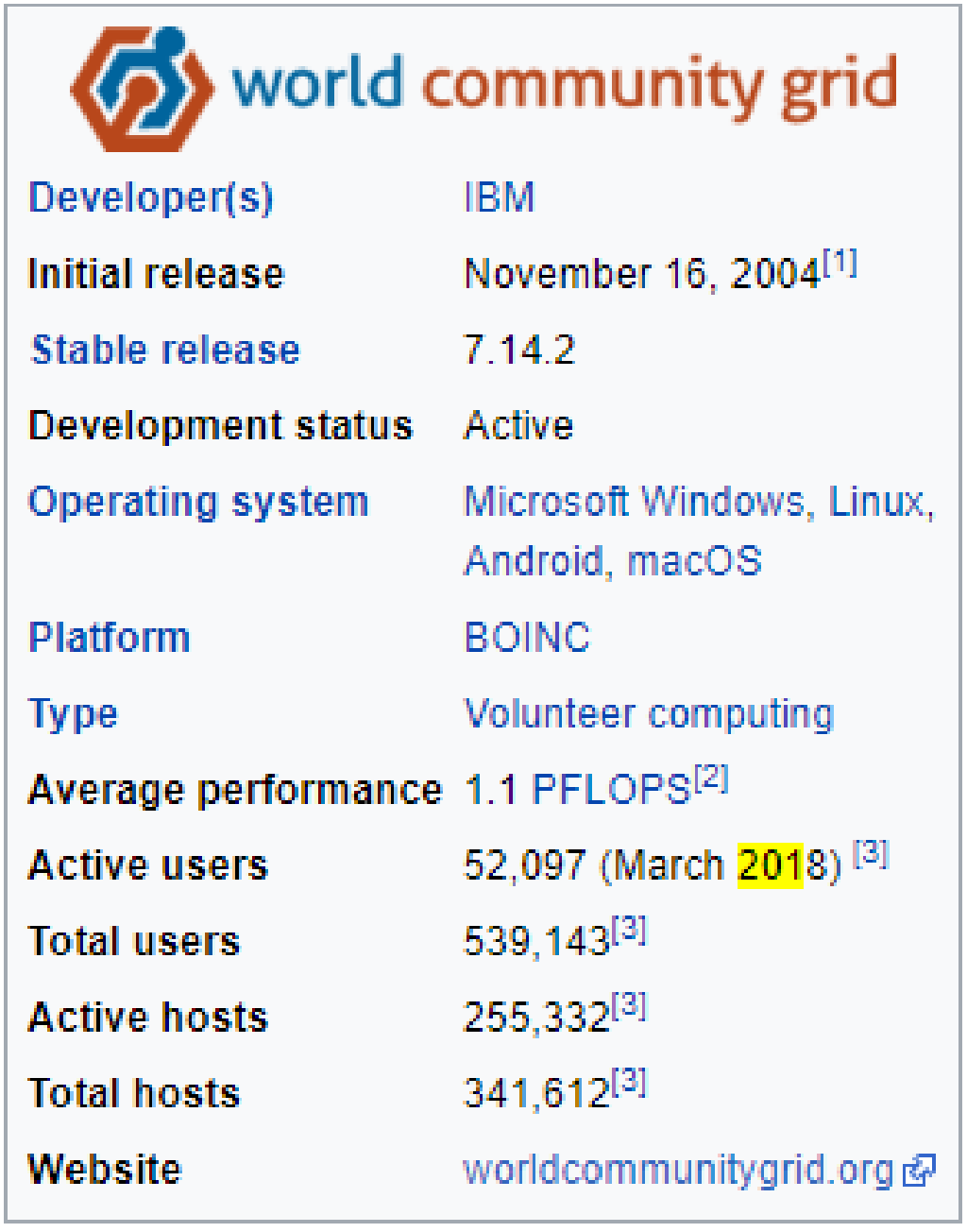
1999 ~ 2006 1 PFLOPS



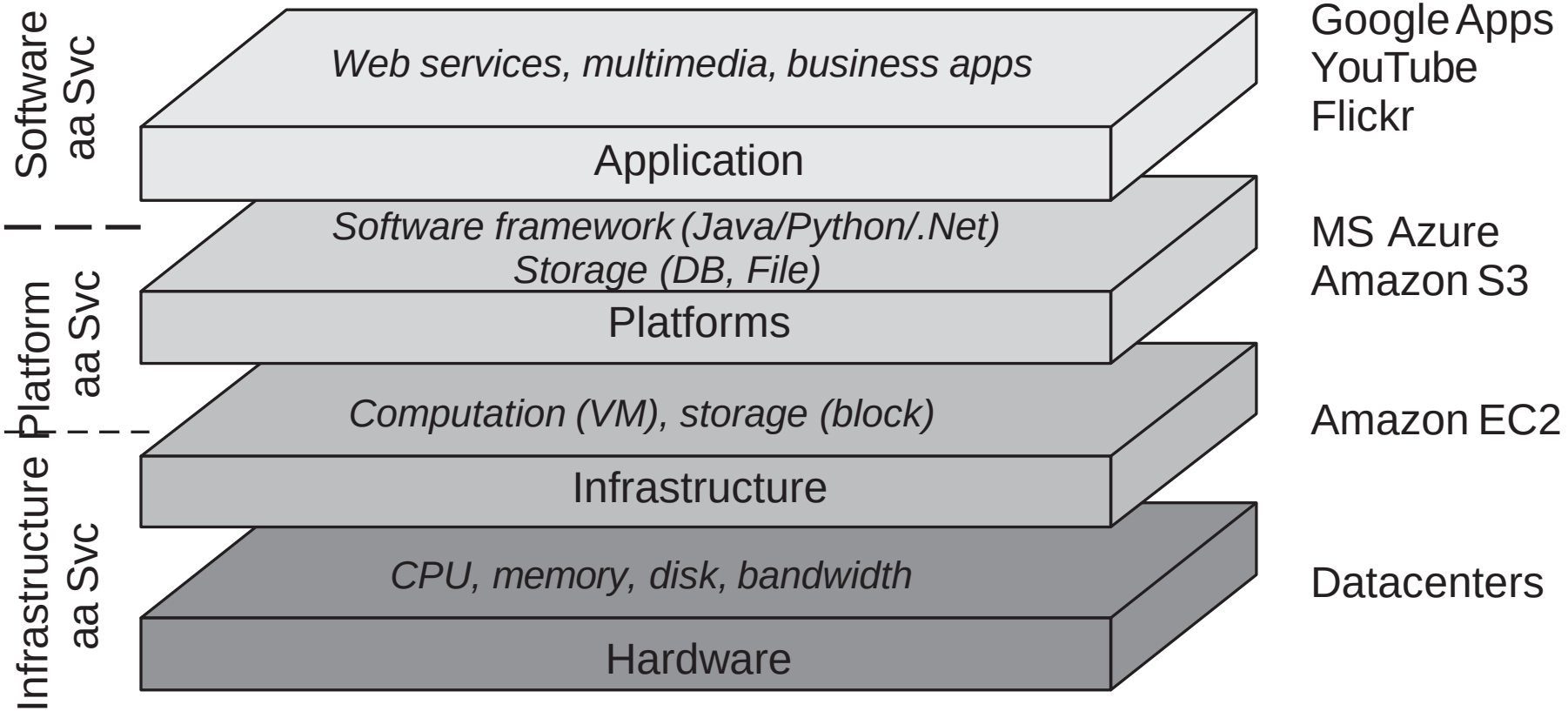
2003 ~ 2009



2000 ~ 2020+ 1100 PFLOPS



Computação em Nuvem



IaaS: cobre as camadas de hardware (escondida do cliente) e infraestrutura.

PaaS: cobre a camada de plataforma.

SaaS: cobre a camada de aplicação.

Computação em Nuvem

Computação em nuvem

Faz uma distinção entre quatro camadas:

Hardware processadores, roteadores, energia, sistemas de refrigeração

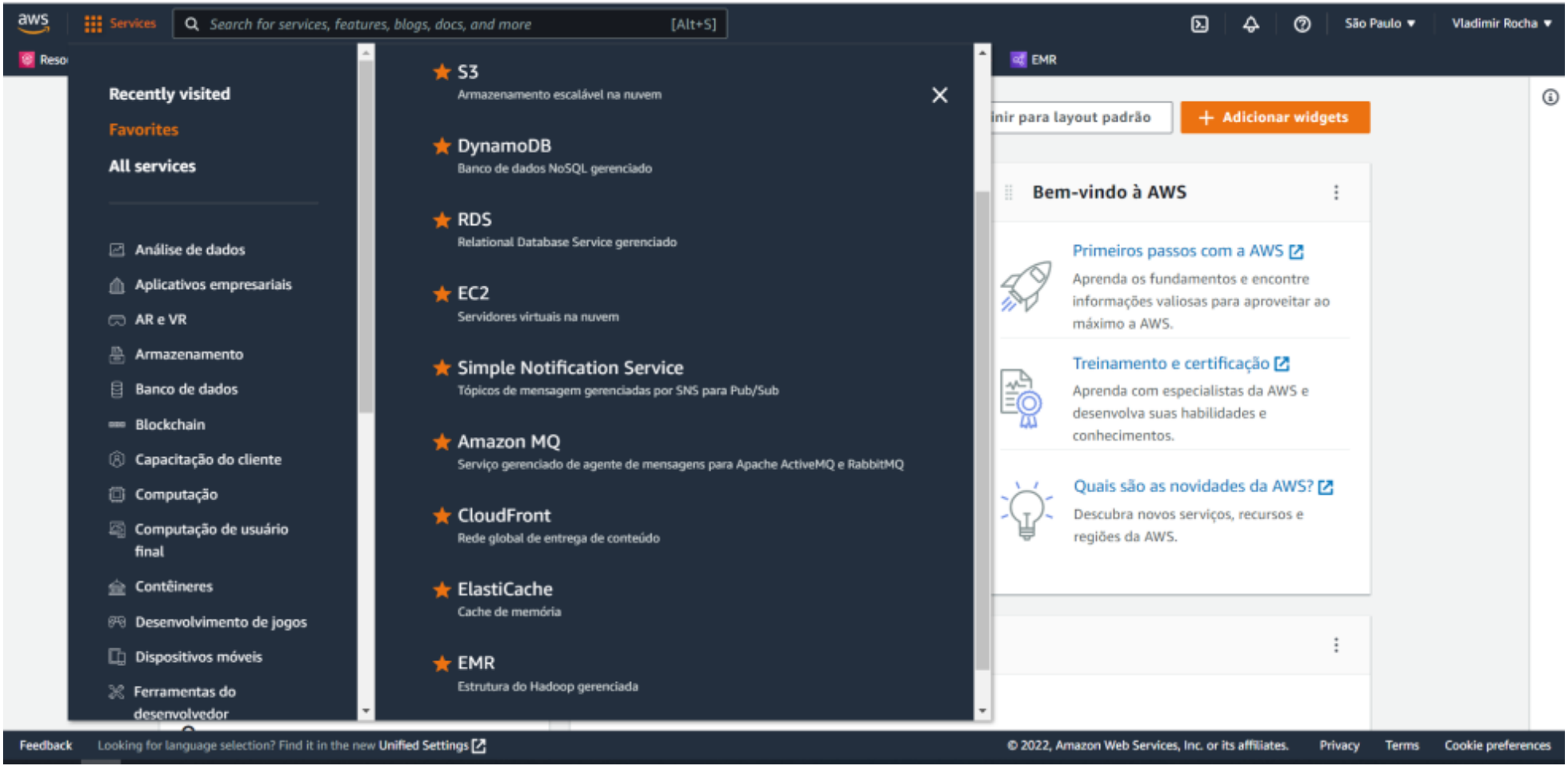
Infraestrutura Utiliza técnicas de virtualização para alocação e gerenciamento de armazenamento e servidores virtuais

Plataforma Provê abstrações de alto nível para os serviços da plataforma. Ex: Amazon S3 para armazenamento de arquivos em *buckets*

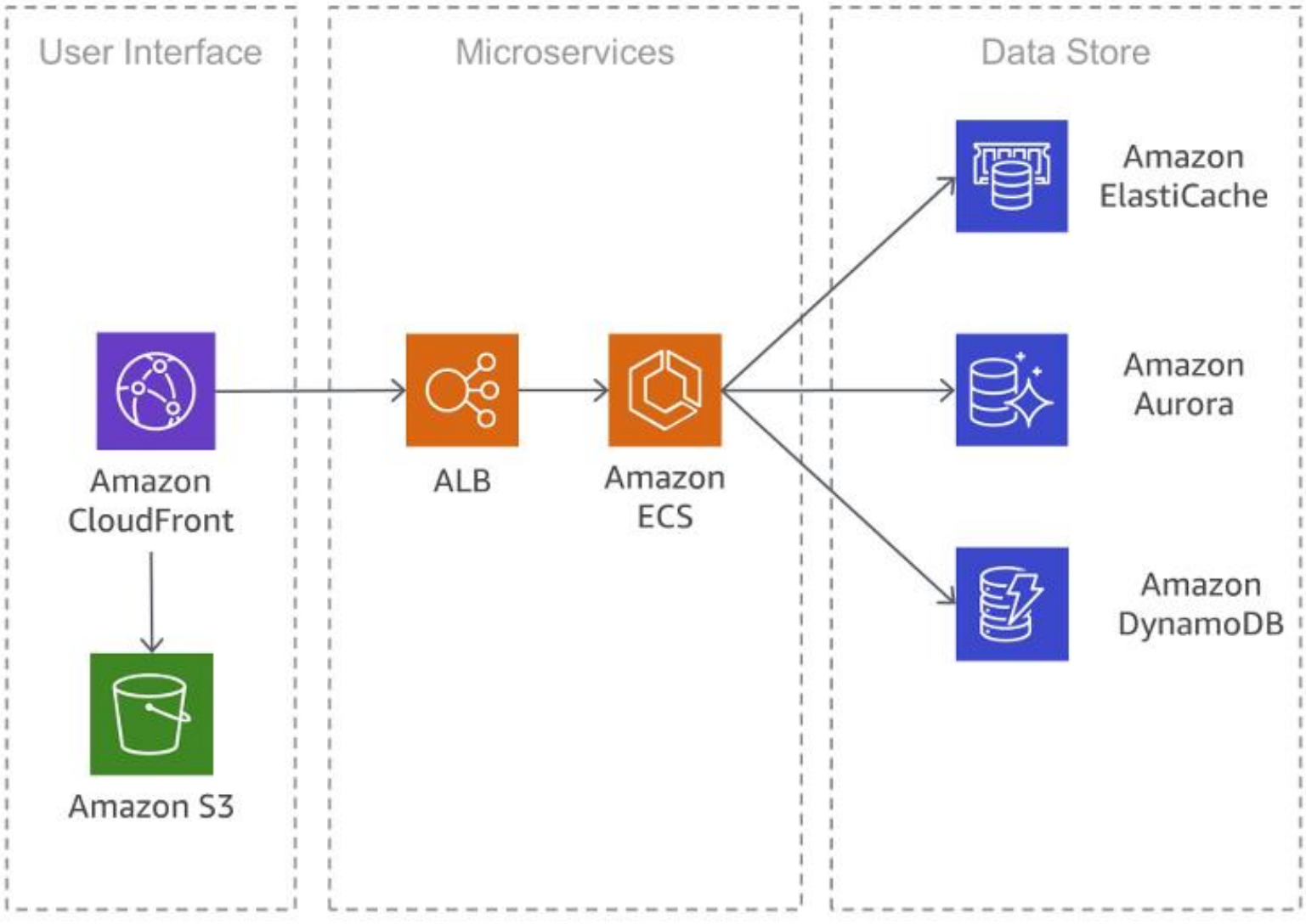
Aplicação as aplicações propriamente ditas, tais como as suítes de aplicativos para escritórios.

Vídeo de como acessar a nuvem da Amazon e criar uma instância (opcional)
https://www.youtube.com/watch?v=_8erPrlPm9Y

Computação em Nuvem



Computação em Nuvem



https://docs.aws.amazon.com/whitepapers/latest/microservices-on-aws/simple-microservices-architecture-on-aws.html

Agenda

Tipos de Sistemas Distribuídos

- Sistemas para computação distribuída de alto desempenho
- Sistemas de informação distribuídos
- Sistemas distribuídos para computação pervasiva

Sistemas de Informação Distribuídos

Observação:

Uma quantidade enorme de sistemas em uso hoje em dia são formas de sistemas de informação tradicionais.

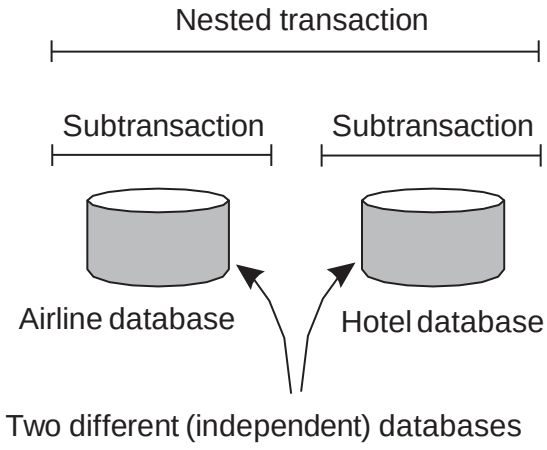
Geralmente são aplicações que se conectam à rede para intercambiar informações.

As palavras-chave que caracterizam os sistemas de informação distribuído são transação e integração.

Sistemas de Informação Distribuídos

Sistemas de processamento de transações.

```
01 BEGIN_TRANSACTION(transaction tx)
02 READ(tx, air_db, data-air)
03 READ(tx, hotel_db, data-hotel)
04 updated := WRITE(tx, air_db, new_data_air)
05 IF NOT updated THEN
06        ABORT_TRANSACTION(tx)
07 ELSE
08        WRITE(tx, hotel_db, new_data_hotel)
09 END_TRANSACTION(tx)
10 END IF
```



Nota:

Transações formam uma operação atômica.

Se cair em qualquer lugar: ABORT_TRANSACTION

Sistemas de Informação Distribuídos: Transações

Uma transação é um conjunto de operações sobre o estado de um objeto (banco de dados, composição de objetos, etc.) que satisfazem as seguintes propriedades (ACID):

Atomicidade ou todas as operações são bem sucedidas, ou todas falham. Quando uma transação falha, o estado do objeto permanecerá inalterado.

Consistência uma transação estabelece um estado de transição válido. Isto não exclui a existência de estados intermediários inválidos durante sua execução.

Isolamento transações concorrentes não interferem entre si. Para uma transação T é como se as outras transações ocorressem ou *antes* de T, ou *depois* de T.

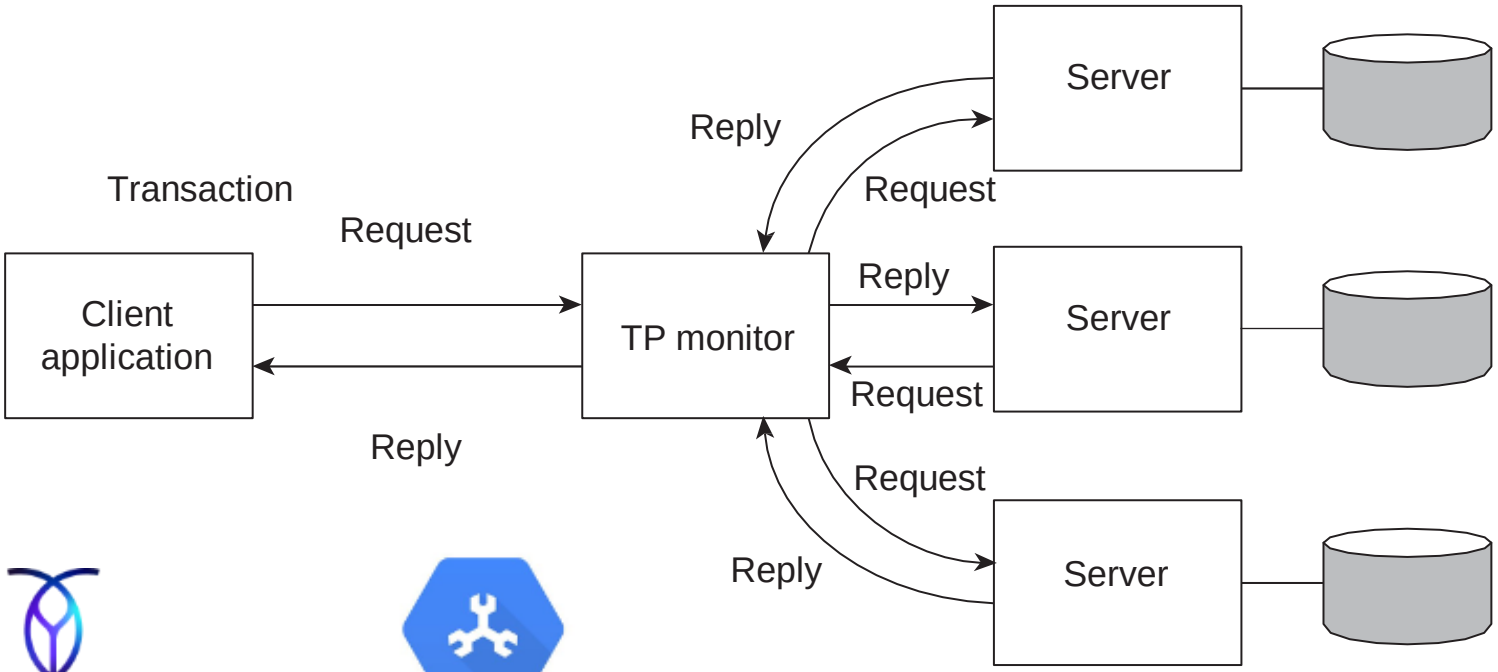
Durabilidade Após o término de uma transação, seus efeitos são permanentes: mudanças de estado sobrevivem a falhas.

Monitor de Processamento de Transações

Capítulo 8

Observação:

Em muitos casos, o conjunto de dados envolvidos em uma transação está distribuído em vários servidores. Um TP Monitor é responsável por coordenar a execução de uma transação.



Distributed Transactions

CockroachDB

[2015 -]

Cloud Spanner

[2012 -]

Integração de Aplicações Corporativas

Situação

As organizações posuem diversas aplicações muitas delas sem interoperabilidade.

Solução básica analisada (monitor)

Combinar as requisições das aplicações em somente um servidor, quem realizará o envio e coleta das respostas, apresentando ao cliente um resultado coerente.

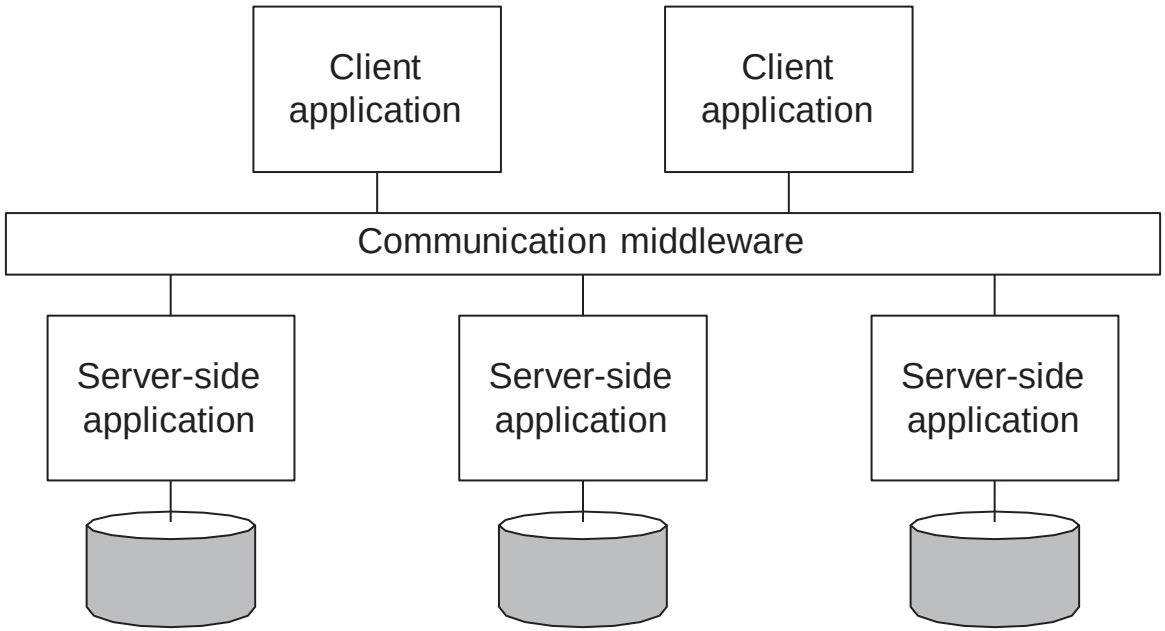
Próximo passo

Integrar direamente a comunicação entre aplicações, levando ao EAI (Enterprise Application Integration).

Integração de Aplicações Corporativas

Problema

Um TP Monitor não basta, também são necessários mecanismos para a comunicação direta entre aplicações.

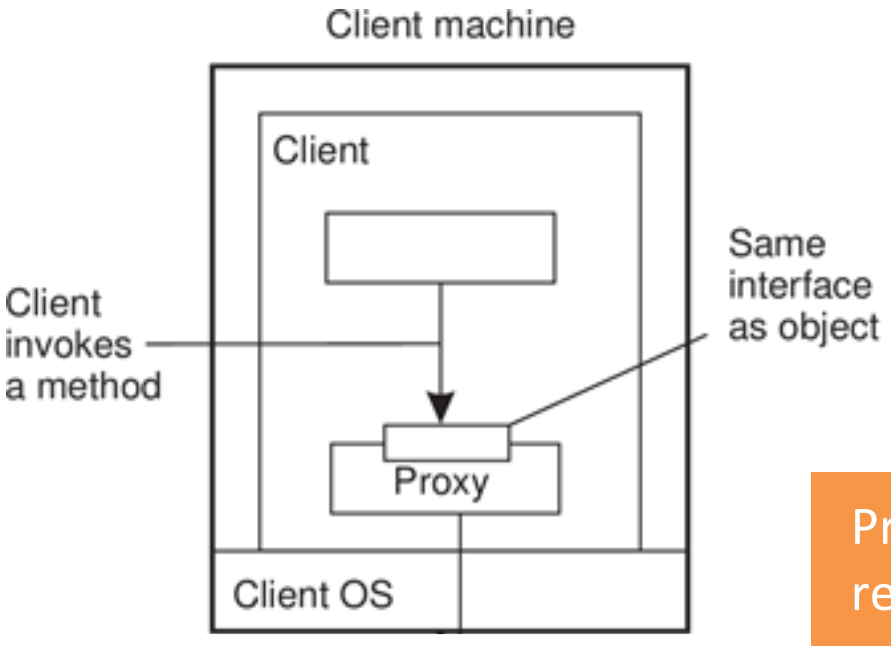


- Chamada de Procedimento Remoto (RPC) [Nelson 1984]

  Java RMI [Sun 2002]

  Google RPC (GRPC) [Google 2015]
- Middleware Orientado a Mensagens (MOM)

Como integrar as aplicações: RPC

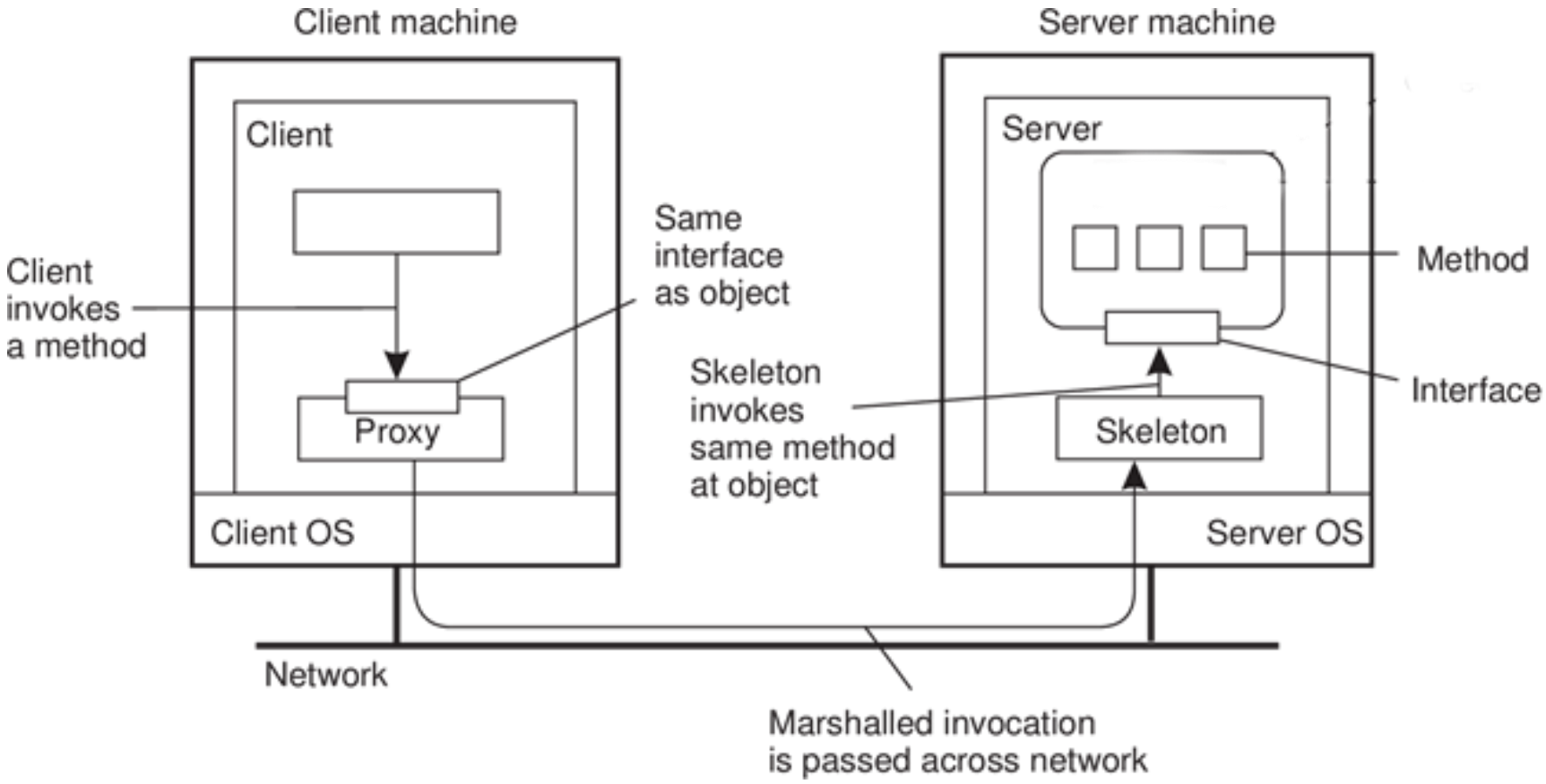


Proxy: componente que encaminha requisições ao servidor responsável.

O cliente faz chamadas ao método como estivesse sendo executado na máquina local.

Mas o método pode estar sendo executado em máquinas remotas.

Como integrar as aplicações: RPC



O cliente faz chamadas ao método como estivesse sendo executado na máquina local.

Mas o método pode estar sendo executado em máquinas remotas.

Agenda

Tipos de Sistemas Distribuídos

- Sistemas para computação distribuída de alto desempenho
- Sistemas de informação distribuídos
- Sistemas distribuídos para computação pervasiva
    - Ubíqua
    - Móvel
    - Sensores

Sistemas Pervasivos

Tendência em sistemas distribuídos; nós são pequenos, móveis e normalmente embutidos em um sistema muito maior.
Associado com a Internet das coisas (IoT – Internet of Things).

Três tipos (com sobreposição):

- Sistemas ubíquos: continuamente presentes
- Sistemas móveis: inerentemente móvel
- Sistemas de sensores: sente e atúa no ambiente.

Sistemas Ubíquos

Alguns requisitos:

- Mudança contextual: o sistema é parte de um ambiente onde mudanças devem ser rapidamente levadas em consideração
- Autonomia: nós operam de forma autônoma sem intervenção humana.
- Inteligência: pode emerger um comportamento inteligente dada a dinâmica e as interações dos nós.

Comportamento emergente: flocking
https://www.youtube.com/watch?v=9zLu5quQfYw



Nest Learning Thermostat + Nest Temperature Sensor

Sistemas Móveis

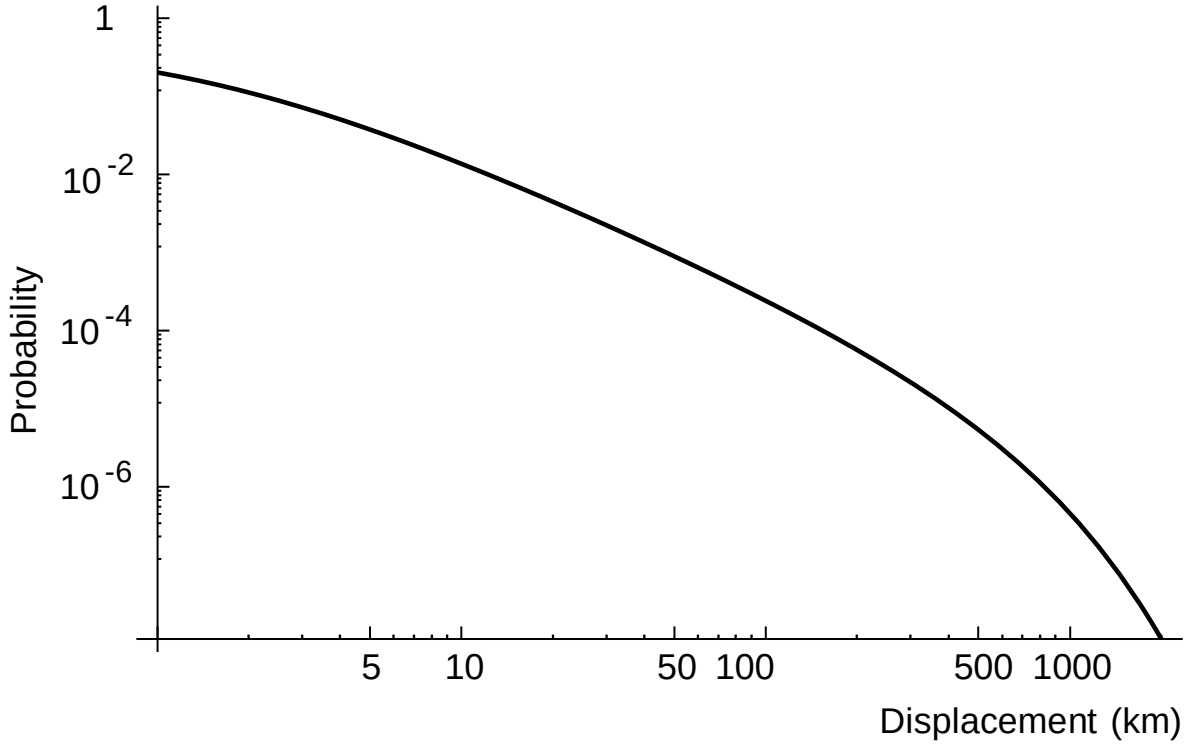
Miríade de dispositivos móveis: smartphones, tablets, óculos AR/VR, etc.

Características:

- Comunicação sem fio
- Mobilidade implica em que a localização do dispositivo mudará no tempo. Palavra-chave: descoberta
- A comunicação pode ser difícil, pois não há uma rota estável. Leva às redes tolerantes a disrupções

Padrões de mobilidade

Quão móveis somos? Rastreando 100 mil celulares durante seis meses [Gonzalez 2008][Schläpfer 2021].



Além disso: as pessoas tendem a voltar ao mesmo lugar depois de 24, 48 ou 72 horas ⇒ não somos tão móveis.

Redes de sensores

Características

Os nós aos quais os sensores estão presos são:

- Muitos (10s–1000s)
- Simples (pouca capacidade de memória/computação/comunicação)
- Normalmente necessitam de uma bateria

Crowd-sourced weather apps claim accuracy, but watch the sky anyway

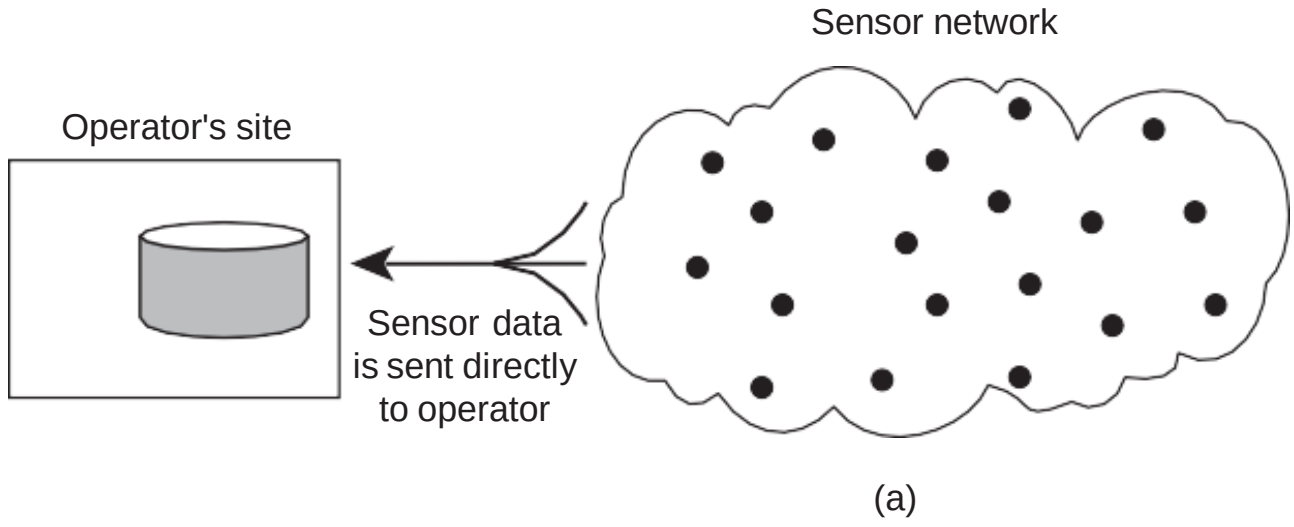
Barometer-equipped smartphones are slowly becoming a vast sensor network for weather data



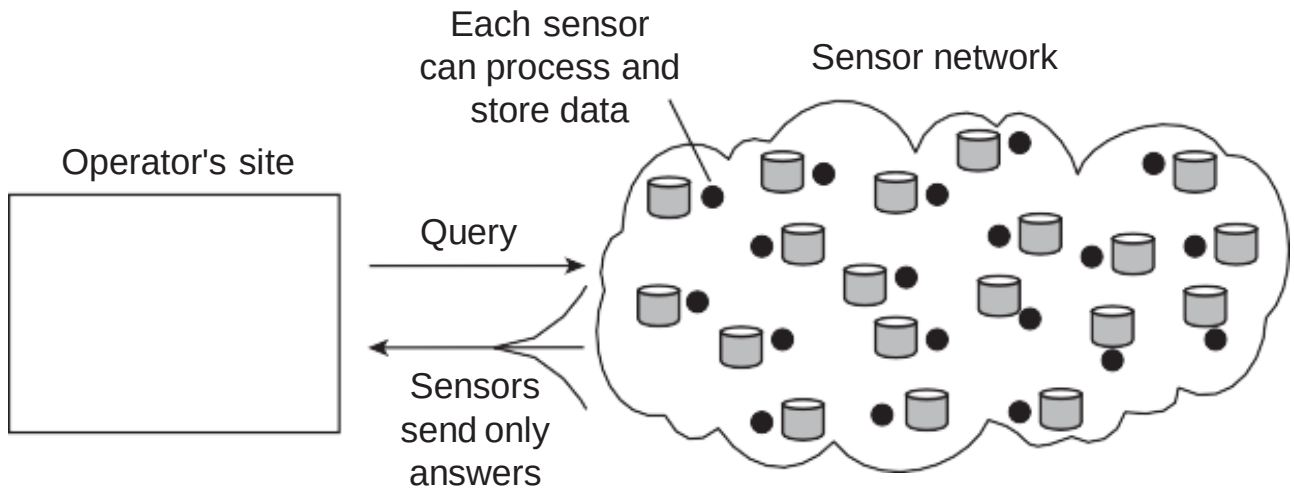
By Tim Hornyak

Tokyo Correspondent, IDG News Service | AUGUST 28, 2015 09:00 AM PT

Redes de sensores como um sistema distribuído



(a)



(b)

Cloud – Edge – Things

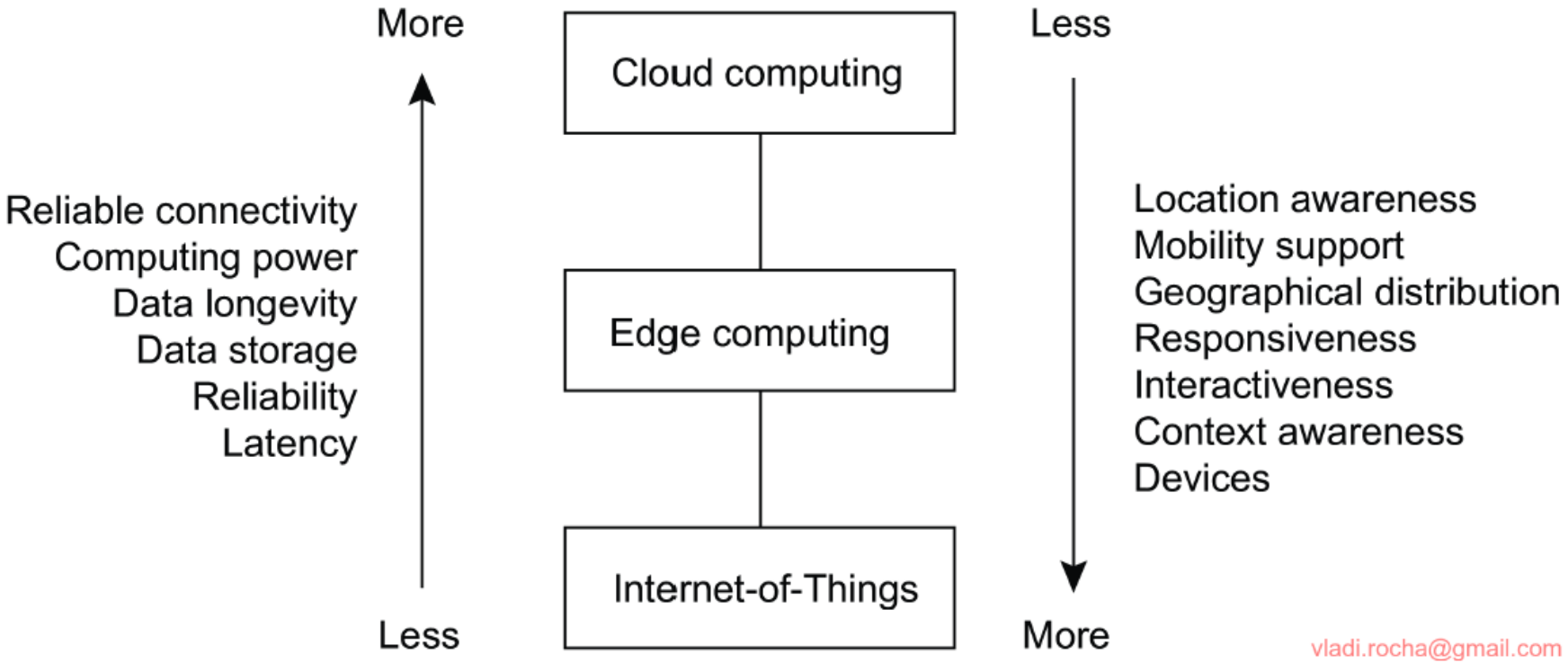


Figure 1.17: A hierarchical view from clouds to devices (adapted from Yousefpour et al. [2019]).

Conceitos adquiridos

- Cluster, grade e nuvem.
- Transações/ACID.
- Proxy.
- Sistemas móveis e redes de sensores.
- Edge, Cloud e IoT

Sistemas Distribuídos

Arquiteturas de Sistemas Distribuídos

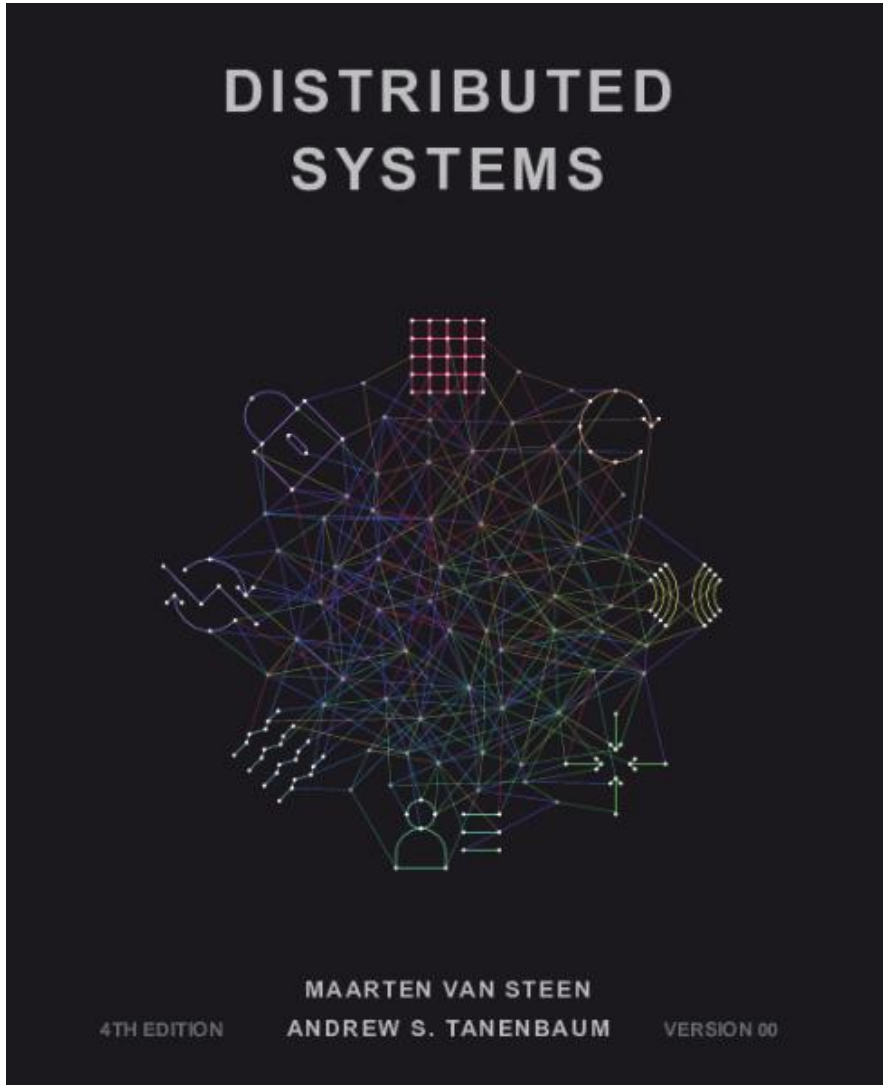
ARCHITECTURES

Vladimir Rocha (Vladi)

CMCC - Universidade Federal do ABC

Distributed Systems 4th edition (free)

https://www.distributed-systems.net/index.php/books/ds4/



4.03 (2025)

Disclaimer

- Estes slides foram baseados nos do professor **Emilio Francesquini** para o curso de Sistemas Distribuídos na UFABC.
- Este material pode ser usado livremente desde que sejam mantidos, além deste aviso, os créditos aos autores e instituições.
- Estes slides foram adaptados daqueles originalmente preparados (e gentilmente cedidos) pelo professor **Daniel Cordeiro, da EACH-USP** que por sua vez foram baseados naqueles disponibilizados online pelos autores do livro "Distributed Systems", 3ª Edição em: https://www.distributed-systems.net.

Agenda

- Estilos arquiteturais
- Arquiteturas de software
- Arquiteturas versus middleware
- Sistemas distribuídos autogerenciáveis

Agenda

- Estilos arquiteturais
- Arquiteturas de software
- Arquiteturas versus middleware
- Sistemas distribuídos autogerenciáveis

Estilos arquiteturais

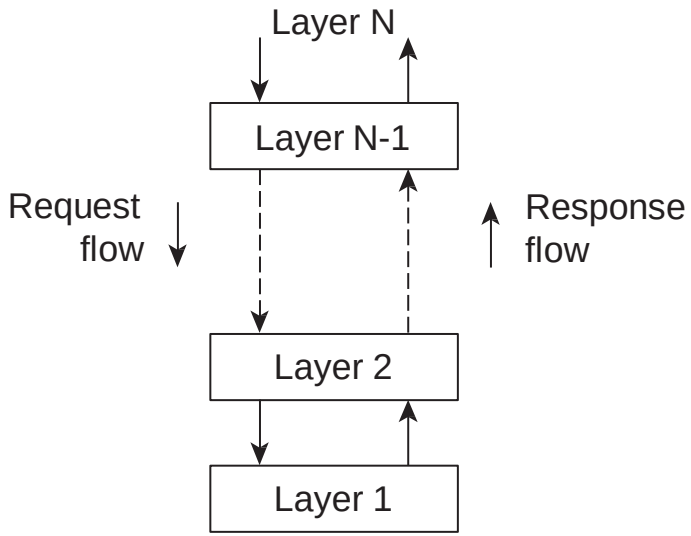
Estilo: conjunto de componentes interligados e que interagem entre si

1. Arquiteturas em camadas
2. Arquiteturas baseadas em objetos
3. Arquiteturas baseadas em eventos e dados
4. Arquiteturas centradas em recursos

1. Estilo arquitetural: camadas

Ideia básica

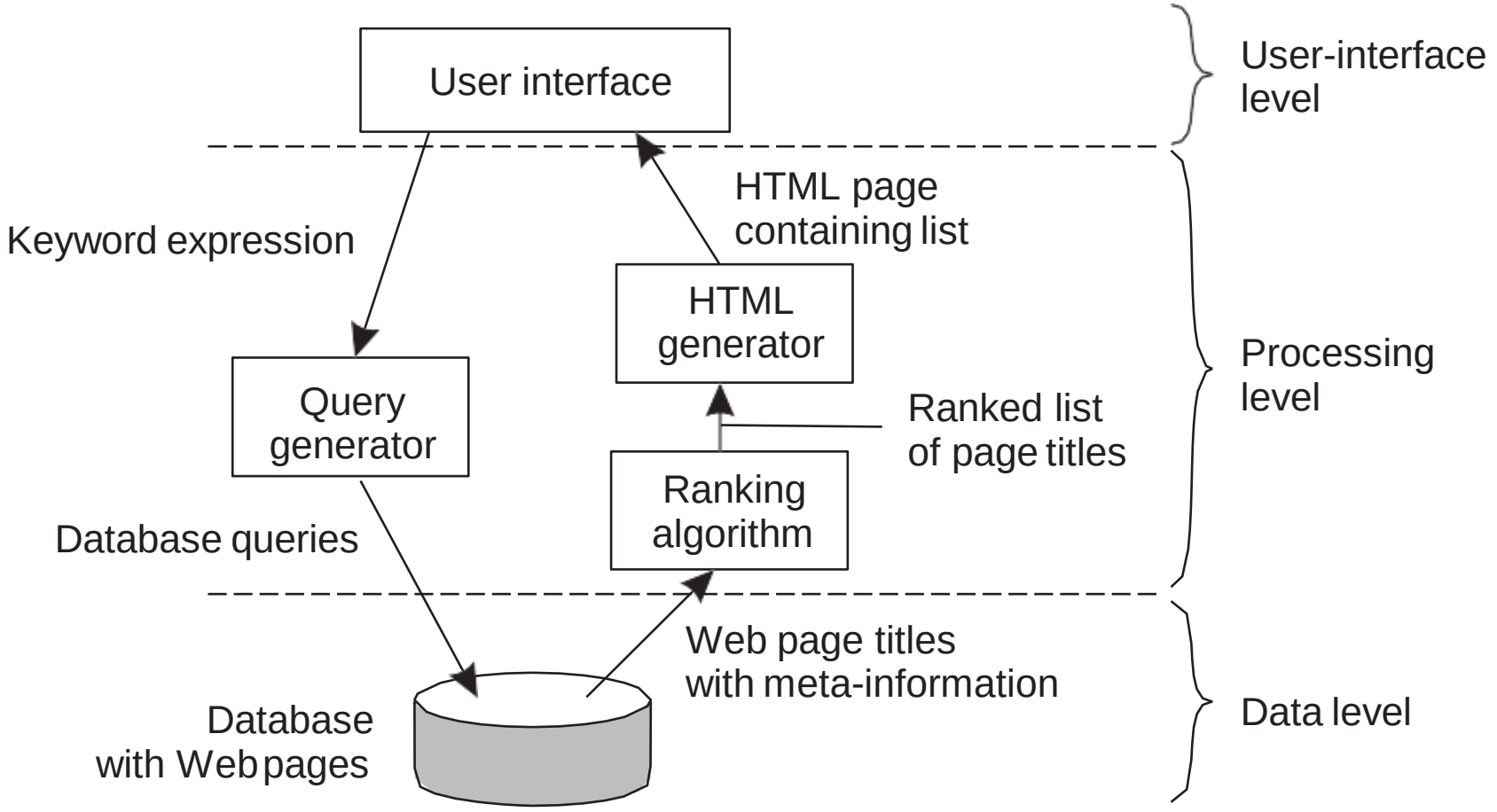
Organize em componentes logicamente diferentes e os distribua entre as máquinas disponíveis.



(a)

(a) Estilo em camadas é usado em sistemas cliente-servidor

Estilo arquitetural: exemplo de camadas

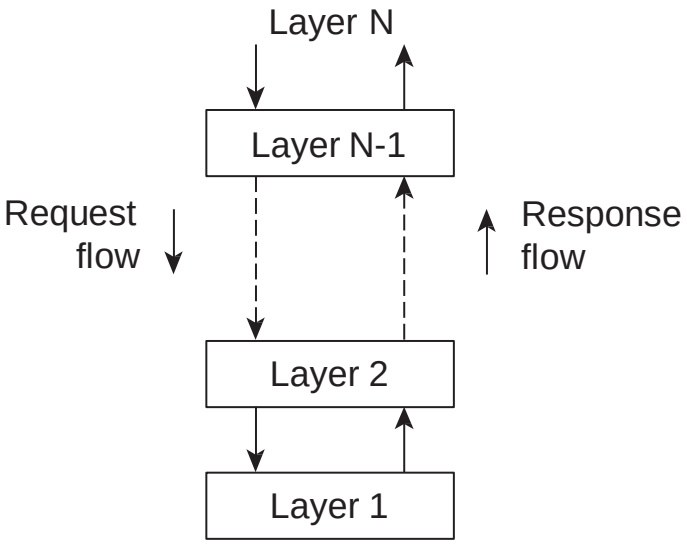


Several servers / layers

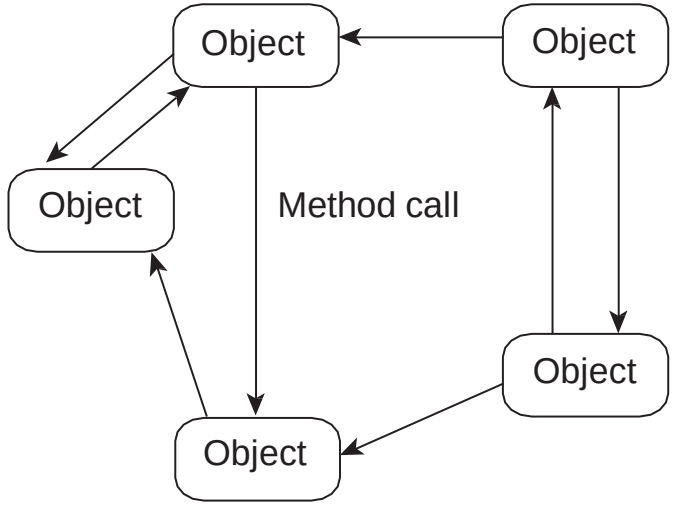
2. Estilo arquitetural: objetos

Ideia básica

Organize em componentes logicamente diferentes e os distribua entre as máquinas disponíveis.



(a)

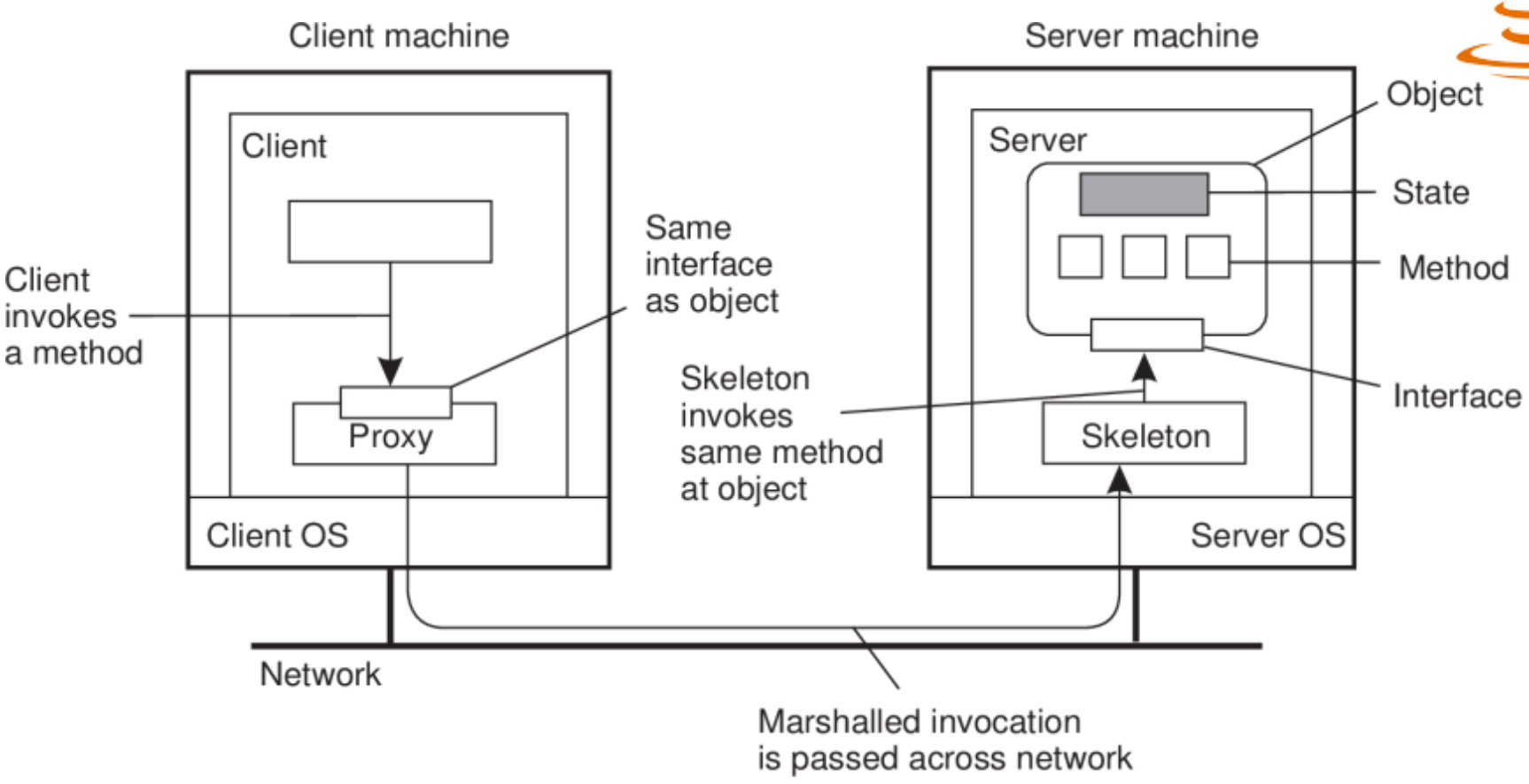


(b)

(a) Estilo em camadas é usado em sistemas cliente-servidor

(b) Estilo orientado a objetos usado em sistemas de objetos distribuídos e **microsserviços**.

Estilo arquitetural: exemplo de objetos distrib. (RMI)



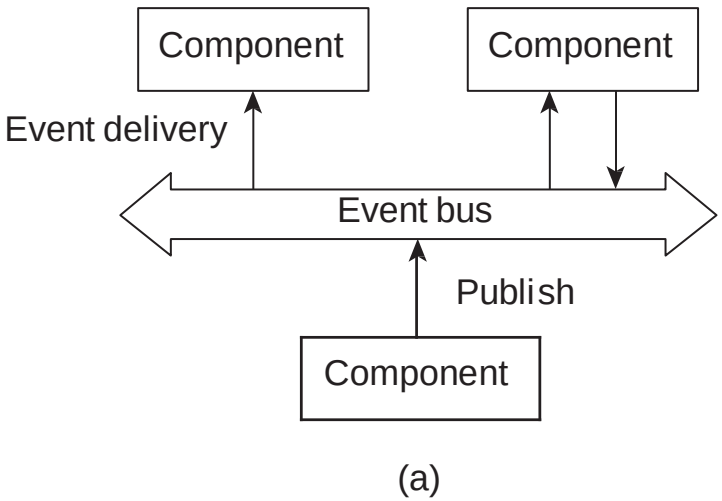
Encapsulamento

Os objetos ficam distribuídos pelo sistema (servidores). Apesar do usuário fazer chamadas que são equivalentes a chamadas locais, elas podem estar sendo feitas em objetos remotos.

3. Estilo arquitetural: eventos e dados

Ideia básica

Desacoplar processos na referência (anônimos)

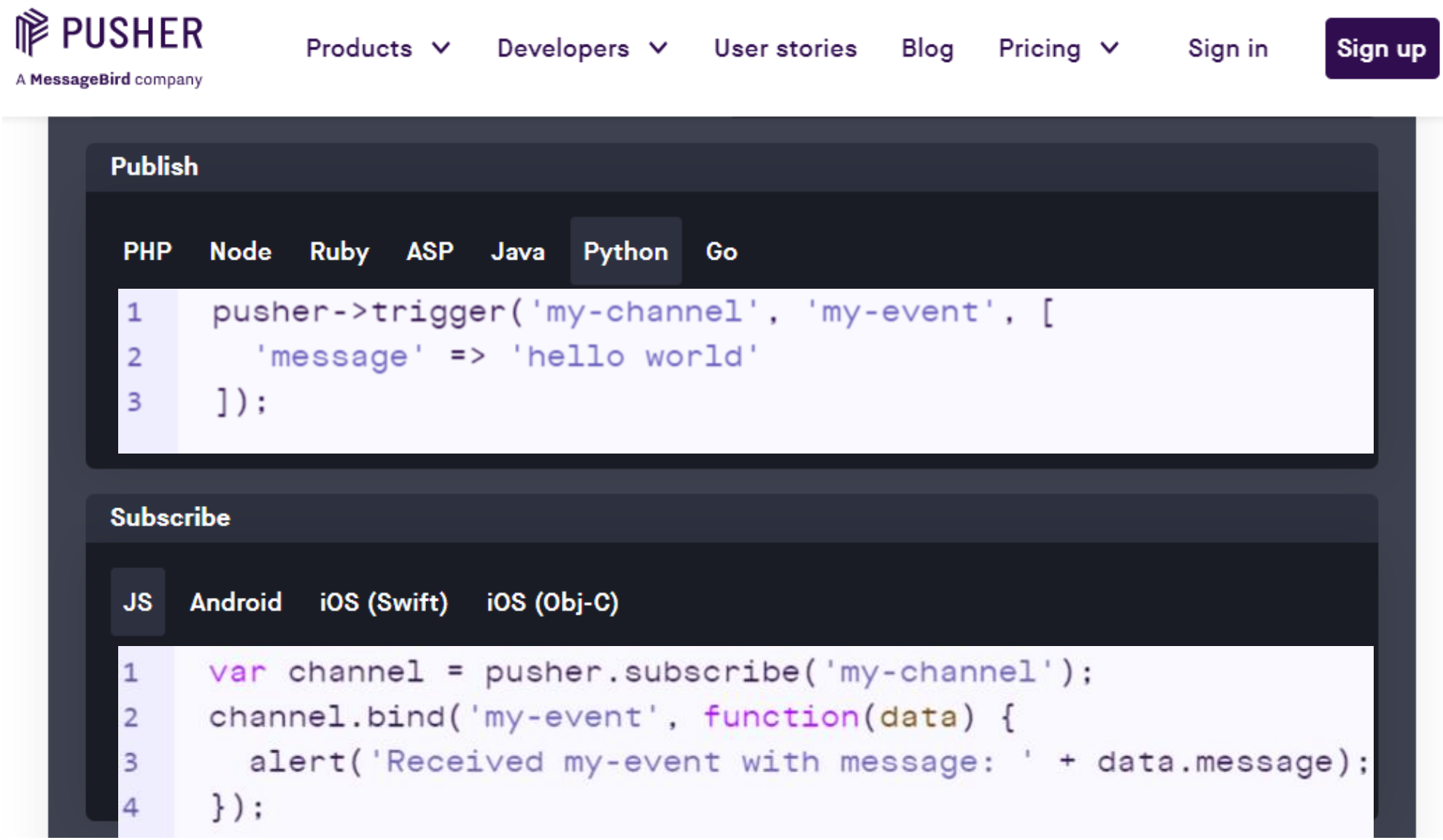


(a)

(a) Publish/subscribe [desacoplado na referência]

Building Next-Generation Applications with EDA • Eric Johnson • GOTO 2024
Observability in an Asynchronous World • James Eastham • GOTO 2024

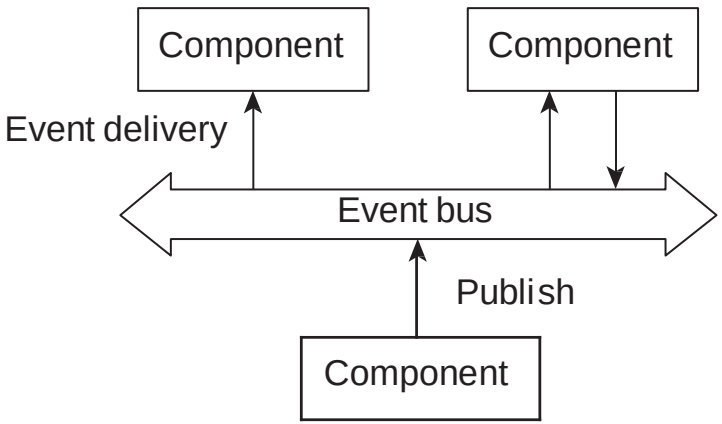
Estilo arquitetural: exemplo eventos (publish/subscribe)



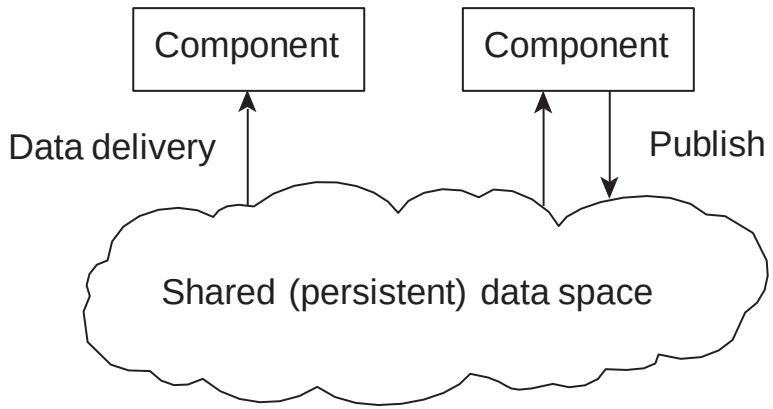
3. Estilo arquitetural: eventos e dados

Ideia básica

Desacoplar processos na referência (anônimos) e tempo (assíncronos)



(a)

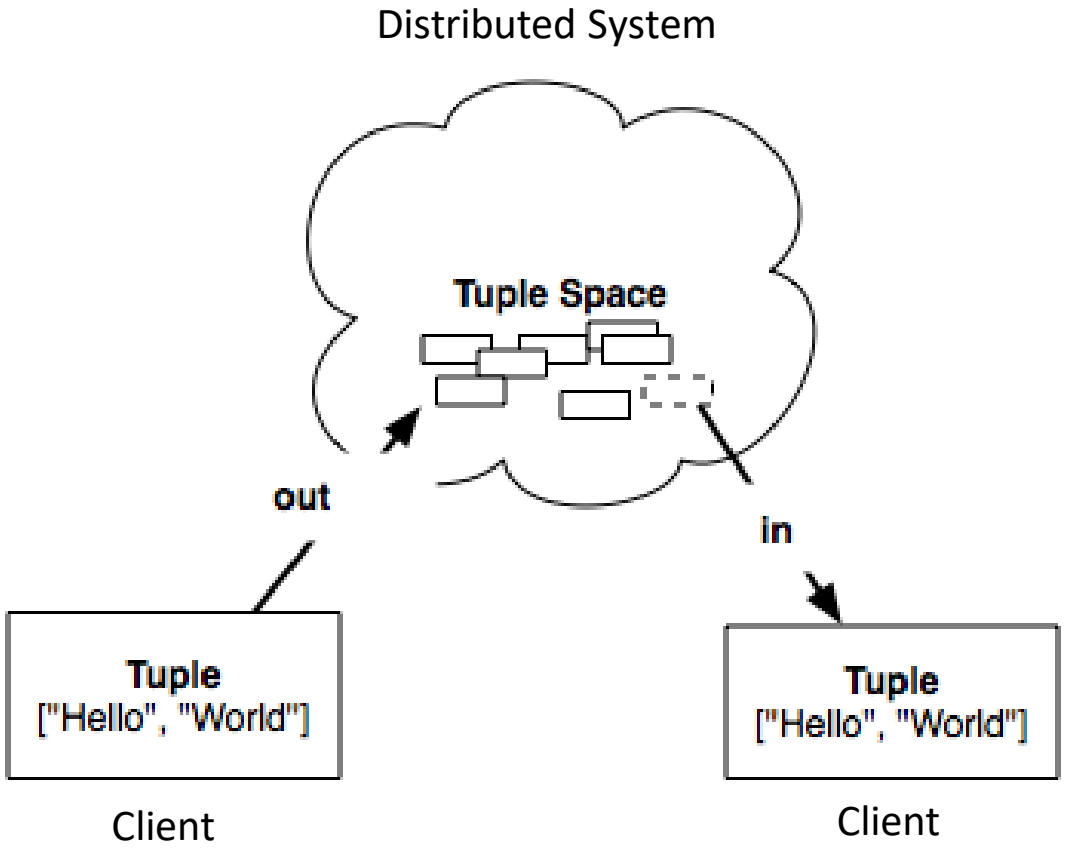


(b)

(a) Publish/subscribe [desacoplado na referência]
(b) Espaço de dados compartilhados [desacoplado na ref. e tempo]

Estilo arquitetural: exemplo de dados compartilhados

GIGASPACES
innovate with confidence



https://www.druby.org/sidruby/6-1-introducing-linda-and-rinda.html

4. Estilo arquitetural: recursos

Vê um sistema distribuído como uma coleção de recursos gerenciados individualmente por componentes. Recursos podem ser adicionados, removidos, recuperados e modificados por aplicações remotas - REST [Fielding 2000].

1. Recursos são identificados usando um único esquema de nomeação
2. Todos os serviços oferecem a mesma interface
3. Após a execução de uma operação em um serviço, o componente esquece tudo sobre quem chamou a operação

Operações básicas

Operação	Descrição
PUT	Cria um novo recurso
GET	Recupera o estado de um recurso usando um tipo de representação
DELETE	Apaga um recurso
POST	Modifica um recurso ao transferir um novo estado

Estilo arquitetural: exemplo de recursos



Essência

Objetos (arquivos) são armazenados em buckets (diretórios).
Operações em ObjectName em BucketName requerem o seguinte identificador:

http://BucketName.s3.amazonaws.com/ObjectName

Todas as operações são realizadas via interfaces com requisições HTTP:

- Criar um bucket/objeto: PUT + URI
- Listar objetos: GET em um nome de bucket
- Ler um objeto: GET em uma URI completa

Normalmente as interfaces de acesso são consideradas Wrappers.

Agenda

- Estilos arquiteturais
- Arquiteturas de software
- Arquiteturas versus middleware
- Sistemas distribuídos autogerenciáveis

Arquiteturas

- Arquiteturas centralizadas e multicamadas (multidivididas)
- Arquiteturas decentralizadas
- Arquiteturas híbridas

Arquiteturas

- Arquiteturas centralizadas e multicamadas (multidivididas)
- Arquiteturas decentralizadas
- Arquiteturas híbridas

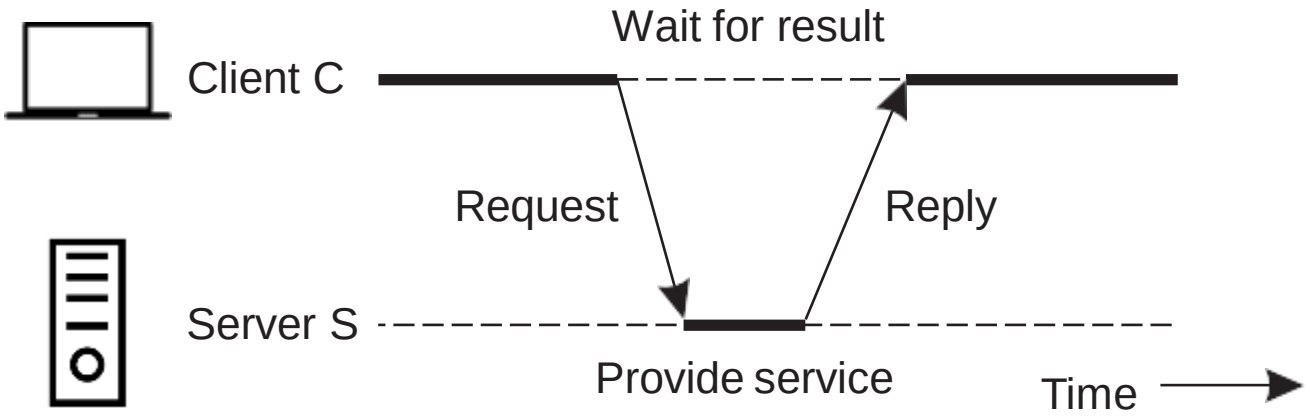
Arquiteturas centralizadas





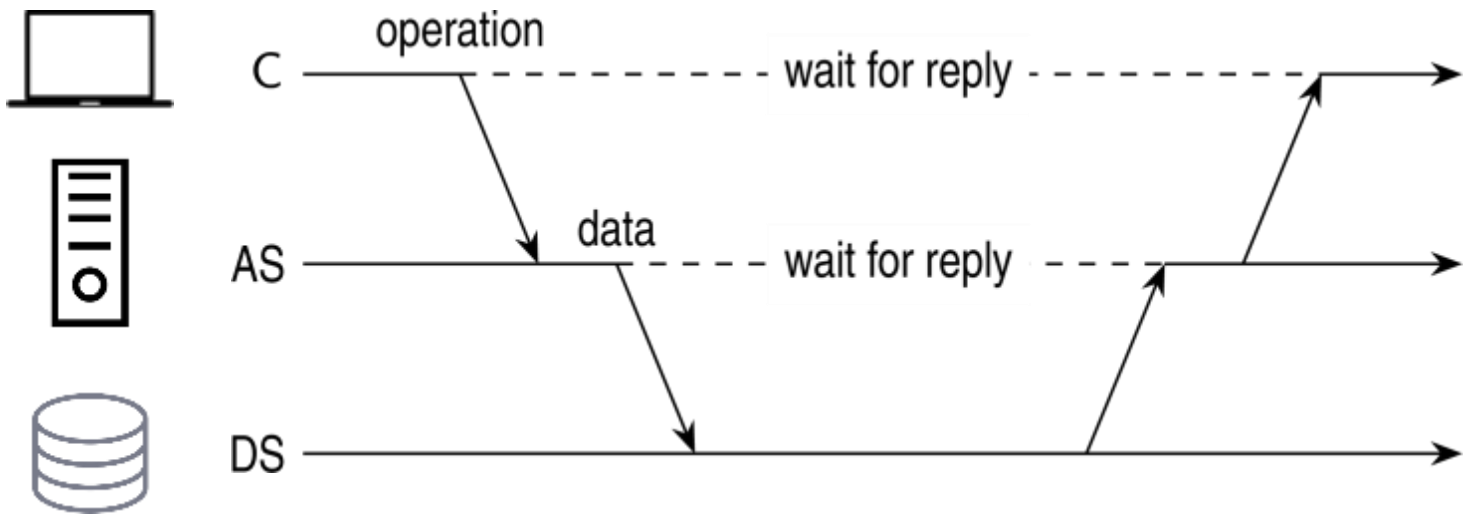
Características do modelo Cliente–Servidor [desde os anos 70]

- Existem processos que oferecem serviços (servidores)
- Existem processos que usam esses serviços (clientes)
- Clientes e servidores podem estar em máquinas diferentes
- Clientes seguem um modelo requisição/resposta ao usar os serviços



Arquiteturas centralizadas

Sendo Cliente e Servidor ao mesmo tempo



Arquiteturas multicamada

Visão tradicional em três camadas lógicas

- A camada de apresentação (user interface) contém o necessário para a aplicação poder interagir com o usuário
- A camada de negócio (application) contém as funções de uma aplicação
- A camada de dados (database) contém os dados que o cliente quer manipular através dos componentes da aplicação

Observação

Estas camadas são encontradas em muitos sistemas de informação distribuídos, que usam tecnologias de bancos de dados tradicionais e suas aplicações auxiliares.

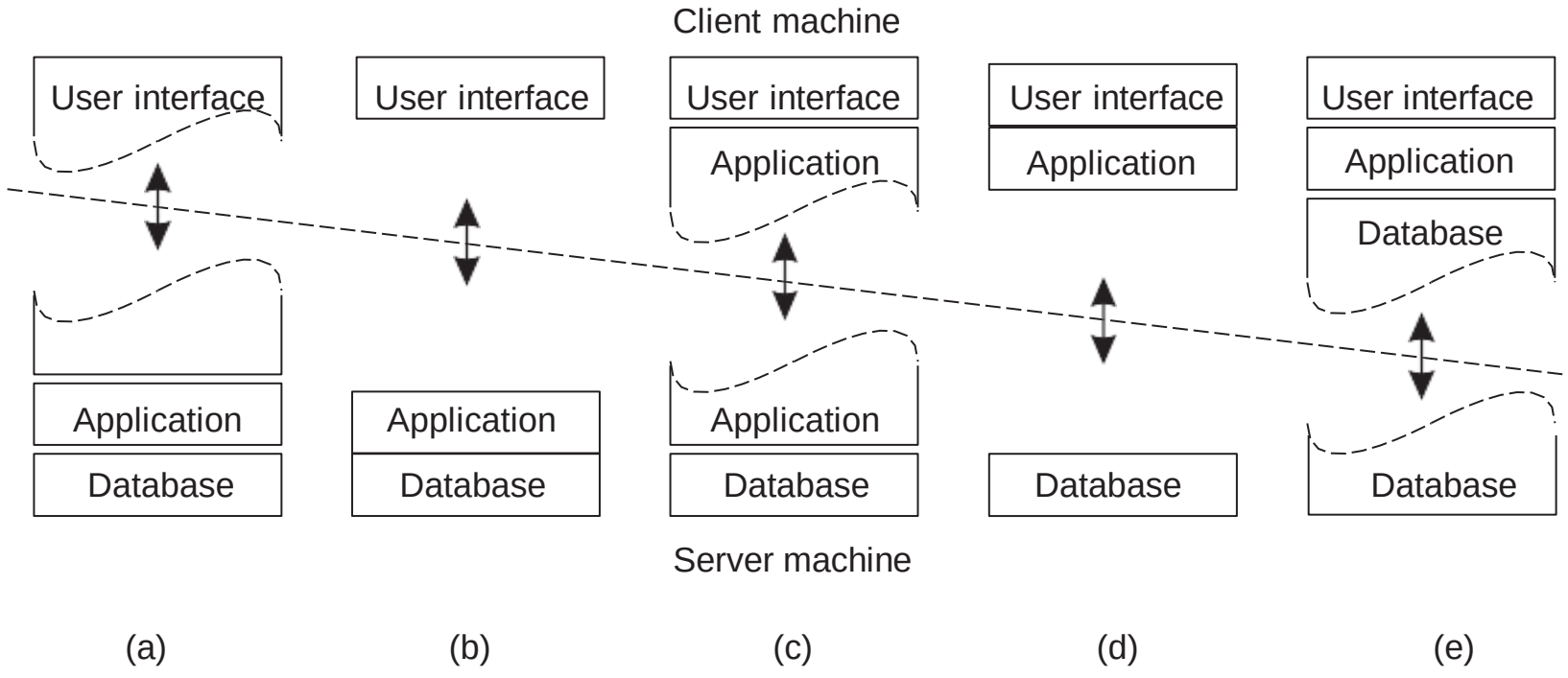
Arquiteturas multicamada

uma camada física: configurações de terminal burro/mainframe

duas camadas físicas: configuração cliente–servidor único.

três camadas físicas: cada camada em uma máquina separada

Configurações tradicionais em 2 camadas físicas + 3 lógicas:



Arquiteturas

- Arquiteturas centralizadas e multicamadas (multidivididas)
- Arquiteturas decentralizadas
- Arquiteturas híbridas

Arquiteturas decentralizadas (Peer-to-Peer)

P2P estruturado os nós são organizados seguindo uma estrutura de dados distribuída específica



amazon DynamoDB

P2P não-estruturado os nós selecionam aleatoriamente seus vizinhos





P2P híbrido alguns nós são designados, de forma organizada, para executar funções especiais





Nota:

Praticamente todos os casos são exemplos de redes overlay: dados são roteados usando conexões definidas pelos nós

Arquiteturas decentralizadas

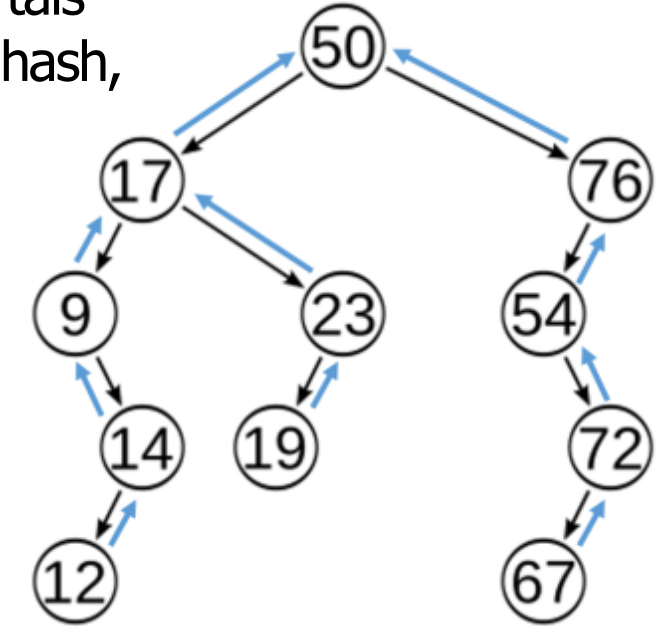
P2P estruturado os nós são organizados seguindo uma estrutura de dados distribuída específica



[Amazon 2012]

O nó é denominado de **Peer**, que atua como cliente e como servidor ao mesmo tempo

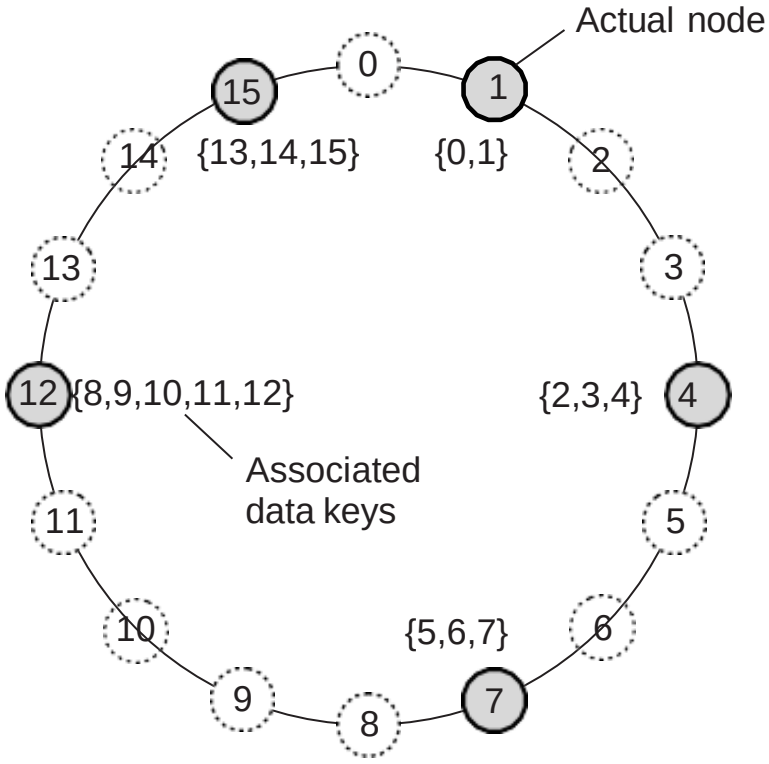
Diversas estruturas distribuídas podem ser criadas, tais como fila, pilha, árvore ABB, árvore B+, tabelas de hash, etc.



Sistemas P2P estruturados

Ideia básica

Organizar os nós em uma rede overlay estruturada, tal como um anel lógico, e fazer com que alguns nós se tornem responsáveis por alguns serviços baseado unicamente em seus Ids [Stoica 2001].



Nota

O sistema provê uma operação LOOKUP(key) que irá fazer o roteamento de uma requisição até o nó correspondente usando as conexões lógicas.

Sistemas P2P estruturados

Como percorrer um anel lógico usando orientação a eventos (comunicação assíncrona – também chamada "síncrona diferida")

1. Considere o código a partir de um nó
2. Crie o método main (ou run) e métodos locais
3. Crie métodos para enviar e receber mensagens
   - Pode ter muitos métodos enviar e receber com diferentes assinaturas
   - Usar RPC, no capítulo "Communication"
4. Crie a estrutura do nó (com atributos)

Sistemas P2P estruturados

Como percorrer um anel lógico usando orientação a eventos (sem RPC) (comunicação assíncrona – também chamada "síncrona diferida")

```
No.run() // método que inicializa o No
    realizarProcessamentoInicial() // método local
    SendMsg(proximoNo, "segue", NULL)

No.SendMsg(no, nomeMetodo, params) // método que envia a msg
    send(no, nomeMetodo, params)

No.ReceiveMsg(nomeMetodo, params) // método que recebe a msg
    IF proximoNo != NULL SendMsg(proximoNo, nomeMetodo, NULL)


Node {
    Node proximoNo
    /* faltam informações de como acessar o No
       (lembrar de redes de computadores) */
} No;
```

Sistemas P2P estruturados

Como percorrer um anel lógico usando orientação a eventos (sem RPC) (comunicação assíncrona – também chamada "síncrona diferida")

Supondo que precisa encontrar um valor armazenado por um dos nós do anel.

- Modificar a classe Node.
- Modificar `ReceiveMsg` verificando localmente se tem o valor.
- Modificar `SendMsg` para devolver o valor se o tiver
    - Devolver para quem?
    - Como devolver?

Arquiteturas decentralizadas

P2P estruturado os nós são organizados seguindo uma estrutura de dados distribuída específica

P2P não-estruturado os nós selecionam aleatoriamente seus vizinhos



[NullSoft 2000]



[2003]

P2P híbrido alguns nós são designados, de forma organizada, a executar funções especiais



[2008]

Sistemas P2P não-estruturados

Observação

Muitos sistemas P2P não-estruturados tentam manter um grafo aleatório.

Princípio básico

Cada nó deve contactar um outro nó selecionado aleatoriamente:

- Cada participante mantém uma visão parcial da rede, consistindo de *c* outros nós (denominados vizinhos)
- Cada nó *P* seleciona periodicamente um nó *Q* de sua visão parcial
- *P* e *Q* trocam informação e participantes de suas respectivas visões parciais

Gossip

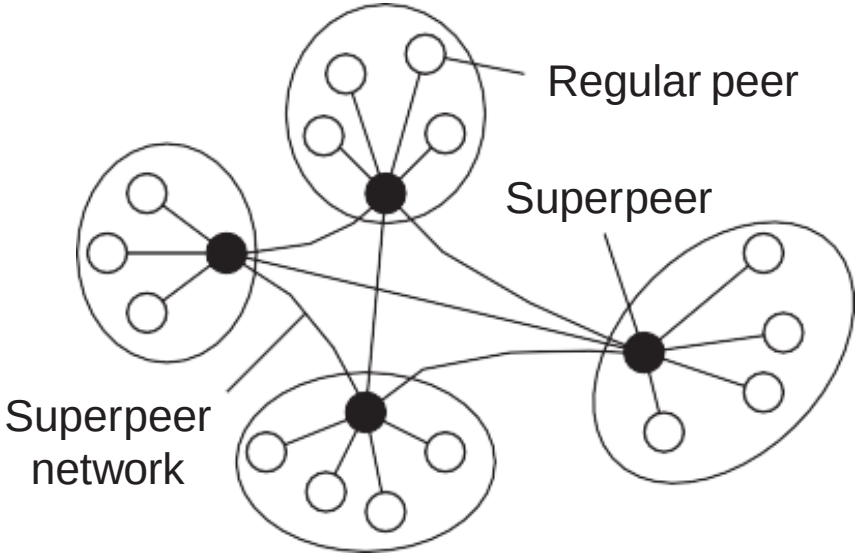
Buscas por dados específicos

Flooding e Random Walk

Sistemas P2P não-estruturados: Superpeers

Observação

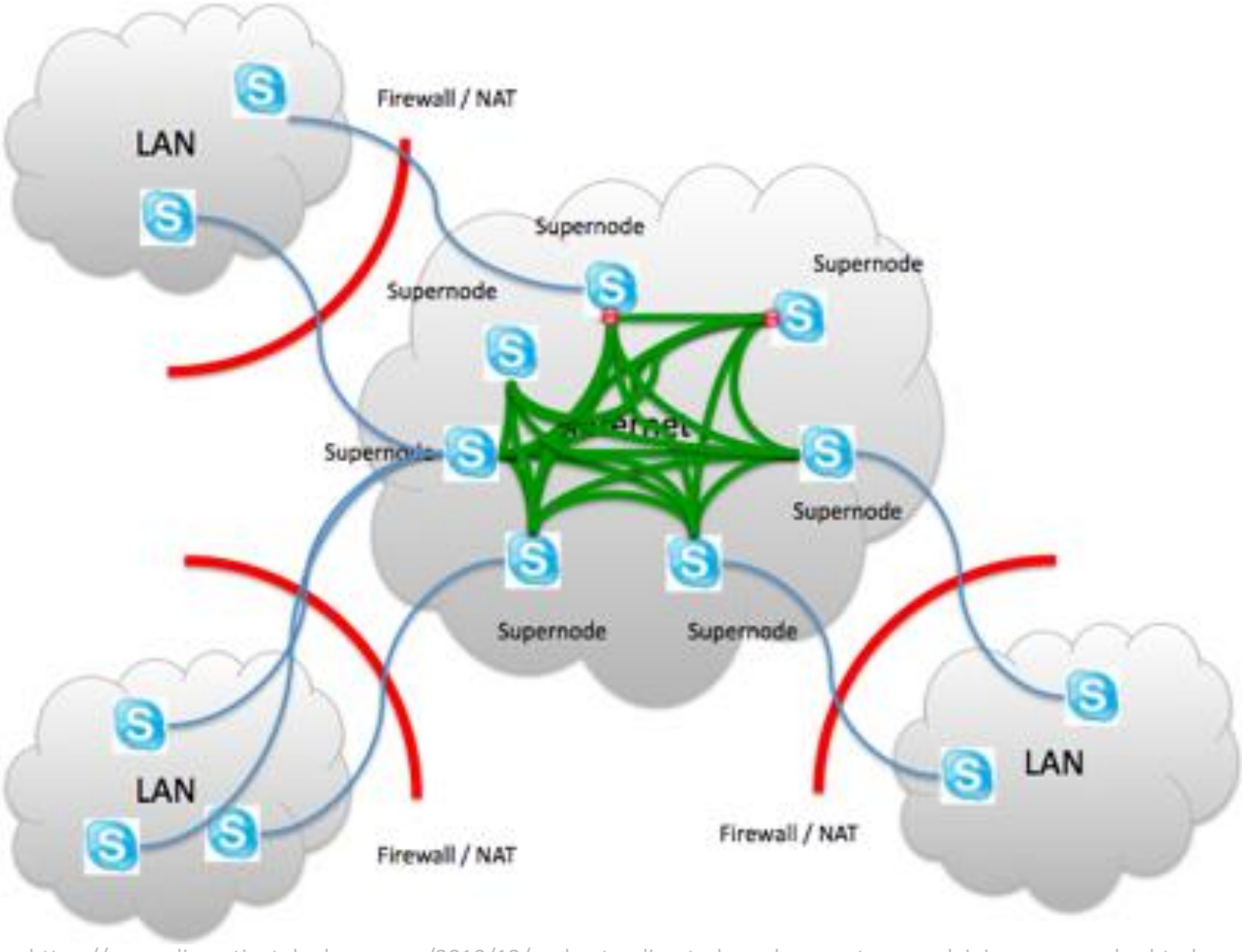
Às vezes, selecionar alguns nós para realizar algum trabalho específico pode ser útil.



Exemplos:

- Peers para manter um índice (para buscas)
- Peers para monitorar o estado da rede
- Peers capazes de configurar conexões

Princípio de operação do Skype: A quer contactar B



https://www.disruptivetelephony.com/2010/12/understanding-todays-skype-outage-explaining-supernodes.html

Arquiteturas decentralizadas

P2P estruturado os nós são organizados seguindo uma estrutura de dados distribuída específica

P2P não-estruturado os nós selecionam aleatoriamente seus vizinhos

P2P híbrido alguns nós são designados, de forma organizada, a executar funções especiais

Cliente-Servidor + P2P



[Amazon 2012]



[2015]

Arquiteturas híbridas: C–S com Egde e Cloud– CDNs

Exemplo:

Arquiteturas de servidores de borda (*edge-servers*), utilizados com frequência como Content Delivery Networks (redes de distribuição de conteúdo).

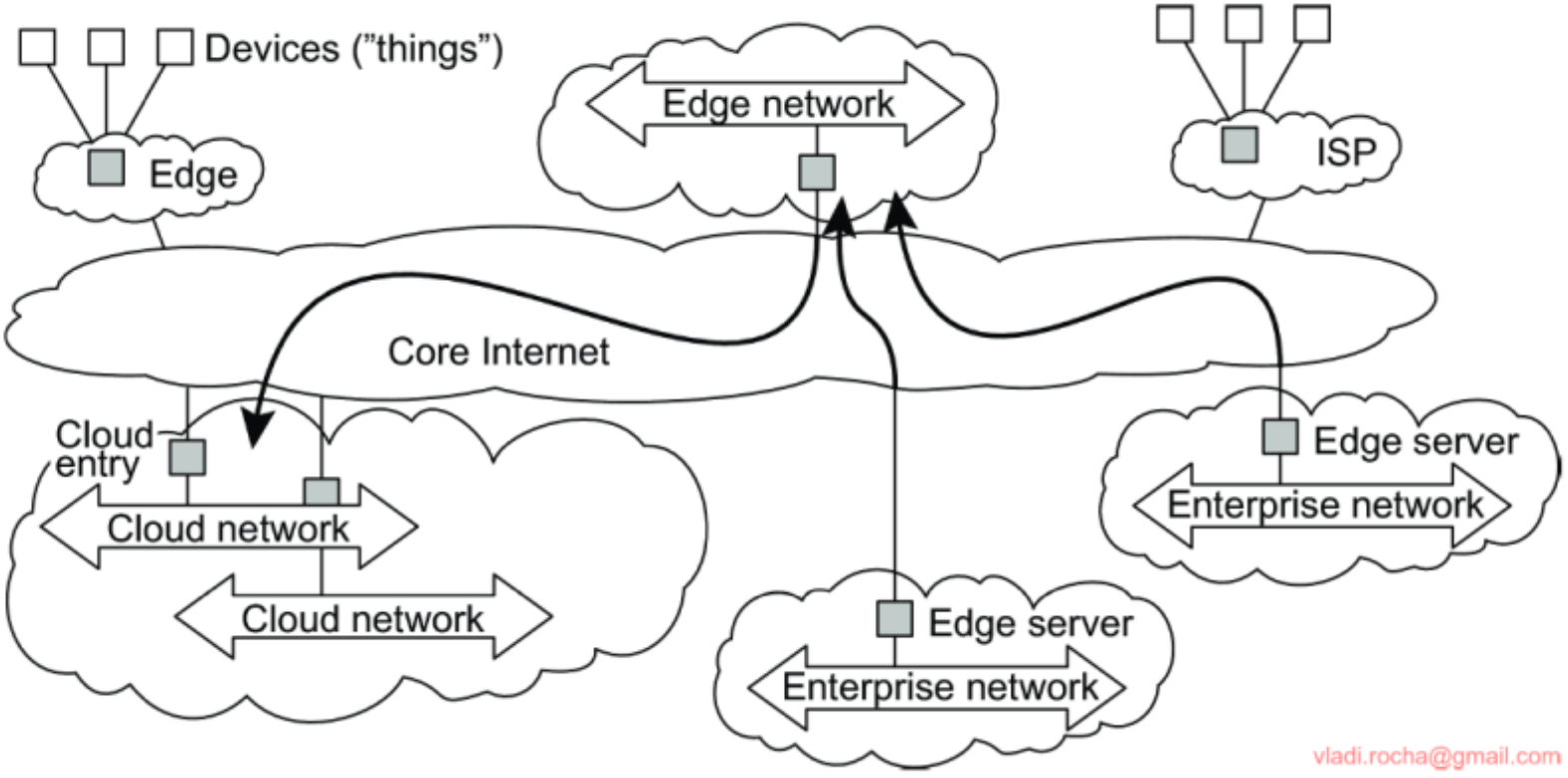
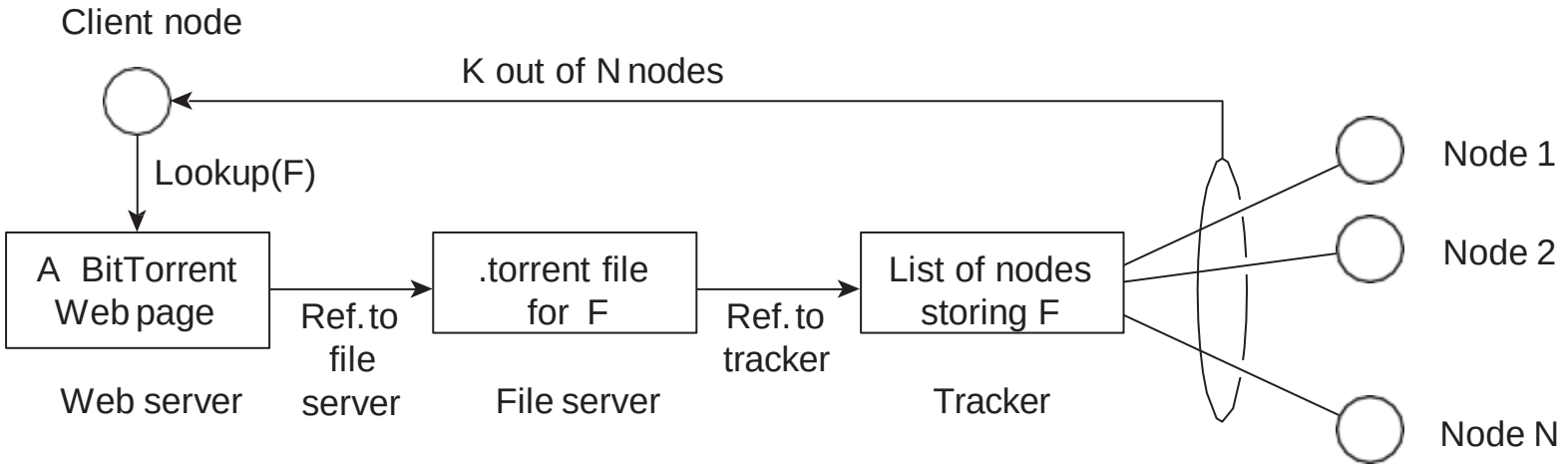


Figure 2.28: A collection of infrastructures involving edge devices, edge infrastructures and cloud infrastructures

Arquiteturas híbridas: C–S com P2P – BitTorrent



Ideia básica

Assim que um nó identifica de onde o arquivo será baixado, ele se junta a uma swarm (multidão) de pessoas que, em paralelo, receberão pedaços do arquivo da fonte e redistribuirão esses pedaços entre os outros [Cohen 2001].

Agenda

- Estilos arquiteturais
- Arquiteturas de software
- Arquiteturas versus middleware
- Sistemas distribuídos autogerenciáveis

Arquiteturas versus Middleware

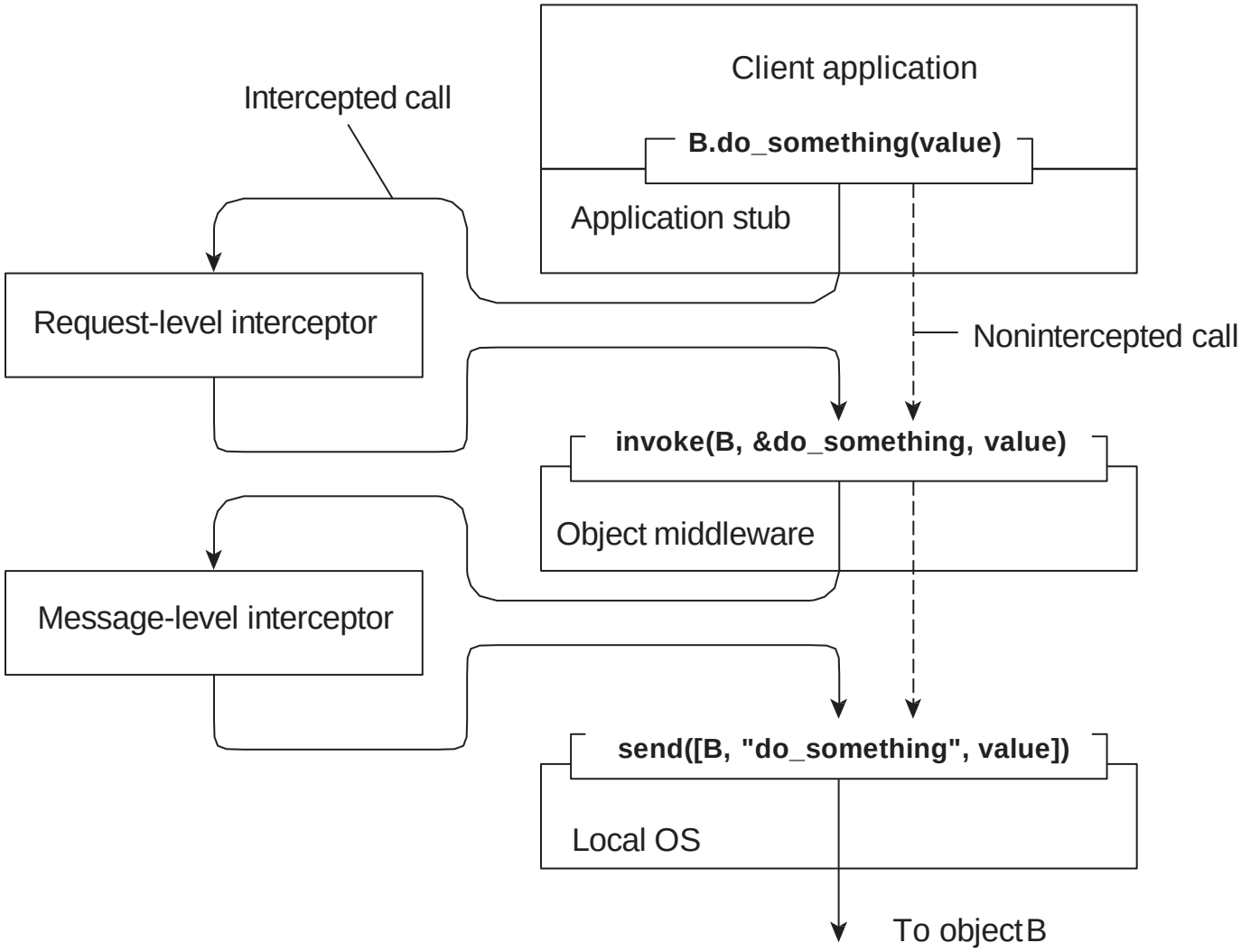
Problema

Em muitos casos, arquiteturas/sistemas distribuídos são desenvolvidos de acordo com um estilo arquitetural específico. O estilo escolhido pode não ser o melhor em todos os casos ⇒ é necessário adaptar o comportamento da arquitetura usando um middleware (dinamicamente).

Interceptors

Interceptam o fluxo de controle normal quando um objeto remoto for invocado.

Interceptors



Wrapper versus Brokers

Problema

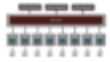
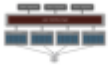
Como ter acesso a funcionalidades de diferentes aplicações?

Possíveis soluções

Usando *wrappers* ou *brokers* que façam conexões entre as aplicações.

Estilos Arquiteturais (Mark Richards)

Fundamentals of Software Architecture: An Engineering Approach (2021)

	 layered	 modular monolith	 microkernel	 microservices	 service-based	 service-oriented	 event-driven	 space-based
agility	★	★★	★★★	★★★★★	★★★★	★	★★★	★★
abstraction	★	★	★★★	★	★	★★★★★	★★★★	★
configurability	★	★	★★★★	★★★	★★	★	★★	★★
cost	★★★★★	★★★★★	★★★★★	★	★★★★	★	★★★	★★
deployability	★	★★	★★★	★★★★★	★★★★	★	★★★	★★★
domain part.	★	★★★★★	★★★★★	★★★★★	★★★★★	★	★	★★★★★
elasticity	★	★	★	★★★★★	★★	★★★	★★★★	★★★★★
evolvability	★	★	★★★	★★★★★	★★★	★	★★★★★	★★★
fault-tolerance	★	★	★	★★★★★	★★★★	★★★	★★★★★	★★★
integration	★	★	★★★	★★★	★★	★★★★★	★★★	★★
interoperability	★	★	★★★	★★★	★★	★★★★★	★★★	★★
performance	★★★	★★★	★★★	★★	★★★	★★	★★★★★	★★★★★
scalability	★	★	★	★★★★★	★★★	★★★	★★★★★	★★★★★
simplicity	★★★★★	★★★★★	★★★★	★	★★★	★	★	★
testability	★★	★★	★★★	★★★★★	★★★★	★	★★	★
workflow	★	★	★★	★	★	★★★★★	★★★★★	★

Conceitos adquiridos

- Arquitetura em camadas, objetos, eventos e recursos.
- Acoplamento na referência e no tempo.
- Arquitetura cliente-servidor.
- Arquitetura de 3 camadas.
- P2P (Peer-to-Peer).
- Interceptor.
- Wrapper e Broker.

Sistemas Distribuídos

Processos, Threads e Migração

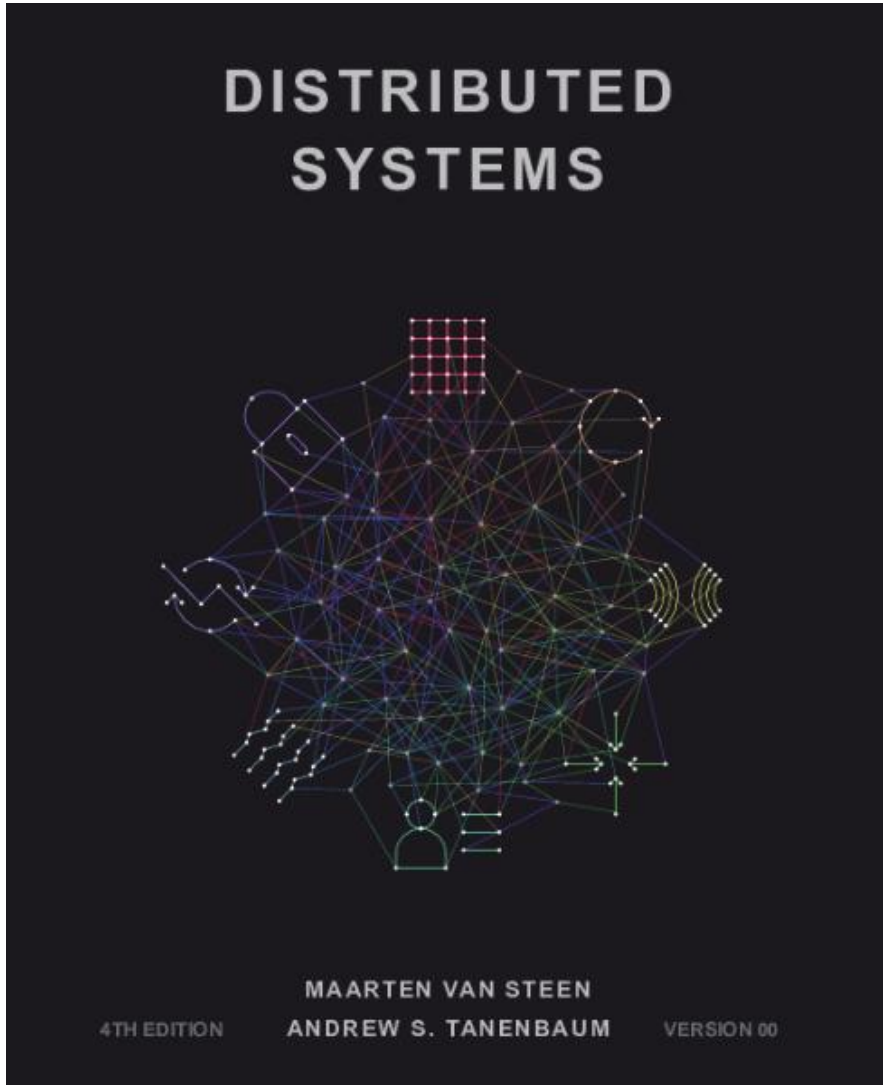
Processes

Vladimir Rocha (Vladi)

CMCC - Universidade Federal do ABC

Distributed Systems 4th edition (free)

https://www.distributed-systems.net/index.php/books/ds4/



4.03 (2025)

Disclaimer

- Estes slides foram baseados nos do professor **Emilio Francesquini** para o curso de Sistemas Distribuídos na UFABC.
- Este material pode ser usado livremente desde que sejam mantidos, além deste aviso, os créditos aos autores e instituições.
- Estes slides foram adaptados daqueles originalmente preparados (e gentilmente cedidos) pelo professor **Daniel Cordeiro, da EACH-USP** que por sua vez foram baseados naqueles disponibilizados online pelos autores do livro "Distributed Systems", 3ª Edição em: https://www.distributed-systems.net.

Agenda

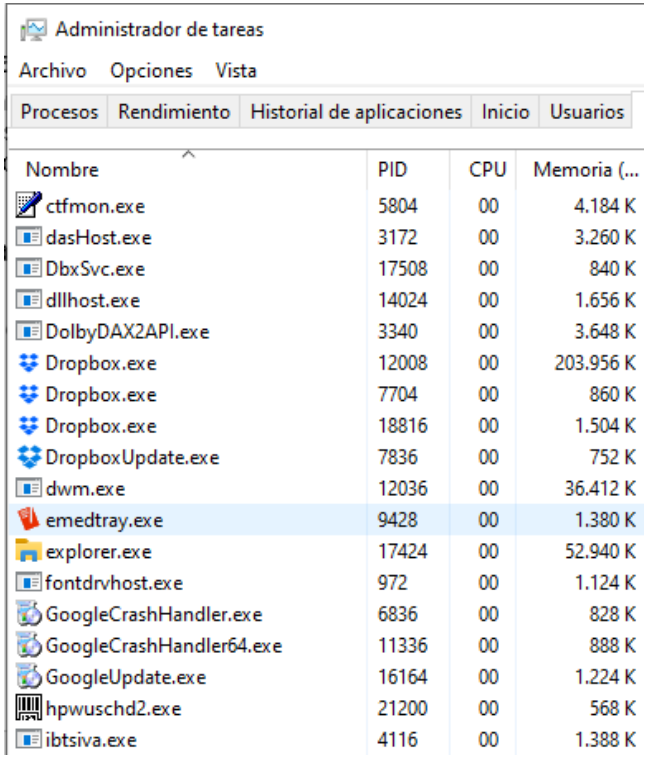
- Introdução à processos e threads
- Threads em sistemas distribuídos
- Clientes
- Servidores
- Migração de código

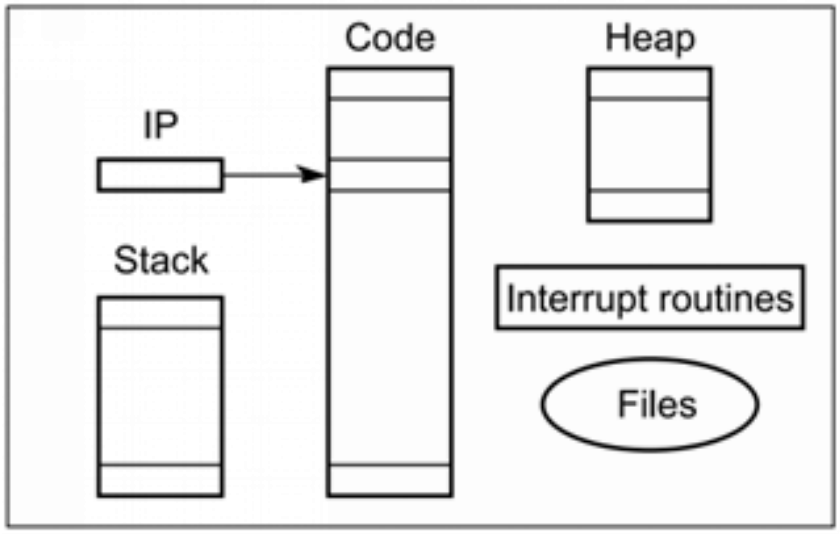
Agenda

- Introdução à processos e threads
- Threads em sistemas distribuídos
- Clientes
- Servidores
- Migração de código

Processos

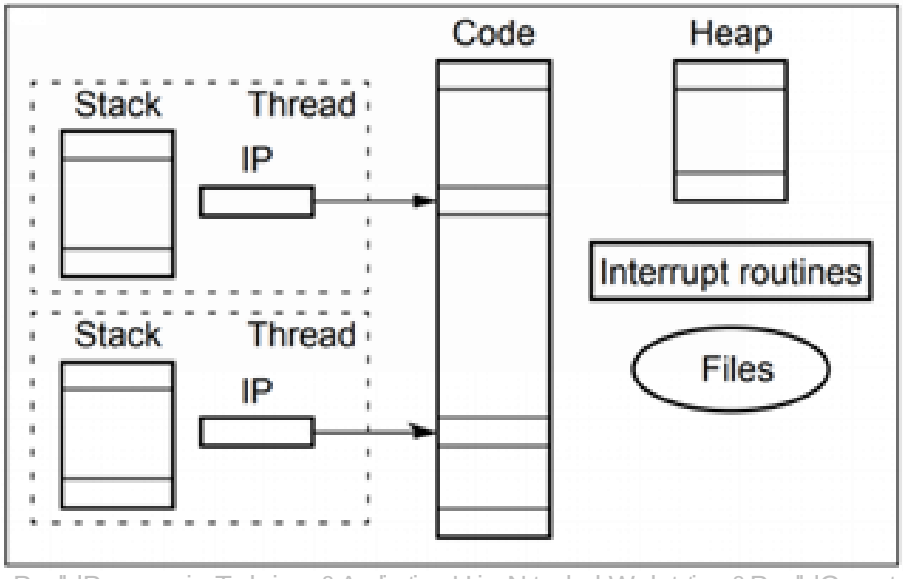
- Uma das abstrações mais importantes de um SO
  - Representam a execução de um programa
  - Execuções simultâneas de um programa são representadas por diversos processos
- Por segurança, os espaços de memória de cada processo são isolados
- Evita problemas que seriam causados por ataques deliberados ou por bugs
- Cada processo é uma linha de execução independente escalonada pelo SO



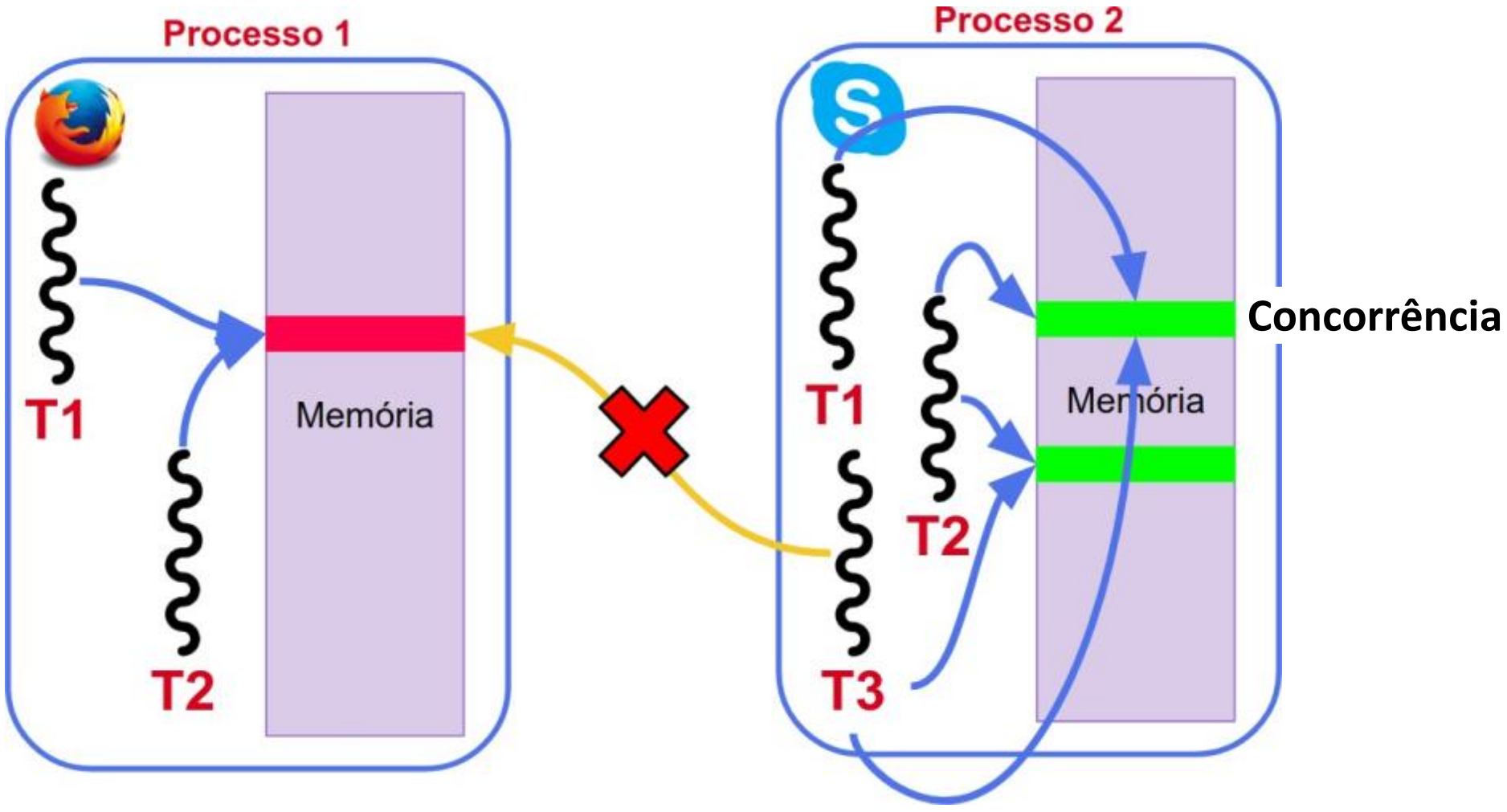


Threads

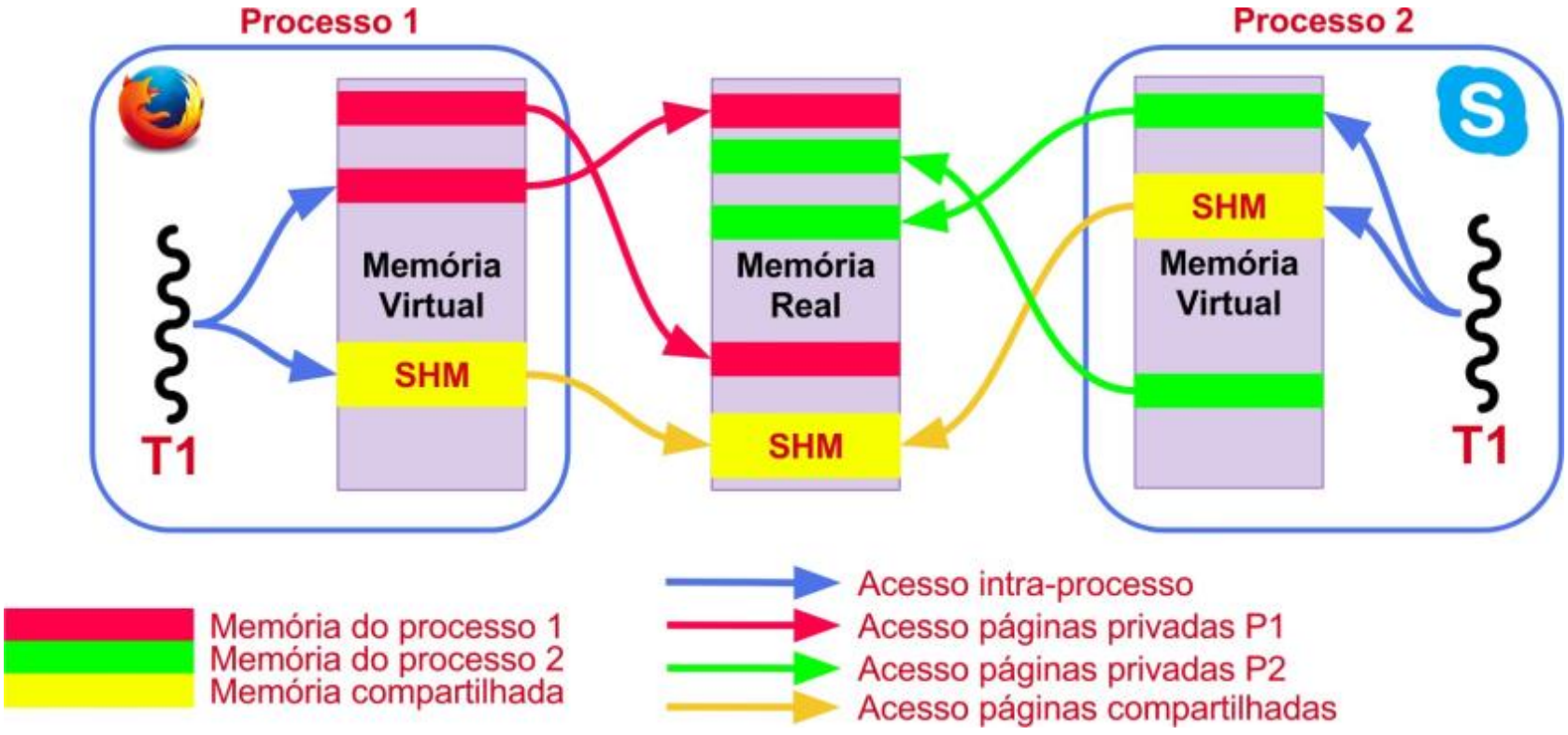
- Frequentemente um mesmo processo precisa fazer mais de uma coisa por vez. Ex.: Navegador de Internet
- Da mesma maneira que processos fornecem múltiplas linhas de execução em uma máquina, threads permitem múltiplas linhas de execução em um só processo
- Como efetivamente todas as threads são o mesmo processo, todas têm acesso ao mesmo espaço de memória e a todos os recursos disponíveis neste processo



Processos vs. Threads - Versão Simplificada



Processos vs. Threads - Versão (menos) Simplificada



Concorrência

- As threads, ao terem acesso ao mesmo espaço de memória, acessarão estruturas e métodos de forma concorrente.

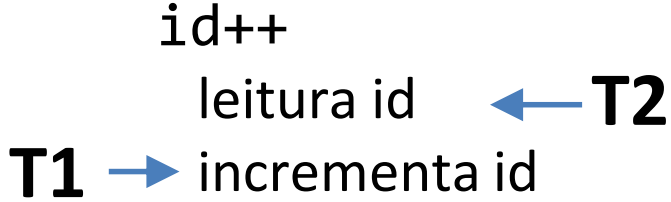
- Como visto em sistemas operacionais, o acesso concorrente pode levar a problemas de consistência pela condição de corrida (*race condition*).

```
int id;

public int inc() {

    int newID = id++;

    return newID;
}
```



Concorrência

- Assim, é necessário utilizar mecanismos para garantir a execução cooperativa e sincronizada, tais como Semáforos, Monitores, etc, evitando deadlocks.

```
int id;

public int inc() {

   int newID = id++;

   return newID;
}
```

```
int id;

public int inc() {
   lock.lock();
   int newID = id++;
   lock.unlock();
   return newID;
}
```

Processos vs. Threads

- SO provê mecanismos para dividir o tempo do processador entre processos e threads, escalonando-os nas unidades de processamento disponíveis
  - Bloqueios por alguma operação de E/S causam uma troca do processo/thread pelo próximo na fila
- Trocas de contexto entre threads são **baratas**
  - Basta trocar o IP e mais alguns registradores
- Trocas de contexto entre processos são mais **caras**
  - Exigem troca da tabela de páginas, troca de IP, troca de rotinas de manuseio de interrupções, ...
- Ainda assim há momentos que o uso de processos pode ser preferível. Exemplo: Google Chrome

Processos vs. Threads

Benchmark	Operating System	Operation	Time per Op
Spawn New Process	NT	spawnl()	12.0 ms
	Linux	fork()/exec()	6.0 ms
Clone Current Process	NT	NONE	N/A
	Linux	fork()	1.0 ms
Spawn New Thread	NT	CreateThread()	0.9 ms
	Linux	pthread_create()	0.3 ms
Switch Current Process (20 runnable processes)	NT	Sleep(0)	0.010 ms
	Linux	sched_yield()	0.019 ms
Switch Current Thread (20 runnable threads)	NT	Sleep(0)	0.006 ms
	Linux	sched_yield()	0.019ms

Spawn: cria e executa

No linux:

fork (C) os.fork (Python): para processos

p_thread (C), Thread (Python e Java): para threads

https://www.linuxjournal.com/article/2941

Agenda

- Introdução à processos e threads
- Threads em sistemas distribuídos
- Clientes
- Servidores
- Migração de código

Threads em sistemas distribuídos

Razões para utilizar várias threads:

- Evita bloqueios: para programas com uma thread, caso haja uma chamada a IO, o SO bloqueará a thread.
- Explora o paralelismo: as threads, em um programa multi-thread, podem ser executadas em paralelo em diversos processadores ou cores.
- Evita troca de contexto entre processos: leva à **programação concorrente com memória compartilhada**.

Threads no cliente

Clientes web multithreaded — escondem a latência da rede:

- Navegador analisa a página HTML sendo recebida e descobre que *muitos outros arquivos devem ser baixados*.
- Cada arquivo é baixado por uma thread separada; cada uma realiza uma requisição HTTP (bloqueante)
- A medida que os arquivos chegam, o navegador os exibem.

Múltiplas chamadas requisição–resposta (RPC) para outras máquinas

- Um cliente faz várias chamadas simultâneas, cada uma em uma thread diferente
- Ele espera até que todos os resultados tenham chegado.
- Obs: se as chamadas são a servidores diferentes, você pode ter um speed-up linear

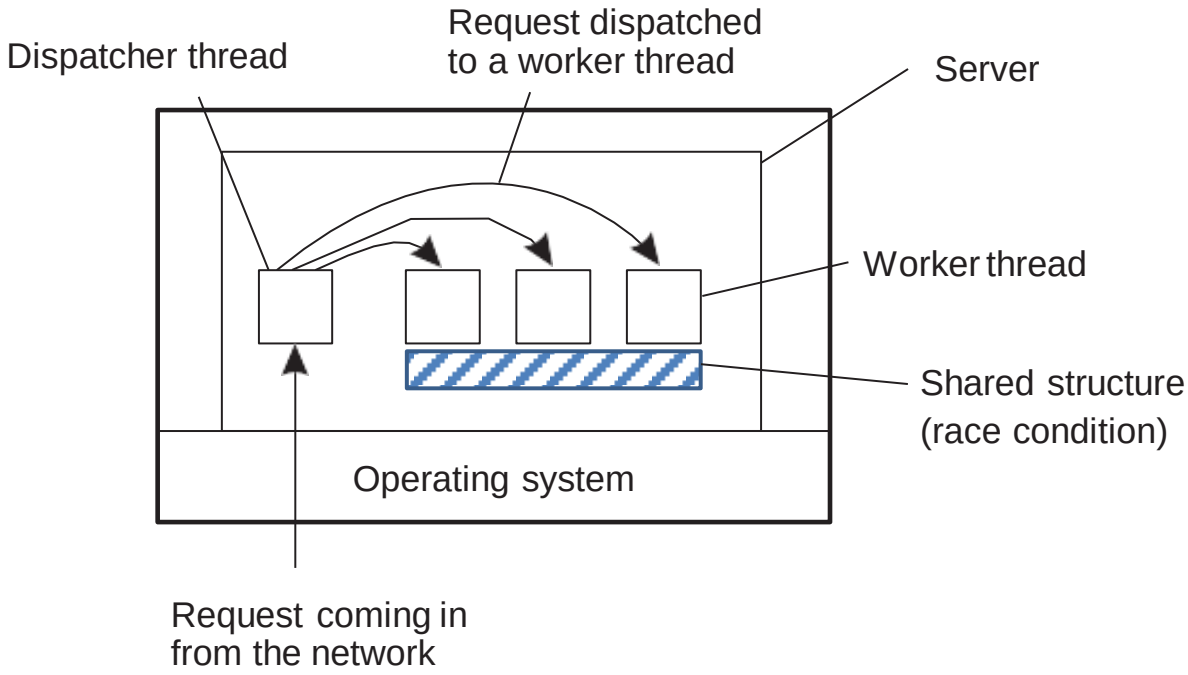
Threads no servidor

Melhorias no desempenho

- Iniciar uma thread é **muito** mais barato do que iniciar um novo processo
- Ter servidores single-threaded impedem o uso de sistemas que possuem vários processadores (multi-core)
- Tal como os clientes: esconda a latência da rede reagindo à próxima requisição enquanto a anterior está enviando sua resposta.

Threads no servidor

Modelo despachante / operário (dispatcher/worker)

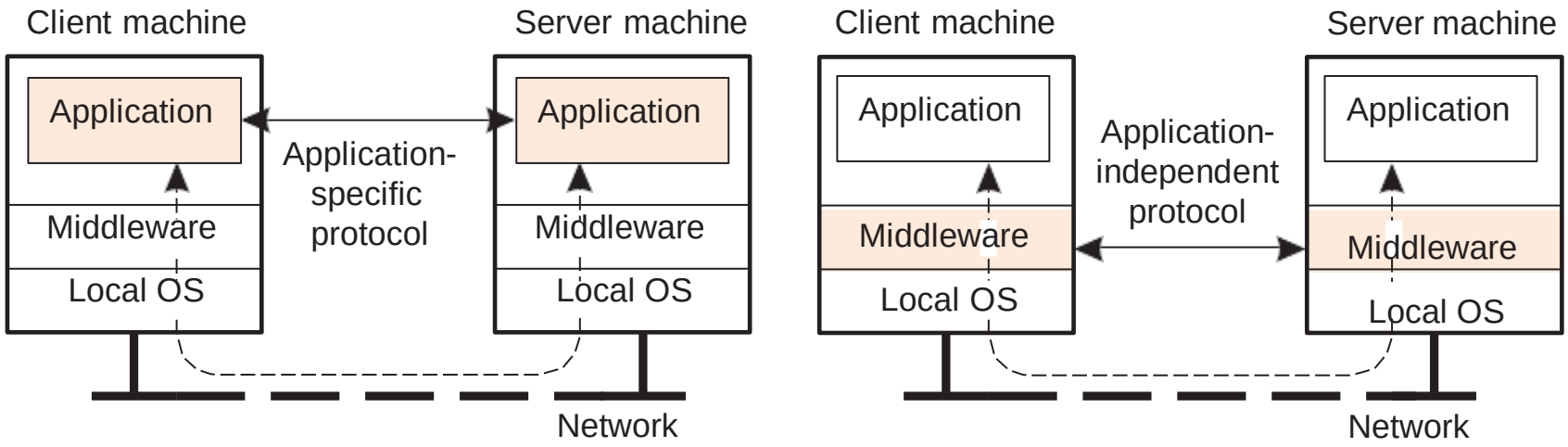


Agenda

- Introdução à processos e threads
- Threads em sistemas distribuídos
- Clientes
- Servidores
- Migração de código

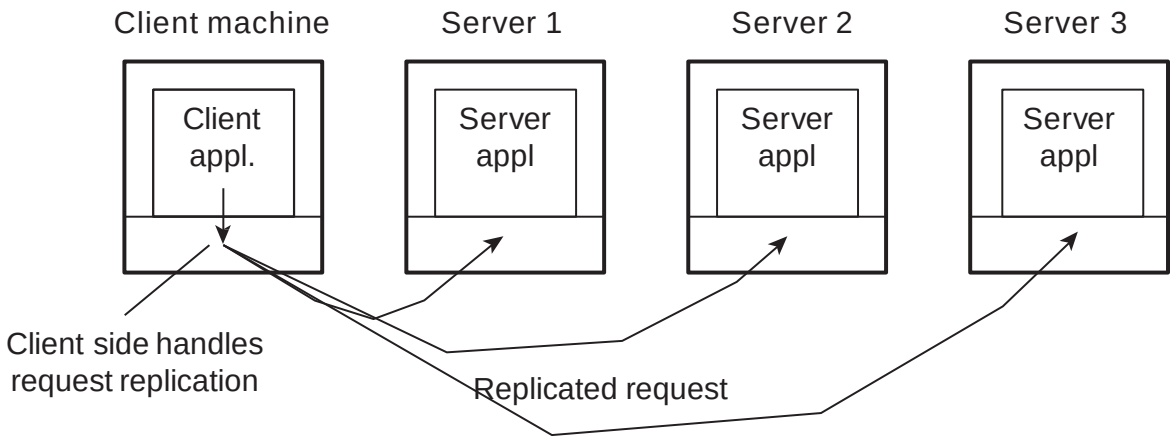
Clientes: interação com servidor

Cliente se comunica com o servidor
Usando protocolos de aplicação ou de middleware



Clientes: transparência de distribuição

- transparência de acesso: *stubs* do cliente para requisições RPC
- transparência de localização/migração: deixe o software cliente manter o controle sobre a localização atual
- transparência de replicação: múltiplas requisições são gerenciadas pelo stub do cliente:



- transparência de falhas: podem geralmente ser responsabilidade só do cliente (que tenta esconder falhas de comunicação e do servidor)

Agenda

- Introdução à processos e threads
- Threads em sistemas distribuídos
- Clientes
- Servidores
- Migração de código

Servidores: organização geral

Modelo básico

Um processo que implementa um serviço específico em nome de uma coleção de clientes.

Ele espera pela requisição de um cliente, garante que a requisição será tratada e, em seguida, passa a esperar pela próxima requisição.

Servidores: organização geral

Dois tipos básicos:

Servidores iterativos o servidor trata uma requisição antes de atender a próxima

Servidores concorrentes usa um despachante (*dispatcher*), que toma uma requisição e repassa seu tratamento a uma *thread*/processo separado

Observação

É mais comum encontrarmos servidores concorrentes: eles podem tratar múltiplas requisições mais facilmente, principalmente se for necessário realizar operações bloqueantes (em discos ou outros servidores).

Servidores e estado

Página inicial sem cookies e sem logar



Servidores *stateless*

Não mantém informação exata sobre o status de um cliente após ter processado uma requisição:

- Não guarda se um arquivo foi aberto (simplesmente fecha-o e abre de novo se necessário)
- Não promete invalidar o cache do cliente
- Não rastreia os seus clientes

Servidores e estado

Servidores *stateless*

Não mantém informação exata sobre o status de um cliente após ter processado uma requisição:

- Não guarda se um arquivo foi aberto (simplesmente fecha-o e abre de novo se necessário)
- Não promete invalidar o cache do cliente
- Não rastreia os seus clientes

Consequências

- Clientes e servidores são completamente independentes (evita coordenação)
- Inconsistências de estado devido a problemas no cliente ou servidor são reduzidas
- Possível perda de desempenho. Um servidor não pode antecipar o comportamento do cliente (ex: *prefetching*)

Servidores e estado

Home após logar



Servidores com estado (*stateful*)

Guardam o status de seus clientes:

- Registram quando um arquivo foi aberto para realização de *prefetching*
- Sabem quando o cliente possui cache dos dados e permitem que os clientes mantenham cópias locais de dados compartilhados

Servidores e estado

Servidores com estado (*stateful*)

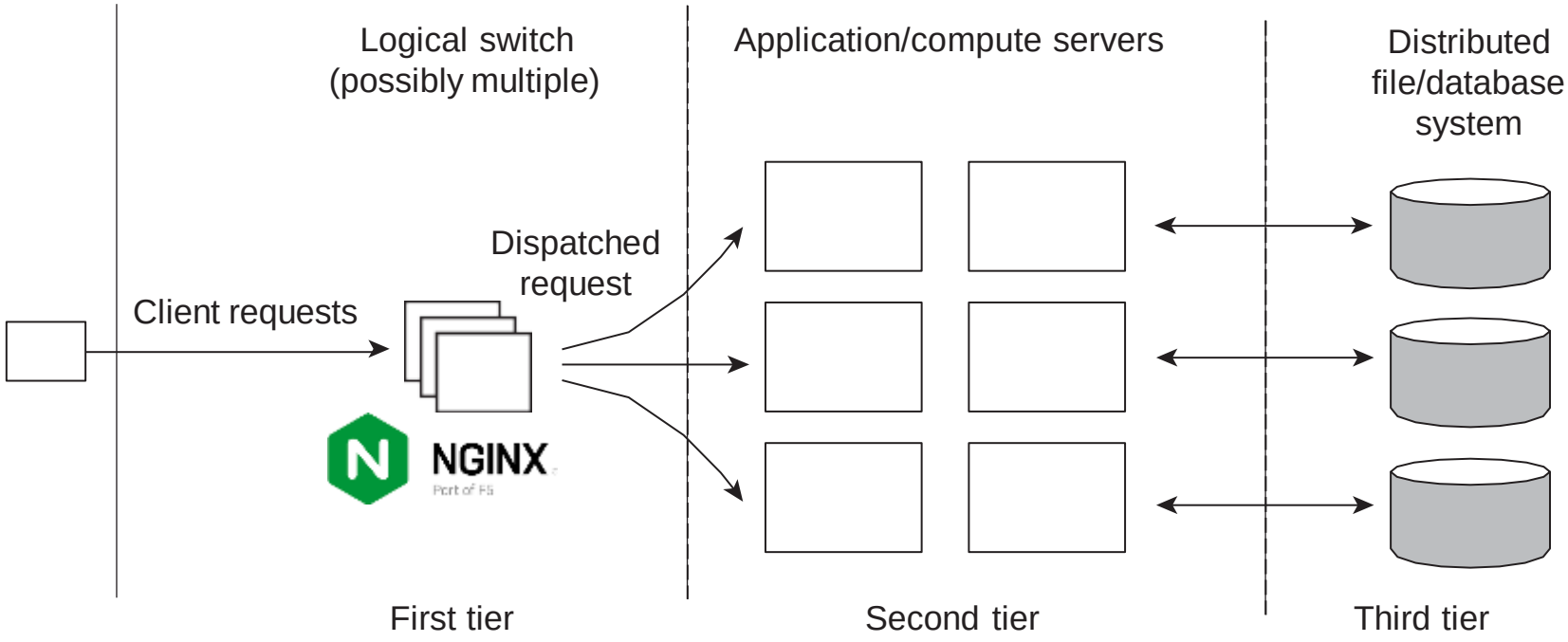
Guardam o status de seus clientes:

- Registram quando um arquivo foi aberto para realização de *prefetching*
- Sabem quando o cliente possui cache dos dados e permitem que os clientes mantenham cópias locais de dados compartilhados

Observação:

O desempenho de servidores *stateful* pode ser extremamente alto, desde que seja permitido que os clientes mantenham cópias locais dos dados. Nesse caso, confiabilidade não é o maior problema e sim a inconsistência.

Aglomerados de servidores: três camadas diferentes



A primeira camada (*frontend*) é responsável por interagir com clientes e repassar requisições para um servidor de lógica de negócios e computação apropriado (*backend*).

O logical switch pode ser um proxy (componente intermediário que encaminha requisições) conhecido como *load balancer*.

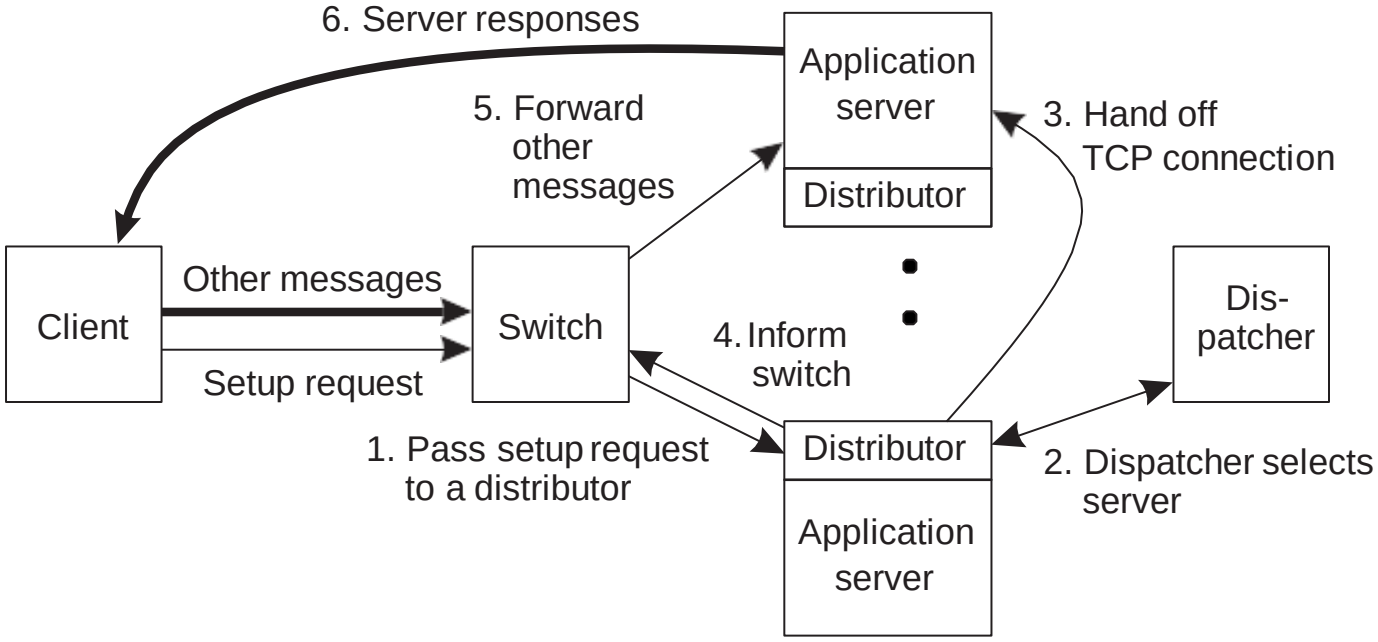
O *load balancer* utiliza diversas estratégias (round-robin, ociosidade, sessão, etc.) para encaminhar a requisição ao servidor adequado.

Aglomerados de servidores: tratamento de requisições

O frontend (switch do slide anterior) fica sobrecarregado facilmente: medidas precisam ser tomadas

Switch da camada de transporte: Frontend simplesmente redireciona o mensagem a um dos servidores backend (considerando métricas de desempenho)

Distribuição: frontend seleciona o melhor servidor backend de acordo ao conteúdo da mensagem recebida



Agenda

- Introdução à processos e threads
- Threads em sistemas distribuídos
- Clientes
- Servidores
- Migração de código

Migração de código

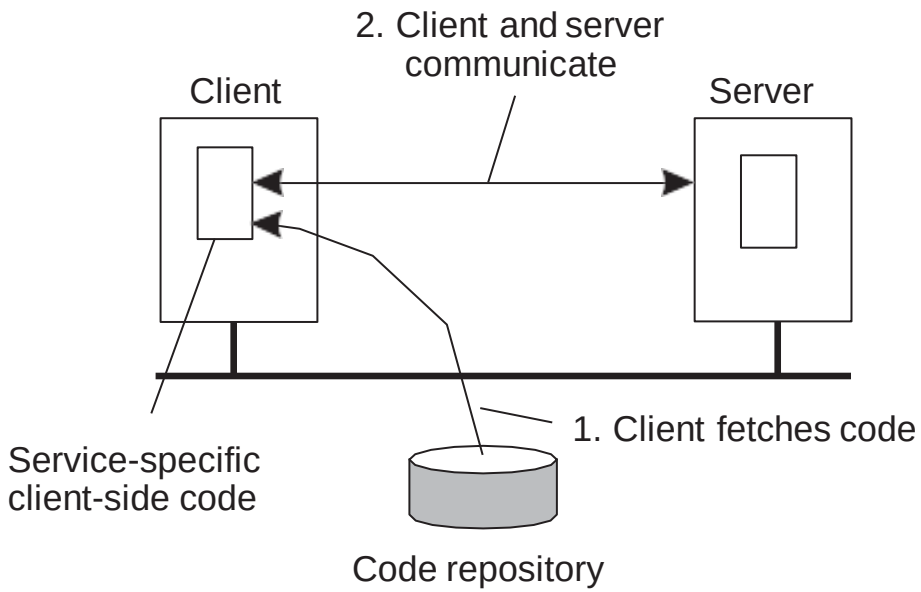
- Abordagens para realização de migração de código
- Migração em sistemas heterogêneos

Migração de código: razões para migrar

Distribuição de carga

Minimizar a comunicação assegurando que os computadores estão próximos aos dados

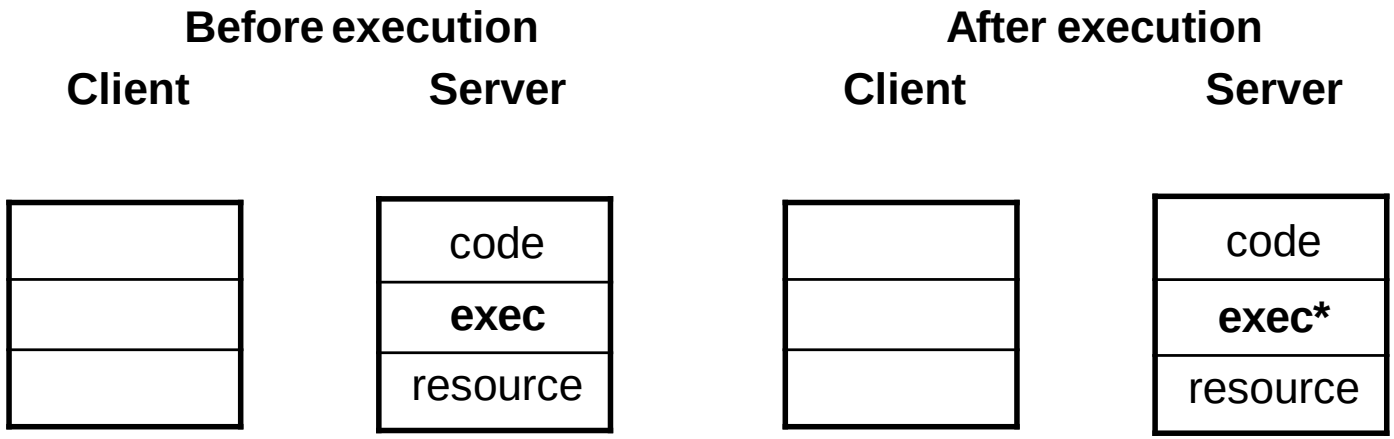
Flexibilidade: mover o código ao cliente quando necessário



Migração de código: contexto

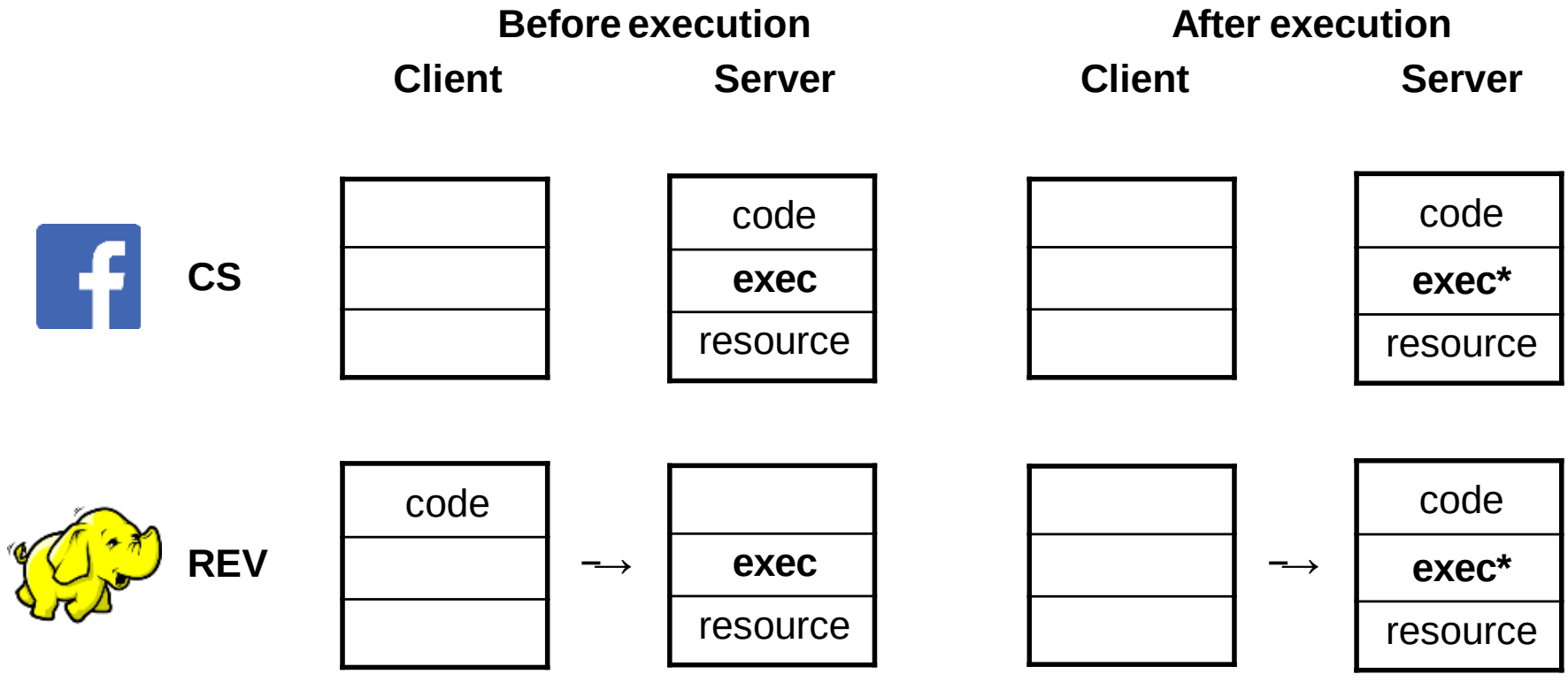


CS



CS: Client-Server

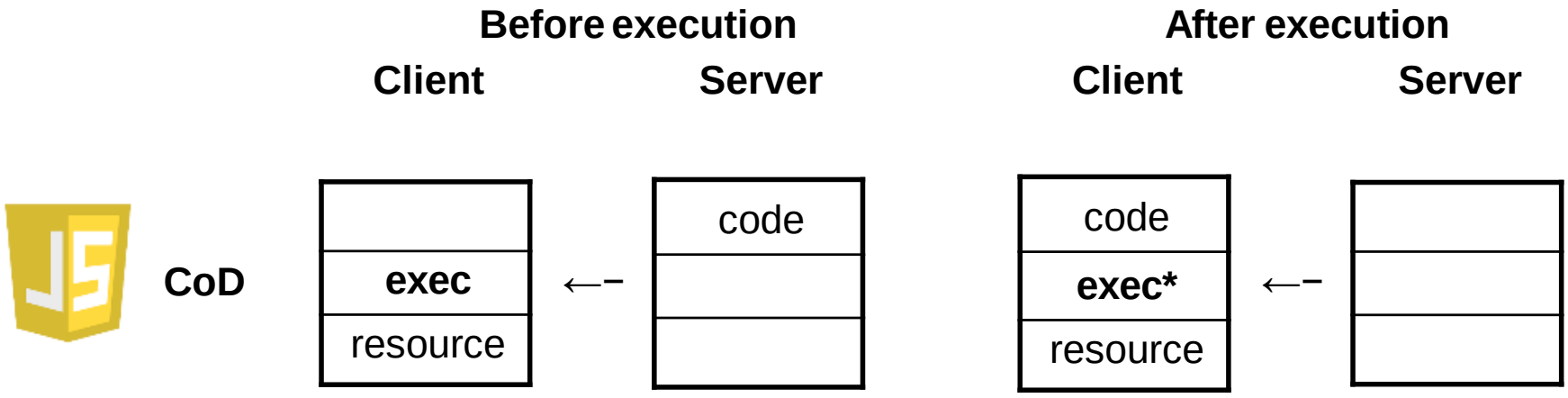
Migração de código: contexto



CS: Client-Server

REV: Remote evaluation

Migração de código: contexto



CoD: Code-on-demand

Migração de código: contexto

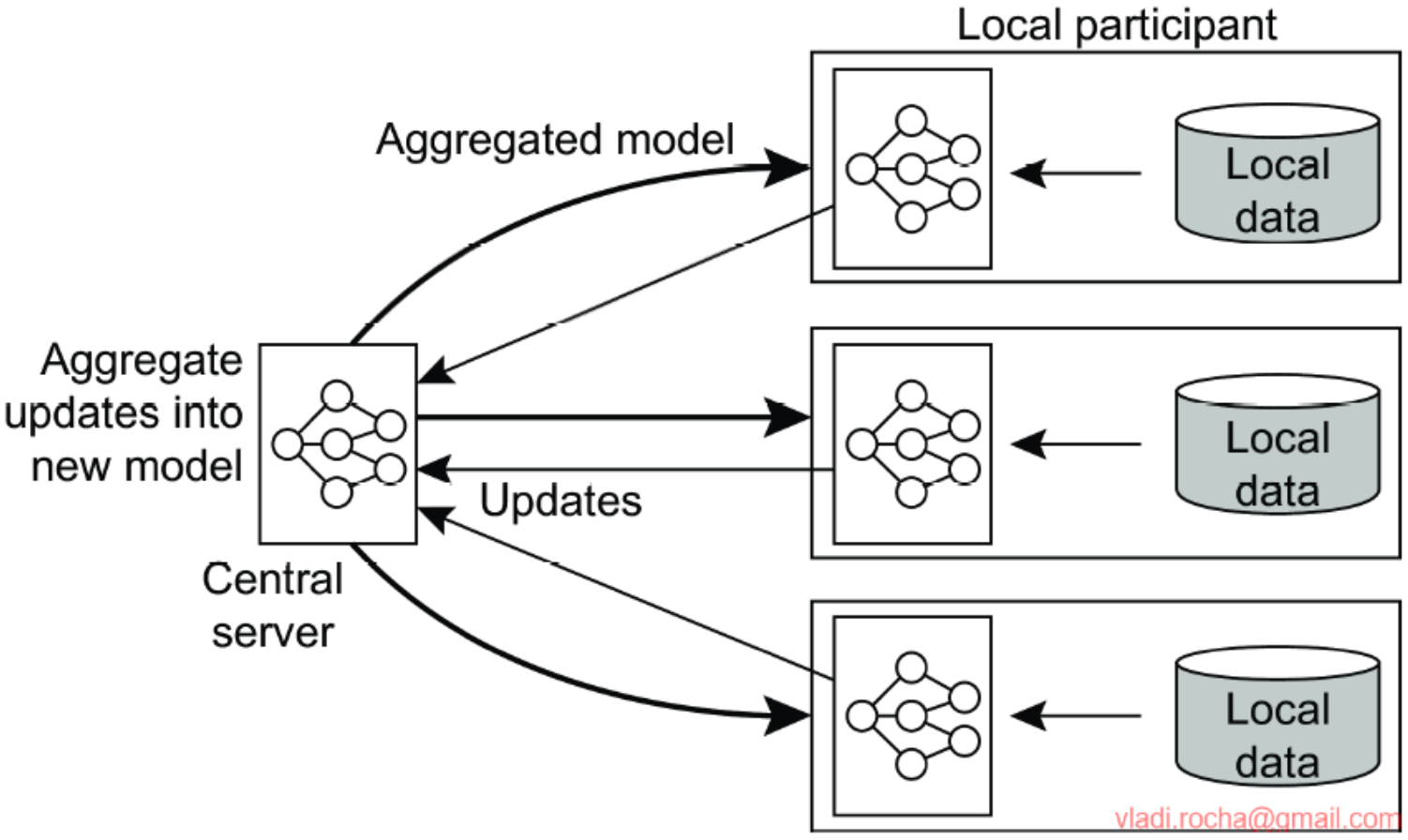
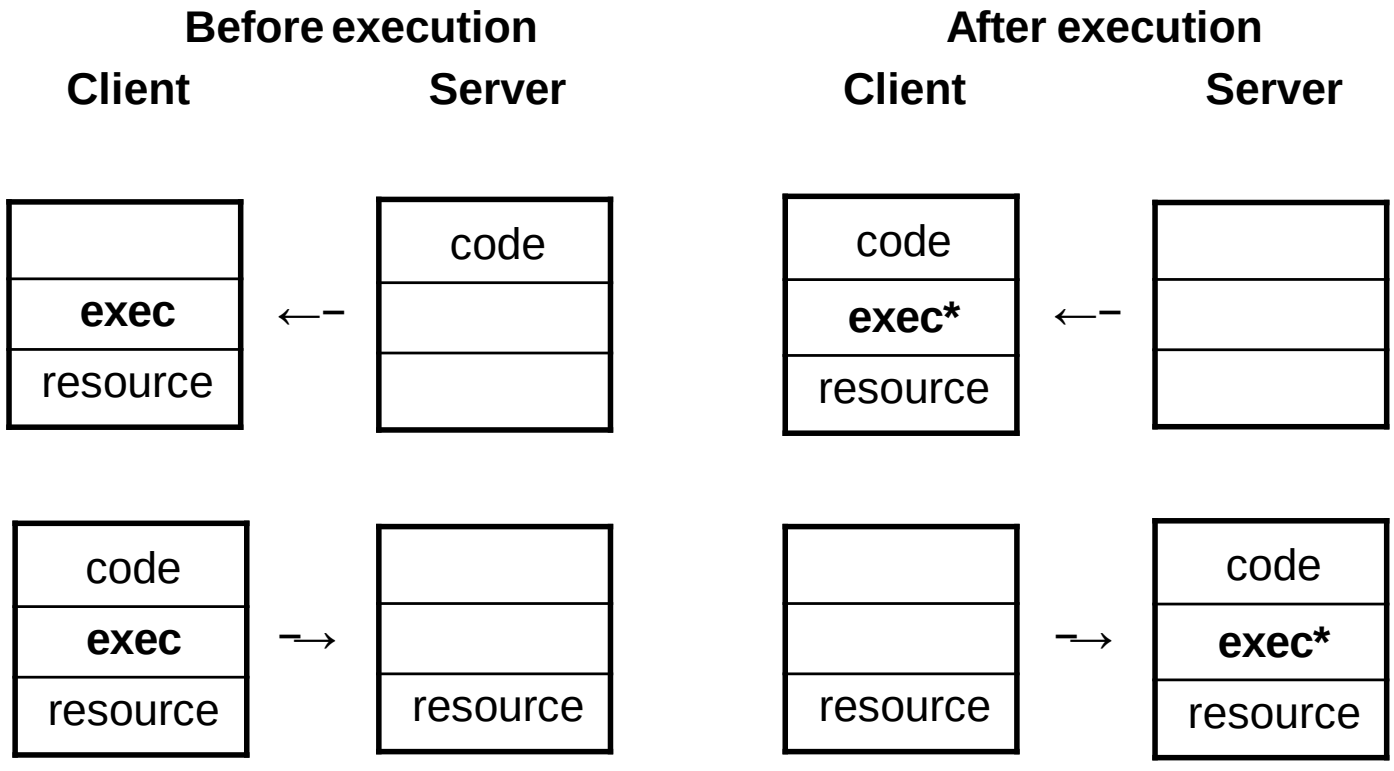


Figure 3.26: The principle of federated learning. (2018)

CoD: Code-on-demand and Machine Learning

Migração de código: contexto





CoD: Code-on-demand

MA: Mobile agents

Migração em sistemas heterogêneos

Problema principal

- A máquina destino pode não ser adequada para executar o código migrado
- A definição de contexto de thread/processo/processador é altamente dependente do hardware, sistema operacional e bibliotecas locais

Solução

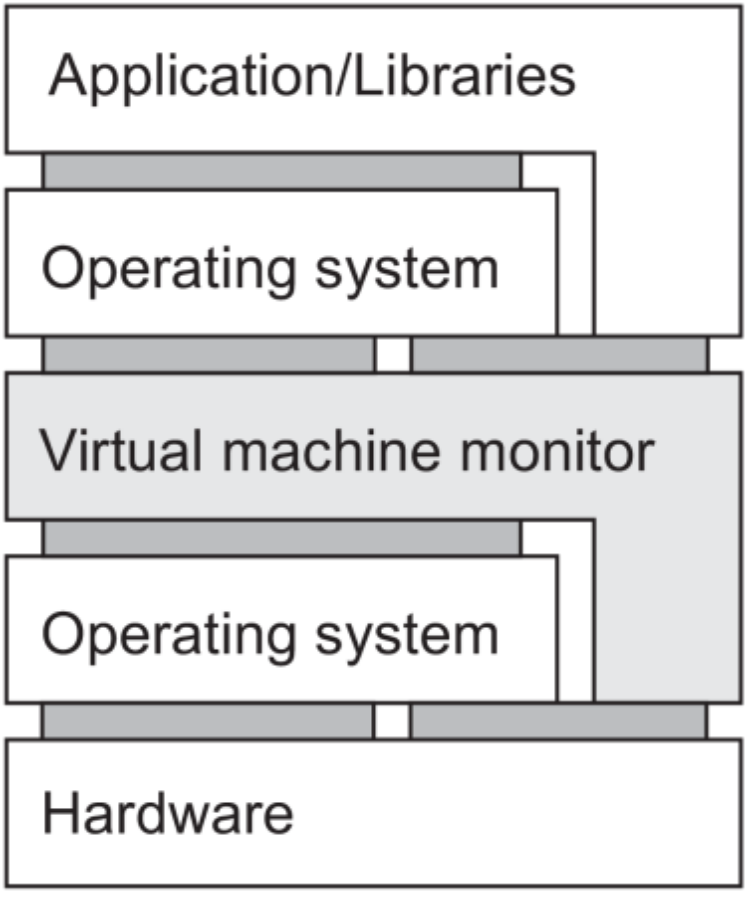
Usar alguma máquina abstrata que é implementada nas diferentes plataformas:

- Linguagens interpretadas, que possuem suas próprias MVs
- Uma MV que represente um SO completo
- Um container que represente um SO completo

Migração em sistemas heterogêneos

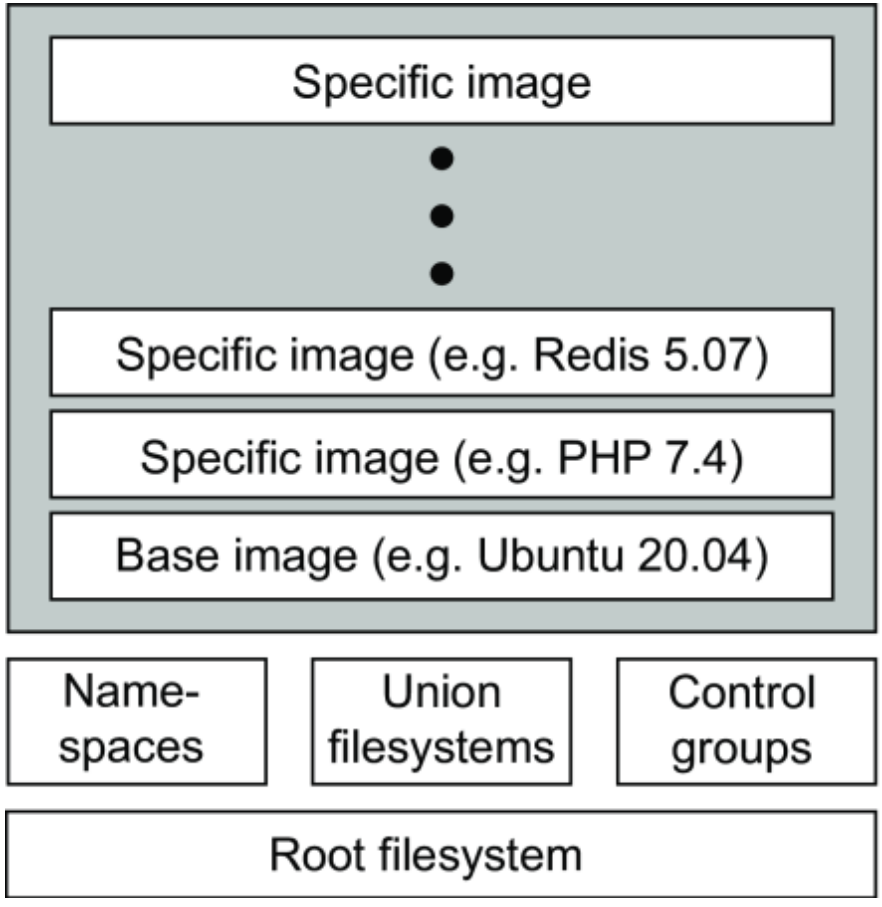
Máquina virtual de um SO

VirtualBox [2007]

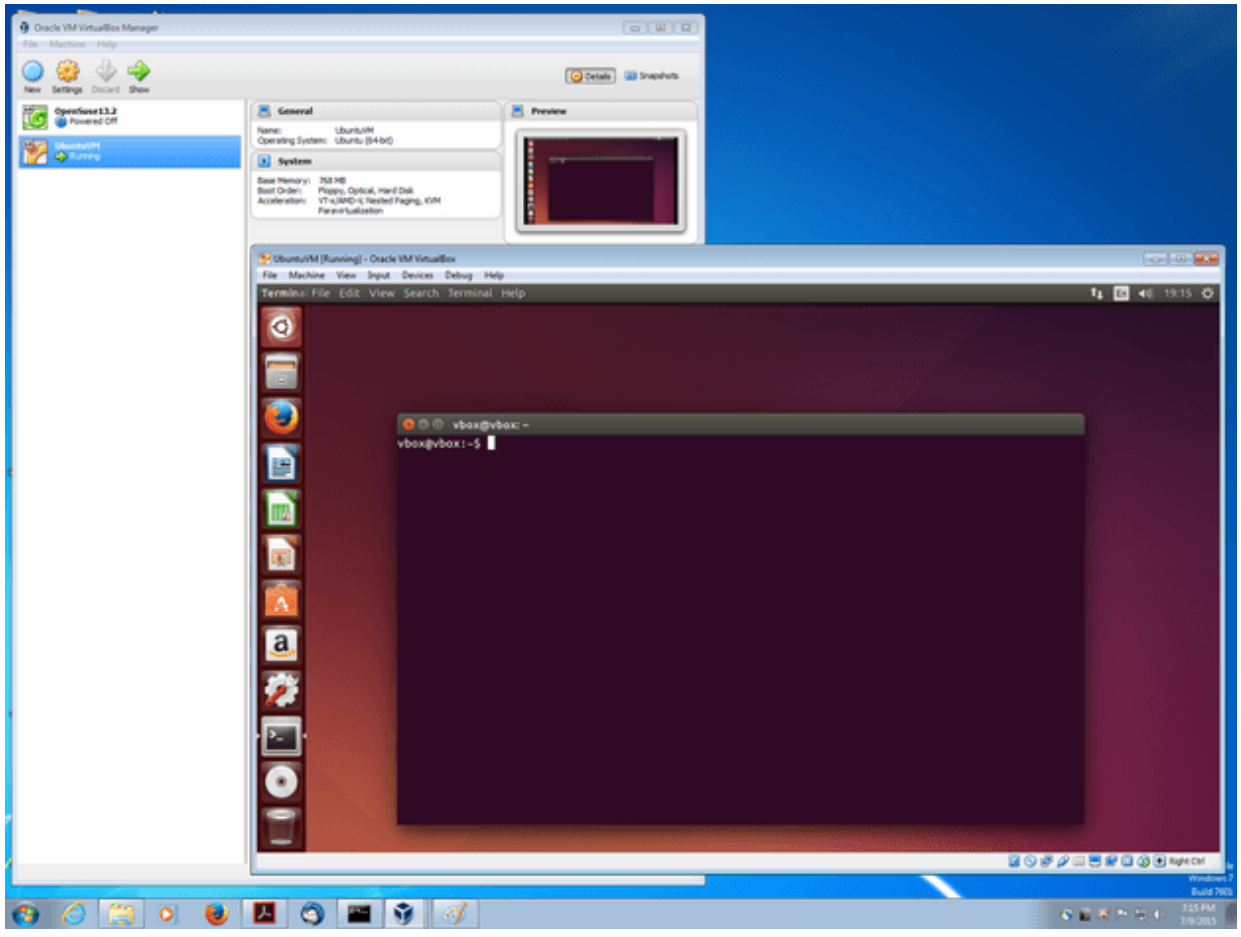


Container de um SO

Docker [2013]



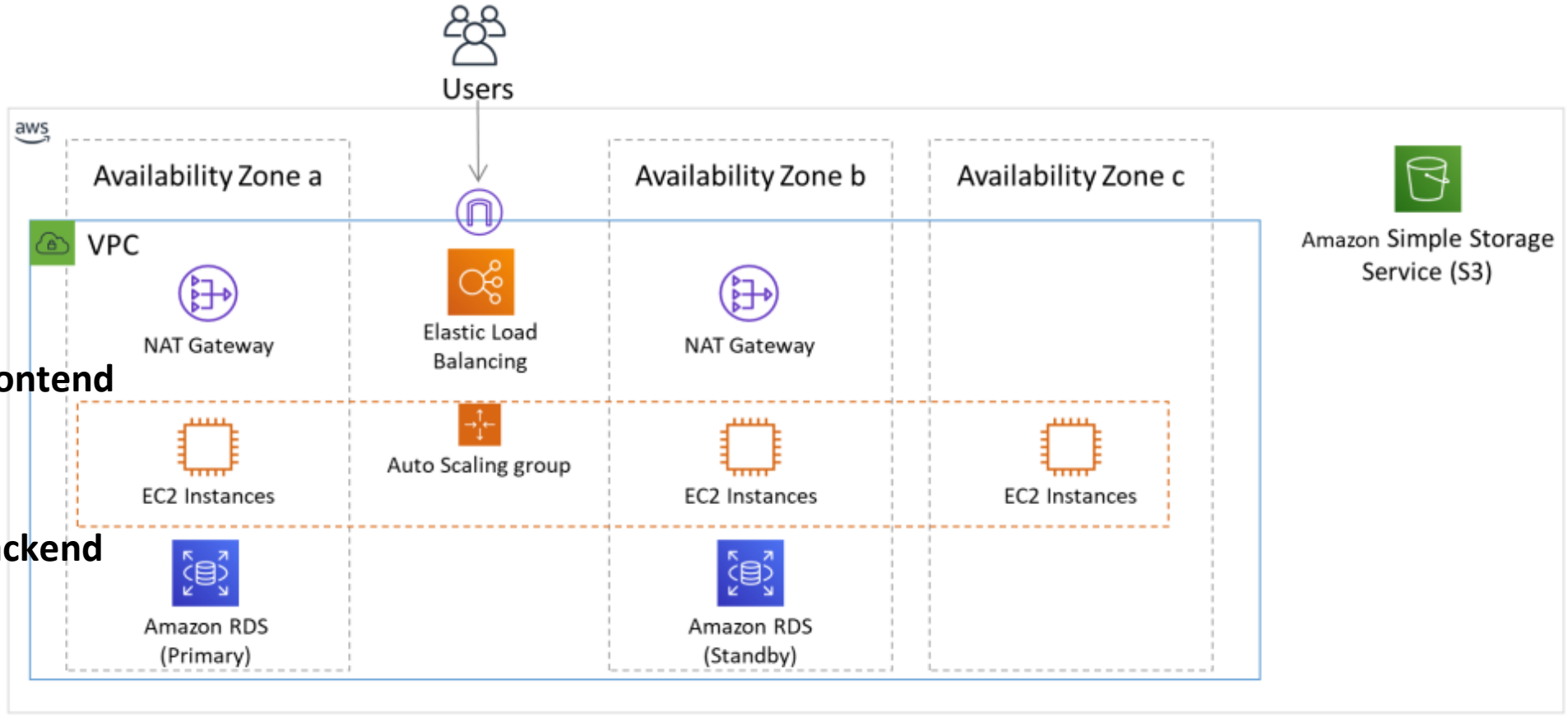
Migração em sistemas heterogêneos



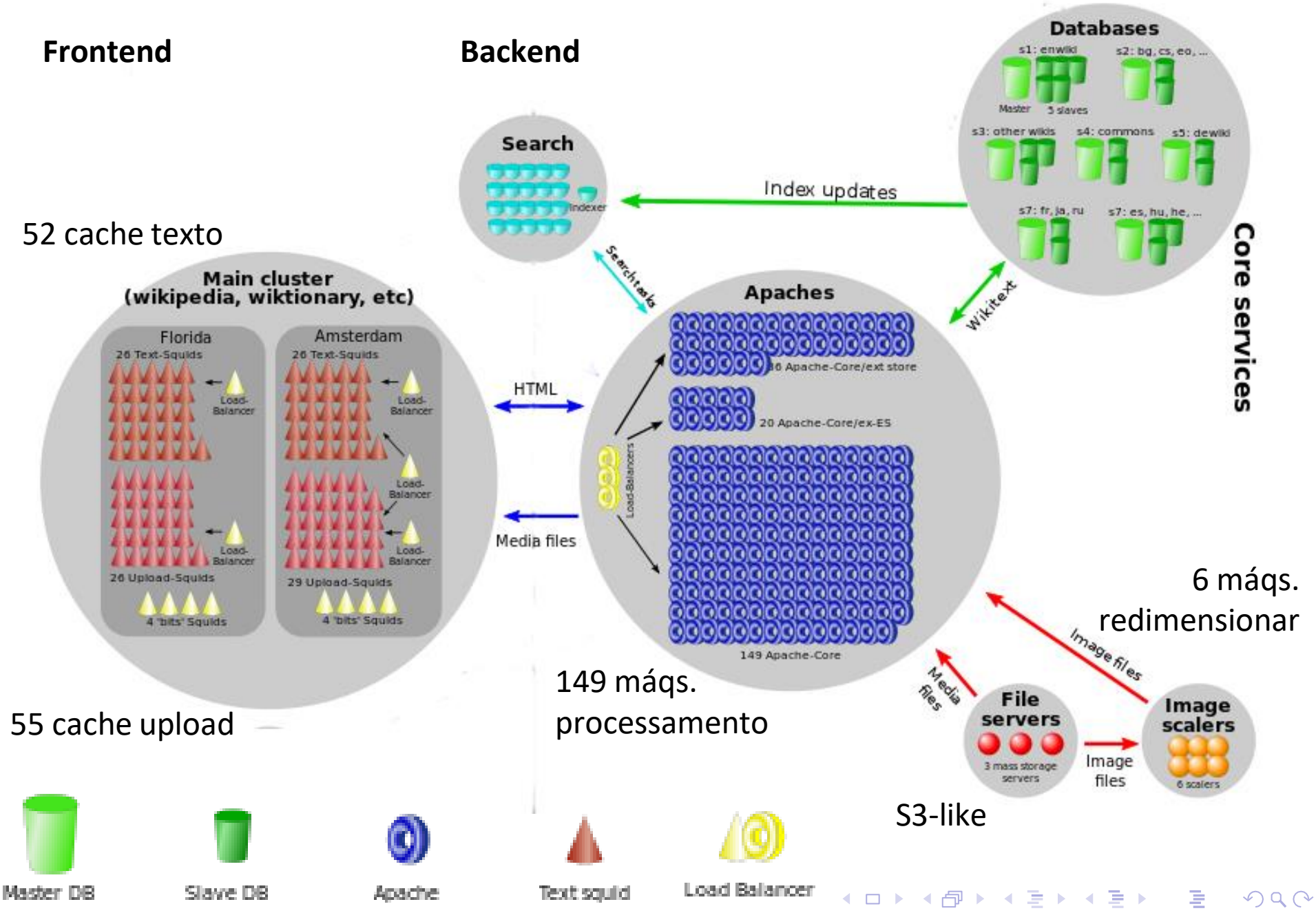
Vídeo de como instalar Ubuntu no Virtualbox (opcional)
https://www.youtube.com/watch?v=i_mNaqvrBe0

Servidores Caso1: Arquitetura Resiliente (AWS)

https://aws.amazon.com/pt/blogs/architecture/building-resilient-well-architected-workloads-using-aws-resilience-hub/



Servidores Caso2: Arquitetura do Wikimedia



Conceitos adquiridos

- Threads no servidor (dispatcher/worker).
- Concorrência
- Stateless e Stateful.
- Load balancer.
- Virtualização.
- Frontend e Backend.

Sistemas Distribuídos

Comunicação

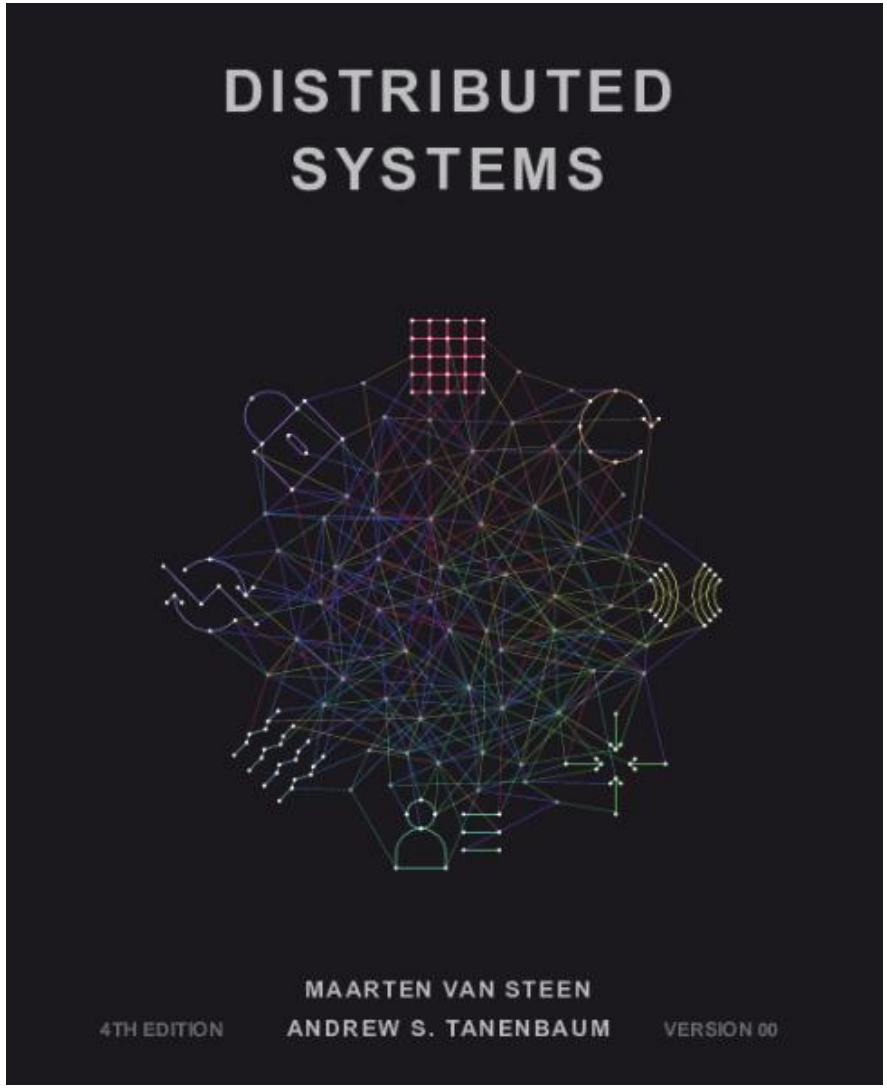
COMMUNICATION

Vladimir Rocha (Vladi)

CMCC - Universidade Federal do ABC

Distributed Systems 4th edition (free)

https://www.distributed-systems.net/index.php/books/ds4/



4.03 (2025)

Disclaimer

- Estes slides foram baseados nos do professor **Emilio Francesquini** para o curso de Sistemas Distribuídos na UFABC.
- Este material pode ser usado livremente desde que sejam mantidos, além deste aviso, os créditos aos autores e instituições.
- Estes slides foram adaptados daqueles originalmente preparados (e gentilmente cedidos) pelo professor **Daniel Cordeiro, da EACH-USP** que por sua vez foram baseados naqueles disponibilizados online pelos autores do livro "Distributed Systems", 3ª Edição em: https://www.distributed-systems.net.

Agenda

- Fundamentos
- Comunicação orientada a procedimento (RPC)
- Comunicação orientada a mensagem (MOM)
- Comunicação multicast (FIFO/Causal/Total – Gossip/Flooding)
- Comunicação orientada a fluxo

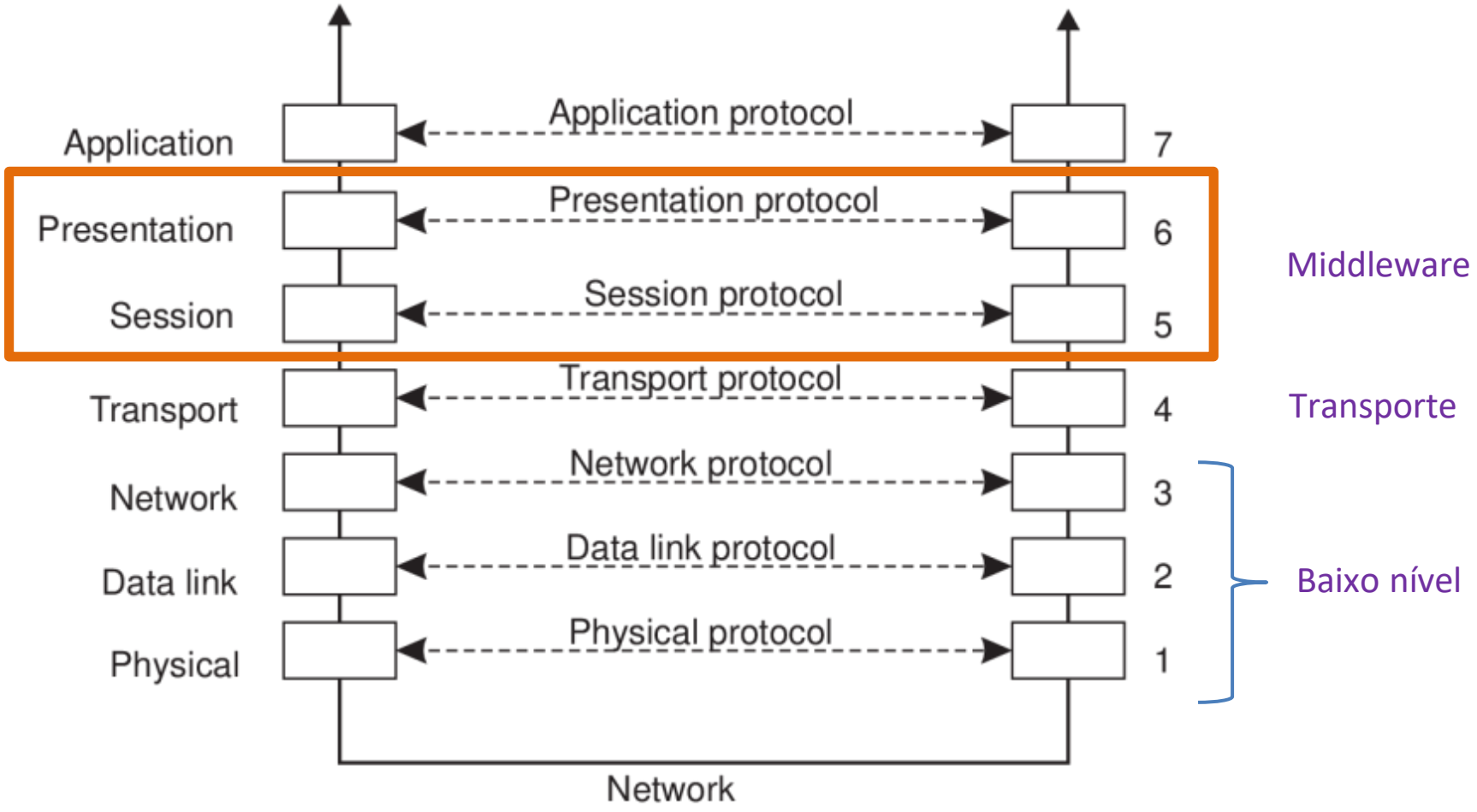
Agenda

- Fundamentos
- Comunicação orientada a procedimento (RPC)
- Comunicação orientada a mensagem (MOM)
- Comunicação multicast (Gossip / Flooding)
- Comunicação orientada a fluxo

Protocolos em camadas

- Camadas de baixo nível
- Camada de transporte
- Camada do middleware
- Camada de aplicação
- Tipos de comunicação

Modelo de comunicação básico



Desvantagens:

- Funciona apenas com passagem de mensagens
- Frequentemente possuem funcionalidades desnecessárias
- Viola a transparência de acesso

Camada de transporte

Importante:

A camada de transporte fornece as ferramentas de comunicação efetivamente utilizadas pela maioria dos sistemas distribuídos.

Protocolos padrões da Internet

TCP: orientada a conexão, confiável, comunicação orientada a fluxo de dados

UDP: comunicação de datagramas não confiável (*best-effort*)

Nota:

IP multicasting é normalmente considerado um serviço padrão (mas essa é uma hipótese perigosa)

Camada de middleware

Middleware foi inventado para prover serviços e protocolos frequentemente usados que podem ser utilizados por várias aplicações diferentes.

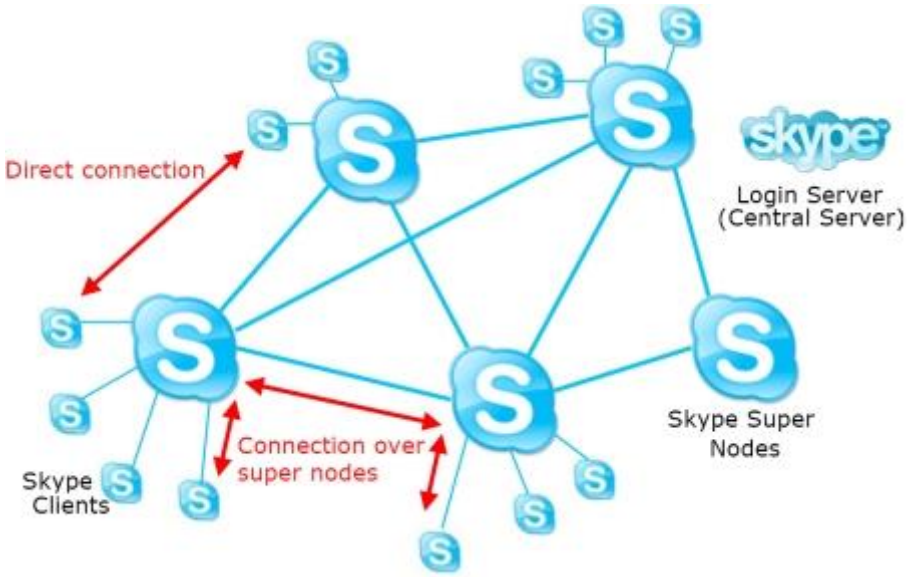
- Um conjunto rico de protocolos de comunicação
- (Des)empacotamento [*(un)marshaling*] de dados, necessários para a integração de sistemas
- Protocolos de gerenciamento de nomes, para auxiliar o compartilhamento de recursos
- Protocolos de segurança para comunicações seguras
- Mecanismos de escalabilidade, como replicação, sharding e caching

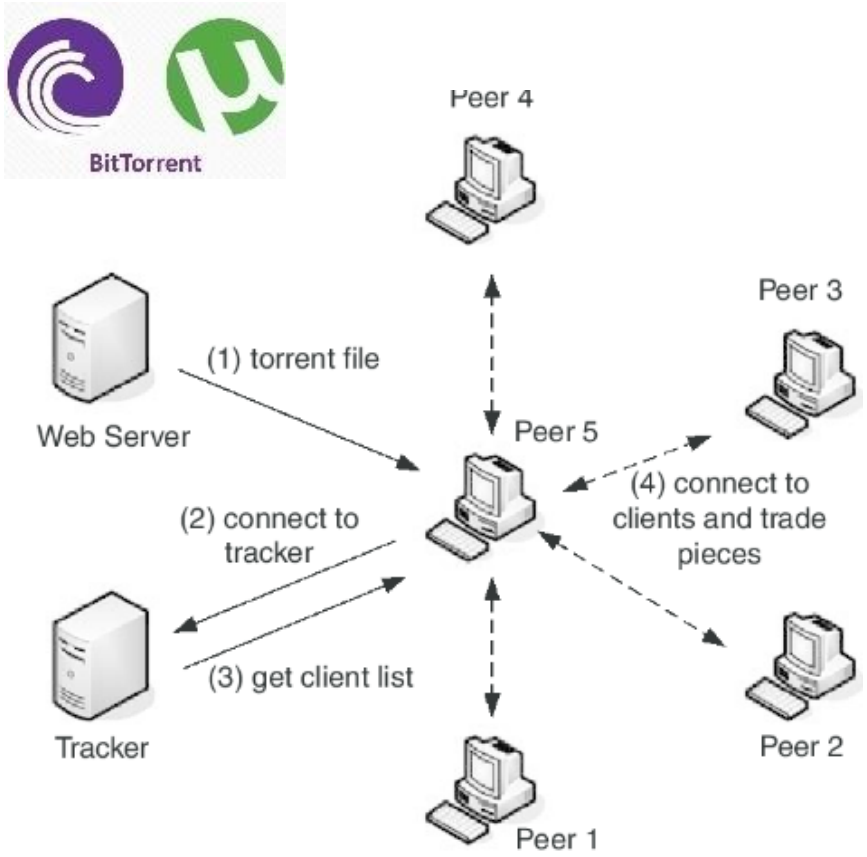
Observação:

O que realmente sobra são protocolos específicos de aplicação.

Camada de aplicação

www.img.lx.it.pt%2F~fp%2Fcomunicacao_multimedia%2Fano%2520lectivo_2016_2017%2FTrabalhos_CMul_2016_2017%2FGrupo%25203%2Farquitetura.html



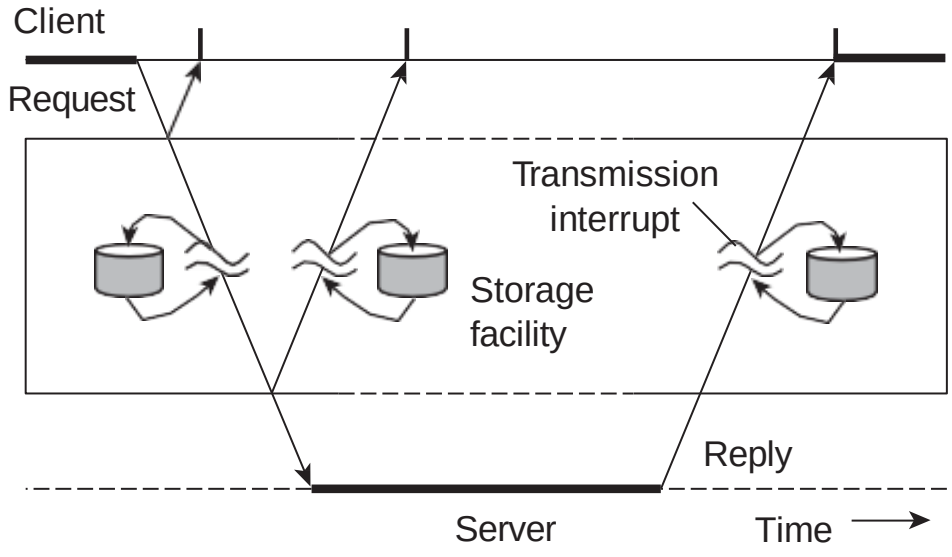


https://www.researchgate.net/publication/225633414_Modeling_energy_efficiency_of_file_sharing/figures?lo=1

Tipos de comunicação

- Comunicação transiente vs. persistente
- Comunicação assíncrona vs. síncrona

Tipos de comunicação

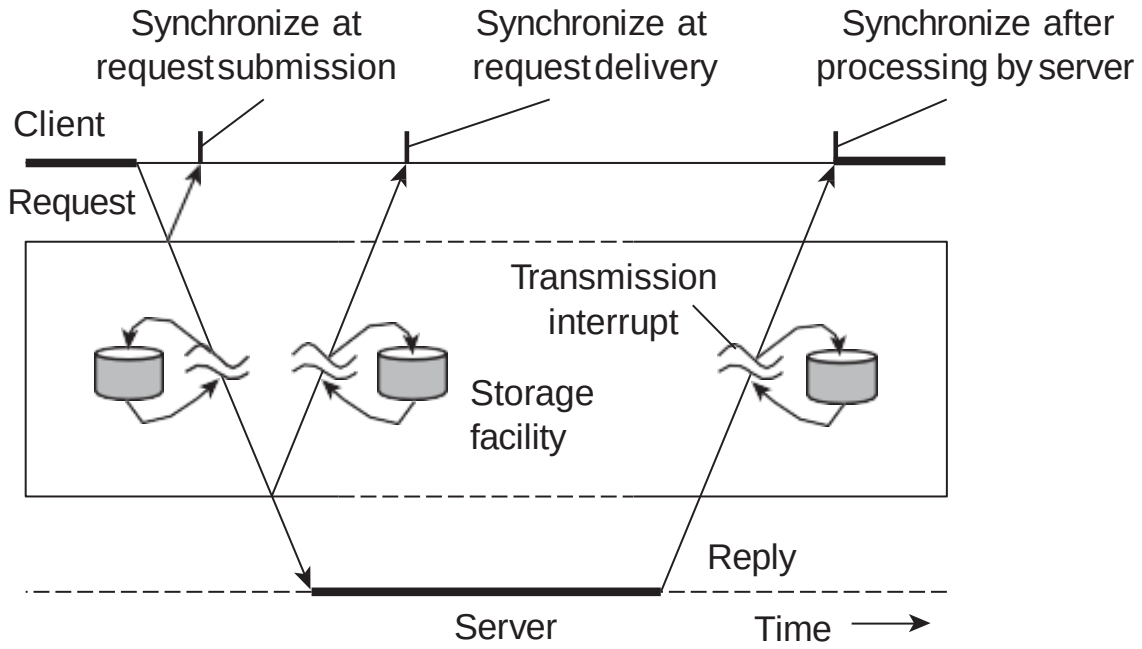


Transiente vs. persistente

Comunicação transiente: remetente descarta a mensagem se ela não puder ser encaminhada para o destinatário

Comunicação persistente: uma mensagem é guardada no remetente pelo tempo que for necessário, até ser entregue no destinatário

Tipos de comunicação



Pontos de sincronização

- No envio da requisição
- Na entrega da requisição
- Após o processamento da requisição

Agenda

- Fundamentos
- Comunicação orientada a procedimento (RPC)
- Comunicação orientada a mensagem (MOM)
- Comunicação multicast (Gossip / Flooding)
- Comunicação orientada a fluxo



RMI

Chamadas a procedimentos remotos (RPC)

- Funcionamento básico de RPCs [Nelson 1984]
- Passagem de parâmetros
- Tipos de comunicação no RPC
- Exemplo

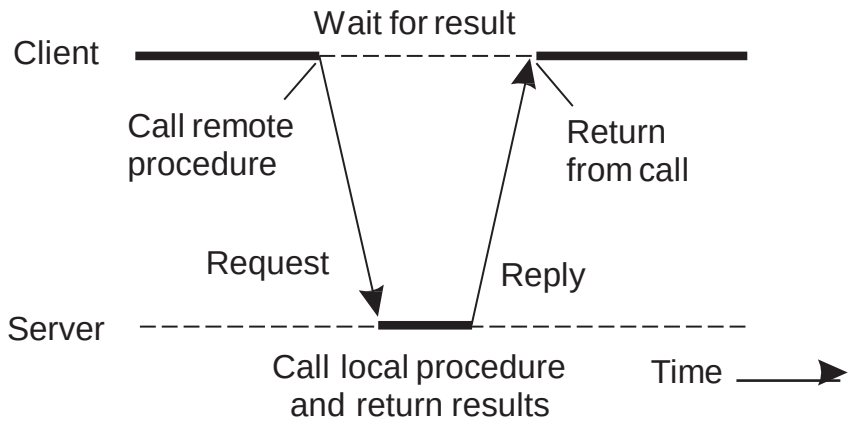
RPC: Funcionamento básico

- Desenvolvedores estão familiarizados com o modelo de procedimentos
- Procedimentos bem projetados operam isoladamente (*black box*)

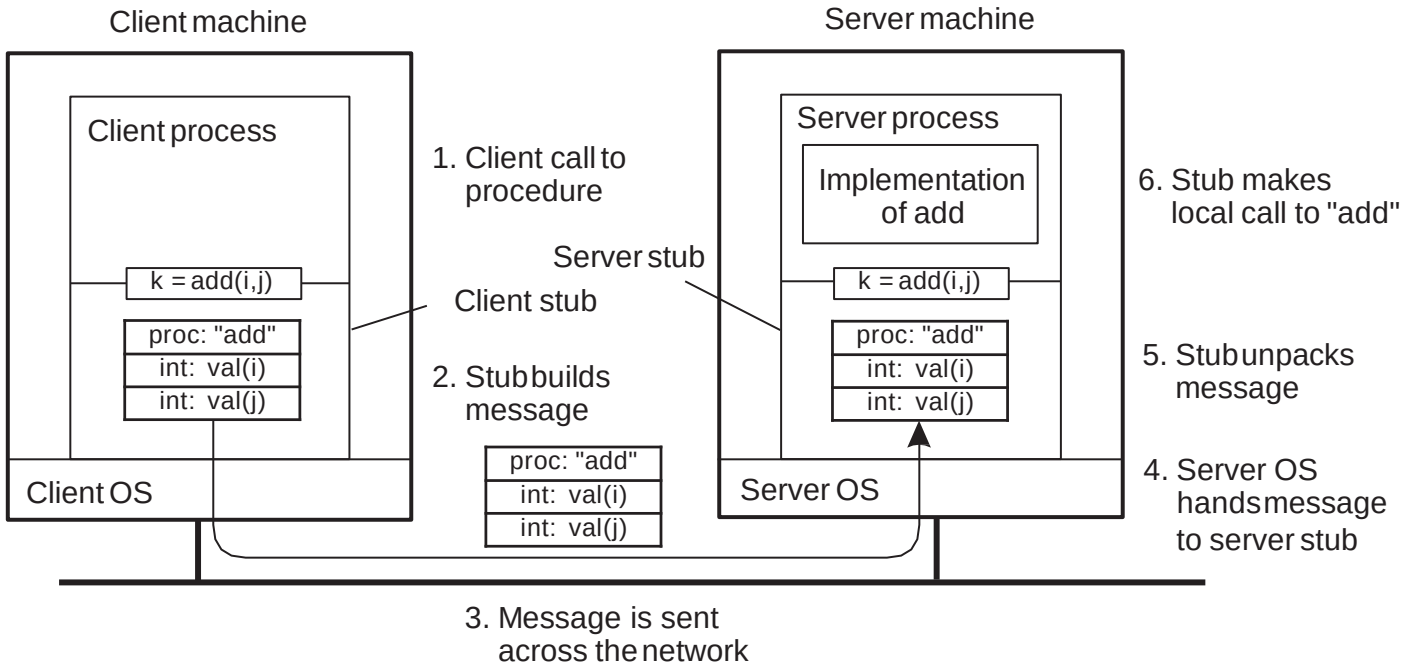
  Precursor dos microsserviços
- Então não há razão para não executar esses procedimentos em máquinas separadas

Conclusão

Comunicação entre o chamador & chamado pode ser escondida com o uso de mecanismos de chamada a procedimentos.



RPC: Funcionamento básico



1. Procedimento no cliente chama o *stub* do cliente
2. *Stub* constrói mensagem; chama o SO local
3. SO envia msg. para o SO remoto
4. SO remoto repassa mensagem para o *stub*
5. *Stub* desempacota parâmetros e chama o servidor
6. Servidor realiza chamada local e devolve resultado para o *stub*
7. *Stub* constrói mensagem; chama SO (volta)
8. SO envia mensagem para o SO do cliente
9. SO do cliente repassa msg. para o *stub*
10. *Stub* do cliente desempacota resultado e devolve para o cliente

RPC: concorrência

Que acontece quando vários clientes acessam o método ?

Depende da implementação. Normalmente (e.g., Java RMI) utiliza-se o modelo dispatcher/worker.

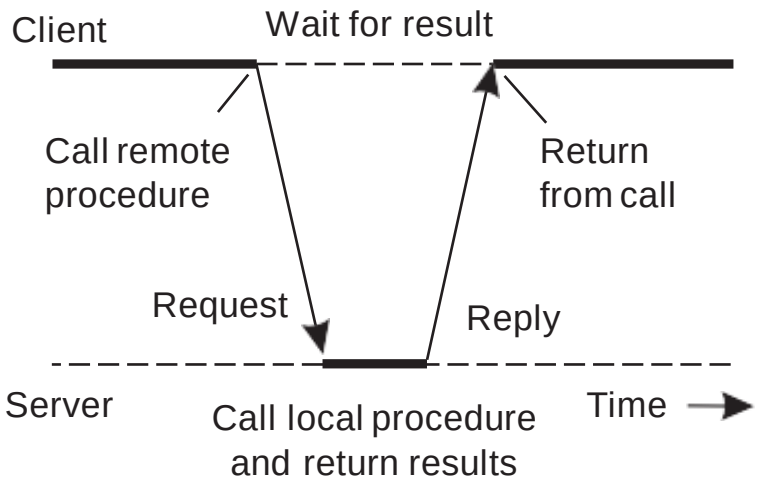
Ou seja, cada requisição do cliente é atendida por uma thread diferente.

Isso quer dizer que é necessário proteger as variáveis e estruturas de acessos concorrentes.

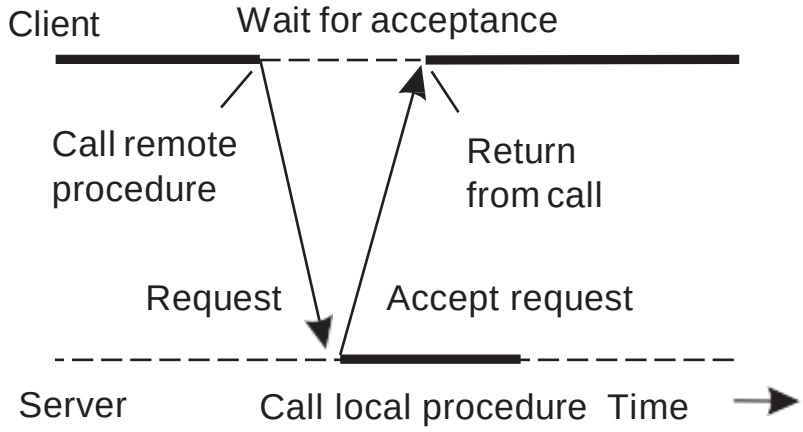
RPC: tipos de comunicação → assíncrono (sem resposta do servidor)

Ideia geral

Tentar se livrar do comportamento estrito de requisição–resposta, permitindo que o cliente continue sem esperar por uma resposta do servidor.



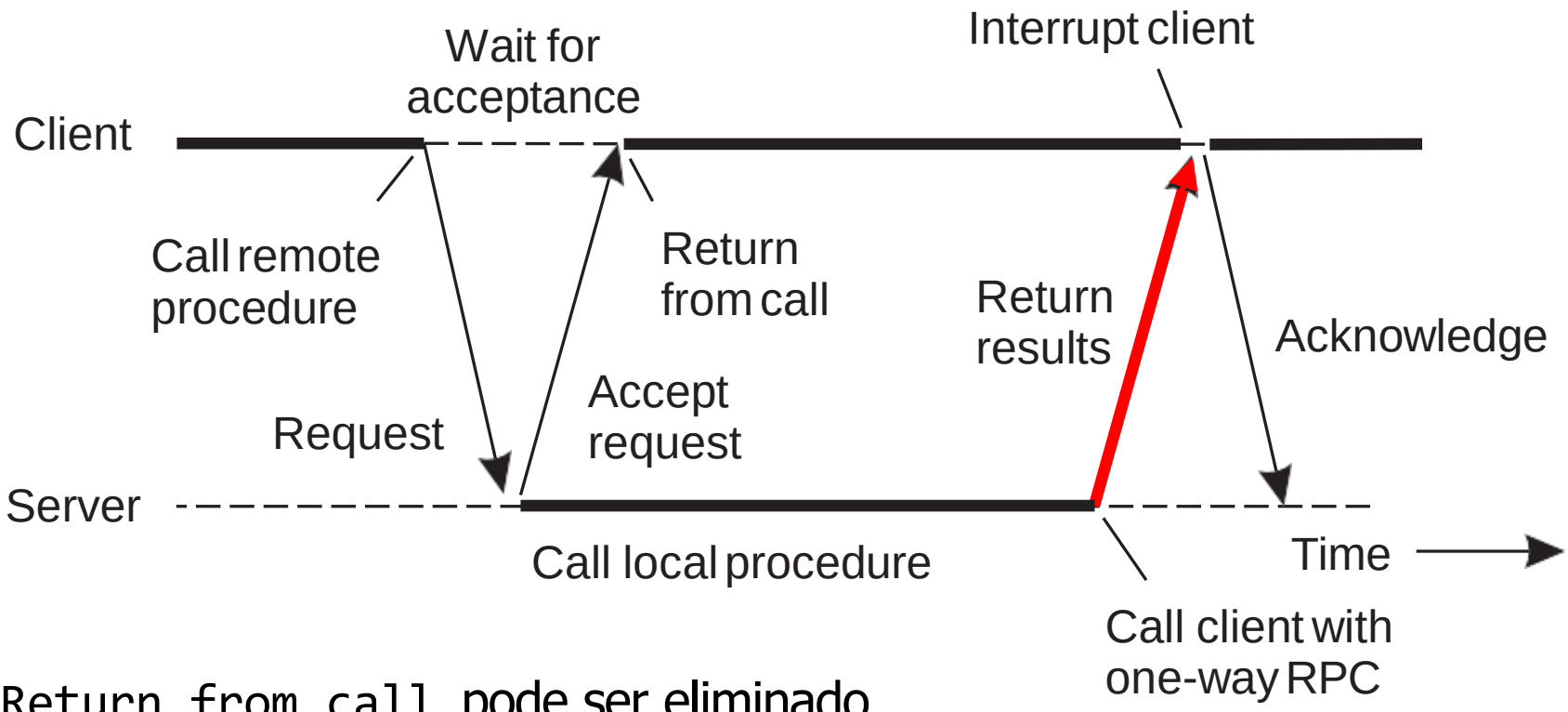
(a)



(b)

`Return from call` pode ser eliminado

RPC: tipos de comunicação → síncrono diferido (também chamado de assíncrono)



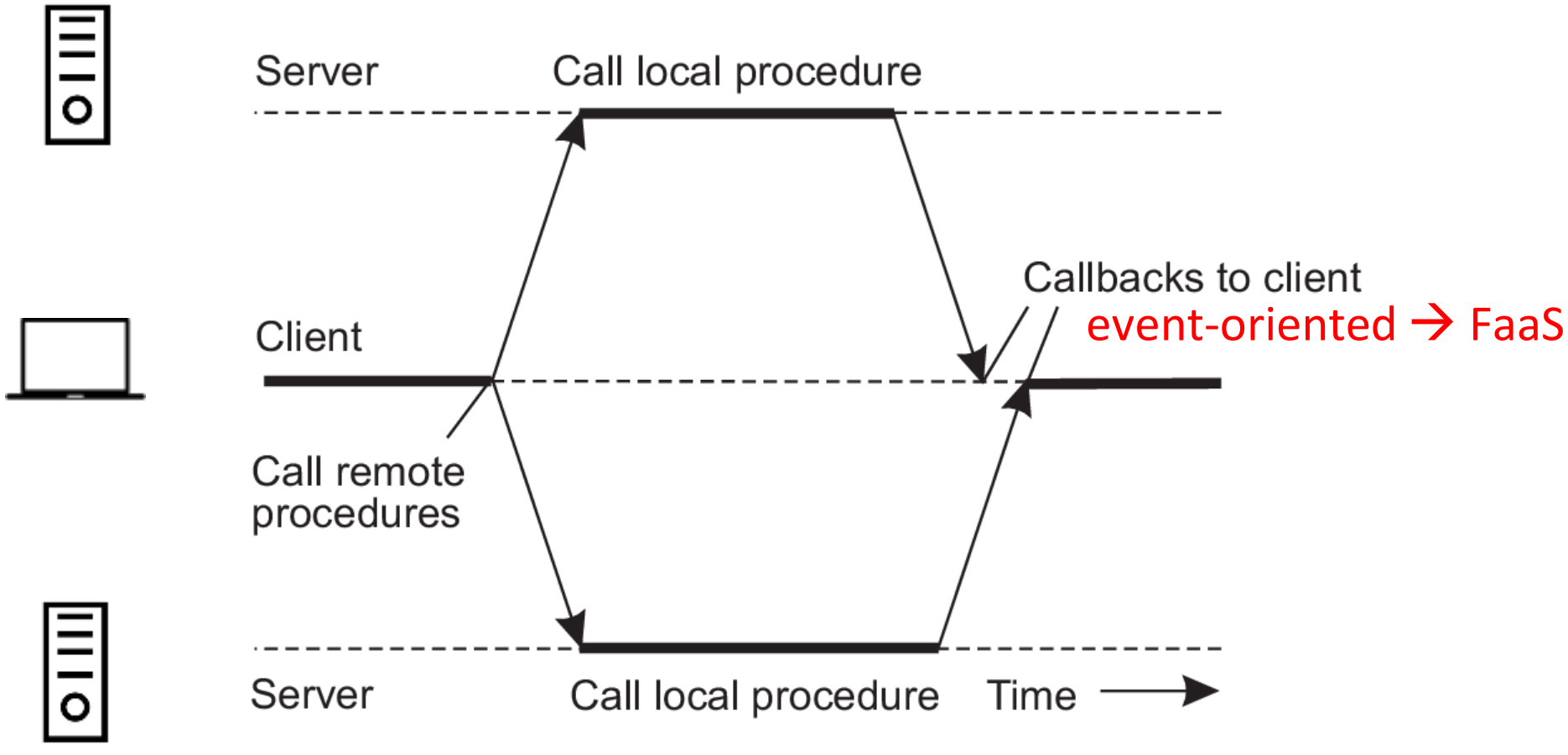
`Return from call` pode ser eliminado

Variação

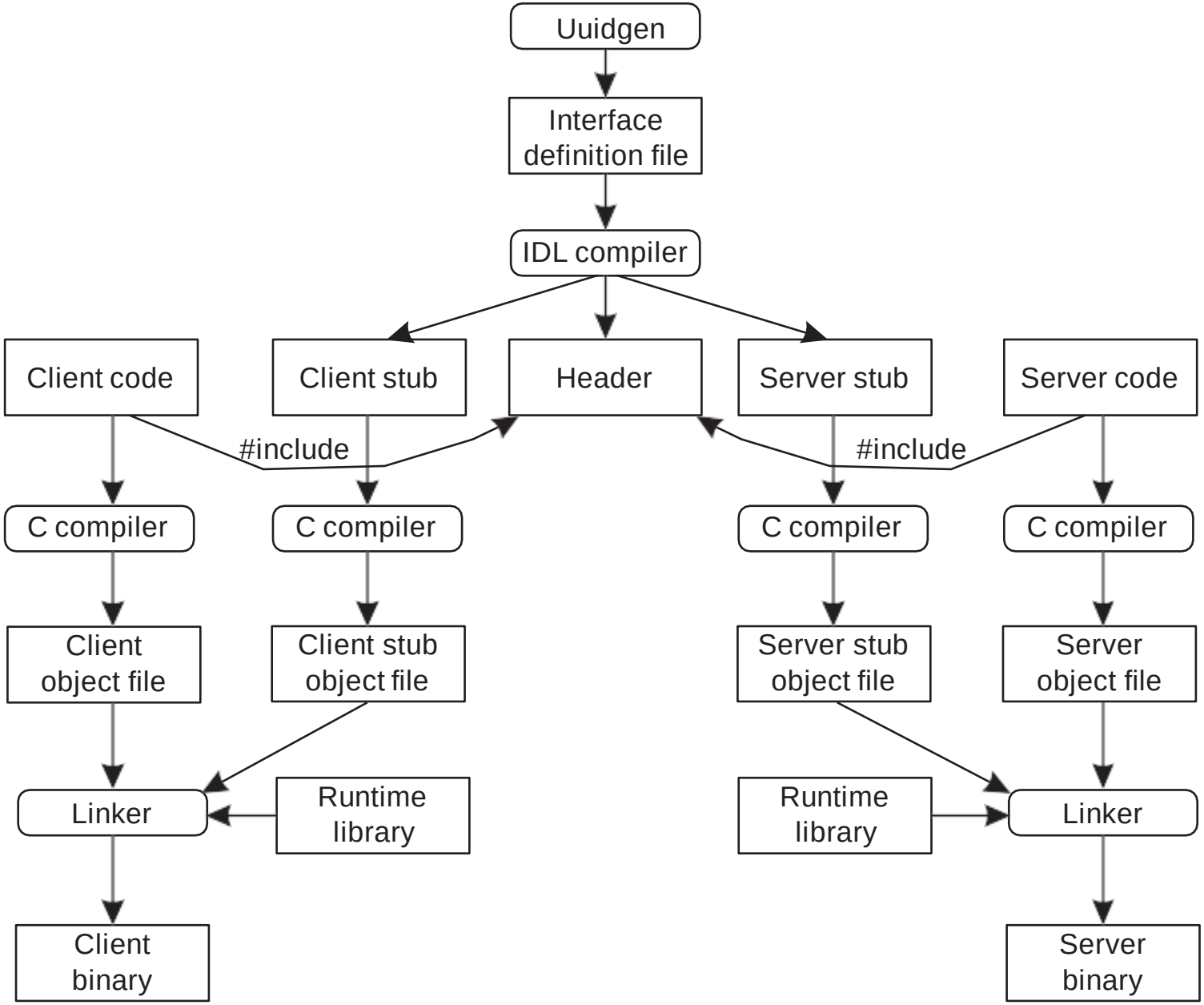
Cliente pode também realizar uma consulta (*poll*) (bloqueante ou não) para verificar se os resultados estão prontos.

RPC: tipos de comunicação → Enviando várias

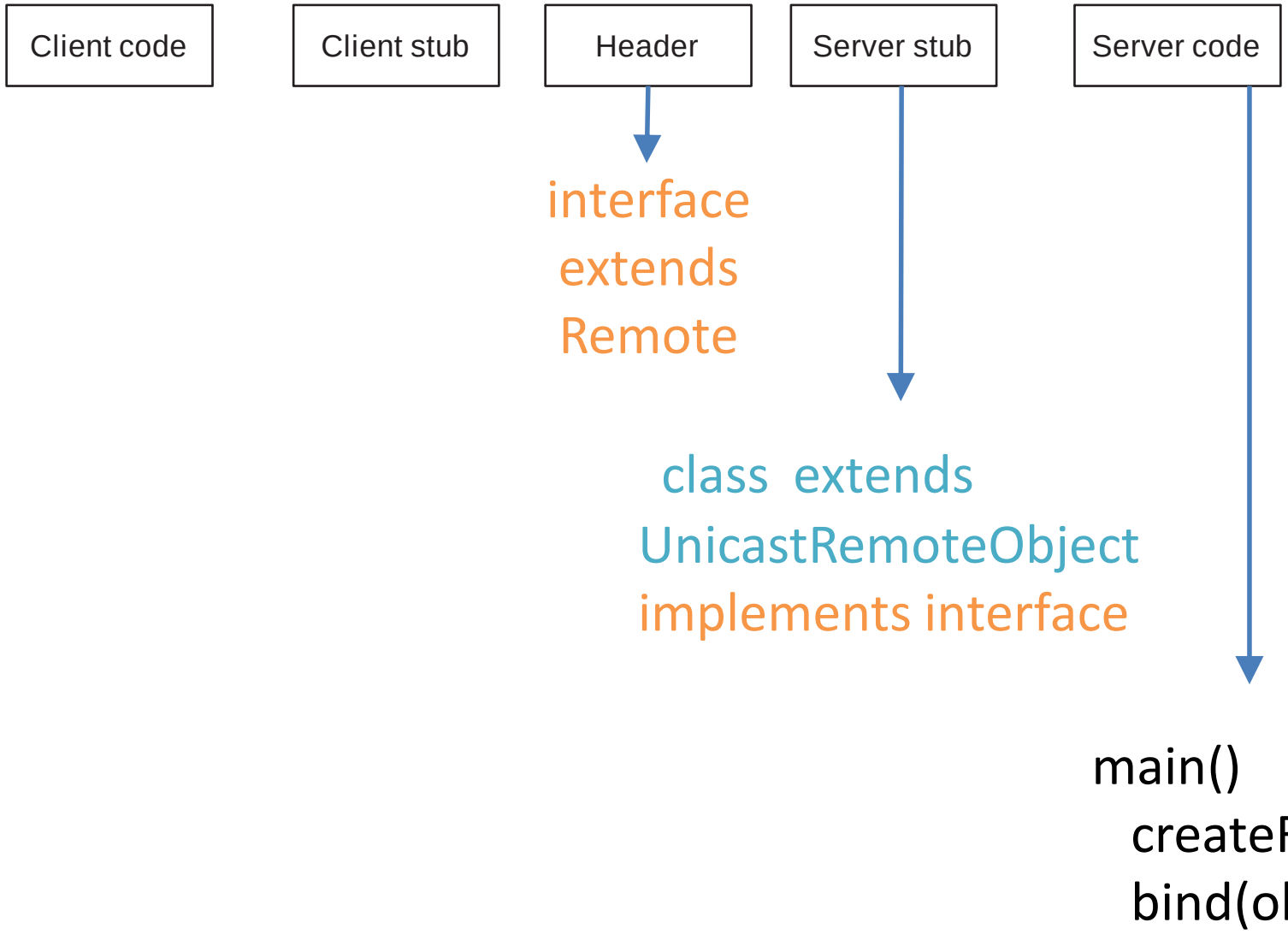
Enviando um pedido de RPC a um grupo de servidores



RPC: Exemplo


Uuidgen
Interface definition file
IDL compiler
Client code
Client stub
Header
Server stub
Server code
#include
#include
C compiler
C compiler
C compiler
C compiler
Client object file
Client stub object file
Server stub object file
Server object file
Linker
Runtime library
Runtime library
Linker
Client binary
Server binary

RPC: Exemplo (RMI – Remote Method Invocation)

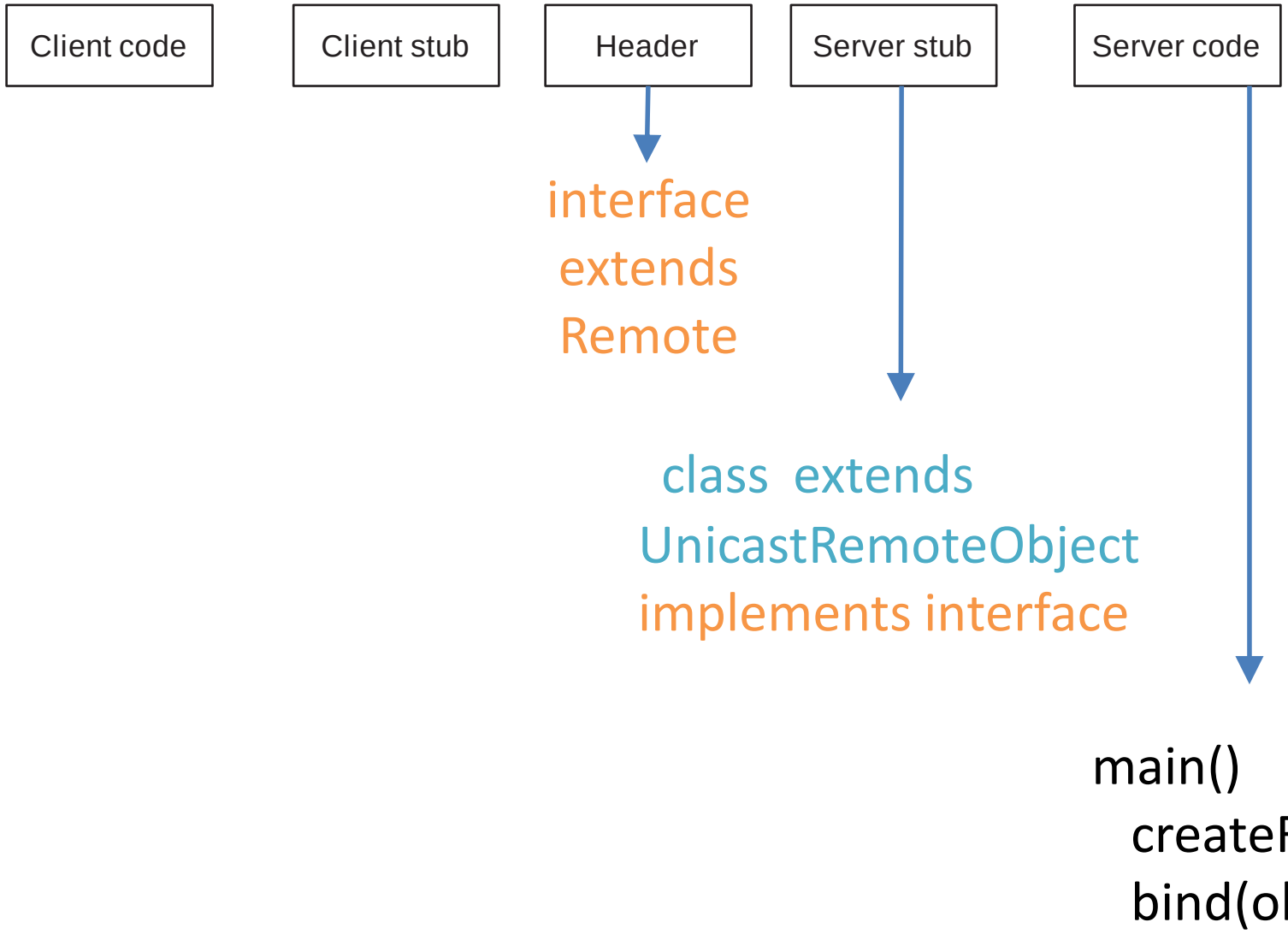


RPC: Exemplo (RMI – Remote Method Invocation)

```
public interface ServicoMatriz extends Remote {

  public int[][] inverte(int[][] matrizGigante)
                                    throws RemoteException;
}
```

RPC: Exemplo (RMI – Remote Method Invocation)



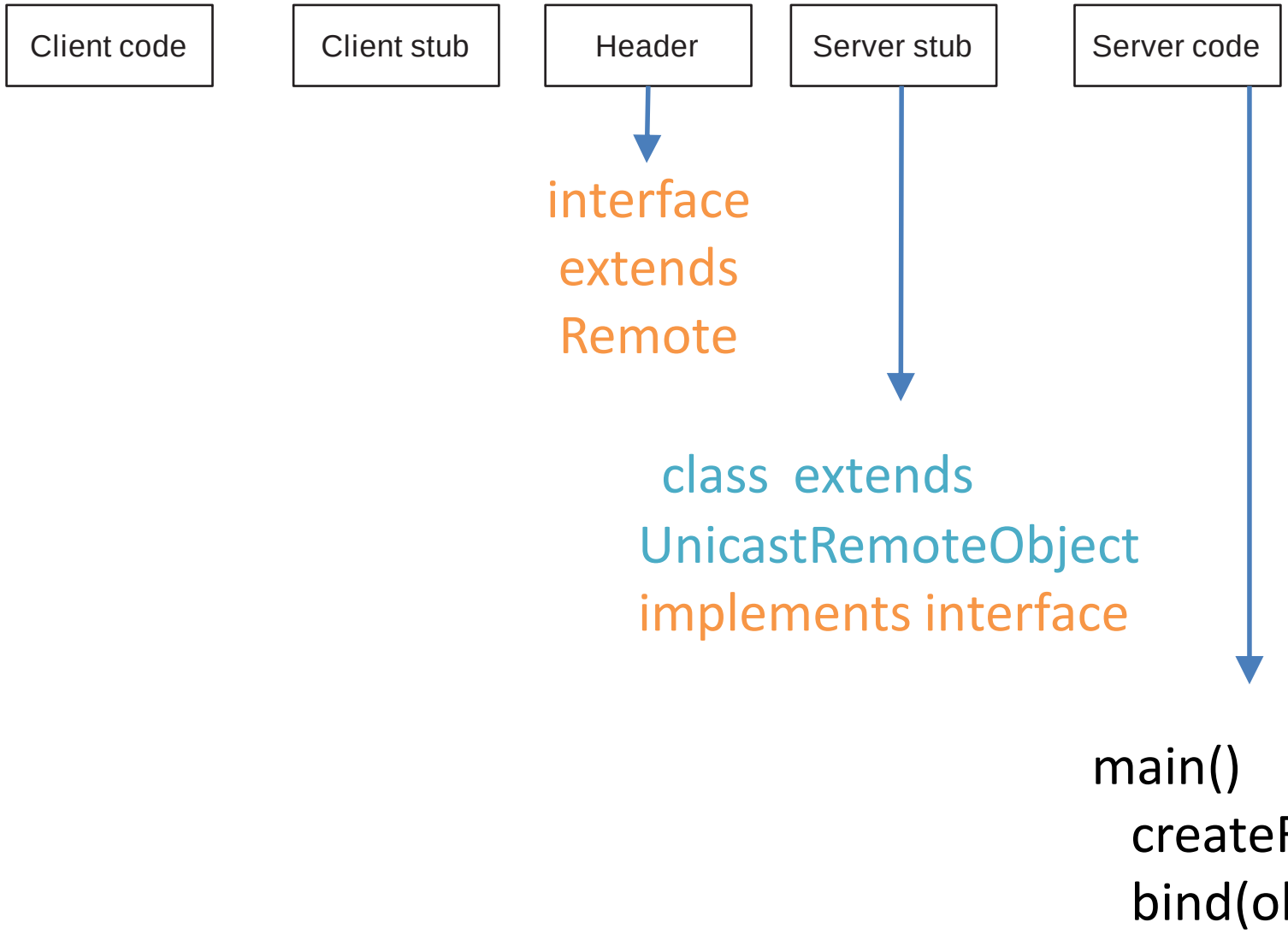
RPC: Exemplo (RMI – Remote Method Invocation)

```
import etml.*
public class ServicoImpl
                extends UnicastRemoteObject
                implements ServicoMatriz {

    public int[][] inverte(int[][] matrizGigante)
                                throws RemoteException {

        Matrix m = new Matrix(matrizGigante);
        return m.invert();
    }
}
```

RPC: Exemplo (RMI – Remote Method Invocation)



RPC: Exemplo (RMI – Remote Method Invocation)

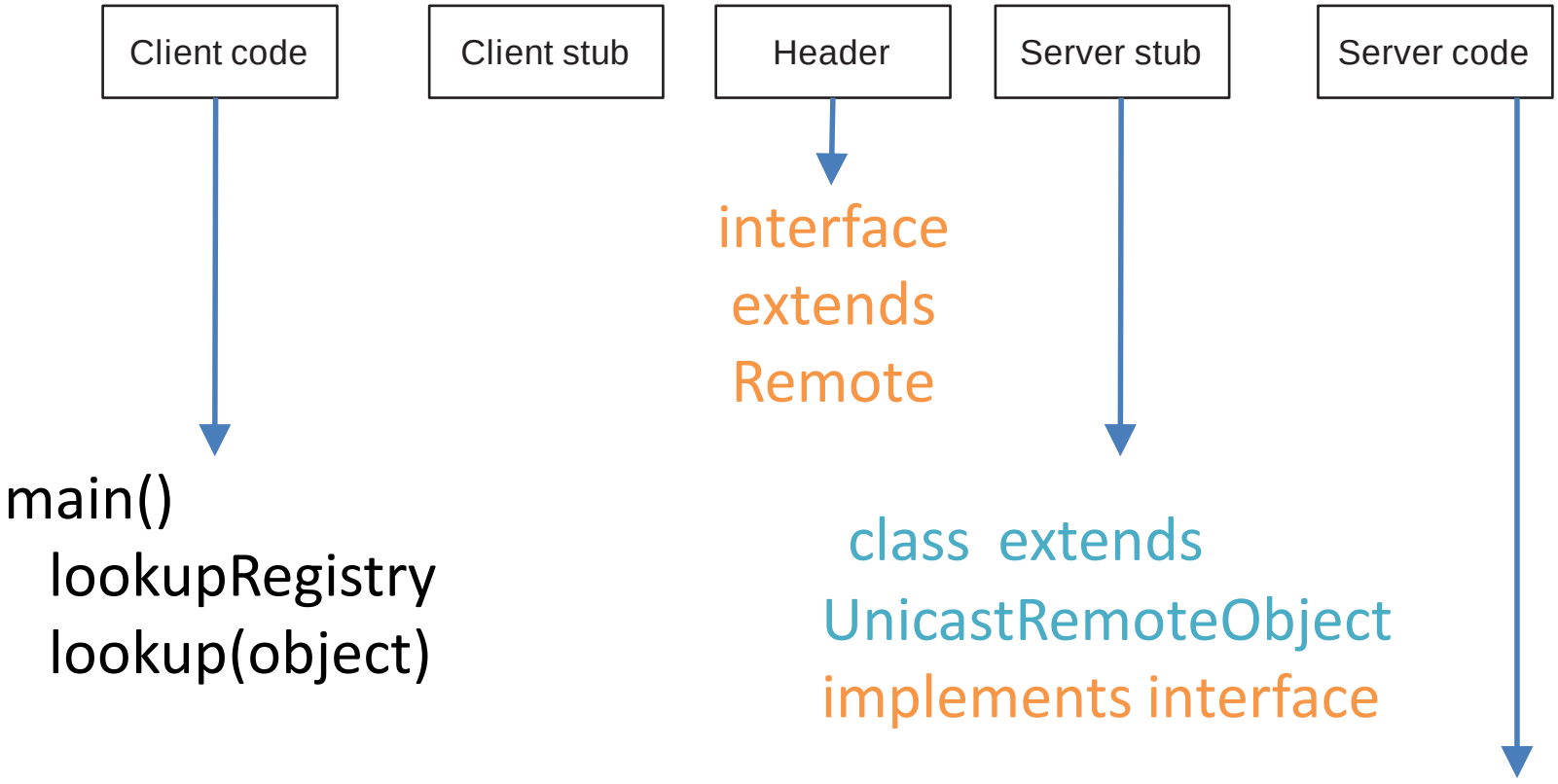
```
public class Servidor {

   public static void main(String[] args) {

      ServicoMatriz sm = new ServicoImpl();
      LocateRegistry.createRegistry(1099);

      Registry reg = LocateRegistry.getRegistry();
      reg.bind("rmi://127.0.0.1/servicomat", sm);
   }
}
```

RPC: Exemplo (RMI – Remote Method Invocation)



RPC: Exemplo (RMI – Remote Method Invocation)

```
public class Cliente {

    public static void main(String[] args) {
        Registry reg = LocateRegistry.getRegistry();
        ServicoMatriz sm = (ServicoMatriz)
        reg.lookup("rmi://127.0.0.1/servicomat");

        int [][] mat = {};
        int[][] inv = sm.inverte(mat);
    }
}
```

RPC: Exemplo (RMI – Remote Method Invocation)

Uma breve introdução da comunicação via RMI pode ser assistida no vídeo do link https://youtu.be/SnJ95lTISRQ

A implementação do exemplo citado no vídeo acima pode ser assistido no link https://youtu.be/knCXDo_NhGA

RPC: Exemplo (gRPC – Google RPC)

Google Remote Procedure Call

Desenvolvido pela Google em 2015.

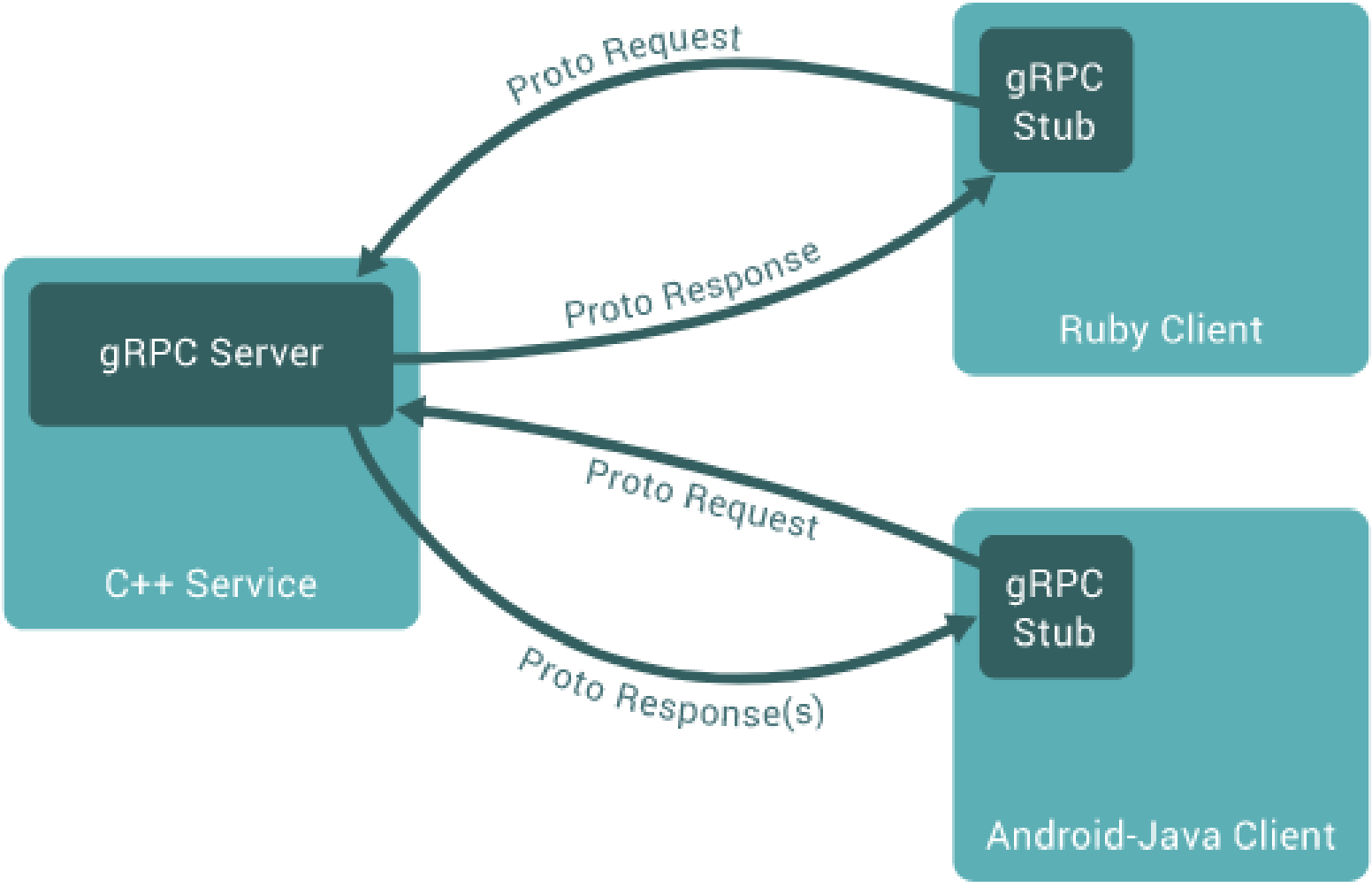
- Mensagens 3 a 10 dez vezes menores
- Enviadas 20 a 100 vezes mais rápido

Cenários comuns de uso:

- Quando precisar de baixa latência e alta escalabilidade.
- Clientes móveis comunicando-se com a nuvem
- Permitir facil extensão para outros componentes (auth, balanceamento, etc).

Usado por empresas tais como: Uber, Netflix, IBM, Dropbox, Cisco ...

RPC: Exemplo (gRPC – Google RPC)

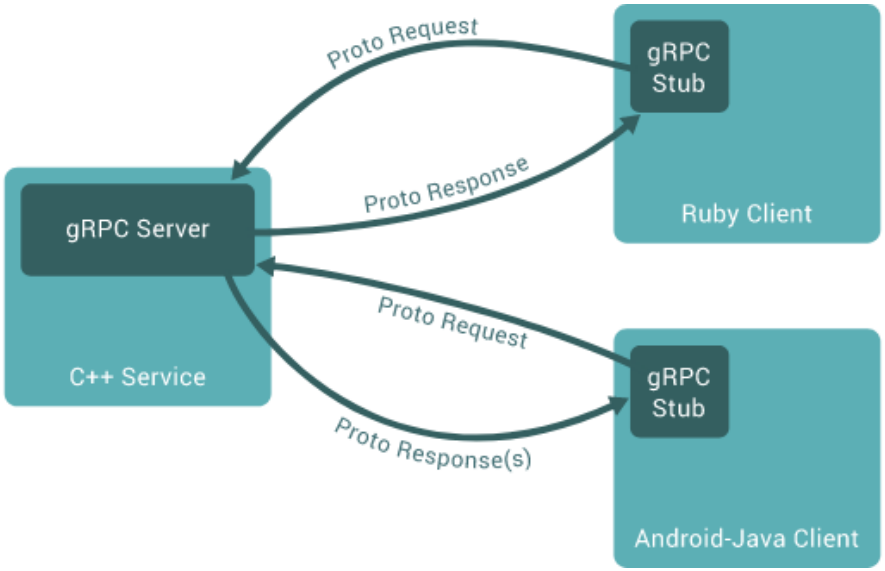


RPC: Exemplo (gRPC – Google RPC)

```
// The greeter service definition.
service Greeter {
    // Sends a greeting
    rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest { string name = 1; }

// The response message containing the greetings
message HelloReply { string message = 1; }
```

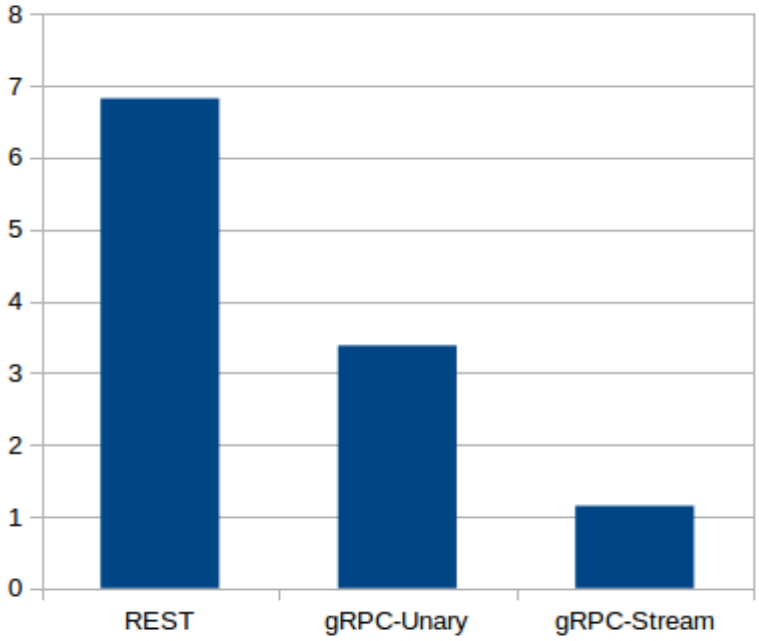


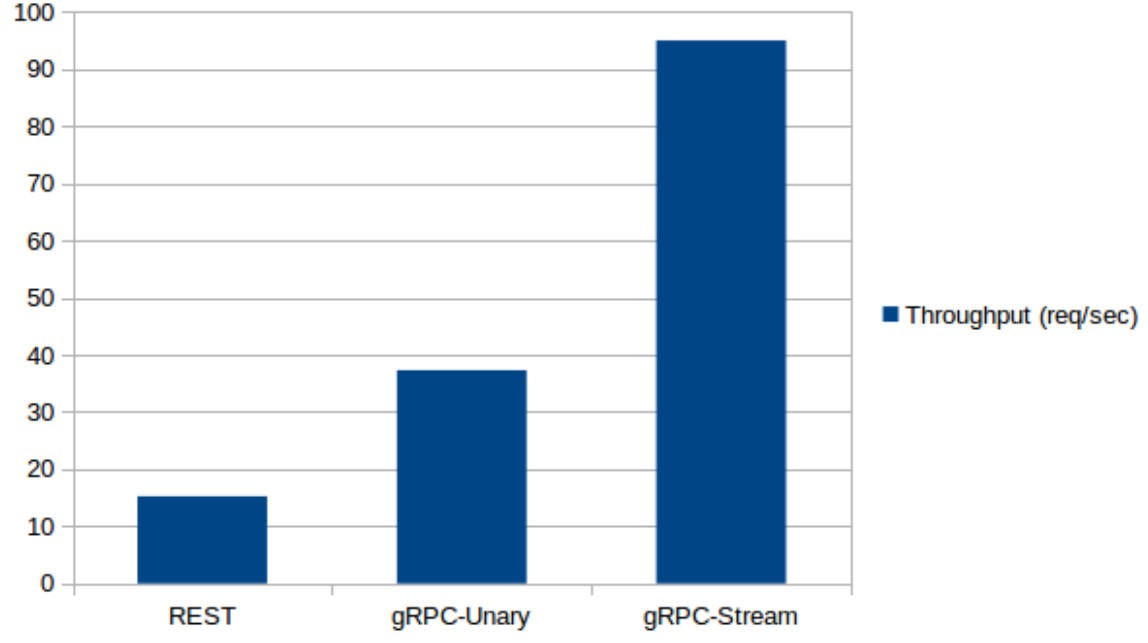
gRPC vs REST

FEATURES	GRPC	REST
Protocol	HTTP/2 (fast)	HTTP/1.1 (slow)
Payload	Protobuf (binary, small)	JSON (text, large)
API contract	Strict, required	Loose, optional (OpenAPI)
Code Generation	Built-in (Protac)	Third-party tools (Swagger)
Security	TLS/SSL	TLS/SSL
Streaming	Bidirectional Streaming	Client-> Server request only
Browser support	Limited(require GPRC-web)	YES

https://medium.com/@LadyNoBug/grpc-v-s-rest-v-s-others-5d8b6eaa61df

gRPC vs REST





https://www.vinsguru.com/grpc-vs-rest-performance-comparison/

Agenda

- Fundamentos
- Comunicação orientada a procedimento (RPC)
- Comunicação orientada a mensagem (MOM) *
- Comunicação multicast (Gossip / Flooding)
- Comunicação orientada a fluxo











* Usados como: event-streaming platform

Comunicação orientada a mensagens

- Middleware orientado a mensagens (filas)
- Message broker
- Exemplo: Apache Kafka

Middleware orientado a mensagens

Ideia geral

Comunicação assíncrona e persistente graças ao uso de filas gerenciadas pelo middleware. Filas correspondem a buffers em servidores de comunicação.

PUT	Adiciona uma mensagem à fila especificada
GET	Bloqueia até que a fila especificada tenha alguma mensagem e remove a primeira mensagem
POLL	Verifica se a fila especificada tem alguma mensagem. Nunca bloqueia
NOTIFY	Instala um tratador para ser chamado sempre que uma mensagem for inserida em uma dada fila

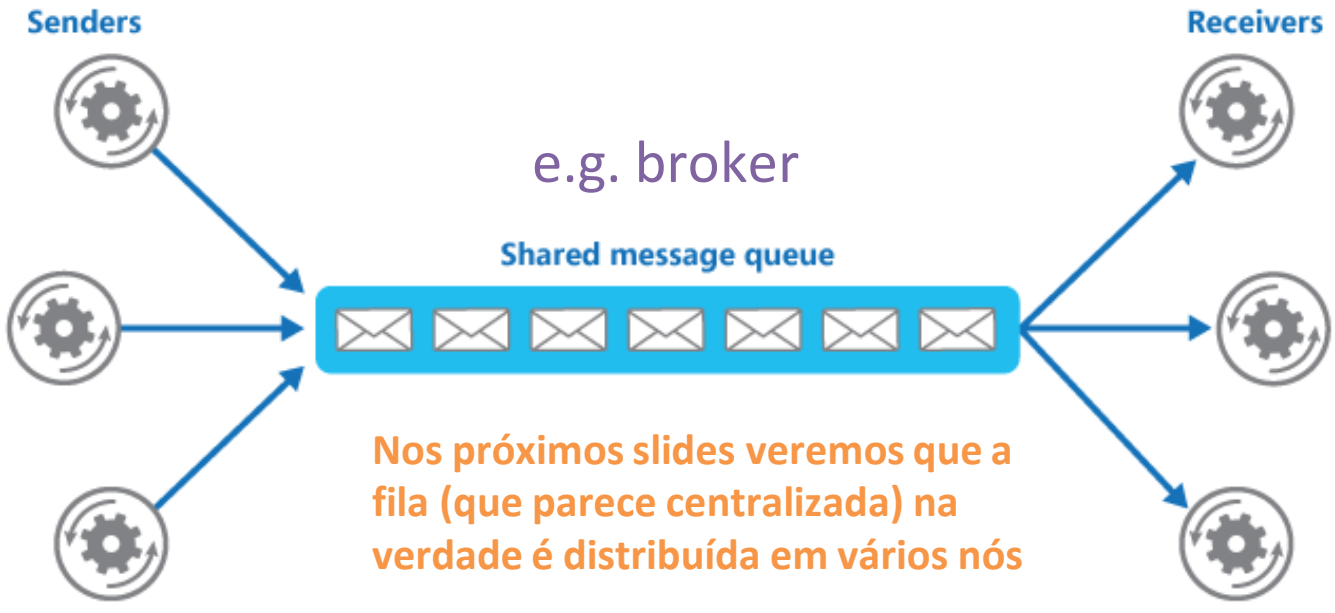
Message broker

Componente centralizado que lida com a heterogeneidade das aplicações:

- transforma as mensagens recebidas para o formato apropriado
- frequentemente funciona como um gerenciador e/ou armazenador da fila

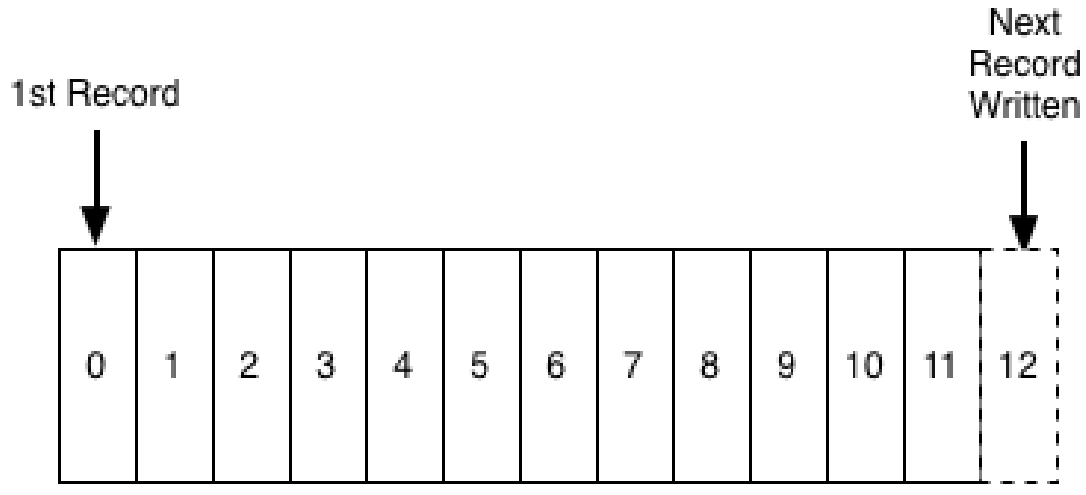
Message broker

- Mensagens específicas da aplicação são colocadas e removidas de filas
- As filas são controladas por um gerenciador de filas (e.g., um broker)
- Processos podem inserir mensagens em filas locais ou remotas, usando um mecanismo de RPC



Apache Kafka

- Exemplo de MOM, desenvolvido pelo LinkedIn [2012]
- Uma das plataformas de mensagerias open source mais usadas
- Utiliza o conceito de *event log* (i.e., estrutura append-only) onde cada registro possui um identificador único que captura a ordem dos eventos.



https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying

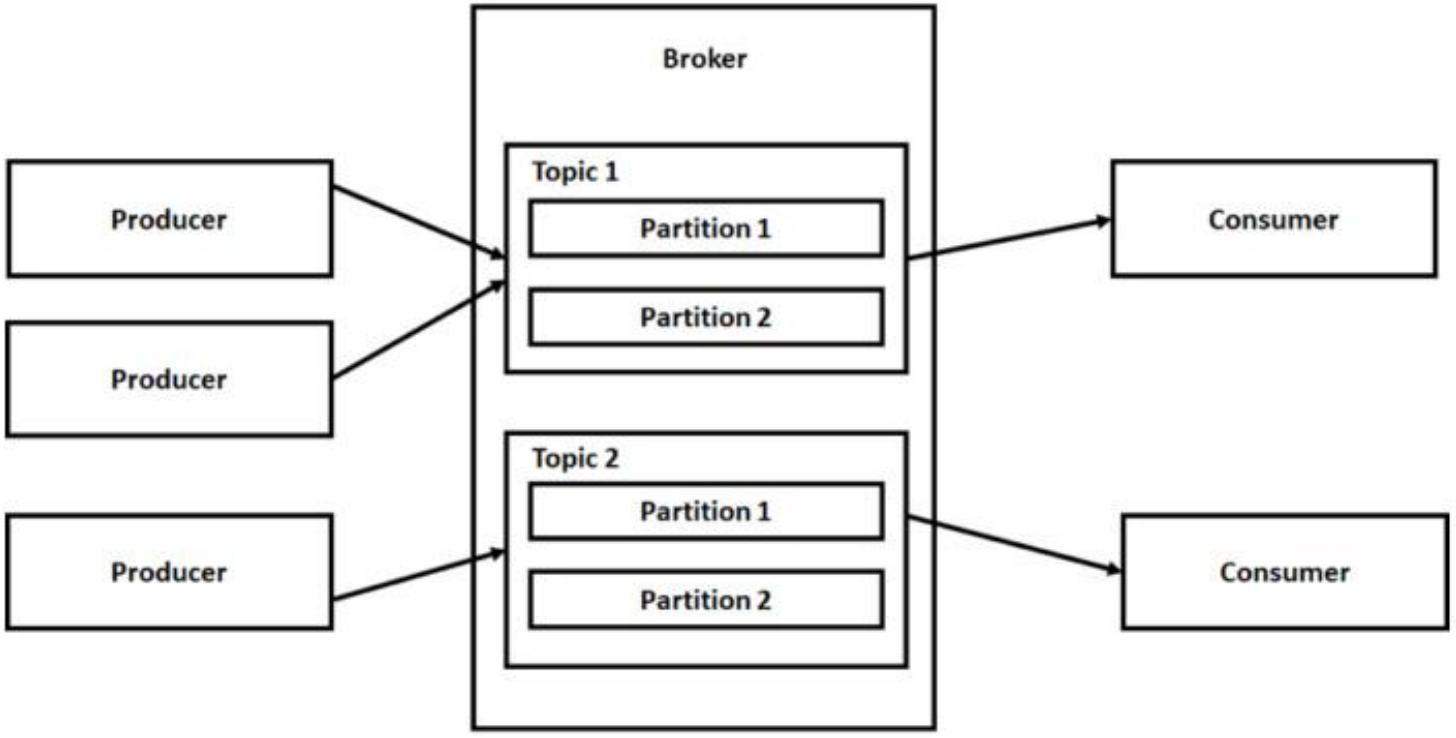
Apache Kafka

- Os logs (i.e., filas) são administrados/persistidos pelo Kafka broker

  O **kafka broker é stateless para requisições de clientes**
- O produtor envia os eventos ao broker de forma assíncrona
- Os eventos são armazenados de forma persistente no log.
- Podem ser lidos por vários consumidores.

  Cada consumidor especifica o log e o índice (offset).

No Kafka, o log é denominado de *Topic*



Apache Kafka

Producer.java

```
Producer producer = new KafkaProducer(props);
producer.send(
    new ProducerRecord<>("topic1", "usr1", "msg1"),
     (event, ex) -> {
       if (ex == null) println("Produced event");
});
```

Consumer.java

```
Consumer consumer = new KafkaConsumer(props);
consumer.subscribe("topic1"));
while (true) {
    ConsumerRecords records = consumer.poll();
    for (ConsumerRecord rec : records) {
       println(rec.key() + rec.value() + rec.offset());
}
```

Apache Kafka

- Para a **escalabilidade**, kafka usa o particionamento (sharding) do tópico
  - Na criação você define a quantidade de partições
  - Permite a escalabilidade horizontal
  - Permite que produtores e consumidores paralelizem as reqs.

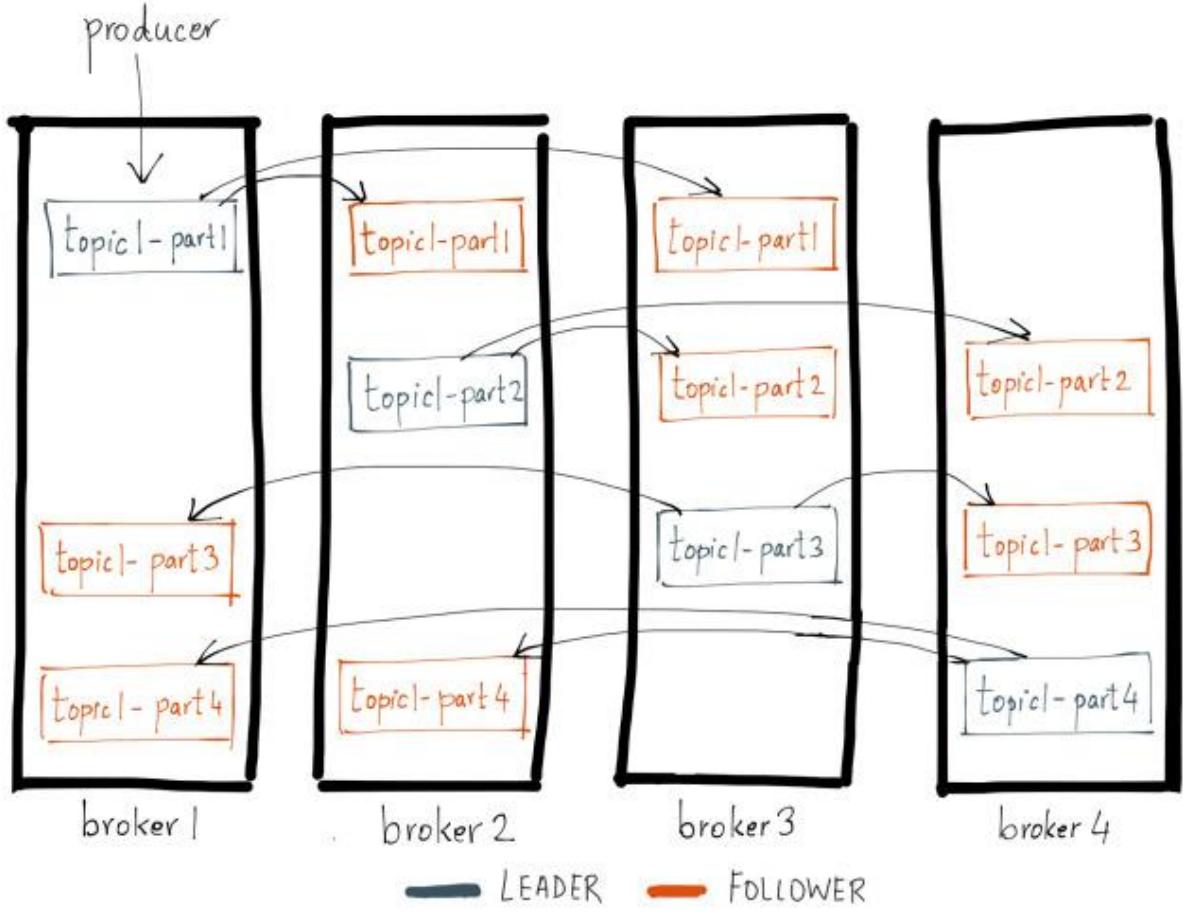
- Como o broker é stateless, cabe ao produtor escolher a partição onde será registrado o evento.

  E.g., Mensagens-SD-DA1, Mensagens-SD-DA2, ...

- Normalmente usa-se o hash de alguma chave (como id) que direciona à partição.

Apache Kafka

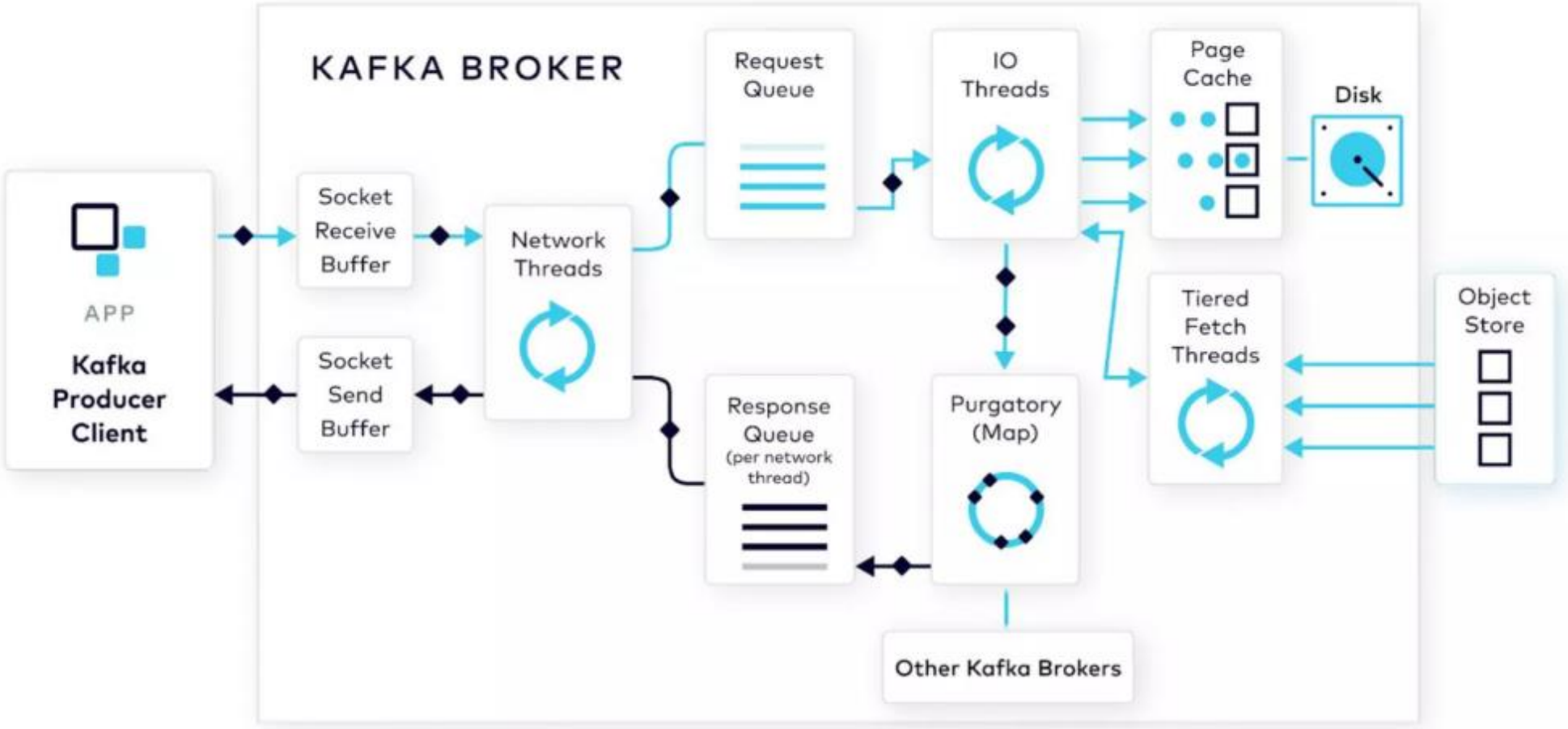
- Para a **disponibilidade**, kafka usa a replicação do tópico
  - Um líder, responsável por atender leituras e escritas.
  - N seguidores, responsáveis por resiliência a falhas do líder. (não aceitam leituras e/ou escritas)



https://www.confluent.io/blog/hands-free-kafka-replication-a-lesson-in-operational-simplicity/

Apache Kafka

- Jornada do Produtor (2023)



Apache Kafka

slack engineering

- Slack (discord-like)
  - Captura eventos de clientes Web que demandarão muito tempo para serem processados.
  - Em 2018, processou mais de 1 bilhão de mensagens por dia usando 16 brokers na AWS

    Instância i3.2xlarge: 8 vCPU, 61 GB RAM.
    Partições por tópico 32.
    Fator de replicação 3.
- Paypal
  - Em 2022 (na black Friday) processou 1.3 trilhões de mensagens por dia usando 85+ clusters totalizando 1500 brokers.

    20.000 tópicos

Apache ActiveMQ



O ActiveMQ é um dos mais populares provedores open source (licence Apache 2.0), utilizado para integrar diferentes linguagens, tais como: Java, .NET, C/C++/C#, Delphi, Perl, Python, entre outras, através do chamado "Cross Language Clients".

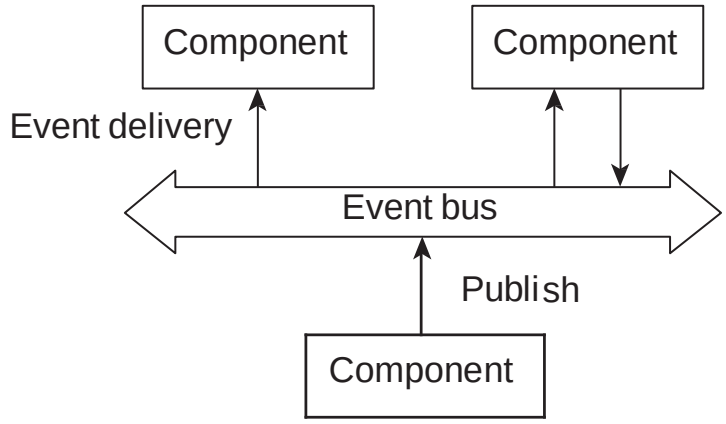
Desde 2004 até hoje

http://activemq.apache.org/

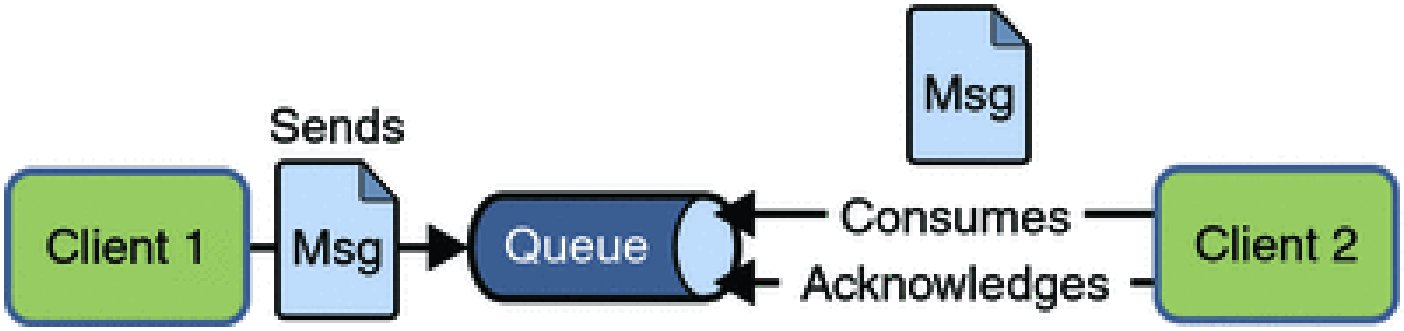
Apache ActiveMQ: modelos

- Modelo ponto a ponto
- Modelo publish/subscribe

Apache ActiveMQ: modelos



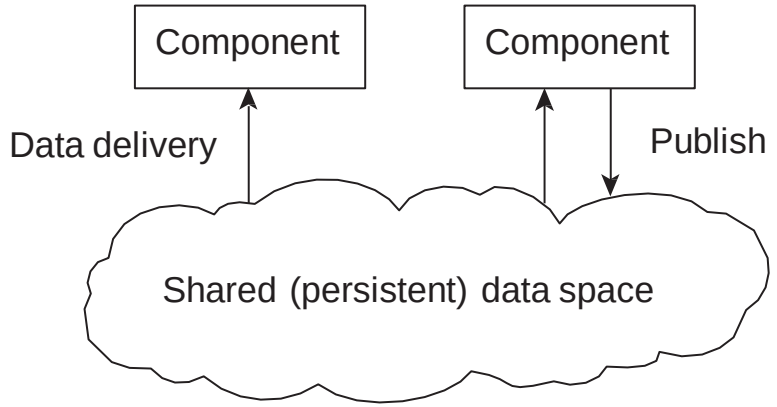
- Modelo ponto a ponto (queue)

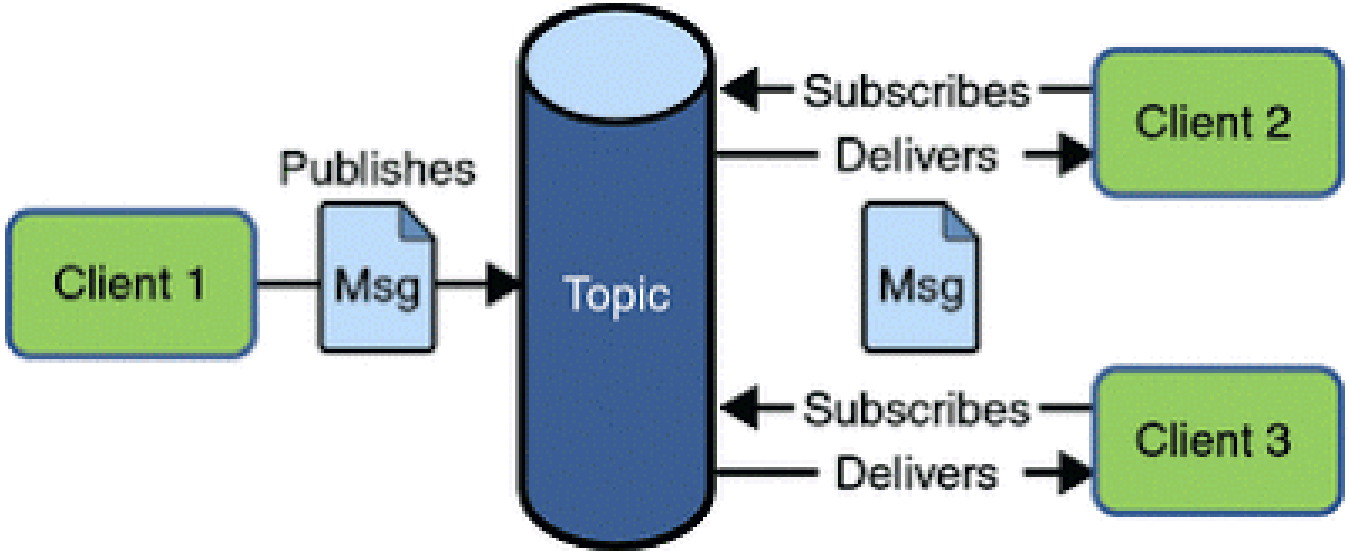


1. O produtor envia uma mensagem para a queue
2. Somente um dos consumidores (que estão escutando) recebe a mensagem

Apache ActiveMQ: modelos

- Espaço de dados compartilhados





1. O produtor envia uma mensagem para o servidor de tópicos
2. Todos os consumidores (que estão escutando) recebem a msg

Vantagens e Desvantagens

- Dentro da Aplicação

  Fácil de usar – Java ExecutorService

  Normalmente armazenamento em memória

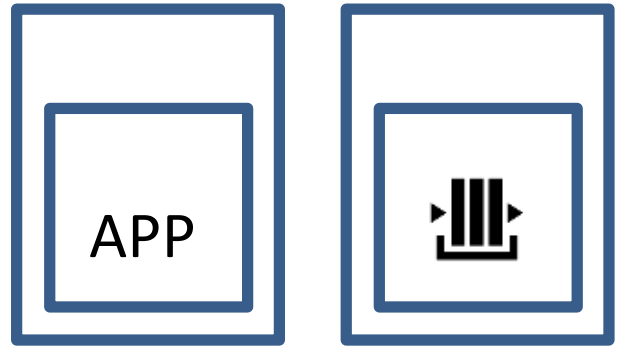
  (perda de mensagens/tarefas se cair a app)



- Fora da Aplicação (separada)

  Armazenamento em disco – Kafka

  Escalabilidade limitada à máquina.

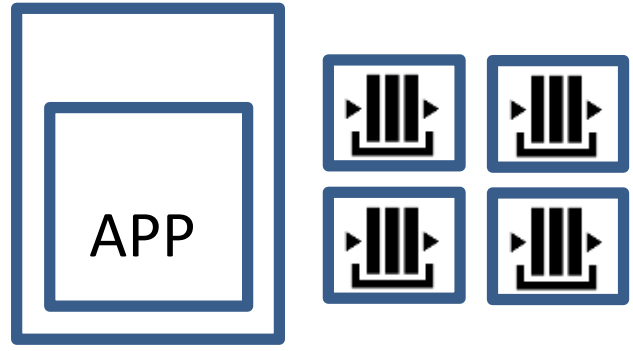


- Fora da Aplicação (distribuída)

  Armazenamento em disco, escalabilidade

  muito grande – Kafka/RabbitMQ/ApacheMQ

  Complexo de administrar, inconsistências



Agenda

- Fundamentos
- Comunicação orientada a procedimento (RPC)
- Comunicação orientada a mensagem (MOM)
- Comunicação multicast
  - FIFO/Causal/Total e sem ordem.
  - Gossip/Flooding
- Comunicação orientada a fluxo

Multicast

- Diferente do unicast (e.g., TCP) no multicast o emissor - sender - envia a mesma mensagem para um grupo de destinos - receivers.
- O multicast pode ser realizado pela camada de rede (difícil de ser implementado) ou pelo middleware.
- Pelo middleware, caberá a este verificar que as mensagens sejam entregues aos destinos

Agenda

- Fundamentos
- Comunicação orientada a procedimento (RPC)
- Comunicação orientada a mensagem (MOM)
- Comunicação multicast
  - FIFO/Causal/Total e sem ordem
  - Gossip/Flooding
- Comunicação orientada a fluxo

Multicast ordem FIFO

Se um processo *p* envia mensagens *m1* e depois *m2*, todos os processos entregarão primeiro *m1* antes de *m2*.

No middleware

Receber: não há controle sobre o que se recebe pela camada de rede.
Entregar: após receber a mensagem, você controla o que fazer com ela
(e.g., entregar para o banco de dados fazer o update)

Entregar ou repassar, ambos termos são usados

Multicast ordem FIFO

Se um processo *p* envia mensagens *m1* e depois *m2*, todos os processos entregarão primeiro *m1* antes de *m2*.

tempo	P1	P2
1	Envia/Entrega m1	Entrega m1
2	Envia/Entrega m2	Entrega m2

tempo	P1	P2	P3
1	Envia/Entrega m1	Entrega m1	
2	Envia/Entrega m2	Entrega m2	Entrega m2
3			Entrega m1

tempo	P1	P2	P3	P4
1	Envia/Entrega m1	Entrega m1	Entrega m3	Envia/Entrega m3
2	Envia/Entrega m2	Entrega m3	Entrega m1	Envia/Entrega m4
3	Entrega m3	Entrega m2	Entrega m2	Entrega m1
4	Entrega m4	Entrega m4	Entrega m4	Entrega m2

Multicast ordem CAUSAL

Se uma mensagem *m1* precede causalmente *m2*, todos os processos entregarão primeiro *m1* antes de *m2*.

No capítulo 6 será definido formalmente o termo causalidade.
Note que não depende do processo origem que enviou o multicast.

Multicast ordem CAUSAL

Detalhes no capítulo 6

Se uma mensagem *m1* precede causalmente *m2*, todos os processos entregarão primeiro *m1* antes de *m2*.

tempo	P1	P2	P3
1	Envia m1		
2		Entrega m1	
3	Entrega m1	Envia m2	Entrega m1
4	Entrega m2		
5		Entrega m2	Entrega m2

tempo	P1	P2	P3
1	Envia m1		
2		Entrega m1	
3	Entrega m1	Envia m2	Entrega m2
4	Entrega m2		
5		Entrega m2	Entrega m1

Multicast ordem TOTAL

Detalhes nos capítulos 6 e 8

Se um processo entrega *m1* antes de *m2*, todos os processos entregarão nessa mesma ordem.

tempo	P1	P2	P3	P4
1	Envia m1			Envia m2
2		Entrega m2	Entrega m2	
3	Entrega m2	Entrega m1	Entrega m1	Entrega m2
4	Entrega m1			Entrega m1

tempo	P1	P2	P3	P4
1	Envia m1			Envia m2
2		Entrega m2	Entrega m2	
3	Entrega m1	Entrega m1	Entrega m1	Entrega m2
4	Entrega m2			Entrega m1

Agenda

- Fundamentos
- Comunicação orientada a procedimento (RPC)
- Comunicação orientada a mensagem (MOM)
- Comunicação multicast
  - FIFO/Causal/Total
  - Gossip/Flooding
- Comunicação orientada a fluxo

Disseminação de dados via Gossip / Flooding

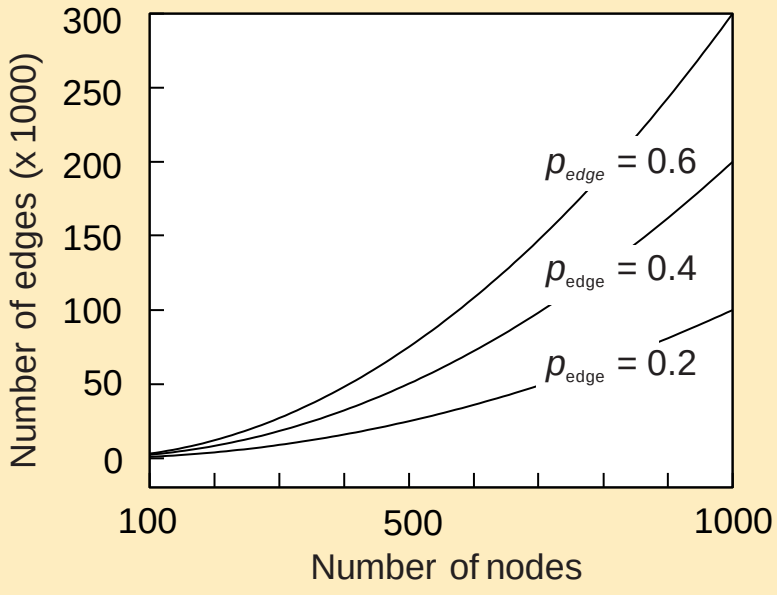
Ideia

Nó *P* envia uma mensagem *m* a seus vizinhos (via **gossip**). Cada vizinho a re-envia (via **flooding**) se não a viu antes.

Desempenho

Maior a quantidade de conexões, mais caro fica

Tamanho de uma overlay aleatória vs o nº de nós



Gossip: Protocolos Epidêmicos

Assuma não há conflito escrita-escrita

- Operações de atualização são realizadas em um só servidor
- Um nó passa informações (estado) a somente alguns nós
- Propagação de atualizações é lenta, i.e., não imediata
  Eventualmente, cada atualização atingirá cada nó

Duas formas de epidemia

Anti-entropy: Cada nó regularmente escolhe outro nó aleatoriamente e intercambia seu estado, levando a estados idênticos
Rumor: Um nó que foi atualizado (i.e., foi contaminado), avisa a outros nó sobre a atualização (contaminando outros).

O que é gossiping?

Node n

Nó, Thread ou Processo

```
n.run():
  while true:
    sleepMs(x)
    executeGossip()
```

Método main executado por Node *n*

Cada certo tempo acorda e faz algo (gossip)

```
n.executeGossip():
  p = selectNode()
  infoLocal = getLocalInfo()
  p.shareInfo(infoLocal, n)
```

Via RPC ao Nó *p*

```
n.shareInfo(info, q):
  addInfo(info)
  infoLocal = getLocalInfo()
  q.updateInfo(infoLocal)
```

Orientado a evento (muitos Nós podem chamá-lo ao mesmo tempo)

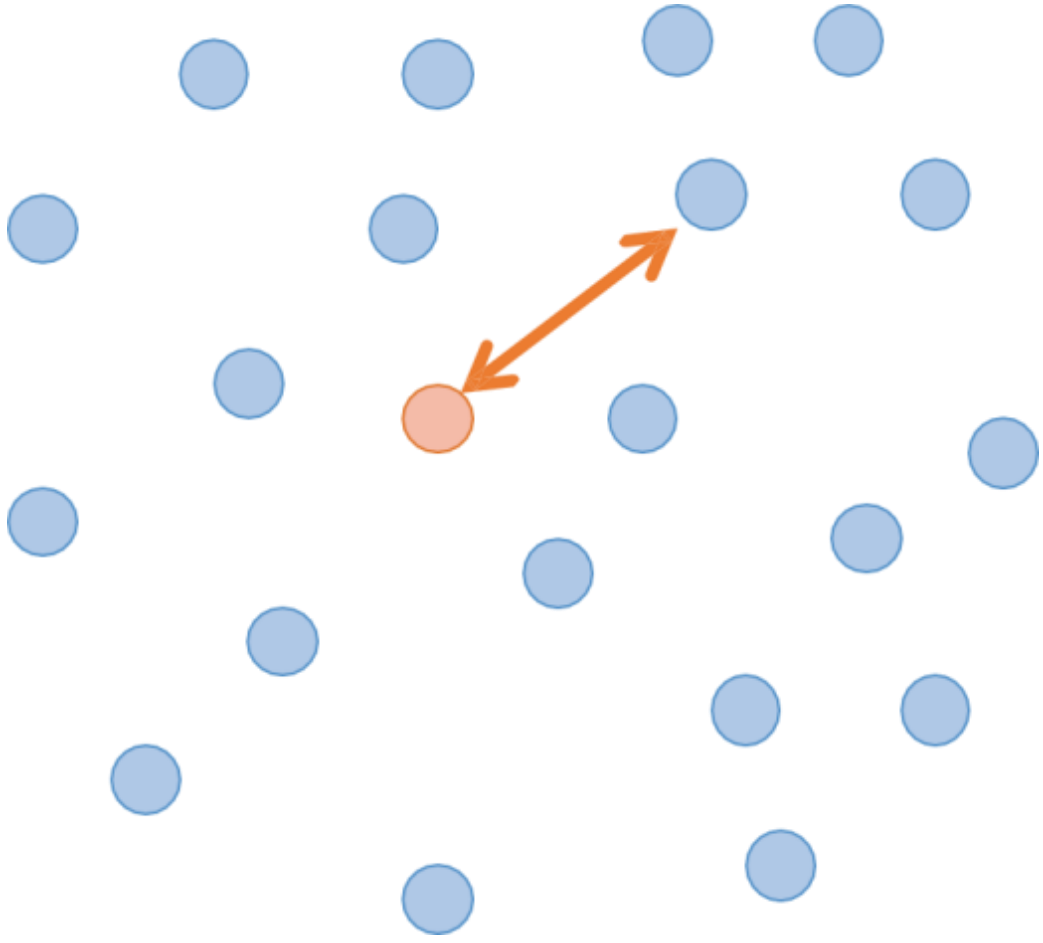
Via RPC ao Nó *q*

```
n.updateInfo(info):
  addInfo(info)
```

Orientado a evento (muitos Nós podem chamá-lo ao mesmo tempo)

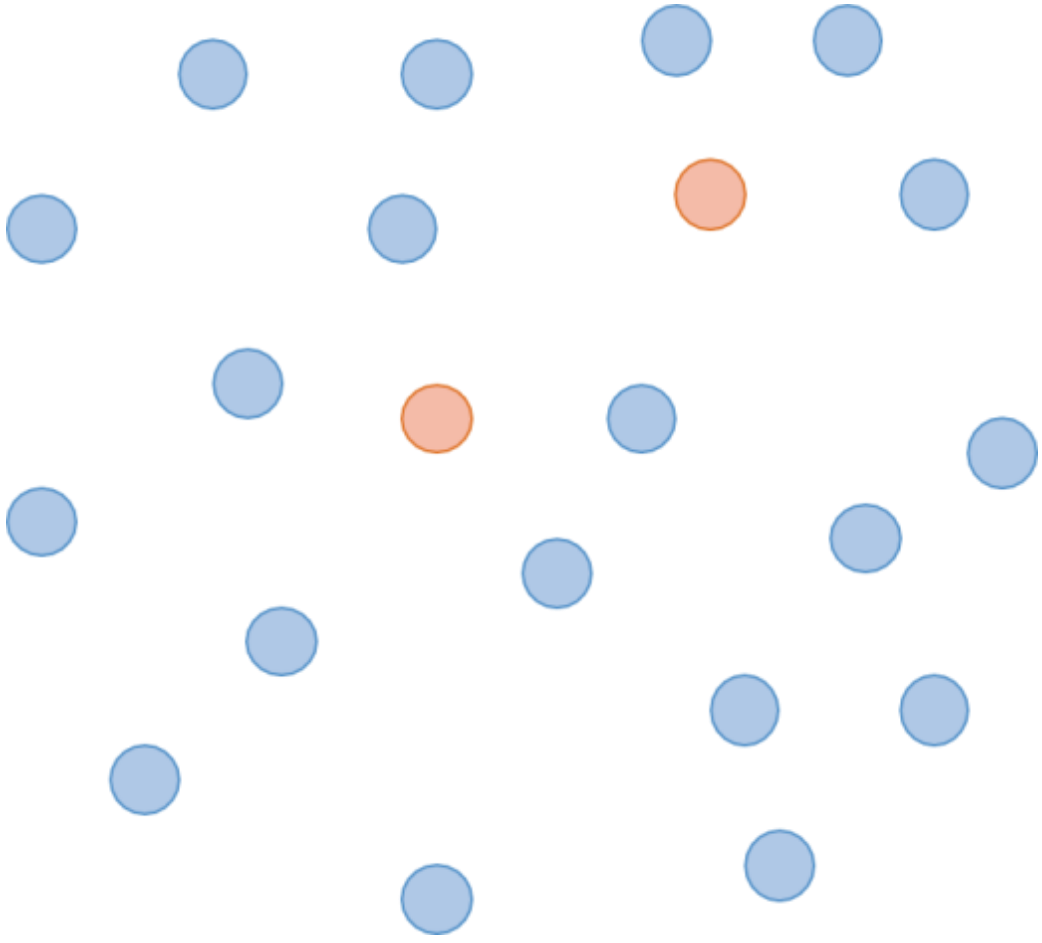
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



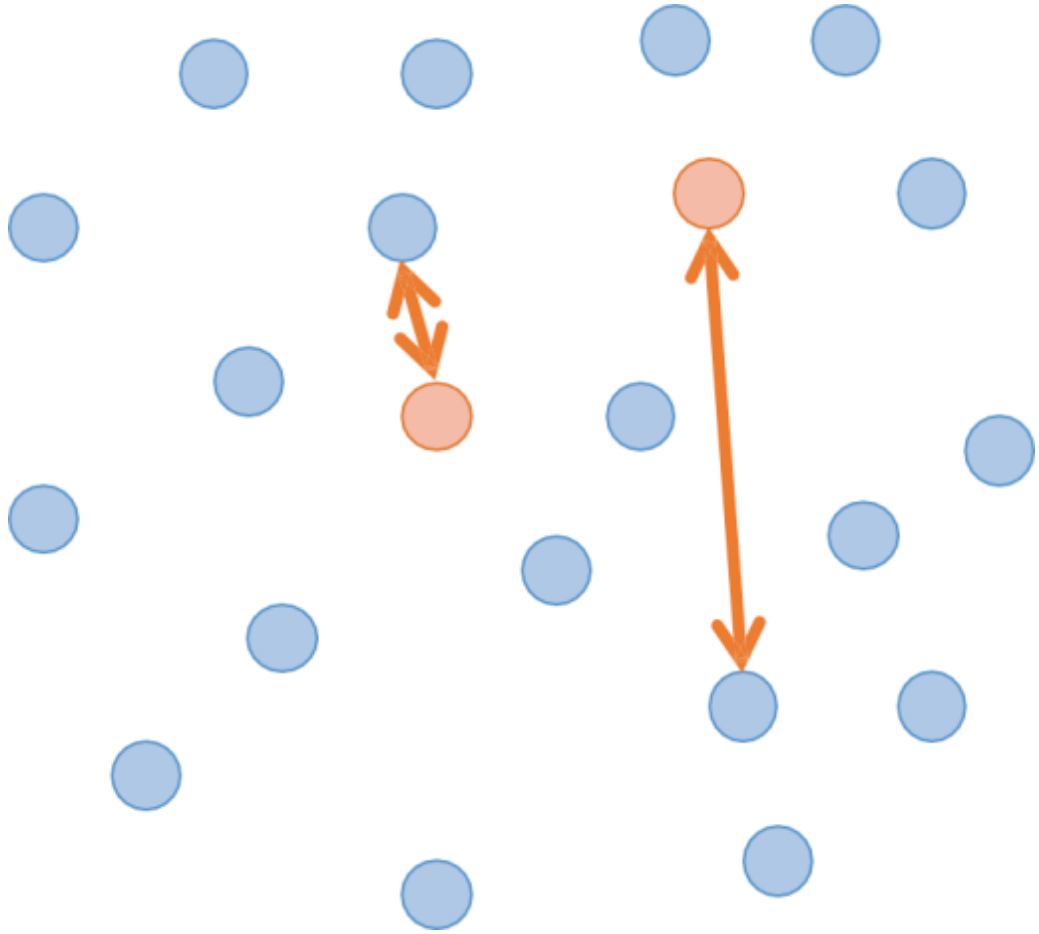
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



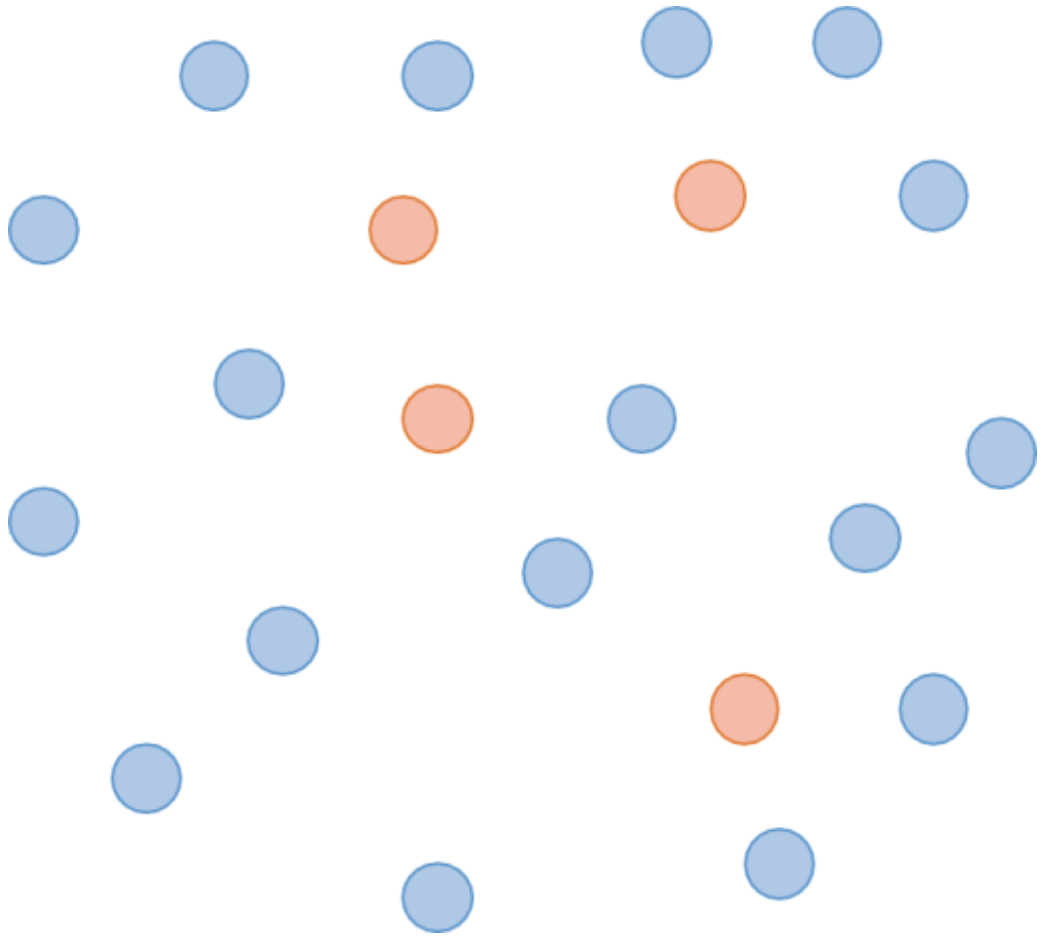
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



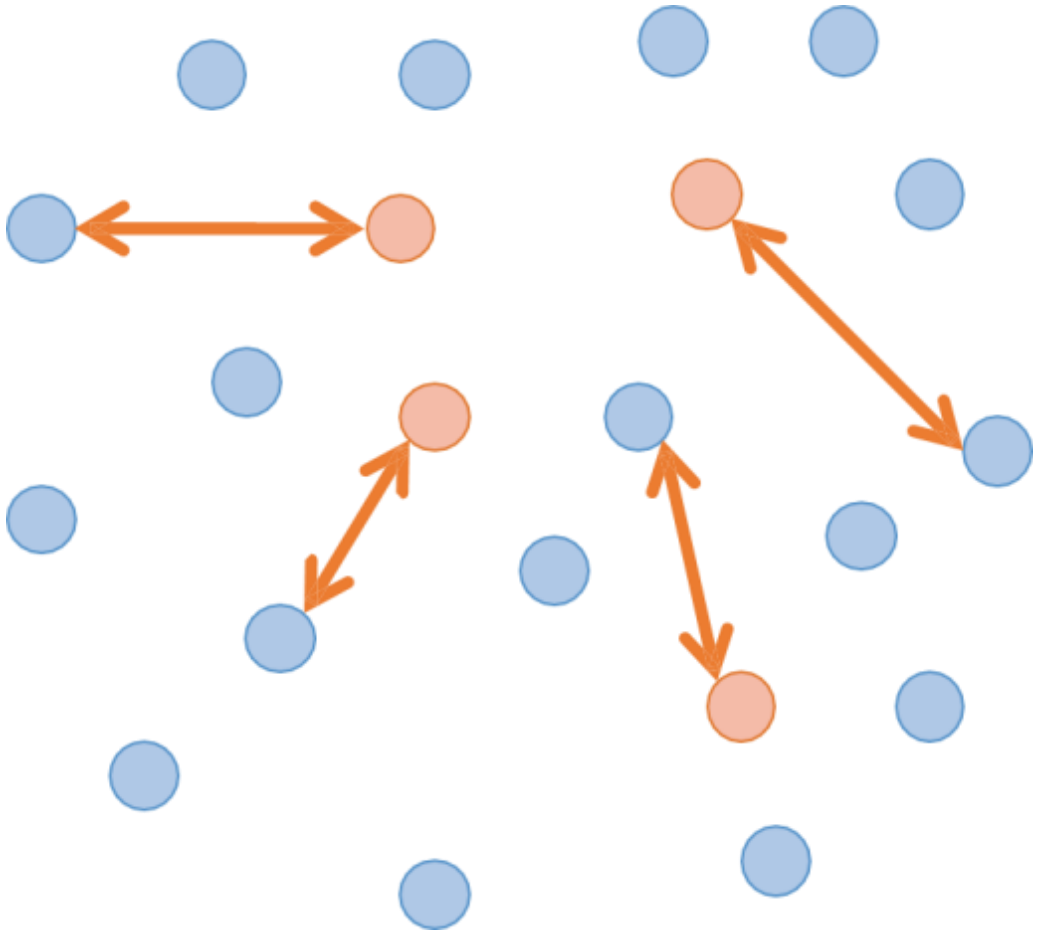
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



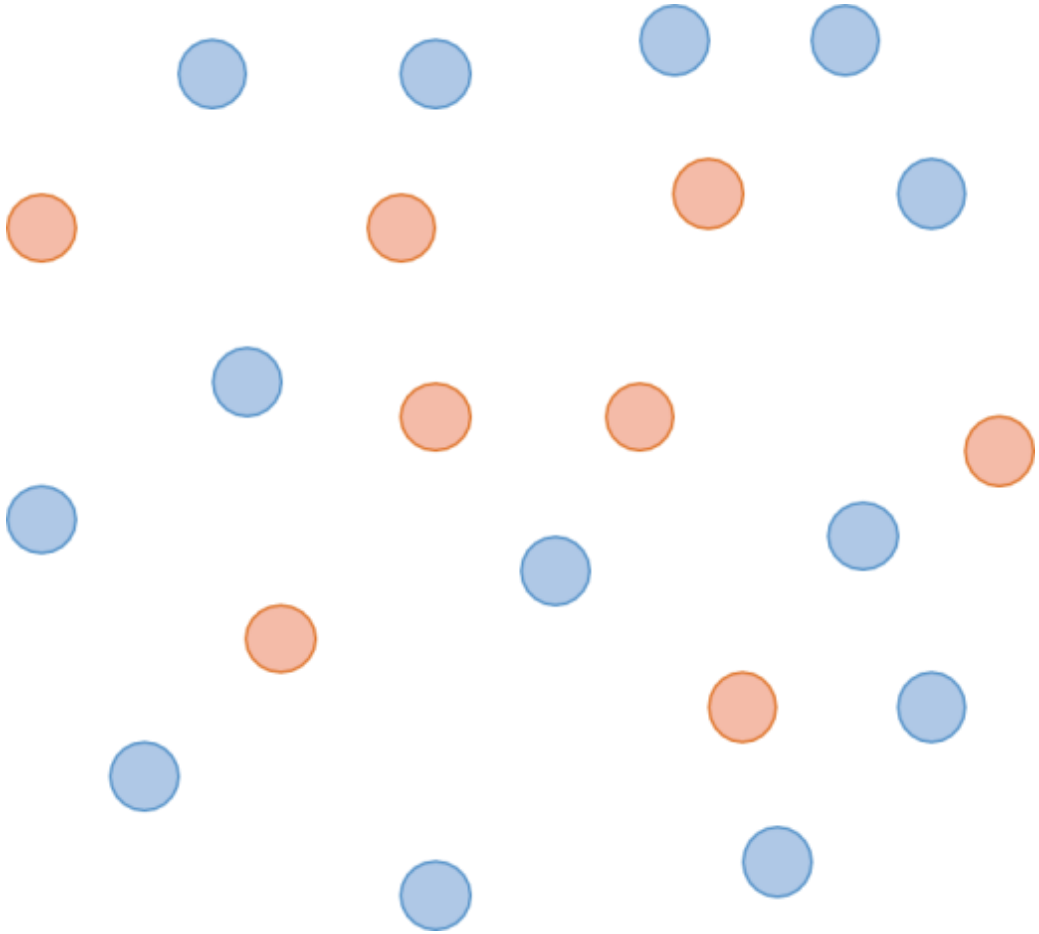
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



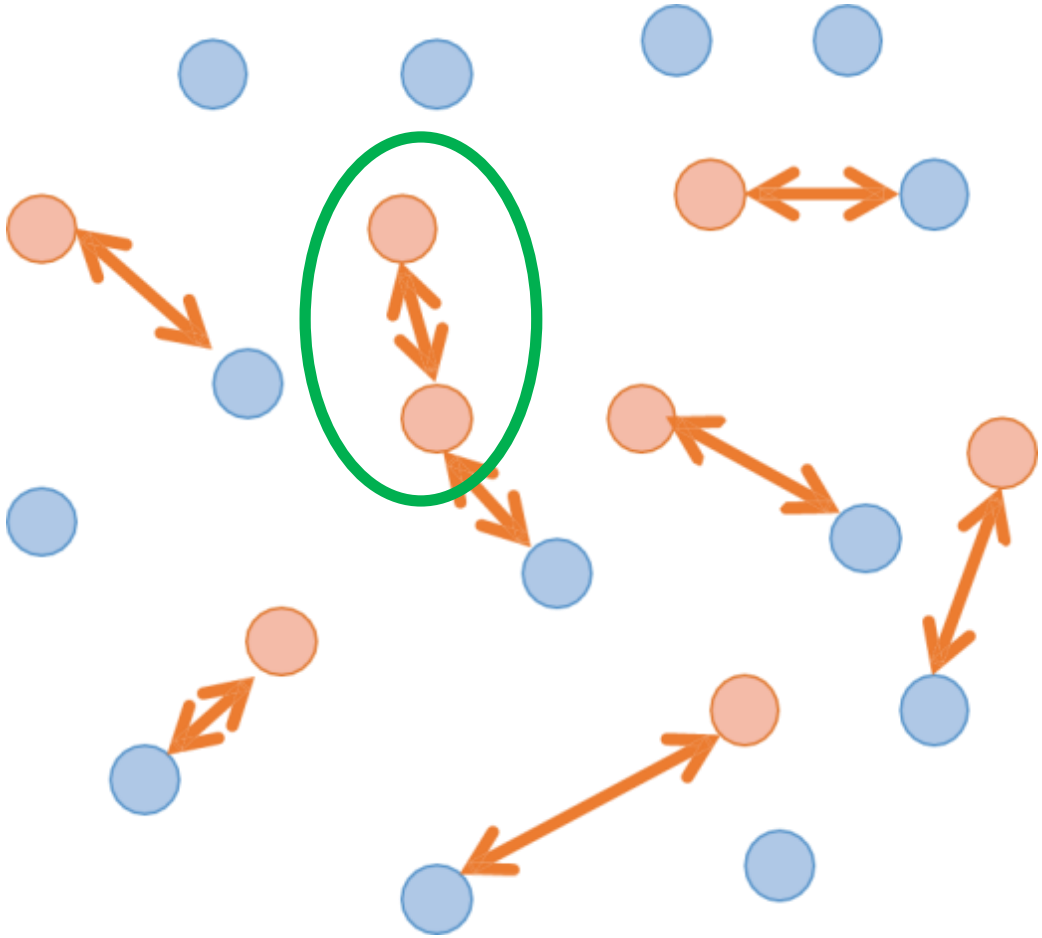
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



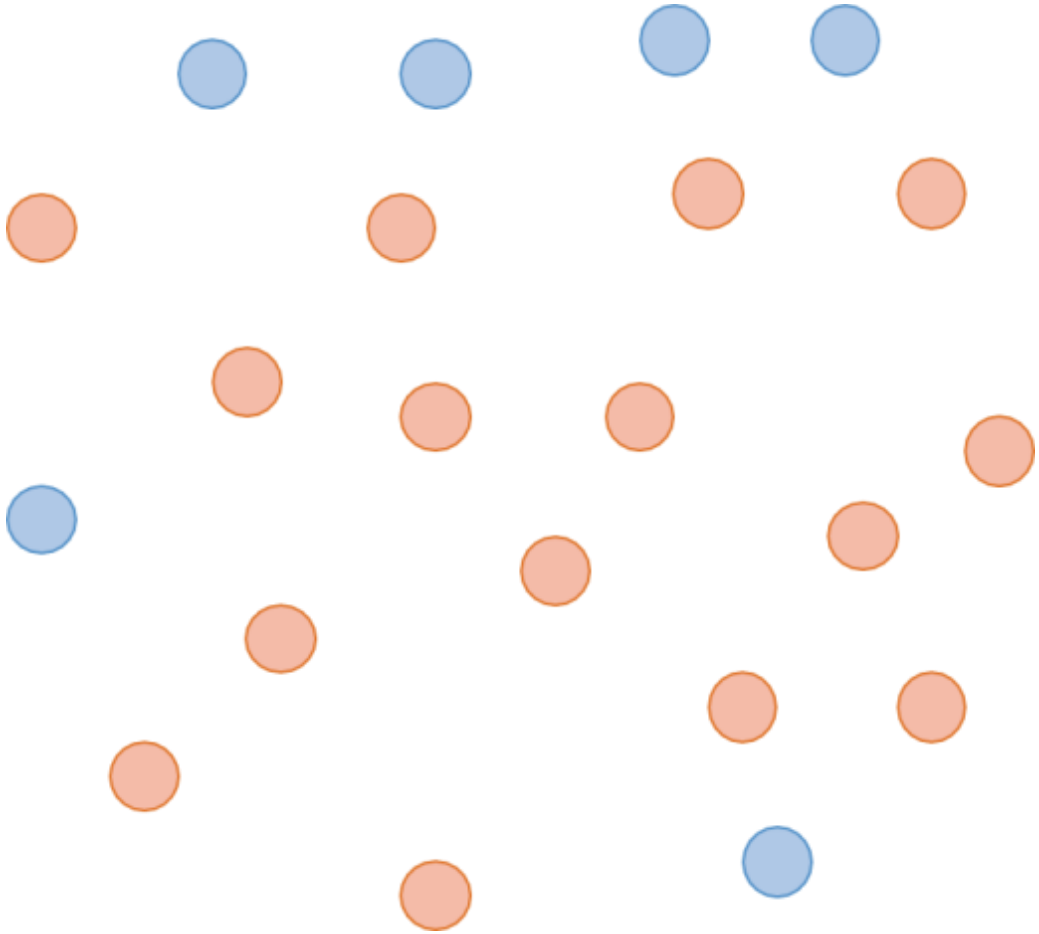
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



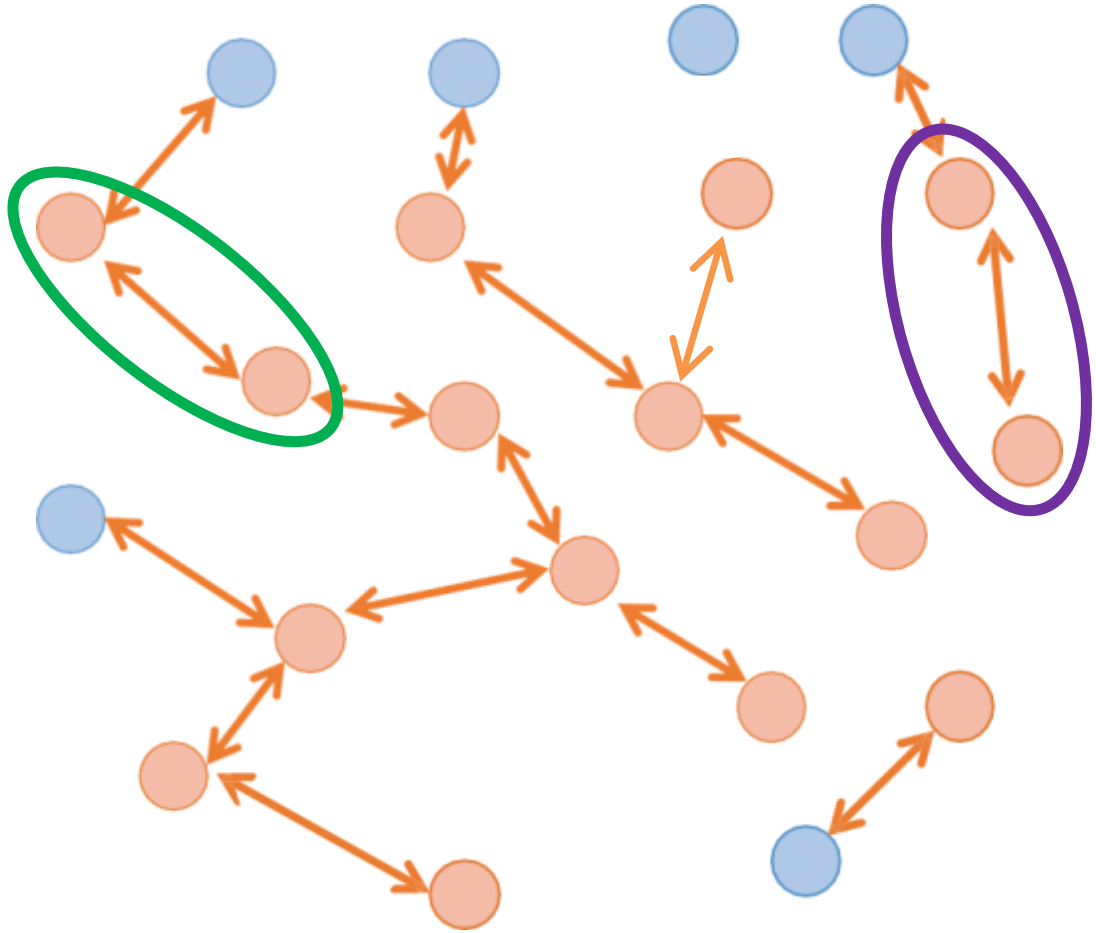
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



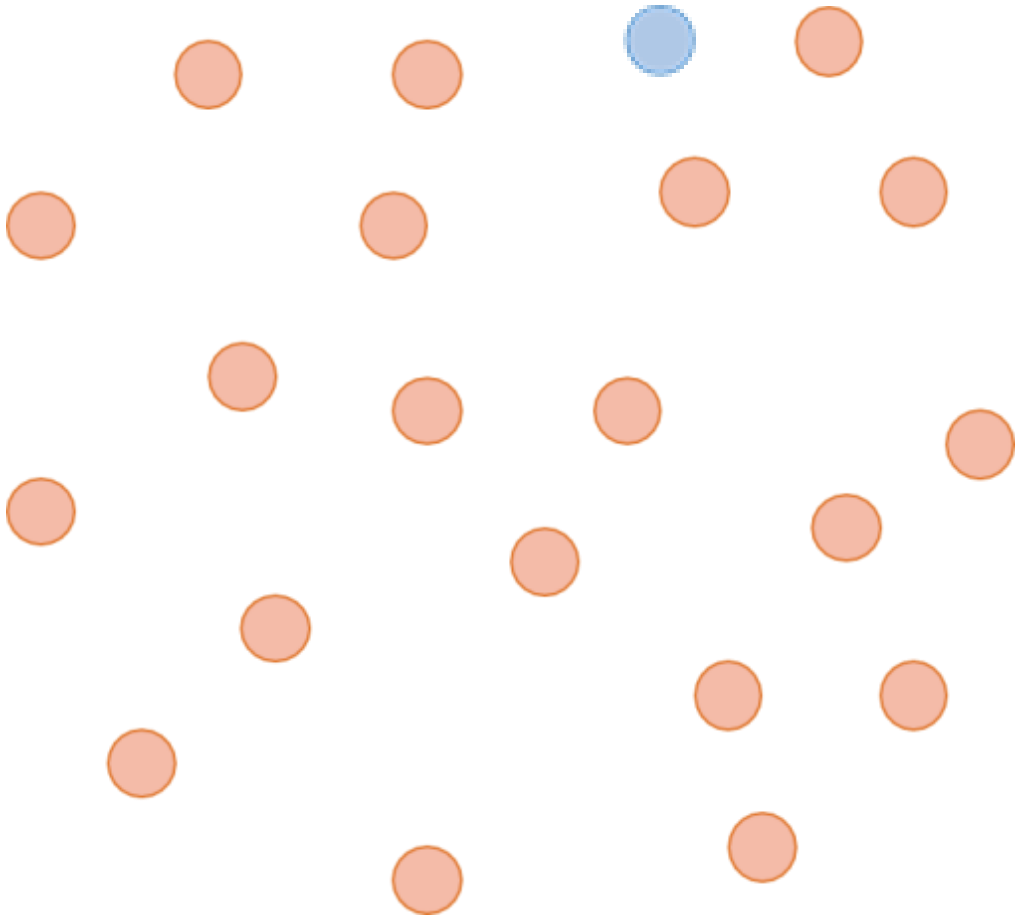
O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



O que é gossiping?

Exemplo de Eugene Bagdasaryan
http://www.cs.cornell.edu/courses/cs6410/2016fa/slides/19-p2p-gossip.pdf



Faltou um

Gossip: Protocolos Epidêmicos → Anti-entropy

Princípios

Um nó *P* seleciona outro nó *Q* do sistema aleatoriamente.

Pull (puxar): *P* somente pede novas atualizações de *Q.*

Push (empurrar): *P* somente envia novas atualizações a *Q,* sem Q ter pedido.

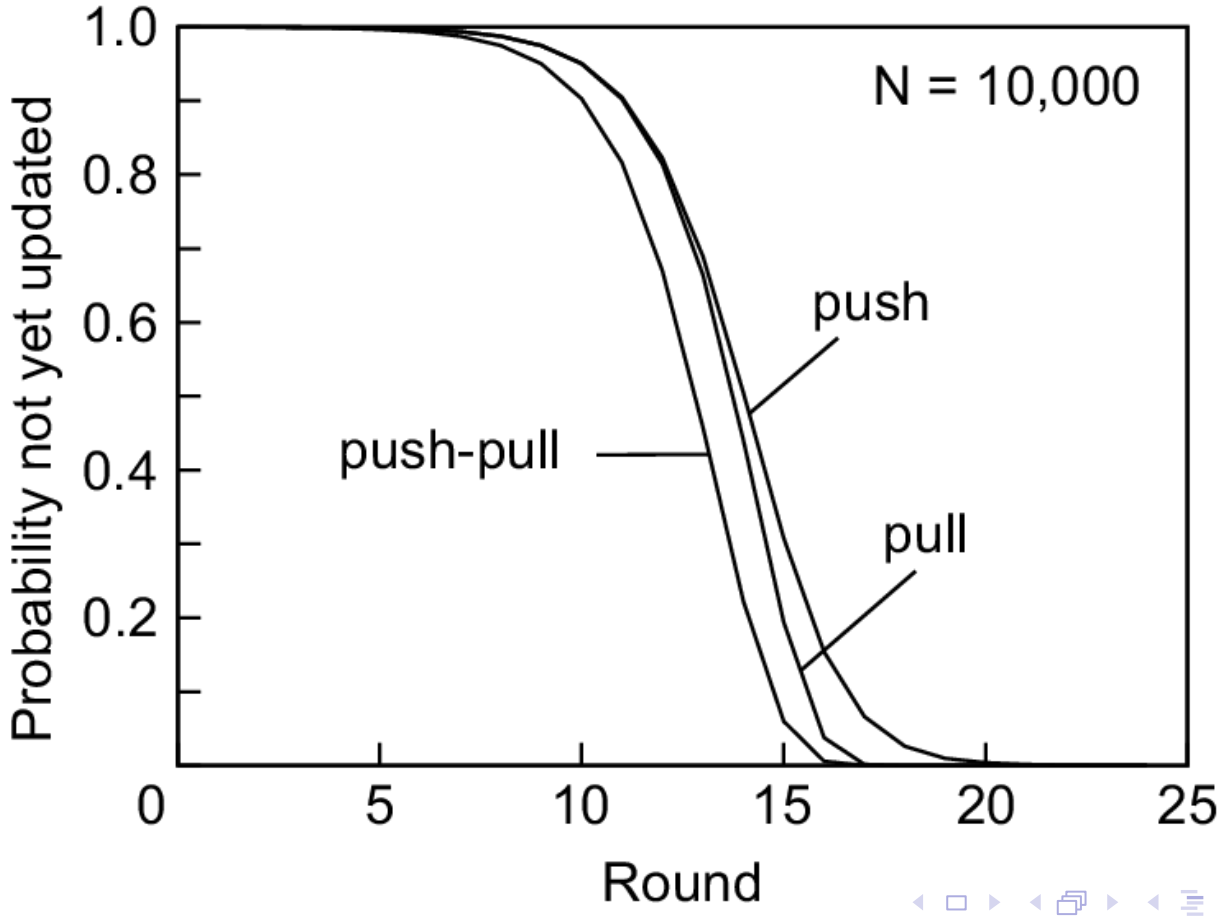
Push-pull: *P* e *Q* enviam atualizações um a outro.

Gossip: Protocolos Epidêmicos → Anti-entropy

Observação

push-pull usa $O(log(N))$ rodadas para disseminar as atualizações nos N nós (rodada = quando cada nó tomou a iniciativa de iniciar o intercâmbio).

Desempenho



Gossip: Protocolos Epidêmicos → Rumor (boato)

Modelo básico

Um nó *S*, com uma atualização a disseminar, contata outros nós. Se o nó contatado já possui a atualização, *S* para de contatar outros nós com probabilidade p_{stop}.

Observação

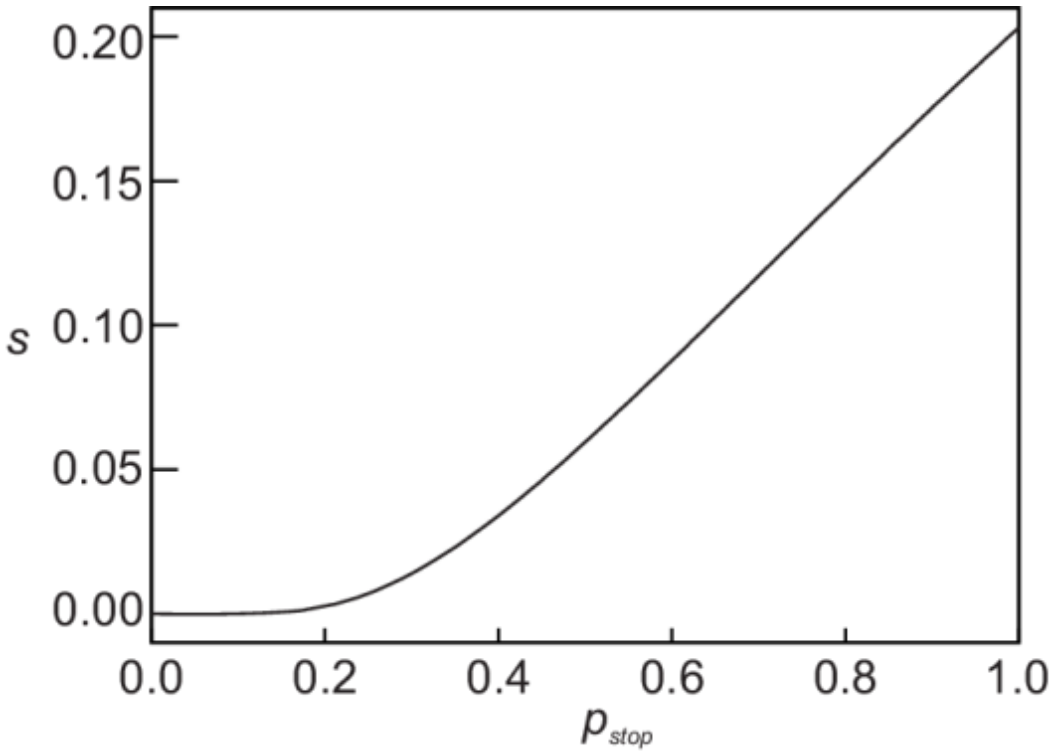
Se *s* é a fração de nós ignorantes (i.e., não tem a atualização), pode ser demonstrado (via SIR) que, para muitos servidores:

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

Gossip: Protocolos Epidêmicos → Rumor (boato)

O efeito de parar

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

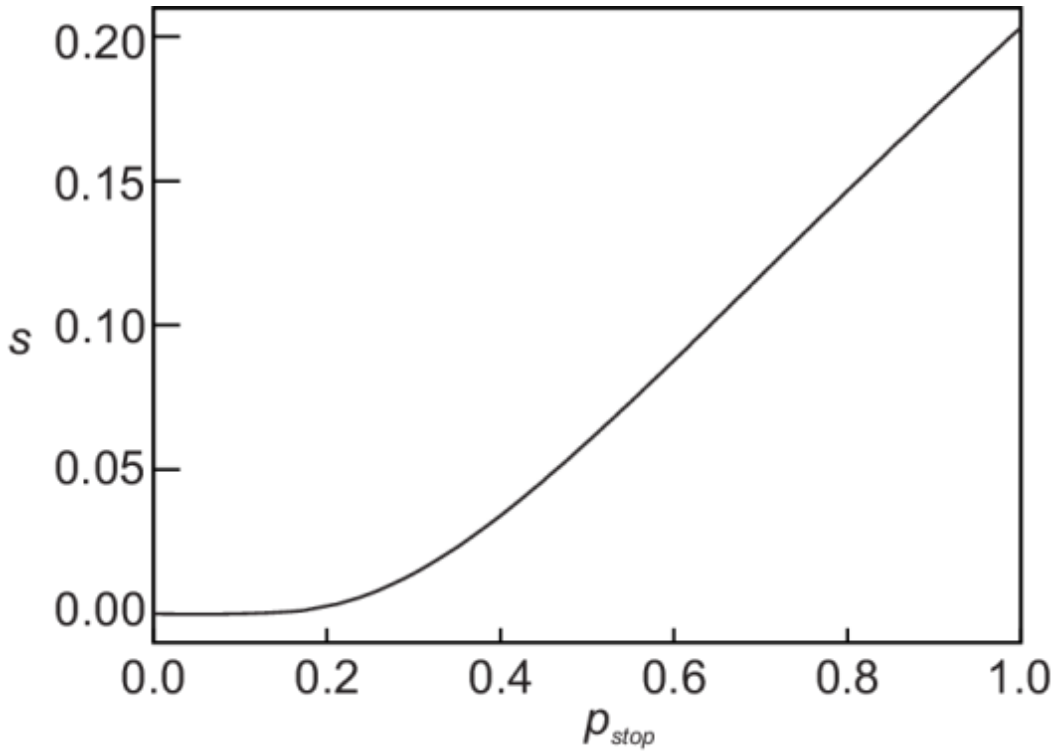


Considere 10,000 nós		
$1/p_{stop}$	s	N_s
1	0.203188	2032
2	0.059520	595
3	0.019827	198
4	0.006977	70
5	0.002516	25
6	0.000918	9
7	0.000336	3

Gossip: Protocolos Epidêmicos → Rumor (boato)

O efeito de parar

$$s = e^{-(1/p_{stop}+1)(1-s)}$$

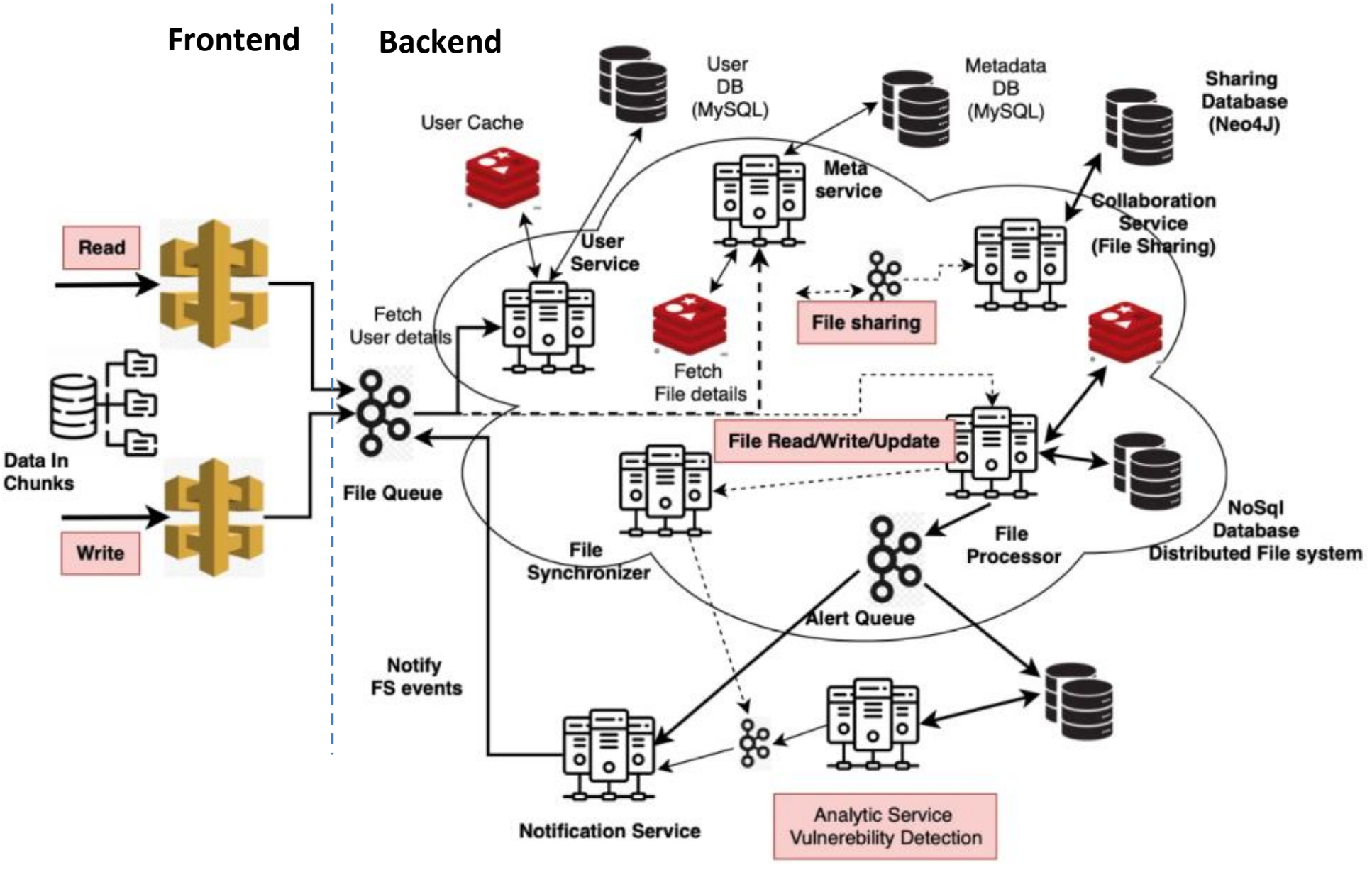


Considere 10,000 nós		
$1/p_{stop}$	s	N_s
1	0.203188	2032
2	0.059520	595
3	0.019827	198
4	0.006977	70
5	0.002516	25
6	0.000918	9
7	0.000336	3

Nota

Se realmente quer estar seguro de atualizar TODOS os nós, o rumor por si só não é suficiente

Arquitetura do Dropbox



Conceitos adquiridos

- Comunicação transiente/persistente, assíncrona/síncrona.
- Comunicação RPC.
- Comunicação MOM (Event Streaming).
- Kafka.
- Multicast com ordem FIFO, Causal e Total.
- Comunicação Gossip: anti-entropy e rumor.

Sistemas Distribuídos

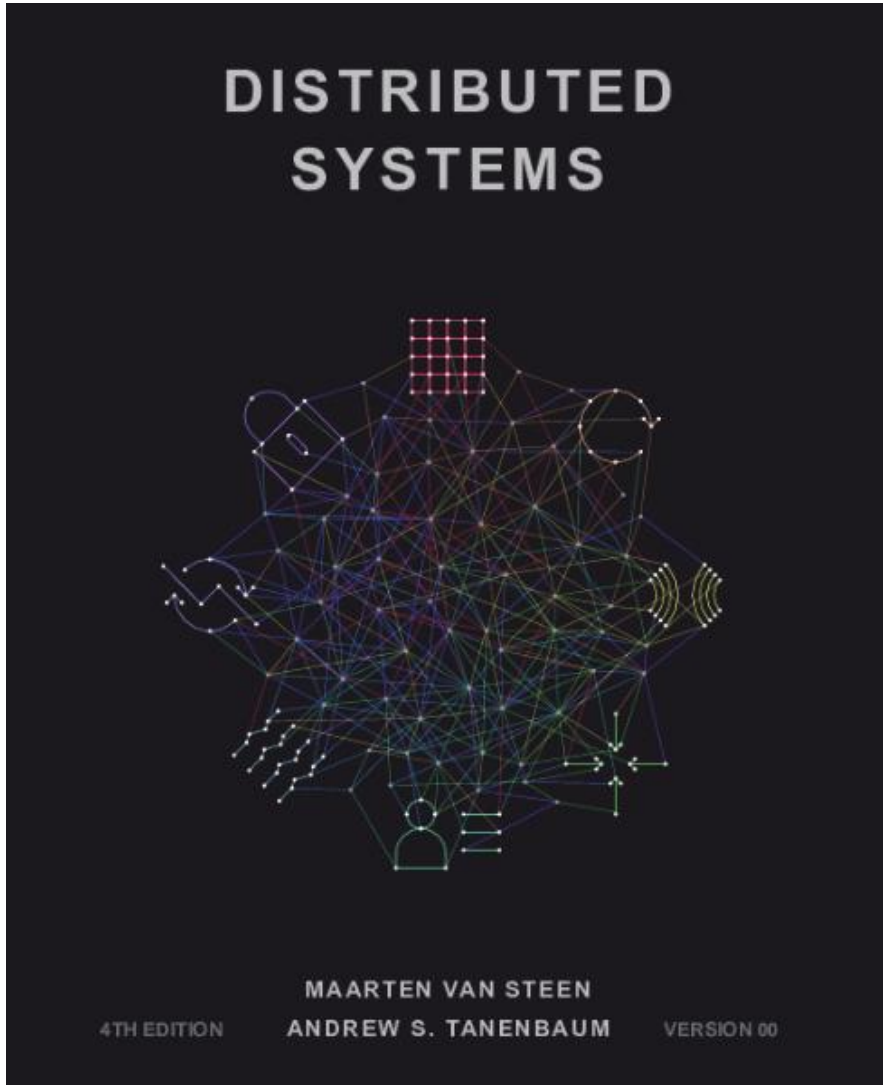
Nomes

NAMING

Vladimir Rocha (Vladi)
CMCC - Universidade Federal do ABC

Distributed Systems 4th edition (free)

https://www.distributed-systems.net/index.php/books/ds4/



4.03 (2025)

Disclaimer

- Estes slides foram baseados nos do professor **Emilio Francesquini** para o curso de Sistemas Distribuídos na UFABC.
- Este material pode ser usado livremente desde que sejam mantidos, além deste aviso, os créditos aos autores e instituições.
- Estes slides foram adaptados daqueles originalmente preparados (e gentilmente cedidos) pelo professor **Daniel Cordeiro, da EACH-USP** que por sua vez foram baseados naqueles disponibilizados online pelos autores do livro "Distributed Systems", 3ª Edição em: https://www.distributed-systems.net.

Agenda

- Definições
- Resolução de nomes (plano)
- Resolução de nomes (estruturado)
- Resolução de nomes (atributo)

Agenda

- Definições: nomes, identificadores e endereços
- Resolução de nomes (simples)
- Resolução de nomes (estruturado)
- Resolução de nomes (atributo)

Nomes

Essencialmente

Nomes são usados para denotar entidades em um sistema distribuído.

Para realizar operações em uma entidade, é preciso ter acesso a ela usando um ponto de acesso.

Pontos de acessos são identificados por um endereço.

Identificadores

Nomes "puros" ou planos

são nomes que não tem significado próprio; são apenas strings aleatórias. Nomes puros podem ser usados apenas para comparação.

Identificador

Um nome que possui as seguintes propriedades:

1. Cada identificador se refere a, no máximo, uma entidade
2. Cada entidade é referenciada por, no máximo, um identificador
3. Um identificador sempre se refere à mesma entidade (reutilização de identificadores é proibida)

Observação

Um identificador não necessariamente precisa ser um nome puro (pode ter conteúdo).

Agenda

- Definições: nomes, identificadores e endereços
- Resolução de nomes (plano)
- Resolução de nomes (estruturado)
- Resolução de nomes (atributo)

Espaço de nomes plano

Problema

Dado um nome não estruturado (ex: um identificador), como localizar seu ponto de acesso?

1. Solução simples (broadcasting / flooding / ponteiros)
2. ~~Abordagens baseadas em um local pré-determinado (*home-based*)~~
3. Tabelas de hash distribuídas (P2P estruturado)
4. Serviço hierárquico de nomes

Soluções simples: broadcasting

Broadcasting

Solução: fazer o *broadcast* do ID, requisitando que a entidade devolva seu endereço

- Não escala para além de redes locais
- Requer que todos os processos escutem e processem os pedidos de localização

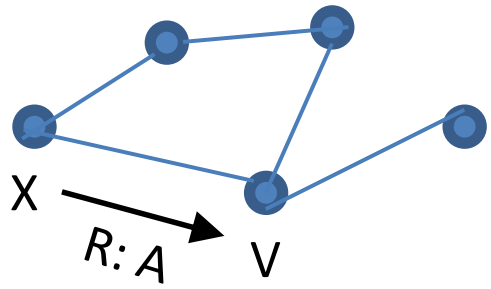
Address Resolution Protocol (ARP)

Para encontrar o endereço MAC associado a um endereço IP, faz o *broadcast* da consulta "quem tem esse endereço IP?" [Plumer/RFC 1982]

Soluções simples: flooding

Flooding (tanenbaum 2018. 3ed pp 85 – section 2.3)

Solução: um nó X envia a requisição R por um recurso A a alguns vizinhos V. O nó V recebe e realiza o seguinte processo:



- Se V já viu a requisição antes, ignora.
- V procura localmente por A.
- Se V não tem o recurso:

  encaminha R para um dos seus vizinhos
- Se V tem o recurso:

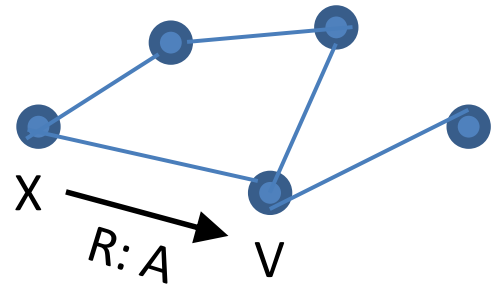
  devolve diretamente para X ou

  devolve para quem fez (encaminhou) a requisição

Soluções simples: flooding

Flooding (tanenbaum 2018. 3ed pp 85 – section 2.3)

Solução: um nó X envia a requisição R por um recurso A a alguns vizinhos V. O nó V recebe e realiza o seguinte processo:



- Se V já viu a requisição antes, ignora.
- V procura localmente por A.
- Se V não tem o recurso:

  encaminha R para um dos seus vizinhos
- Se V tem o recurso:

  devolve diretamente para X ou

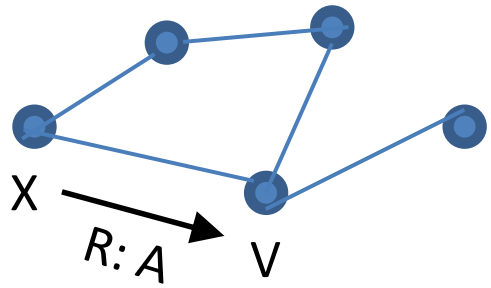
  devolve para quem fez (encaminhou) a requisição

Problema de escalabilidade (de tamanho e geográfica)

Soluções simples: flooding com TTL

Flooding (tanenbaum 2018. 3ed pp 85 – section 2.3)

Solução: um nó X envia a requisição R por um recurso A a alguns vizinhos V. O nó V recebe e realiza o seguinte processo:



- Se V já viu a requisição antes, ignora.
- V procura localmente por A.
- Se V não tem o recurso:

  Se TTL da requisição > 0

  encaminha R para um dos seus vizinhos com TTL-1
- Se V tem o recurso:

  devolve diretamente para X ou

  devolve para quem fez (encaminhou) a requisição

Soluções simples: flooding com TTL

Flooding



[NullSoft 2000]

Gnutella é um sistema distribuído P2P para compartilhamento de arquivos.
A busca era realizada por 'flooding com TTL' com as seguintes características:

- Se V não tem o arquivo, encaminha para 7 vizinhos com TTL=10.
- Se V tem o arquivo, responde de forma inversa pelo mesmo caminho
- Não há loop nas requisições.
- Transferências são realizadas diretamente entre nós via HTTP

Soluções simples: flooding com TTL

Flooding

gnutella

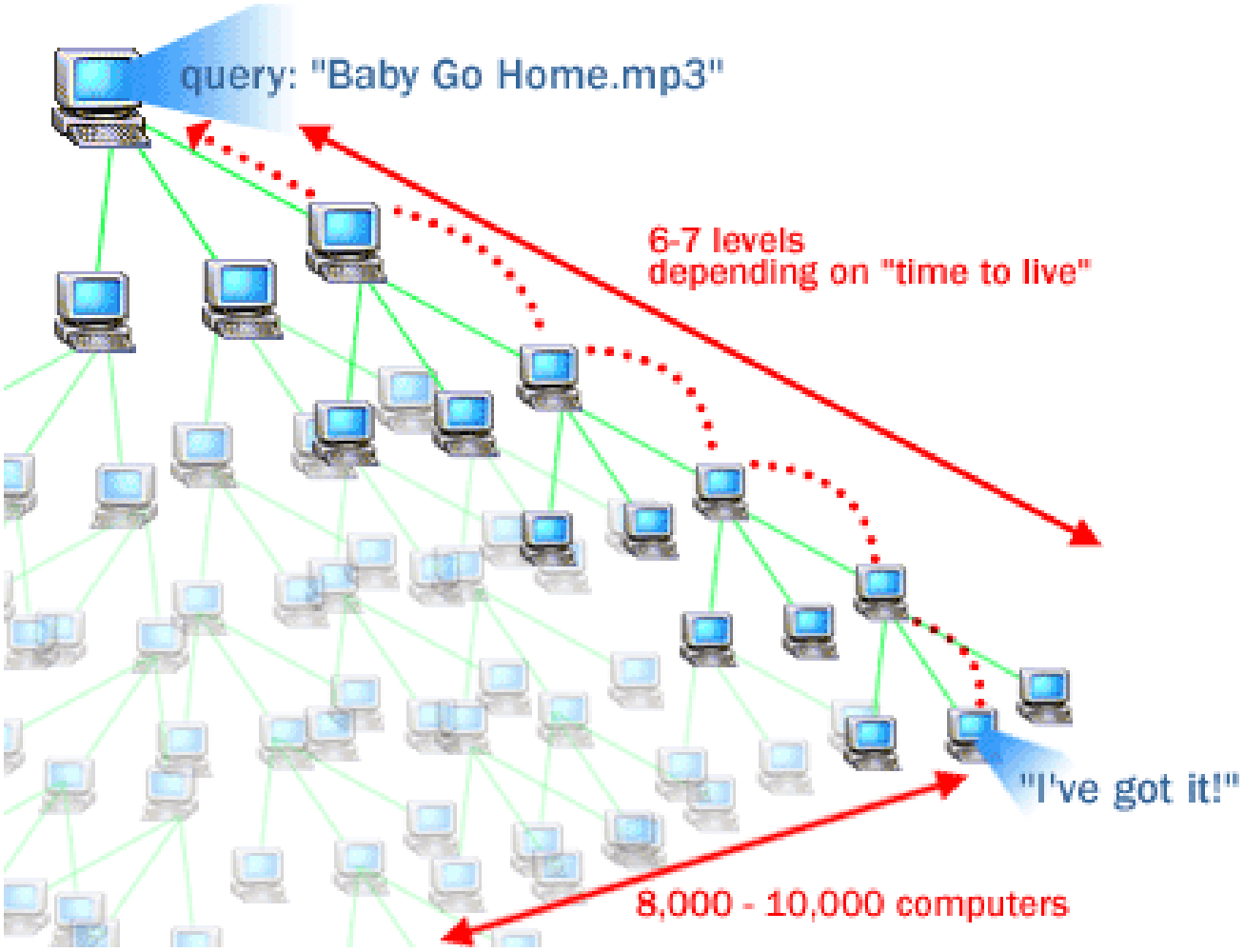


Figure from http://computer.howstuffworks.com/file-sharing.htm

Espaço de nomes plano

Problema

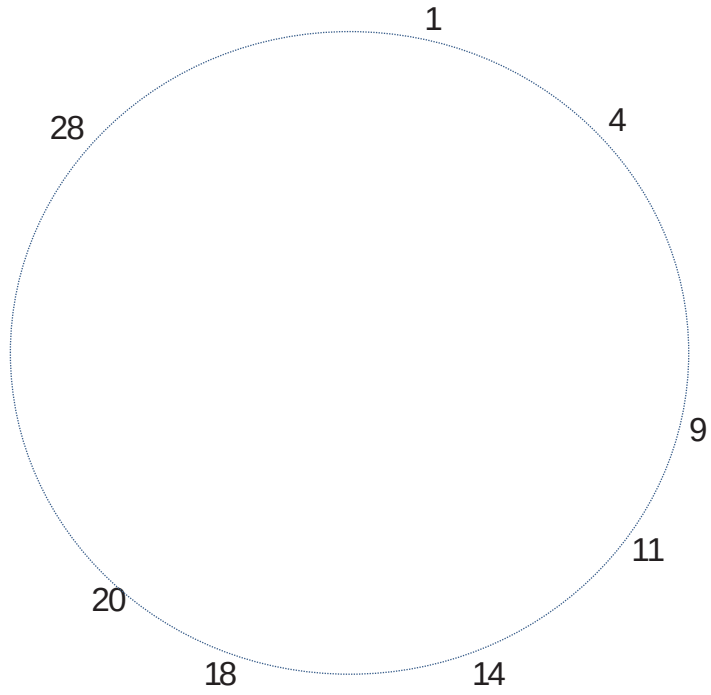
Dado um nome não estruturado (ex: um identificador), como localizar seu ponto de acesso?

1. Solução simples (broadcasting / flooding / ponteiros)
2. Abordagens baseadas em um local pré-determinado (*home-based*)
3. **Tabelas de hash distribuídas (P2P estruturado)**
3. Serviço hierárquico de nomes

Chord

Considere que os nós estejam organizados em forma de anel lógico [Stoica 2003]

- A cada nó é atribuído um identificador aleatório de *m*-bits
- A cada entidade é atribuída uma única chave de *m*-bits
- Entidades com chave *k* estão sob a jurisdição do nó com o menor *id* $\geq$ *k* (seu sucessor)



Busca por uma informação: solução ruim

Faça com que cada nó mantenha o registro de seus vizinhos e faça uma busca linear ao longo do anel

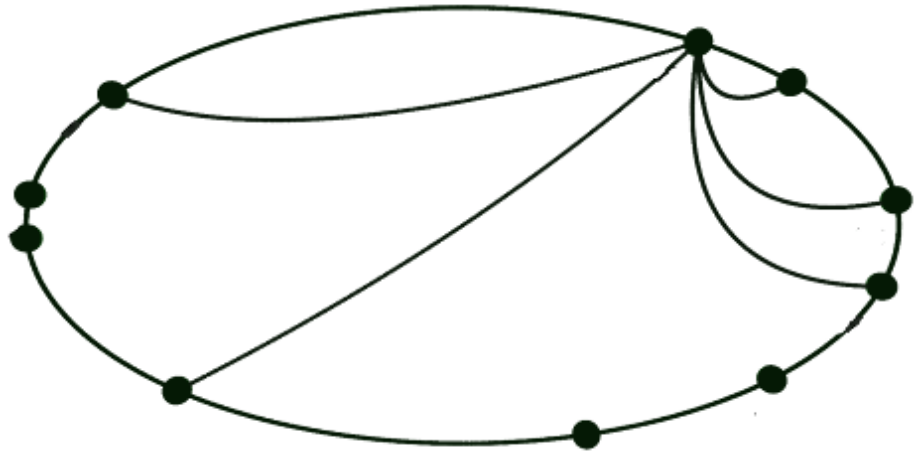
Notação

Chamamos de nó p o nó cujo identificador é p.

Como melhorar o desempenho da busca?

Shortcuts

DHTs: fingertables



Ideia:

- cada nó p mantém um finger table (shortcuts) FT_p [] com um máximo m entradas

$$FT_p[i] = succ(p + 2^{i-1})$$

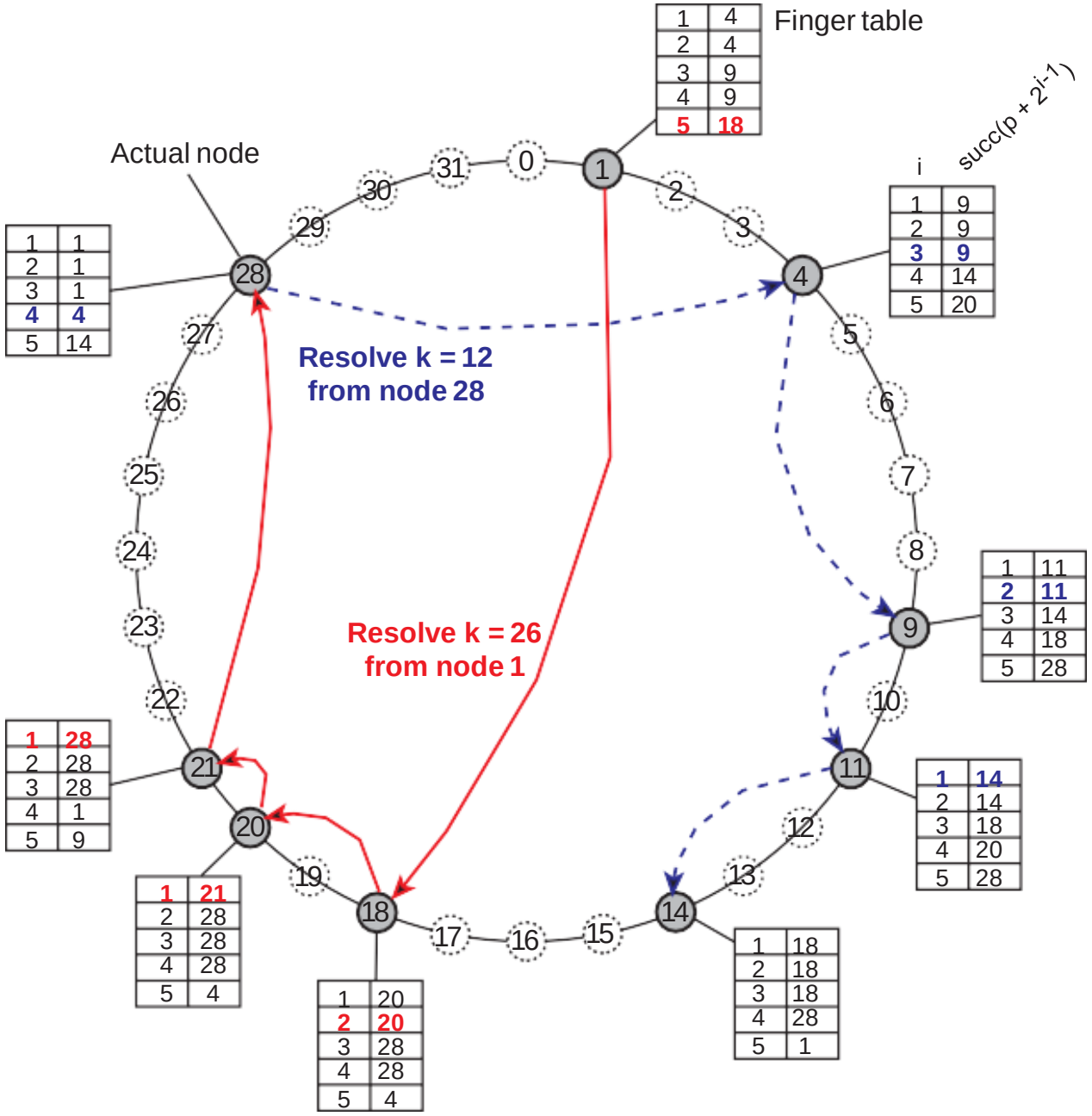
FT_p [i] aponta para o primeiro nó sucessor a p com distância de pelo menos 2^{i-1} posições (distância 2^0, 2^1, 2^2, 2^3, 2^4 ...)

- Para procurar por uma chave k, o nó p encaminha o pedido para o nó com índice j tal que:

$$q = FT_p[j] \leq k < FT_p[j+1]$$

- Se $p < k < FT_p$ [1], a requisição é encaminhada para FT_p [1]

DHTs: fingertables



Espaço de nomes plano

Problema

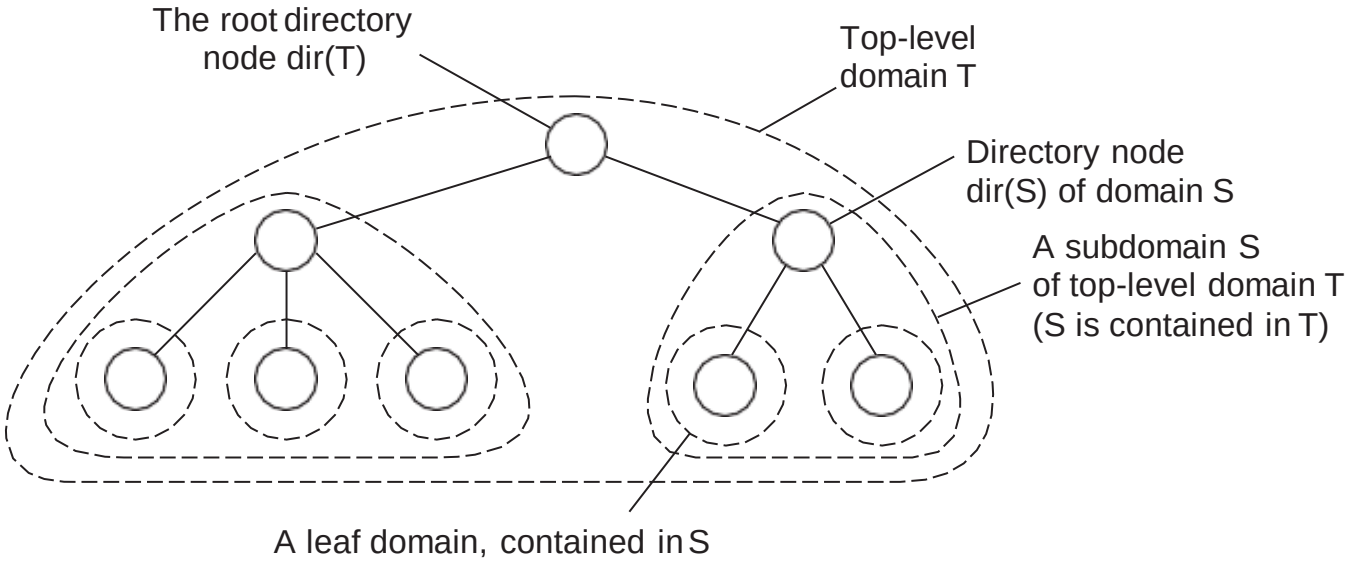
Dado um nome não estruturado (ex: um identificador), como localizar seu ponto de acesso?

1. Solução simples (broadcasting / flooding / ponteiros)
2. Abordagens baseadas em um local pré-determinado (*home-based*)
4. Tabelas de hash distribuídas (P2P estruturado)
4. Serviço hierárquico de nomes

Serviços de localização hierárquicos (HLS)

Ideia:

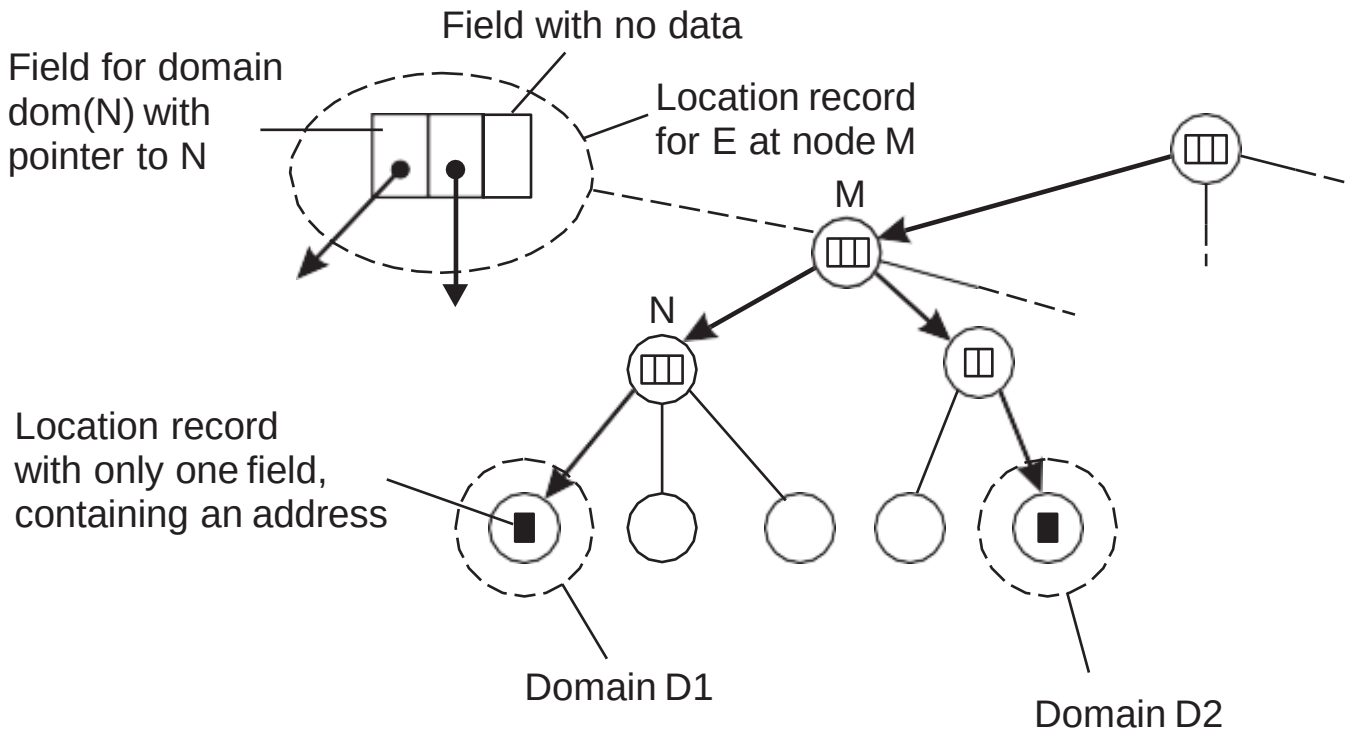
Construir uma árvore de busca em larga escala [van Steen 1998], onde a rede é dividida em domínios hierárquicos.
Cada domínio é representado por um diretório de nós separado.



HLS: organização em árvores

Invariantes

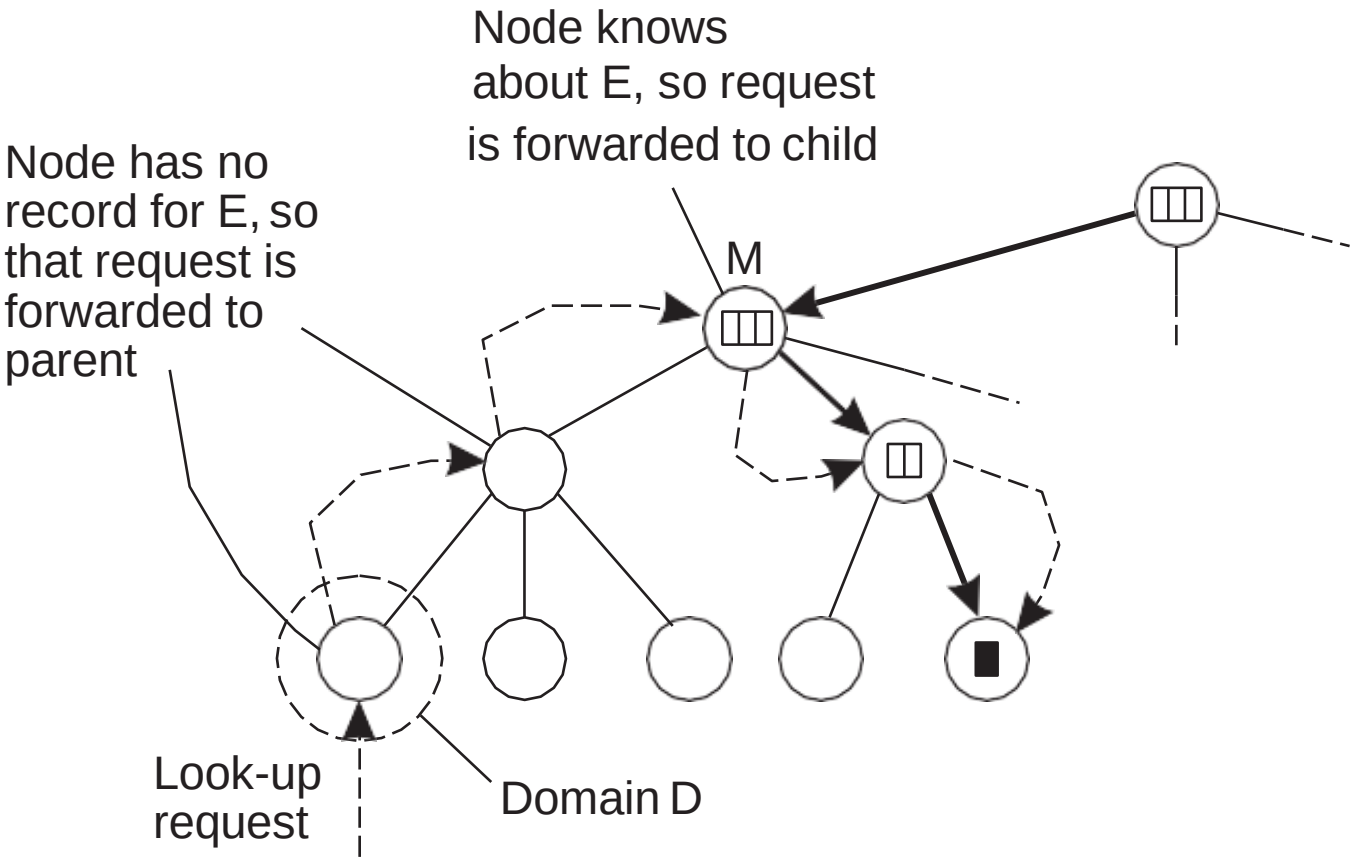
- Endereço da entidade *E* é armazenado numa folha ou nó intermediário
- Nós intermediários contém um ponteiro para um filho *sse* a subárvore, cuja raiz é o filho, contenha o endereço da entidade
- A raiz conhece todas as entidades



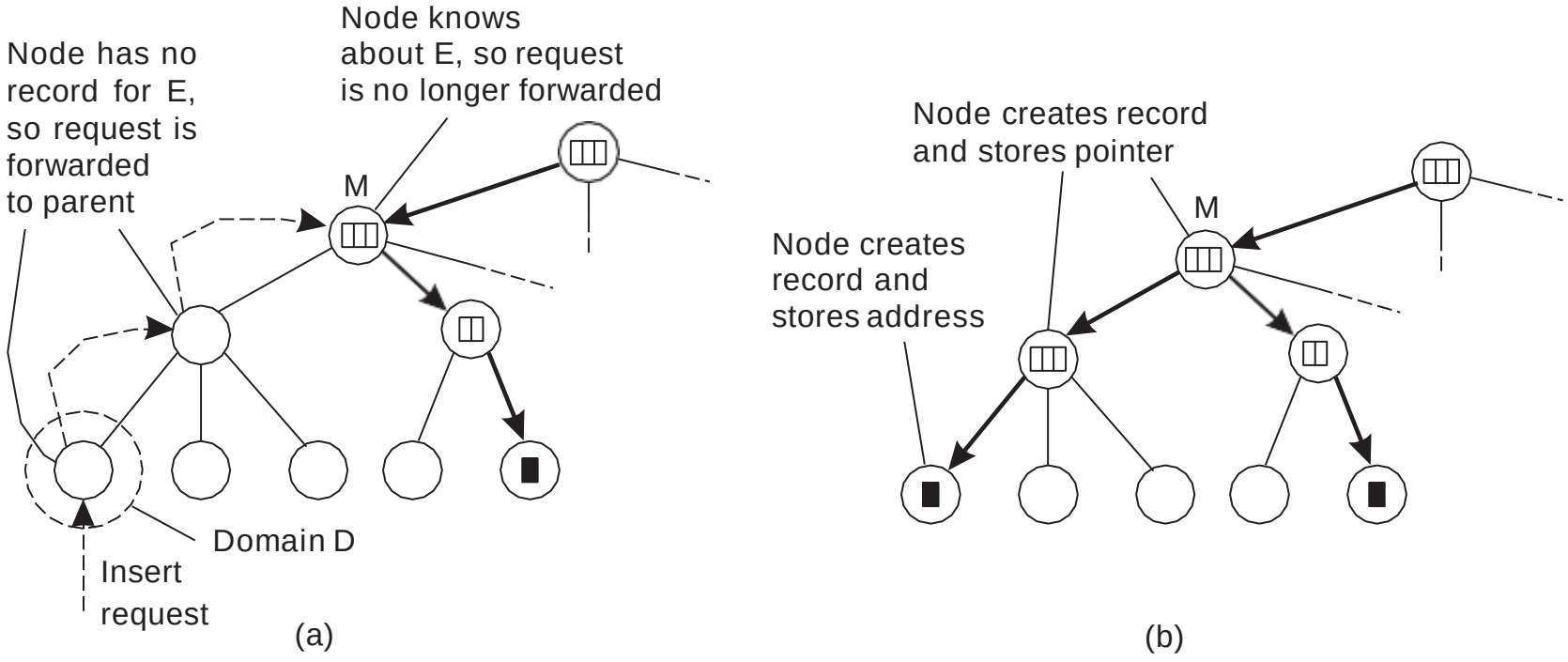
HLS: consulta

Princípios básicos:

- Comece a busca pelo nó folha local
- Se o nó souber sobre *E*, seguir o ponteiro pra baixo, senão suba
- Continue subindo até encontrar a raiz



HLS: inserção



(a) um pedido de inserção é encaminhado ao primeiro nó que conhece a entidade E

(b) uma cadeia de ponteiros de redirecionamento é criada até o nó folha

Agenda

- Definições: nomes, identificadores e endereços
- Resolução de nomes (simples)
- Resolução de nomes (estruturado)
- Resolução de nomes (atributo)

DNS: Implementação de espaços de nomes

A resolução de um nome é o mecanismo para recuperar a entidade que possui o nome

Problema:

Distribuir o processo de resolução de nomes, bem como o gerenciamento do espaço de nomes, em várias máquinas, de forma a distribuir o grafo de nomes para garantir **escalabilidade e disponibilidade.**

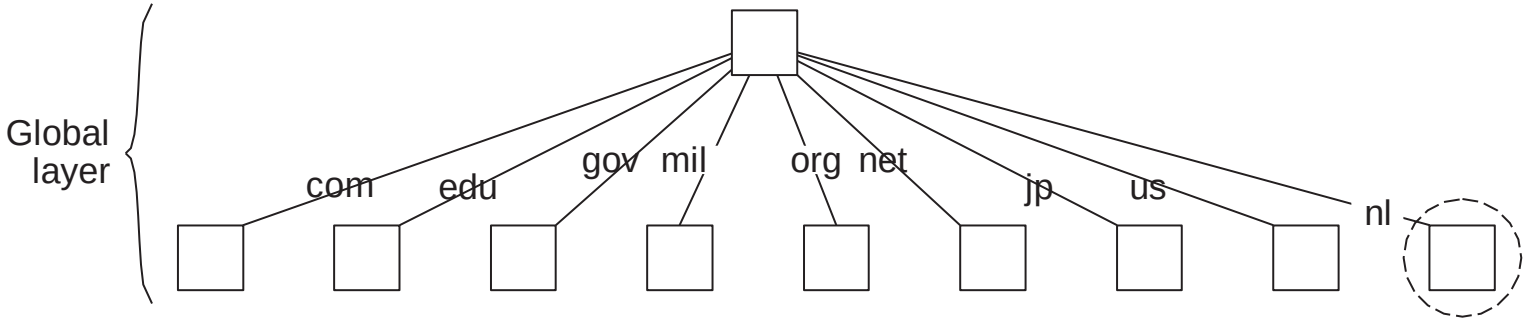
Exemplo a ser analisado:

DNS – Domain Name System [RFC em 1983, implementado em 1984] (transforma um nome a um IP)

DNS: Distribuição

Três níveis distintos:

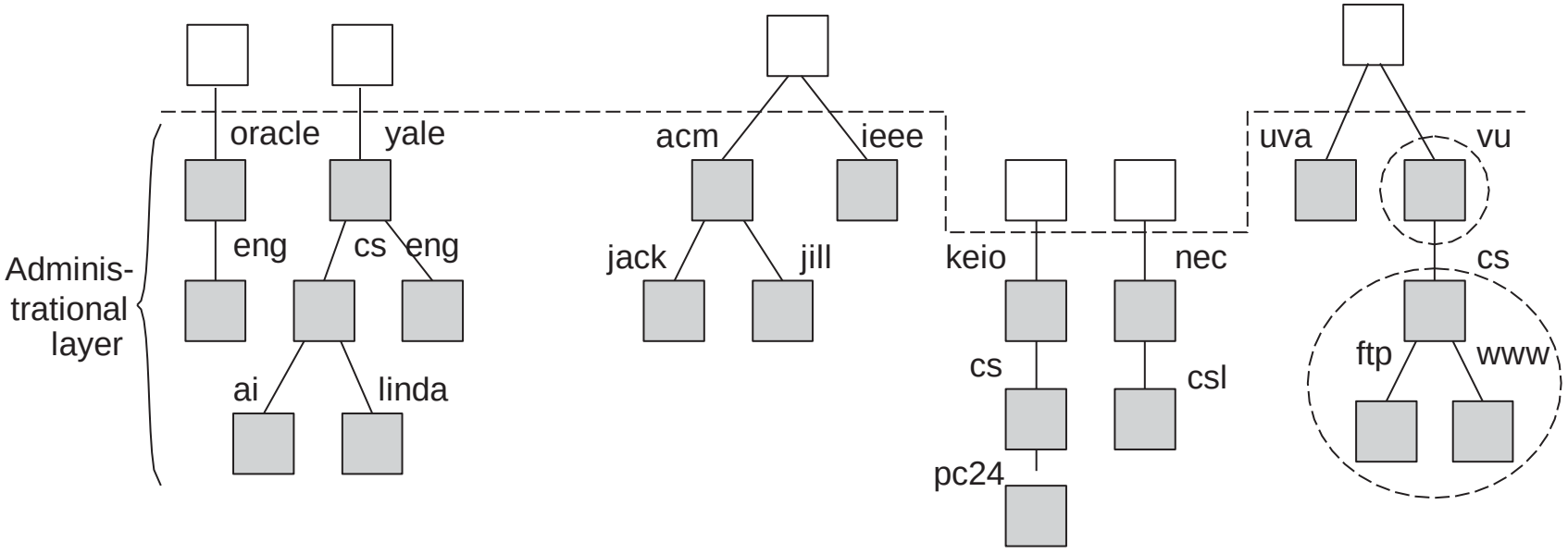
1. Nível global: consiste de um diretório de nós de alto nível. Suas principais características são que os nós do diretório devem ser gerenciados em conjunto por **diferentes administradores** e que há uma relativa **estabilidade nos nomes**, ou seja, as entradas do diretório são modificadas muito raramente



DNS: Distribuição

Três níveis distintos:

2. Nível de administração: nós de diretório de nível intermediário. Podem ser agrupados e **cada grupo pode ser responsabilidade de um administrador diferente.** Apesar de estáveis, mudam com mais frequência do que entradas no nível global

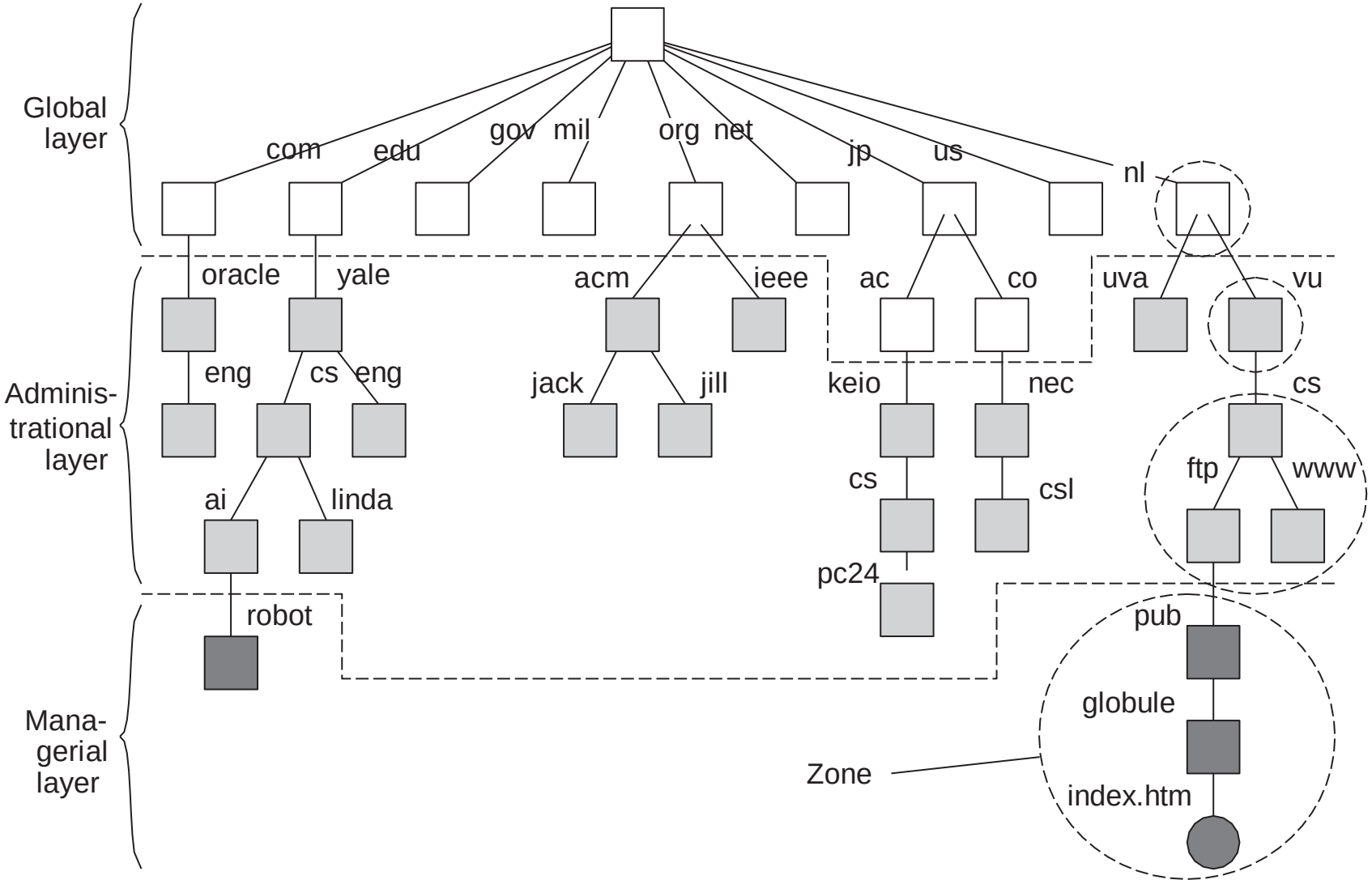


DNS: Distribuição

Três níveis distintos:

3. Nível gerencial: nós de nível inferior que **pertencem a um único administrador.** O problema principal é mapear os nós de diretório aos servidores de nomes local. **Mudanças regulares** ocorrem neste nível.

DNS: Distribuição

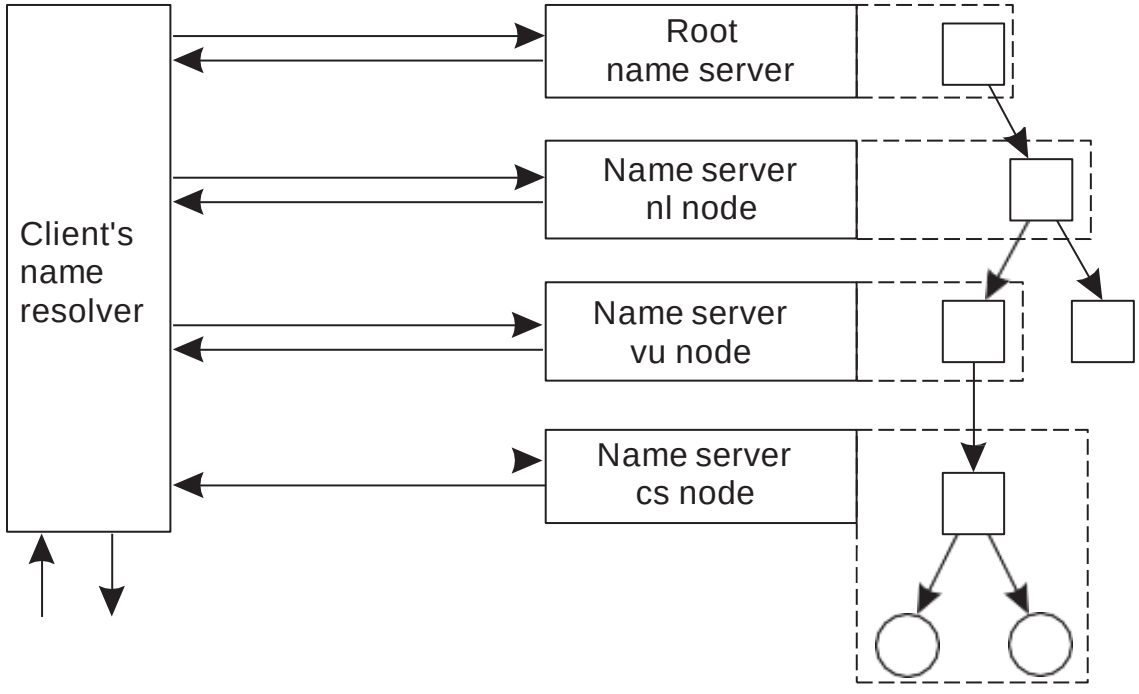


DNS: Comparação

Item	Global	Administração	Gerencial
1	Mundial	Organização	Departamento
2	Poucos	Muitos	Qdes. enormes
3	Segundos	Milissegundos	Imediato
4	Lento	Imediato	Imediato
5	Muitos	Nenhum ou poucos	Nenhum
6	Sim	Sim	Às vezes
1: Escala geográfica 2: # Nós 3: Responsividade		4: Propagação de atualizações 5: # Réplicas 6: Cache no lado do cliente?	

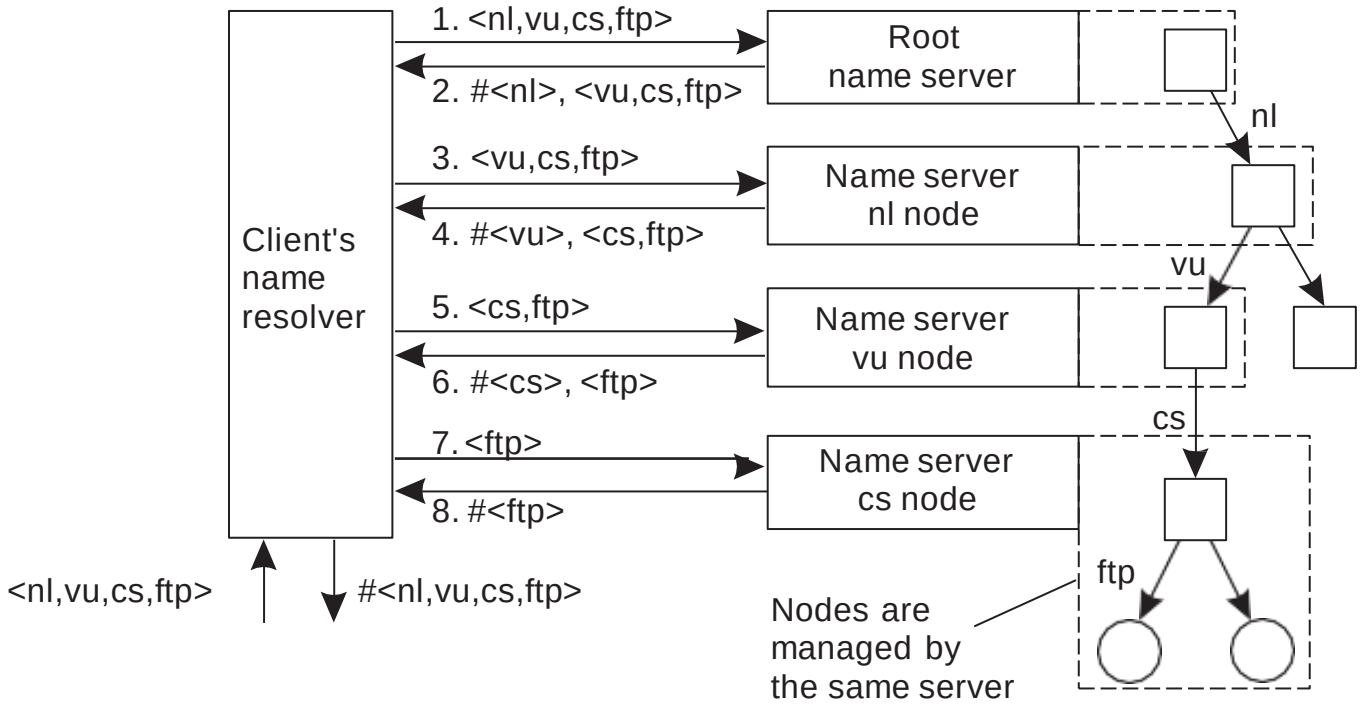
DNS: Resolução de nomes iterativa

1. Cliente envia `resolve(dir,[name1,...,nameK])` para `Server0` responsável por `dir`
2. `Server0` resolve `resolve(dir,name1)` → `dir1`, devolve a identificação (endereço) de `Server1`, que contém `dir1`.
3. Cliente envia `resolve(dir1,[name2,...,nameK])` para `Server1`, etc.



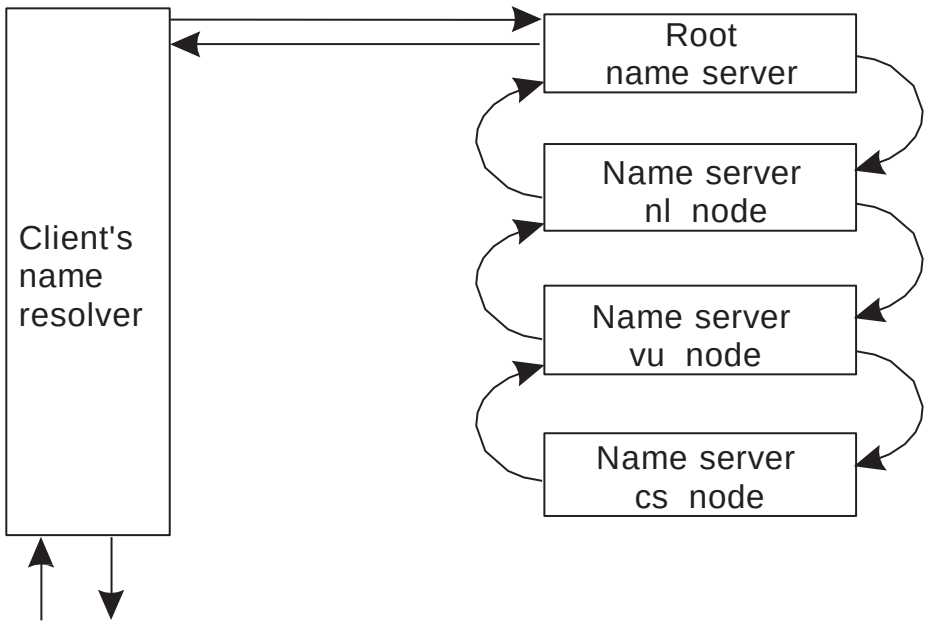
DNS: Resolução de nomes iterativa

1. Cliente envia `resolve(dir,[name1,...,nameK])` para `Server0` responsável por `dir`
2. `Server0` resolve `resolve(dir,name1)` → `dir1`, devolve a identificação (endereço) de `Server1`, que contém `dir1`.
3. Cliente envia `resolve(dir1,[name2,...,nameK])` para `Server1`, etc.



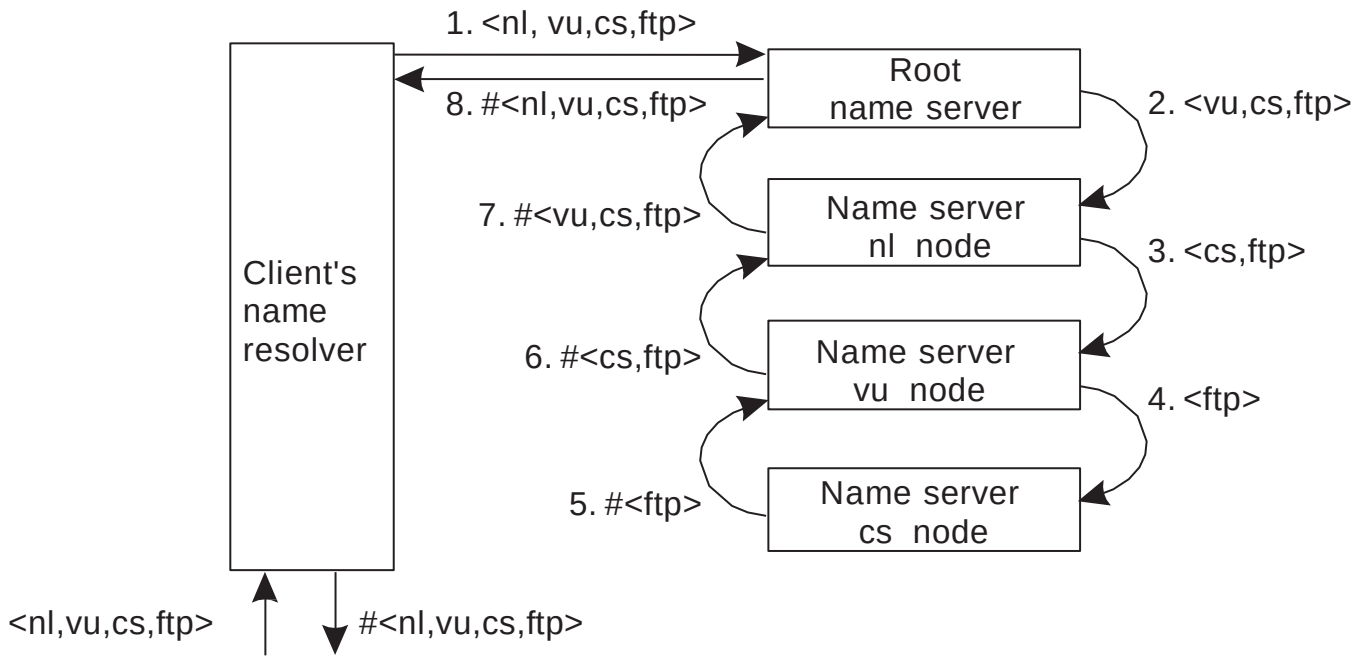
DNS: Resolução de nomes recursiva

1. Cliente envia resolve(dir,[name1,...,nameK]) para Server0, responsável por dir
2. Server0 resolve resolve(dir,name1) → dir1 e envia resolve(dir1,[name2,...,nameK]) para Server1, que contém dir1.
3. Server0 espera pelo resultado de Server1 e o devolve para o cliente.



DNS: Resolução de nomes recursiva

1. Cliente envia resolve(dir,[name1,...,nameK]) para Server0, responsável por dir
2. Server0 resolve resolve(dir,name1) → dir1 e envia resolve(dir1,[name2,...,nameK]) para Server1, que contém dir1.
3. Server0 espera pelo resultado de Server1 e o devolve para o cliente.



DNS: Resolução de nomes recursiva com cache

uso extensivo
do cache
(inconsistências)

Servidor para o nó	Deveria resolver	Procura por	Envia para filho	Recebe e and faz cache	Devolve ao solicitante
cs	<ftp>	#<ftp>	—	—	#<ftp>
vu	<cs,ftp>	#<cs>	<ftp>	#<ftp>	#<cs> #<cs, ftp>
nl	<vu,cs,ftp>	#<vu>	<cs,ftp>	#<cs> #<cs,ftp>	#<vu> #<vu,cs> #<vu,cs,ftp>
root	<nl,vu,cs,ftp>	#<nl>	<vu,cs,ftp>	#<vu> #<vu,cs> #<vu,cs,ftp>	#<nl> #<nl,vu> #<nl,vu,cs> #<nl,vu,cs,ftp>

DNS: Problemas de escalabilidade

Escalabilidade de tamanho

Devemos garantir que os servidores possam lidar com um grande número de requisições por unidade de tempo. Servidores de alto nível podem estar com problemas sérios.

DNS: Problemas de escalabilidade

Escalabilidade de tamanho

Devemos garantir que os servidores possam lidar com um grande número de requisições por unidade de tempo. Servidores de alto nível podem estar com problemas sérios.

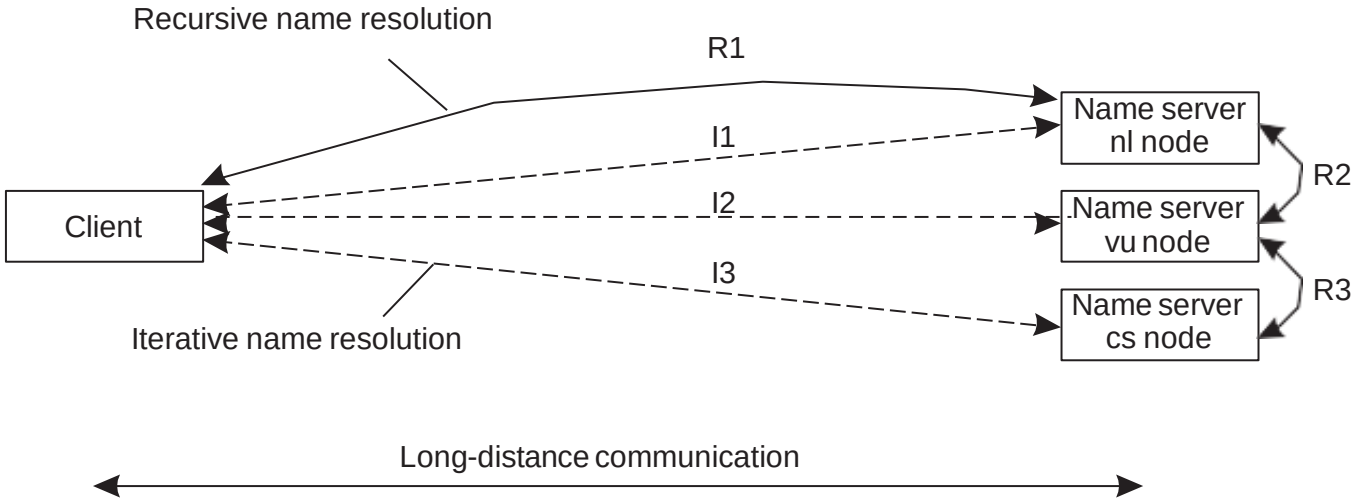
Solução:

Assuma (pelo menos nos níveis global e de administração) que os conteúdos dos nós mudam muito pouco. Assim podemos aplicar replicação extensivamente, mapeando nós a múltiplos servidores e começar a resolução de nomes no servidor mais próximo.

DNS: Problemas de escalabilidade

Escalabilidade geográfica

Precisamos garantir que a resolução de nomes escale mesmo em grandes distâncias geográficas



Porém:

Ao mapear nós em servidores que podem ser localizados em qualquer lugar, nós acabamos introduzindo uma dependência de localização implícita.

DNS: Informação do nó

SOA	Zone	Holds info on the represented zone
A	Host	IP addr. of host this node represents
MX	Domain	Mail server to handle mail for this node
SRV	Domain	Server handling a specific service
NS	Zone	Name server for the represented zone
CNAME	Node	Symbolic link
PTR	Host	Canonical name of a host
HINFO	Host	Info on this host
TXT	Any kind	Any info considered useful

DNS: DIG ufabc.edu.br

;; ANSWER SECTION:

ufabc.edu.br. 86400	IN	SOA	ns1.ufabc.edu.br. root.ufabc.edu.br.
ufabc.edu.br. 86400	IN	NS	ns2.ufabc.edu.br.
ufabc.edu.br. 86400	IN	NS	ns3.ufabc.edu.br.
ufabc.edu.br. 86400	IN	NS	ns1.ufabc.edu.br.
ufabc.edu.br. 86400	IN	A	177.104.50.120
ufabc.edu.br. 86400	IN	MX	smtp.ufabc.edu.br.
ufabc.edu.br. 86400	IN	TXT	v=spf1 mx 177.104.50.110~all

;; ADDITIONAL SECTION:

ns1.ufabc.edu.br 82697	IN	A	177.104.50.101
ns1.ufabc.edu.br. 82697	IN	AAAA	2801:a4:fabc:1034::101
ns2.ufabc.edu.br. 82697	IN	A	177.104.50.102
ns3.ufabc.edu.br. 82697	IN	A	177.104.40.2
smtp.ufabc.edu.br. 86400	IN	A	177.104.50.111

Conceitos adquiridos

- Nome e identificador.
- Flooding.
- DHT e fingertables.
- DNS: níveis, resolução iterativa e recursiva.

Sistemas Distribuídos

DHT Chord

Vladimir Rocha (Vladi)

CMCC - Universidade Federal do ABC

Disclaimer

- Estes slides foram baseados no tutorial do professor Amir Paybereah https://payberah.github.io/files/download/p2p/chord.pdf

Tabela de Hash Distribuída

- Denominada DHT (Distributed Hash Table)
- Sistema P2P estruturado (Peer atua como cliente e servidor)
- Criada em 2001 para encontrar um recurso de forma eficiente: O(log N)
- Provê escalabilidade de tamanho e particionamento (sharding).
- Evita o ponto único de falha (SPoF)
- Usada em:



Conceito

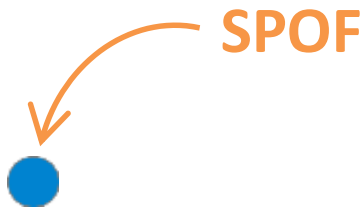
- Considere uma tabela de hash normal.

Key	Value
Fatemeh	Stockholm
Sarunas	Lausanne
Tallat	Islamabad
Cosmin	Bucharest
Seif	Stockholm
Amir	Tehran

Conceito

- Considere uma tabela de hash normal. Se ela estiver só em um servidor, esse se transforma no chamado "ponto único de falha" (SPOF – Single Point of Failure) caso morra.

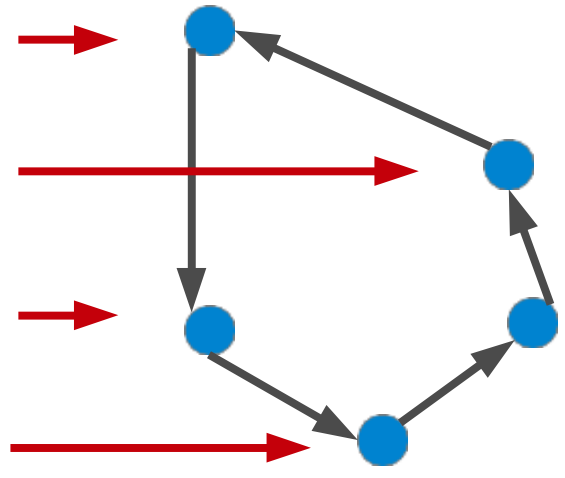
Key	Value
Fatemeh	Stockholm
Sarunas	Lausanne
Tallat	Islamabad
Cosmin	Bucharest
Seif	Stockholm
Amir	Tehran



Conceito

- Então, que tal distribuir a tabela de hash em vários nós?
  Ou seja, teremos uma tabela de hash **distribuída**

Key	Value
Fatemeh	Stockholm
Sarunas	Lausanne
Tallat	Islamabad
Cosmin	Bucharest
Seif	Stockholm
Amir	Tehran



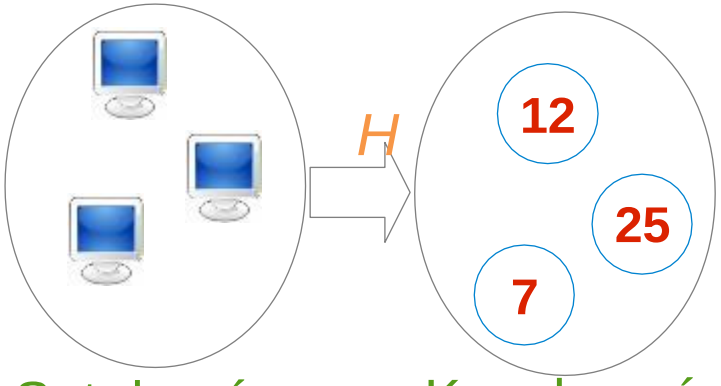
Chord (DHT estruturado em anel)

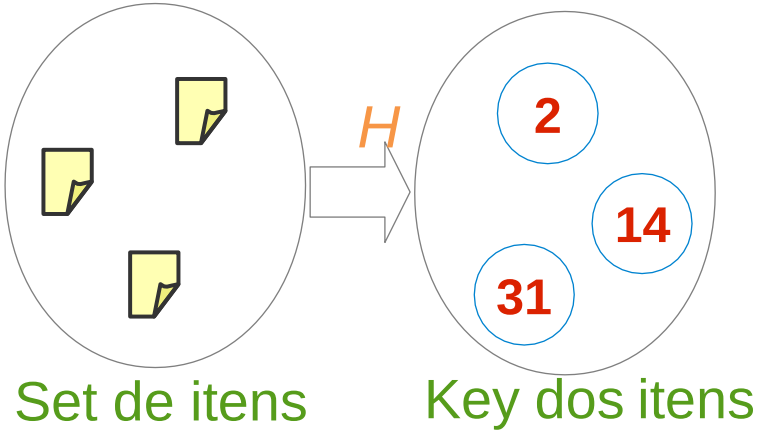
- Visão geral da construção e utilização

Chord (DHT estruturado em anel)

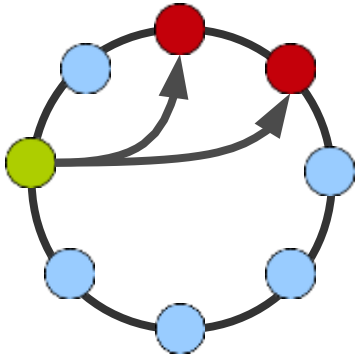
- Visão geral da construção e utilização

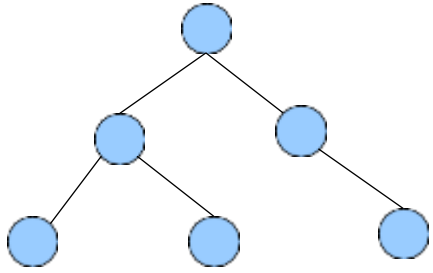
1. Definir o espaço comum de valores para os nós e itens



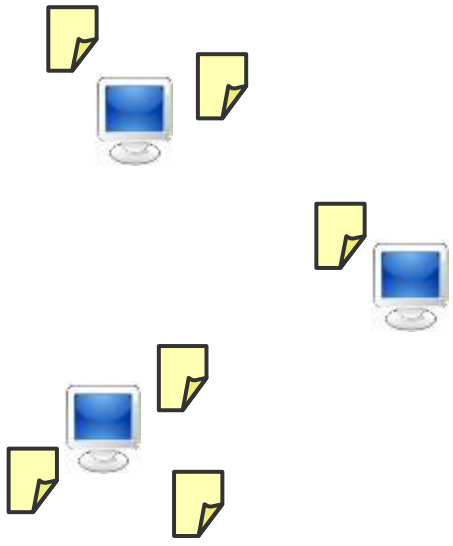


2. Conectar os nós usando um número pequeno de links (minimizar hops)





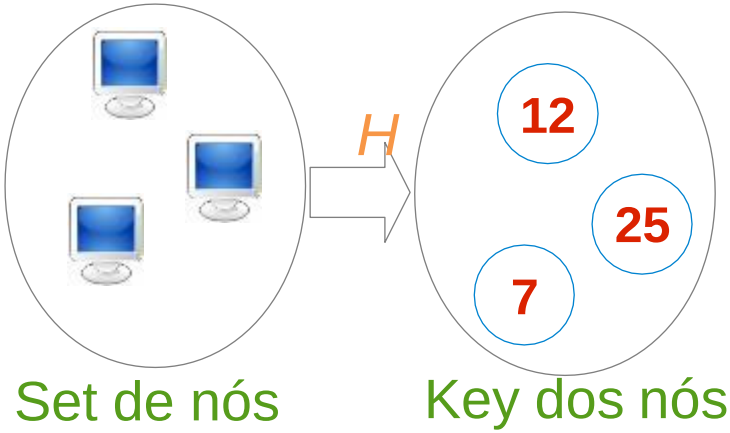
3. Definir a estratégia para designar itens aos nós

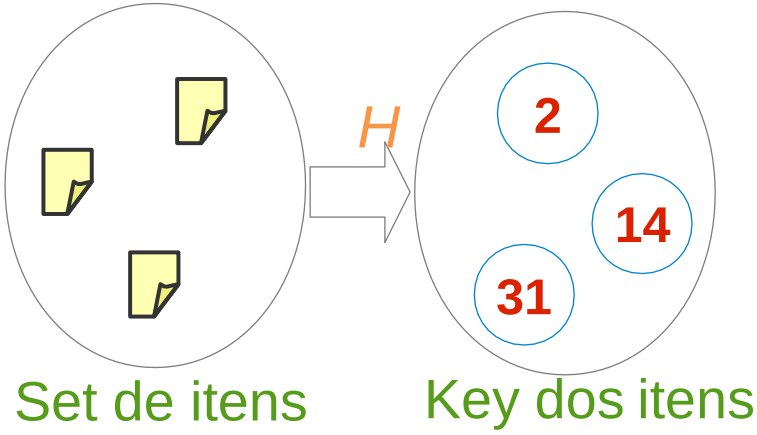


Chord (DHT estruturado em anel)

- Visão geral da construção e utilização

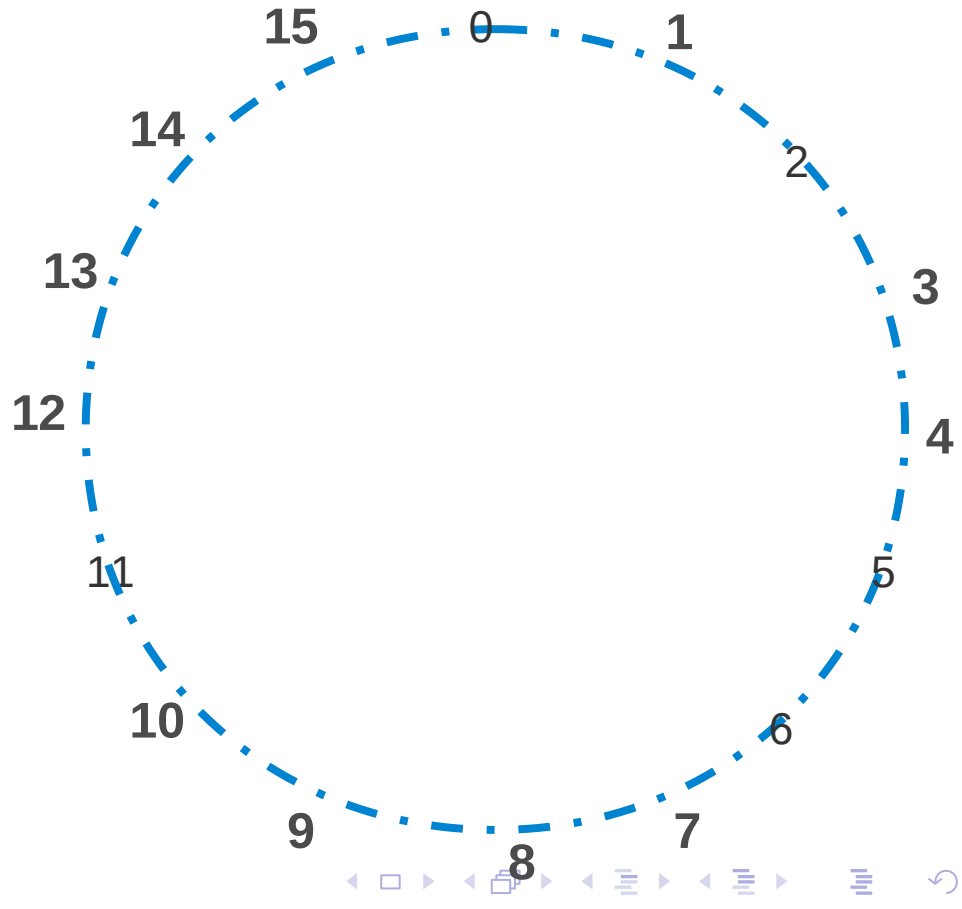
1. Definir o espaço comum de valores para os nós e itens





Construção da DHT Chord

- Usar um espaço de nomes (espaço de identificadores) de {0, 1, 2 ... 2^M-1}
  - Esse espaço de nomes formará um anel lógico módulo 2^M



Construção da DHT Chord

- Usar um espaço de nomes (espaço de identificadores) de {0, 1, 2 ... 2^M-1}
  - Esse espaço de nomes formará um anel lógico módulo 2^M
- O nó terá um nº aleatório obtido de uma função de **hash H**

Exemplo:

Espaço 2^M = 16 {0, ..., 15}

Cinco nós a, b, c, d, e

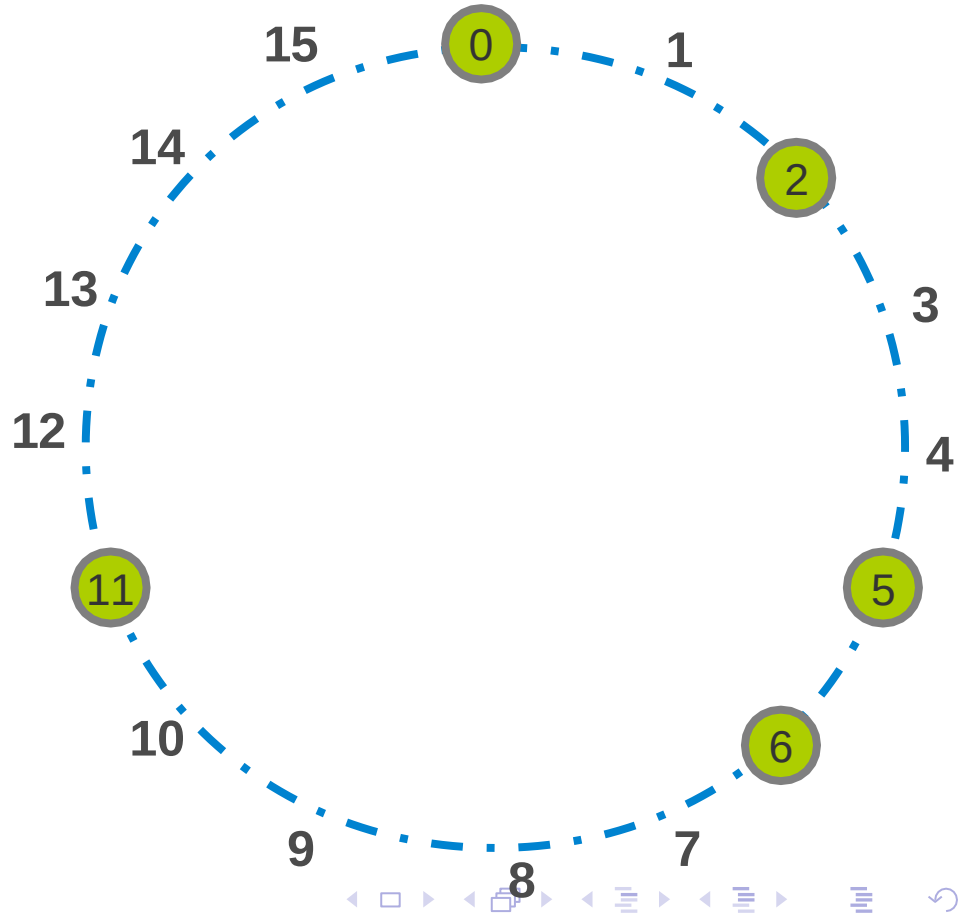
H(a) = 6

H(b) = 5

H(c) = 0

H(d) = 11

H(e) = 2



Construção da DHT Chord

- Usar um espaço de nomes (espaço de identificadores) de {0, 1, 2 ... 2^M-1}
  - Esse espaço de nomes formará um anel lógico módulo 2^M
- O nó terá um nº aleatório obtido de uma função de **hash H**
- O nó será responsável por um conjunto disjunto de identificadores

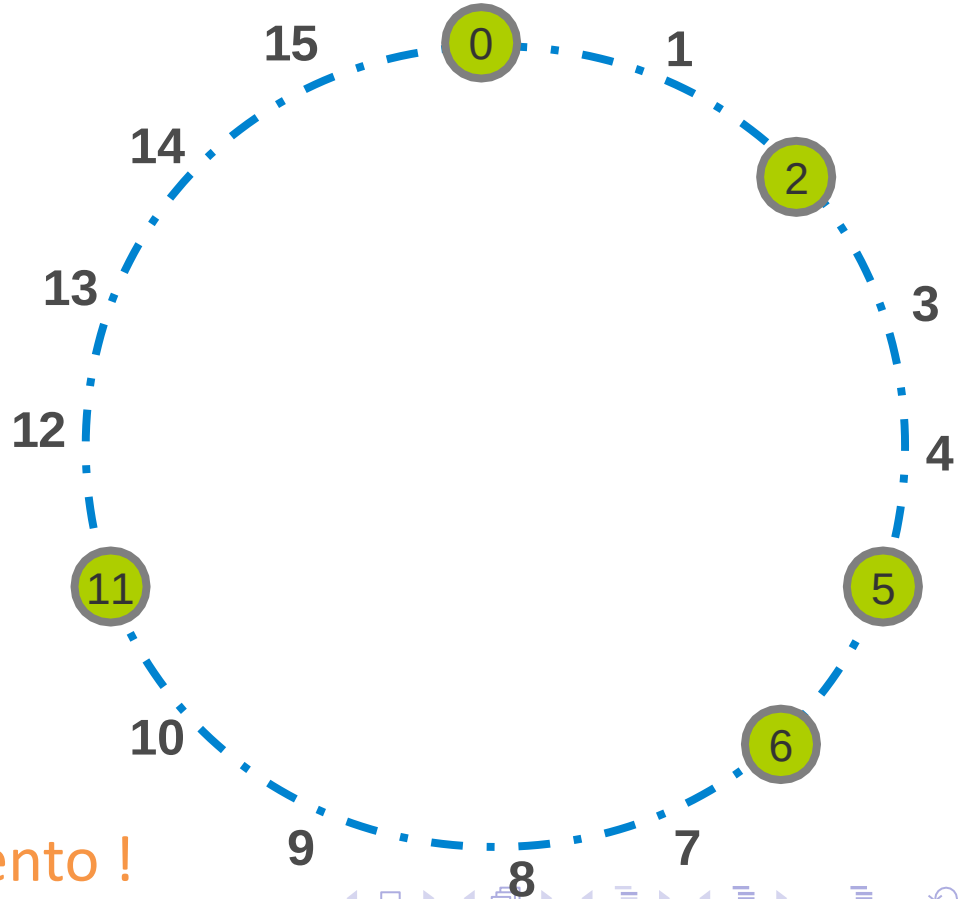
  [nº nó anterior+1, nº dele]

Exemplos:

Nó 0: [12, 0]

Nó 5: [3, 5]

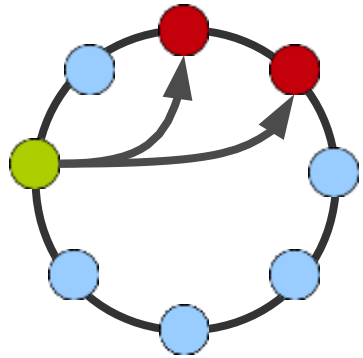
Nó 11: [7,11]



Conjunto disjunto ... Particionamento !

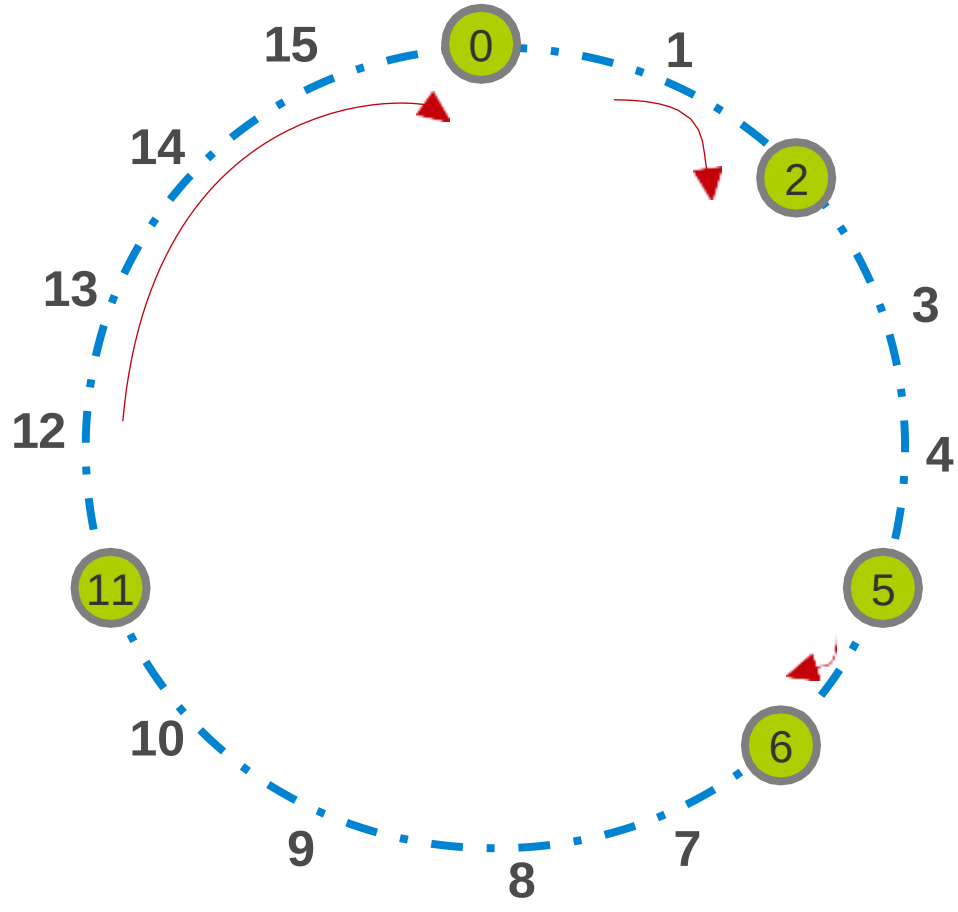
Chord (DHT estruturado em anel)

2. Conectar os nós usando um número pequeno de links (minimizar hops)



Construção da DHT Chord

- O sucessor de um identificador é o primeiro nó indo na direção do relógio começando no identificador



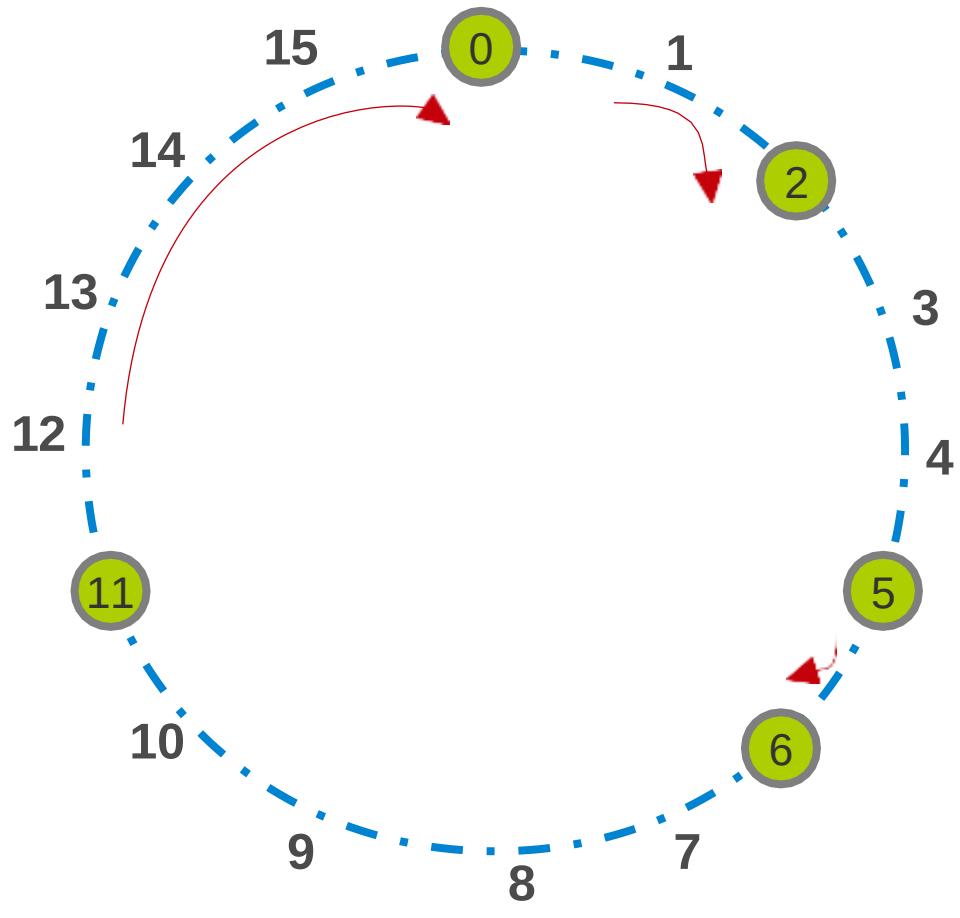
Construção da DHT Chord

- O sucessor de um identificador é o primeiro nó indo na direção do relógio começando no identificador

- `succ(x)` é o primeiro nó no anel com id **maior ou igual** a x

Exemplo:

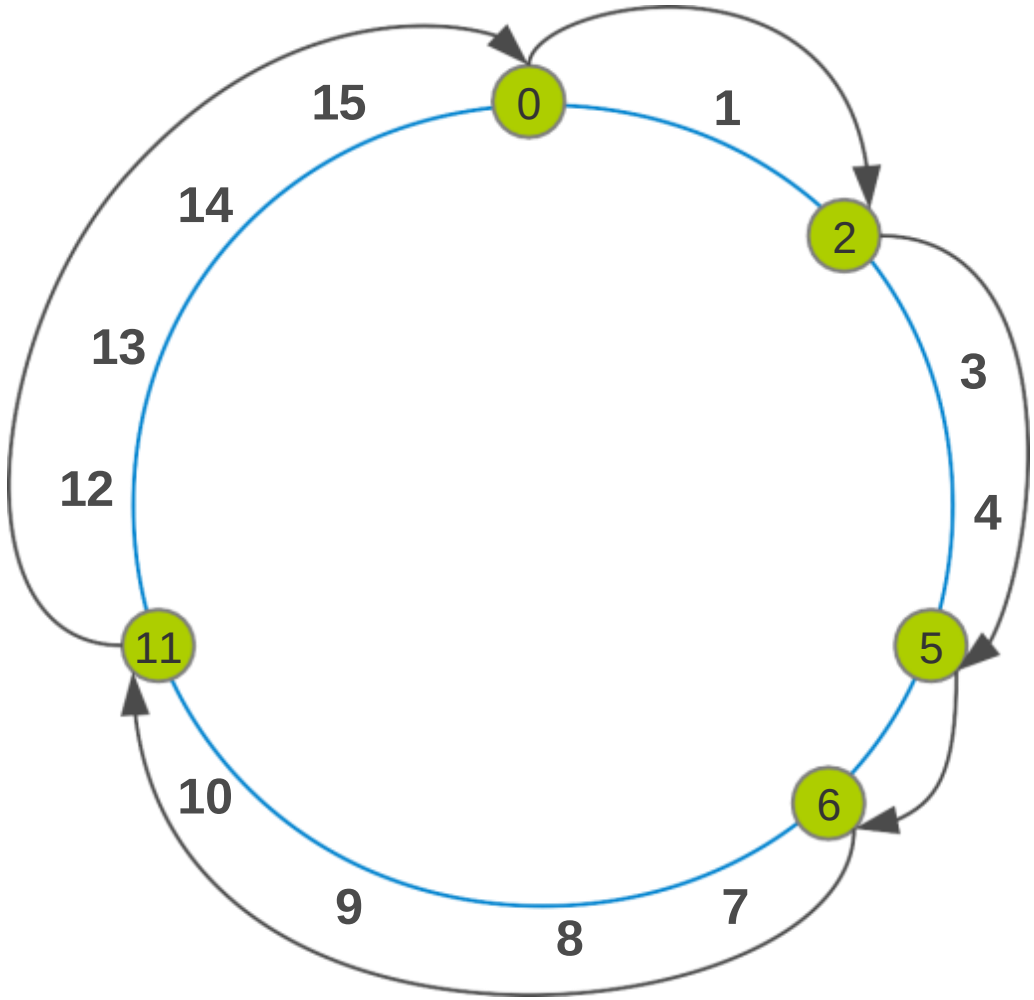
```
succ(12) = 0
succ(1)  = 2
succ(6)  = 6
```



Construção da DHT Chord

- Apontar cada nó *p* ao seu sucessor succ(*p*+1).

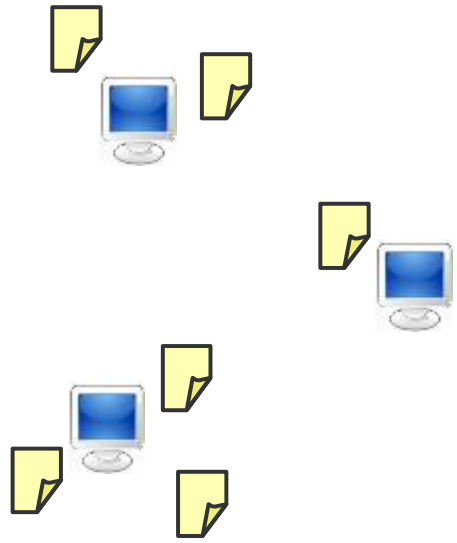
Sucessor do nó 0 é succ(1) = 2
Sucessor do nó 2 é succ(3) = 5
Sucessor do nó 5 é succ(6) = 6
Sucessor do nó 6 é succ(7) = 11
Sucessor do nó 11 é succ(12) = 0



Invariante: um nó sempre deverá ter a conexão com seu sucessor!

Chord (DHT estruturado em anel)

3. Definir a estratégia para designar itens aos nós



Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

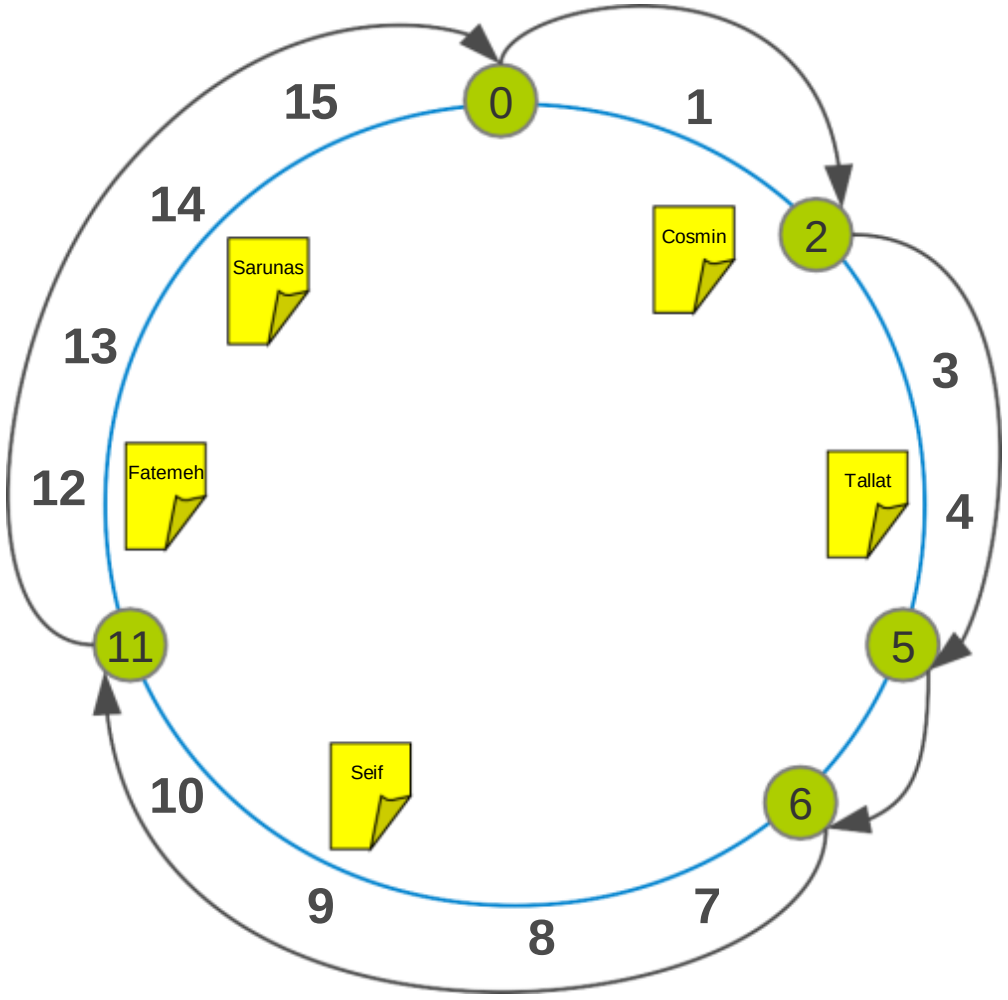
```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```

Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```



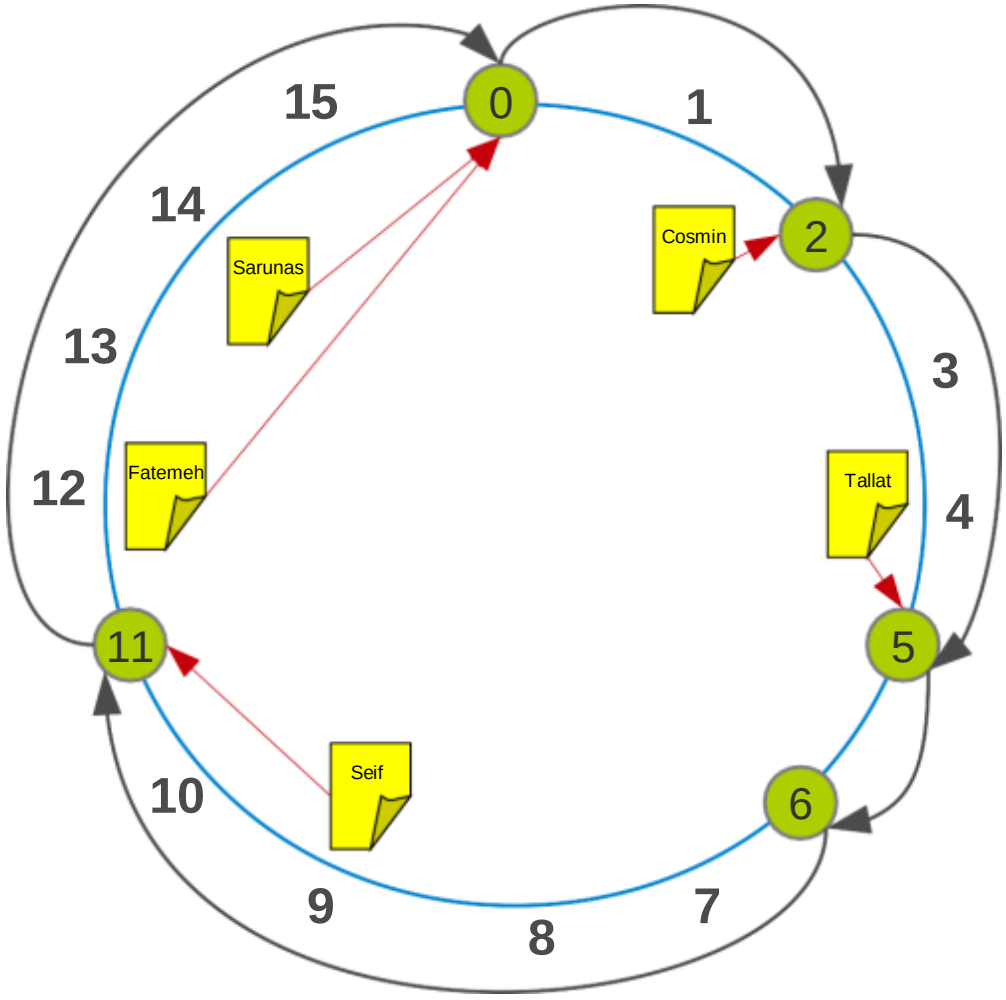
Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```

- Armazene cada item no sucessor



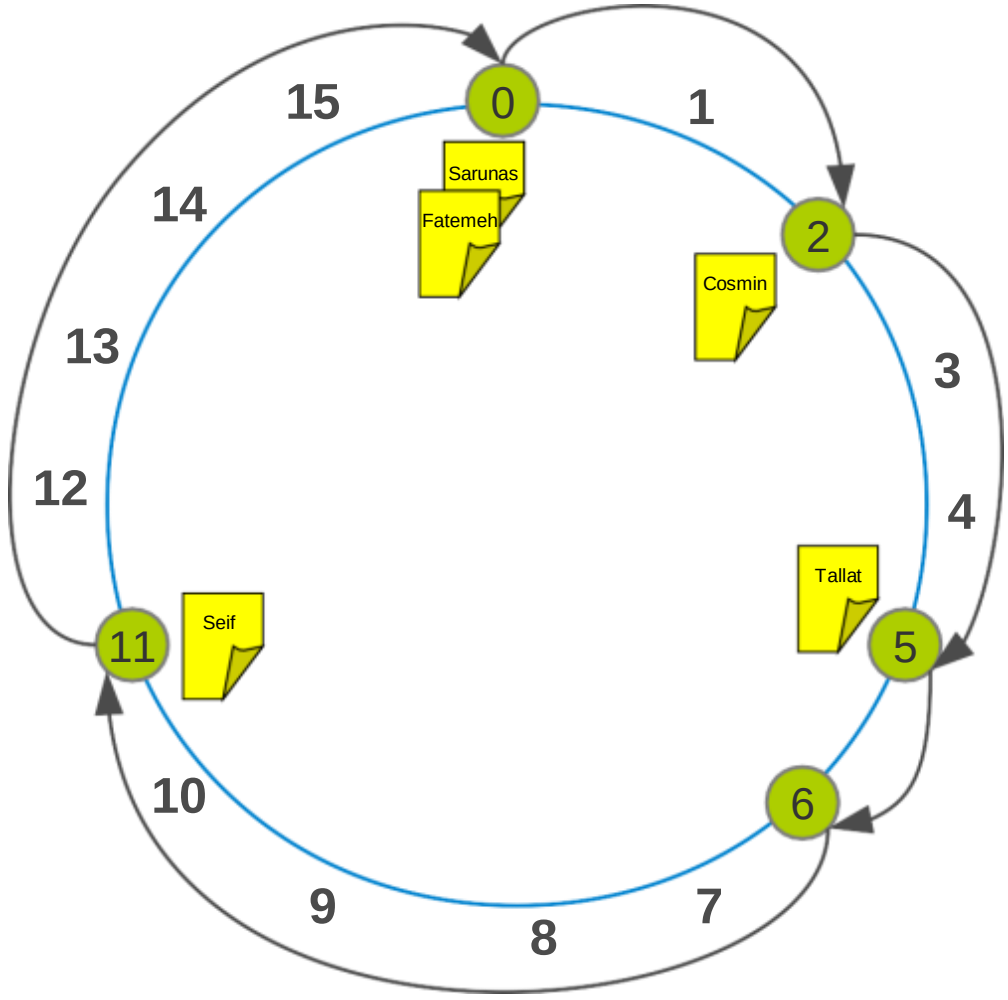
Estratégia para armazenar itens

- Onde armazenar os dados?
  - Utilizando uma função de hash H
  - A mesma da construção

- Cada item <key,value> obterá o identificador H(key) = k

```
H('Fatemeh') = 12
H('Cosmin')  = 2
H('Seif')    = 9
H('Sarunas') = 14
H('Tallat')  = 4
```

- Armazene cada item no sucessor



Estratégia para armazenar itens: *consistent hashing*

- Chord utiliza *consistent hashing* para designar chaves aos nós

- ***Consistent Hashing*** : técnica de hashing que permite que somente O(m/N) pares de chave-valor sejam transferidas na adição ou remoção de nós

  m é o número de chaves armazenadas.
  N é o número de nós.

Para pensar ...

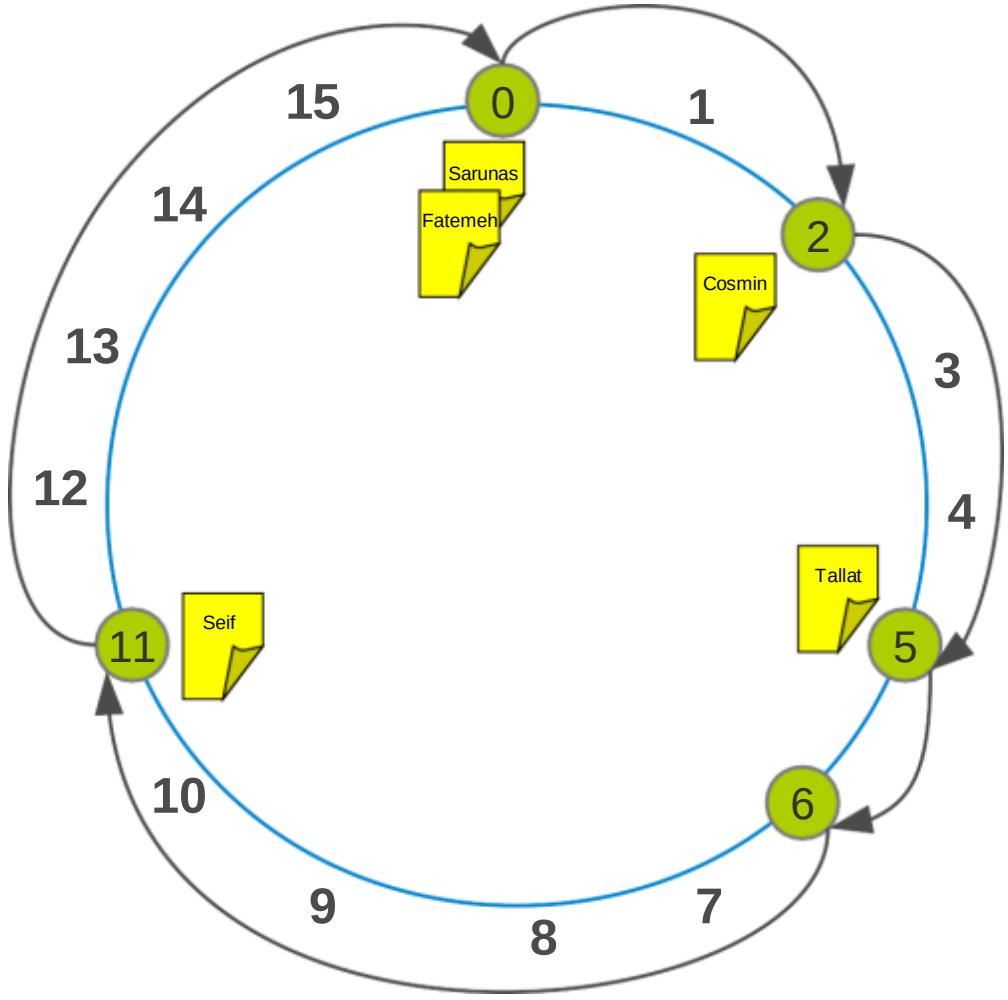
Em vez de armazenar o item no sucessor (como mostrado no slide anterior) qual seria o problema de designar a chave ao nó utilizando a função módulo e não usando o consistent hashing?

Ou seja, qual o problema em armazenar o item no nó: H(key) % N

Como realizar uma busca?

Como realizar uma busca?

- Para buscar uma chave k
  - Pergunte para **qualquer** nó do anel
  - Calcule H(k)
  - Siga os apontadores dos succ até k ser encontrado



Como realizar uma busca?

- Para buscar uma chave k
  - Pergunte para **qualquer** nó do anel
  - Calcule H(k)
  - Siga os apontadores dos succ até k ser encontrado

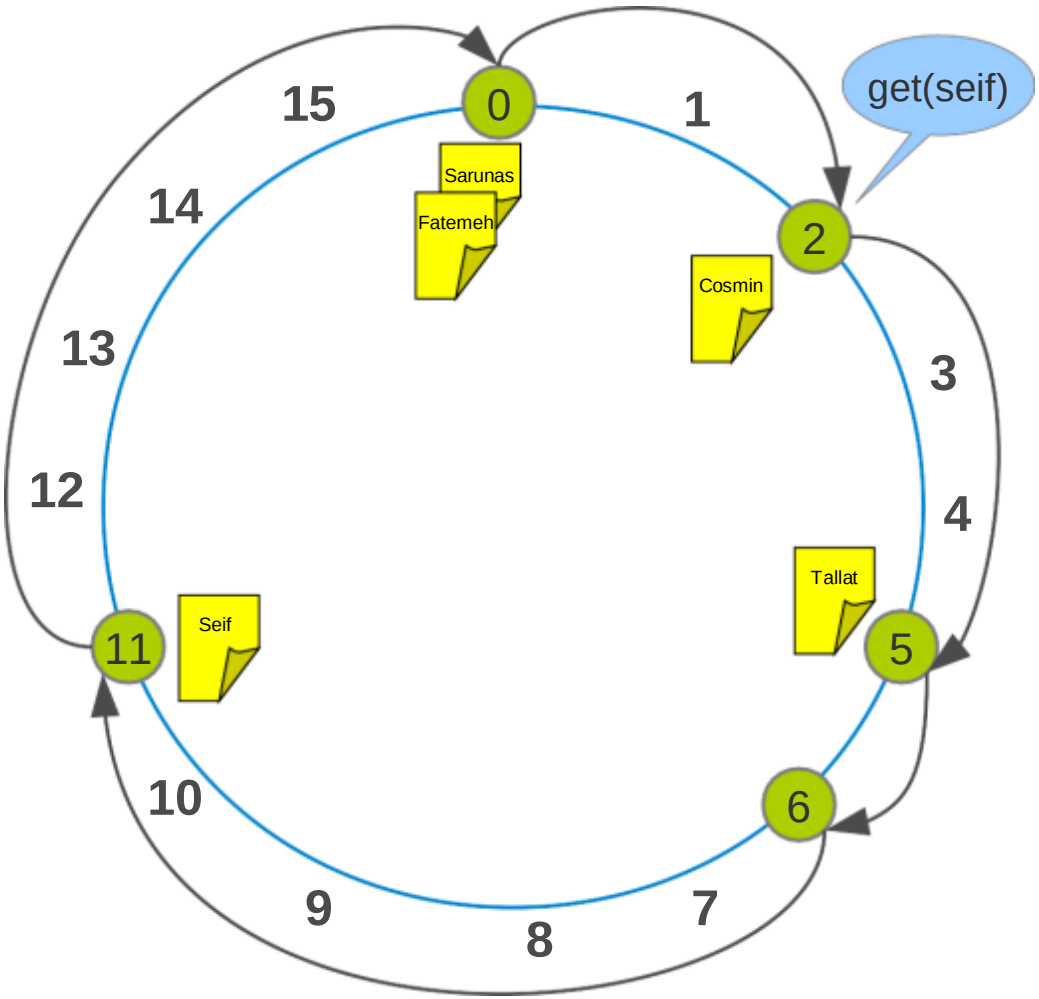
Exemplo, procurando "Seif" em nó 2

```
H('Seif') = 9
```

Percorrer nós 2, 5, 6, 11

Devolver "Stockholm" ao iniciador

Key	Value
Seif	Stockholm



Algoritmos

```
// Estrutura do nó n
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
}
```

Algoritmos

- (a, b] é o segmento do anel que não inclui a mas inclui b
- n.foo(...) é uma chamada RPC ao nó n pelo método foo
- n.bar é uma chamada RPC ao nó n obtendo a variável bar

```
// pergunta ao nó n encontrar o successor do id procurado
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        sucessor.findSucessor(id,ini)
}
```

Algoritmos

```
procedure n.put(id, value) {


}
procedure n.get(id) {


}
```

Algoritmos

```
procedure n.put(id, value) {
   op = put
   findSuccessor(id, this)
}

procedure n.get(id) {
   op = get
   findSuccessor(id, this)
}

procedure n.response(id, node) {
   if (op == put) node.store(id,value)
   else if (op == get) return node.retrieve(id)
}
```

- Ambos métodos put e get utilizam a chamada a findSucessor

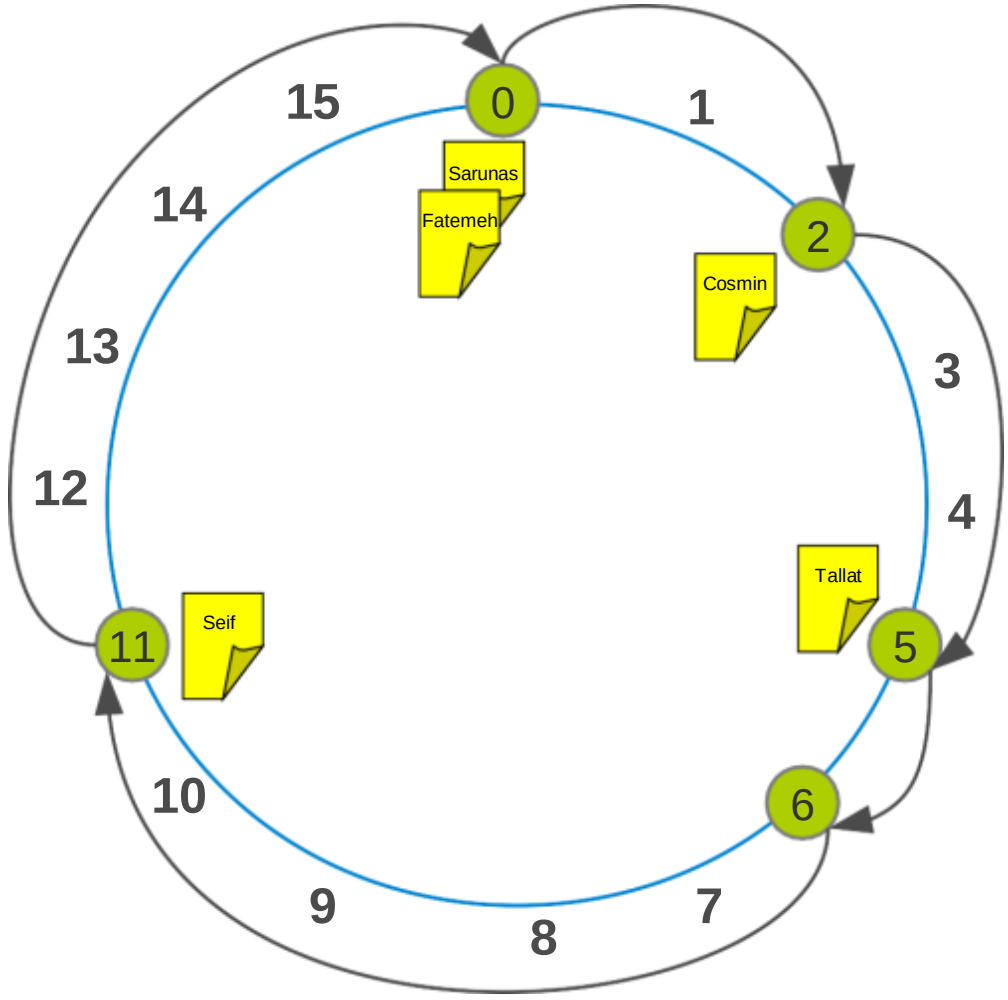
Algoritmos

```
// Estrutura do nó n (inicial)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
}

// Modificando a estrutura do nó n (agora considerando store/retrieve)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
    KV chaves-valor = null
}
```

Buscas mais rápidas

- Se somente o apontador ao sucessor for usado: O(N) para N nós
- Melhor usar "atalhos" (*shortcuts*)

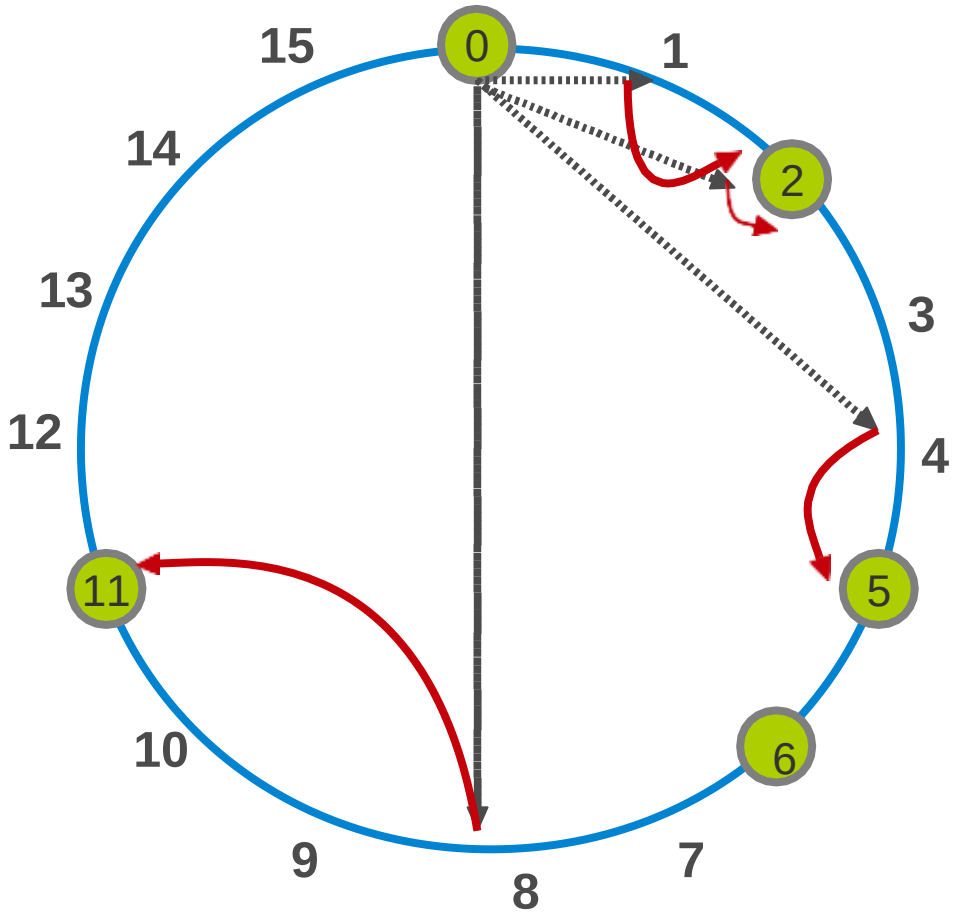


Buscas mais rápidas

- Fingertable

Aponta para succ(p+1)
Aponta para succ(p+2)
Aponta para succ(p+4)
Aponta para succ(p+8)
...
Aponta para succ(p+2^{M-1})

FingerTable nó 0	
i	Succ (n + 2^{i-1})
1	0 + 2^0 = 1 → 2
2	0 + 2^1 = 2 → 2
3	0 + 2^2 = 4 → 5
4	0 + 2^3 = 8 → 11



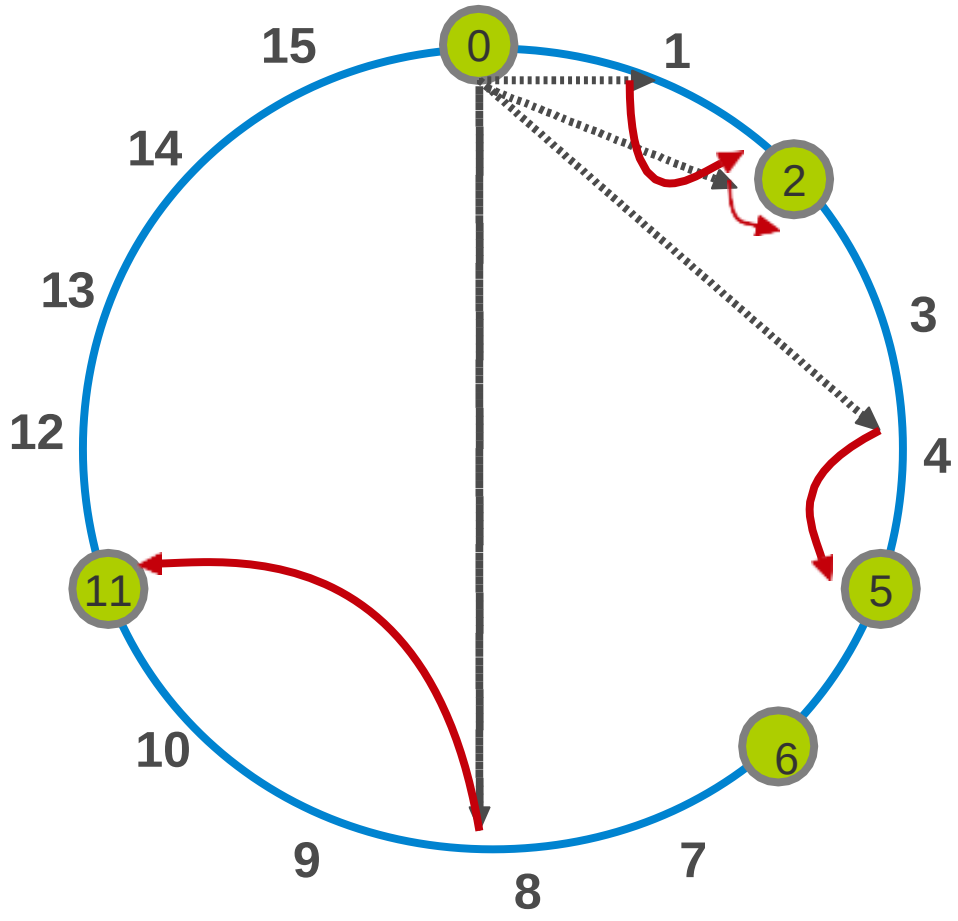
Buscas mais rápidas

- Tamanho da Fingertable é M e o total de chaves armazenadas será 2^M
- Cada nó p conhece o sucessor $(p + 2^{i-1})$ para i = 1 .. M
- $\log_2$ (N) saltos –hops– de um nó a qualquer outro para uma rede com N nós

Por que é importante o $\log_2$?
$\log_2(1000000) \sim 20$

Normalmente nas implementações
M = 128.

Na Figura:
M = 4, armazenando 16 chaves

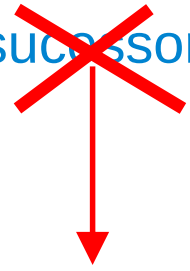


Buscas mais rápidas

```
// pergunta ao nó n encontrar o successor do id
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        sucessor.findSucessor(id,ini)
}
```

Buscas mais rápidas

```
// pergunta ao nó n encontrar o successor do id
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        sucessor.findSucessor(id,ini)
}
```



`closestPrecedingNode(id)`

Buscas mais rápidas

```
// pergunta ao nó n encontrar o successor do id
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}

// procura localmente pelo nó próximo ao id
procedure closestPrecedingNode(id) {
    for (i = M to 1 do) {
        if (FT[i] ∈(n, id)) then
            return FT[i]
    }
    return n // meu id
}
```

Algoritmos

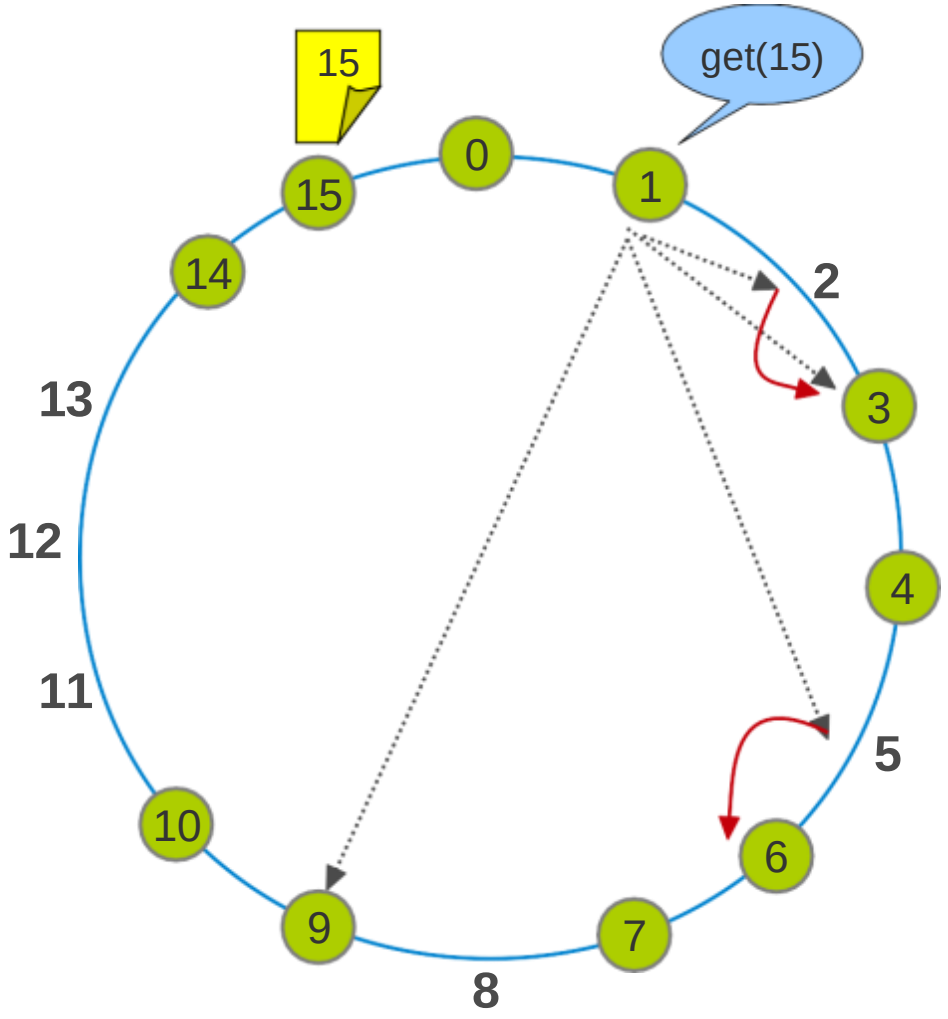
```
// Estrutura do nó n (considerando store)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
    KV chaves-valor = null
}

// Modificando a estrutura do nó n (agora considerando FT)
No {
    int meuId = n
    No predecessor = null
    No sucessor = null
    KV chaves-valor = null
    No[] FT = null
}
```

Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

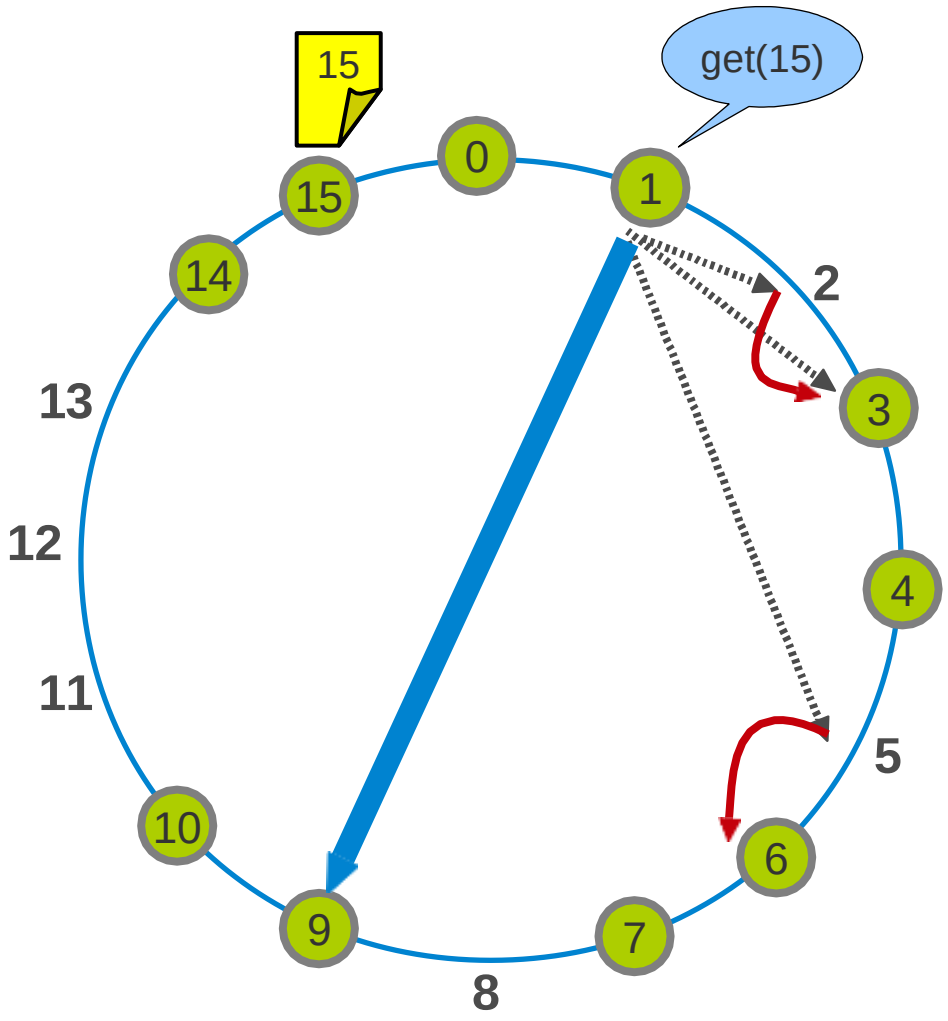
FingerTable nó 1	
i	Succ (n + 2^{i-1})
1	1 + 2^0 = 2 → 3
2	1 + 2^1 = 3 → 3
3	1 + 2^2 = 5 → 6
4	1 + 2^3 = 9 → 9



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

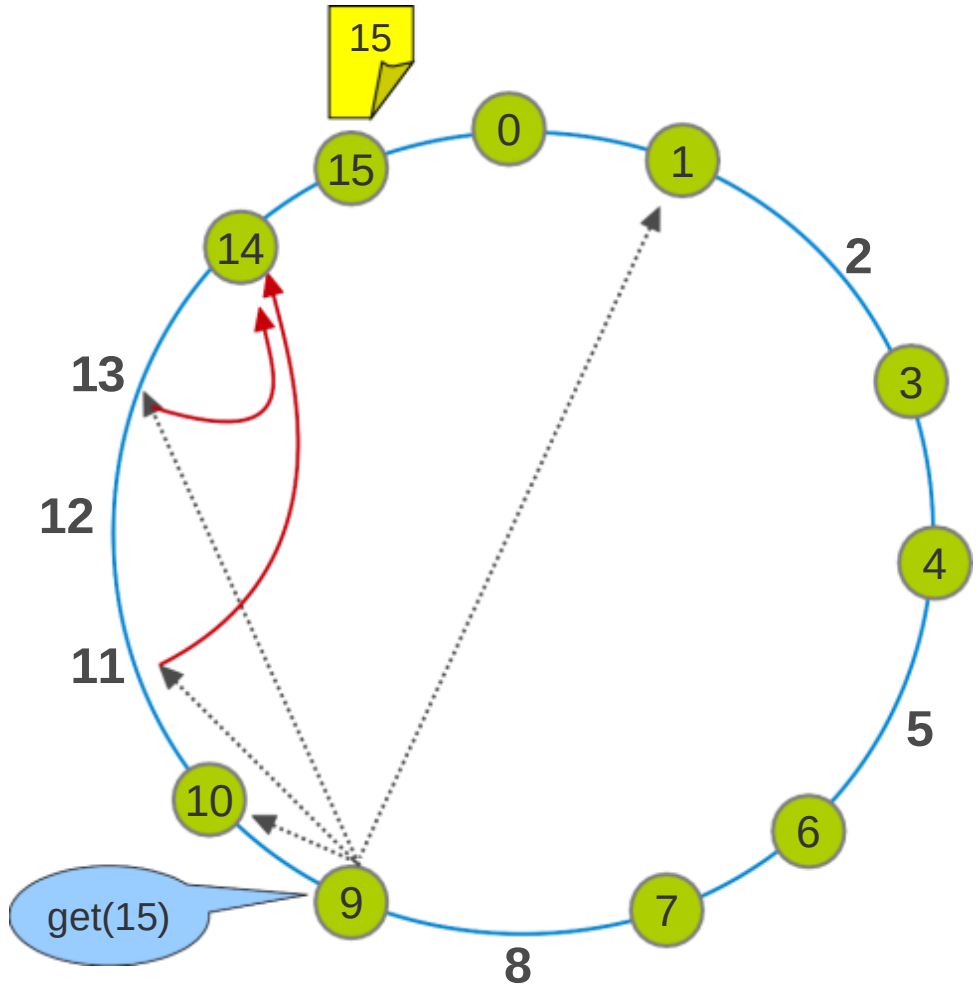
FingerTable nó 1	
i	Succ $(n + 2^{i-1})$
1	$1 + 2^0 = 2 \rightarrow 3$
2	$1 + 2^1 = 3 \rightarrow 3$
3	$1 + 2^2 = 5 \rightarrow 6$
4	$1 + 2^3 = 9 \rightarrow 9$



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

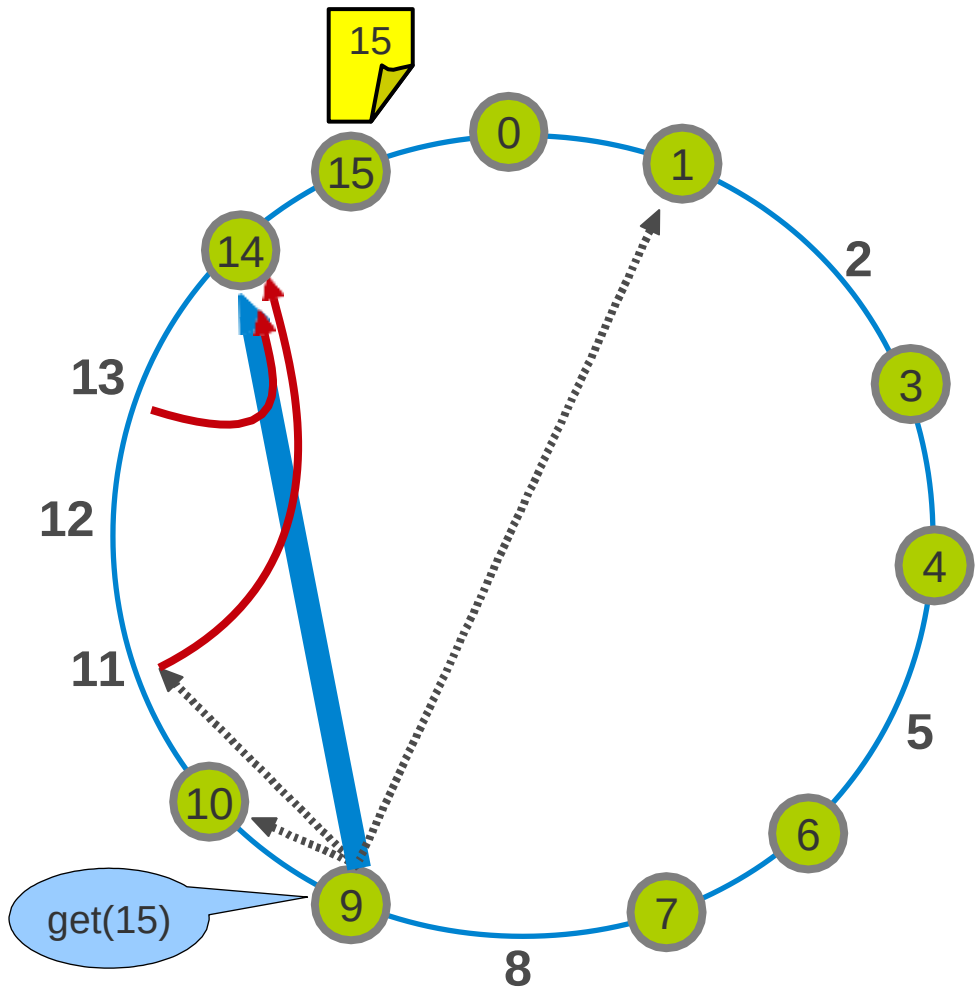
FingerTable nó 9	
i	Succ $(n + 2^{i-1})$
1	$9 + 2^0 = 10 \rightarrow 10$
2	$9 + 2^1 = 11 \rightarrow 14$
3	$9 + 2^2 = 13 \rightarrow 14$
4	$9 + 2^3 = 17 \% 16 \rightarrow 1$



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

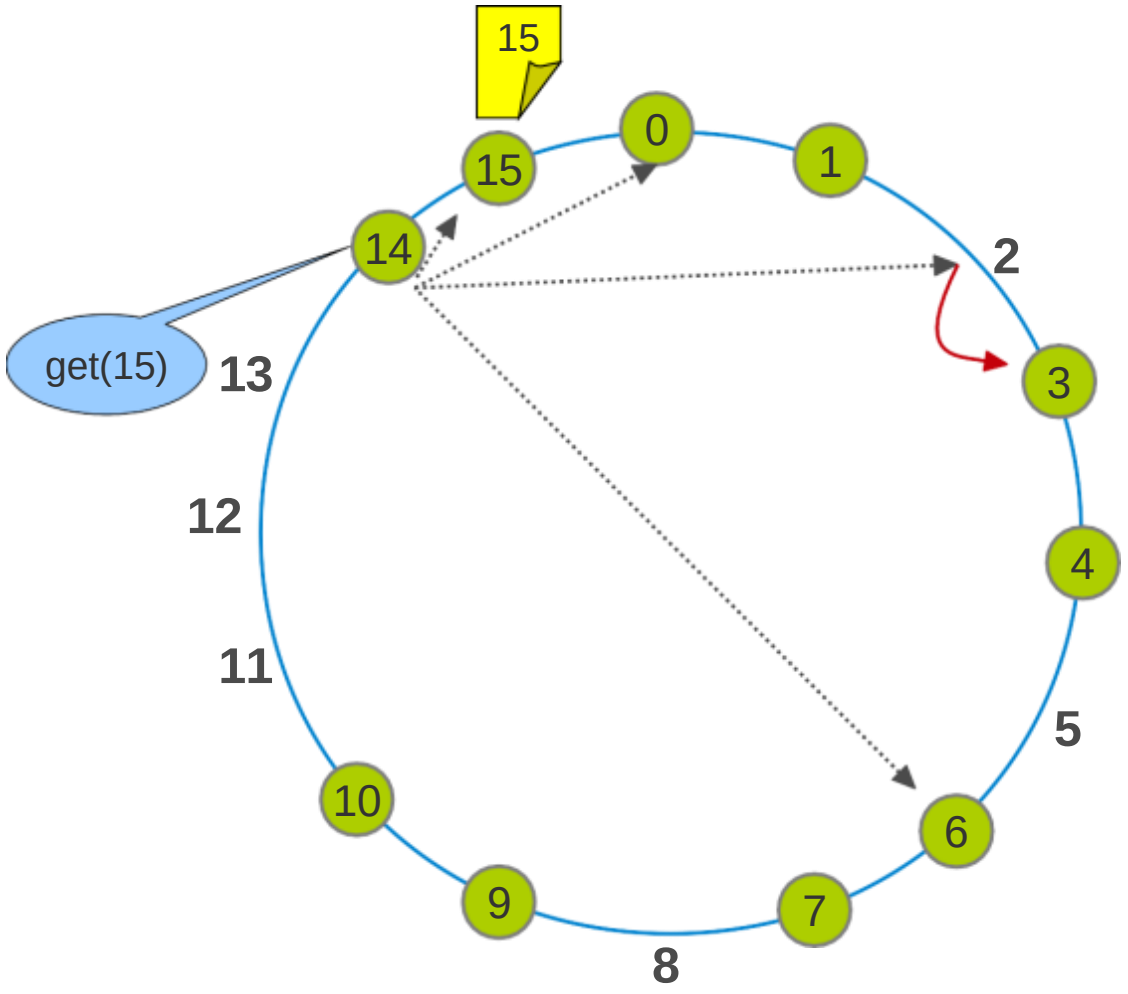
FingerTable nó 9	
i	Succ ($n + 2^{i-1}$)
1	$9 + 2^0 = 10 \rightarrow 10$
2	$9 + 2^1 = 11 \rightarrow 14$
3	$9 + 2^2 = 13 \rightarrow 14$
4	$9 + 2^3 = 17 \% 16 \rightarrow 1$



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

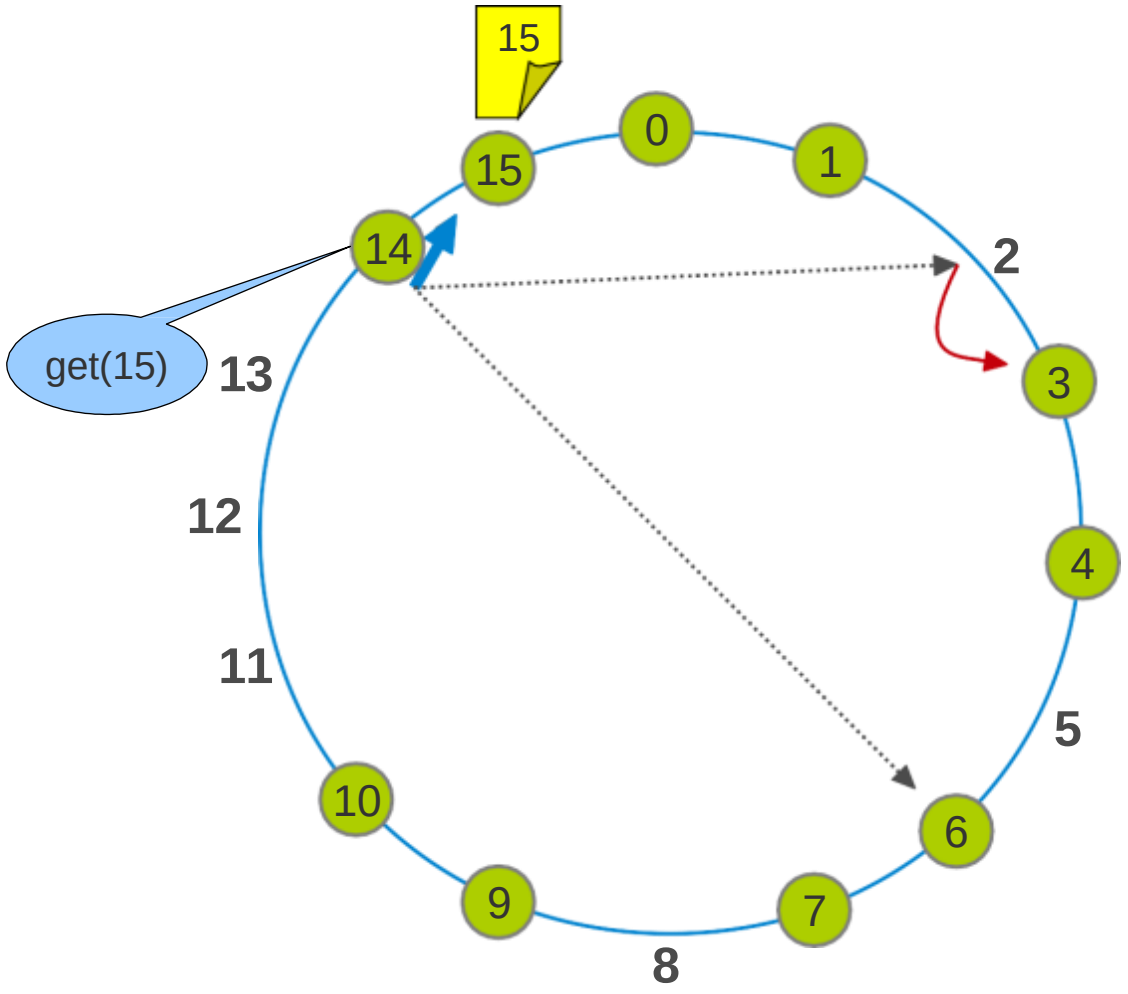
FingerTable nó 14	
I	Succ (n + 2^{i-1})
1	14 + 2^0 = 15 % 16 → 15
2	14 + 2^1 = 16 % 16 → 0
3	14 + 2^2 = 18 % 16 → 3
4	14 + 2^3 = 22 % 16 → 6



Buscas mais rápidas

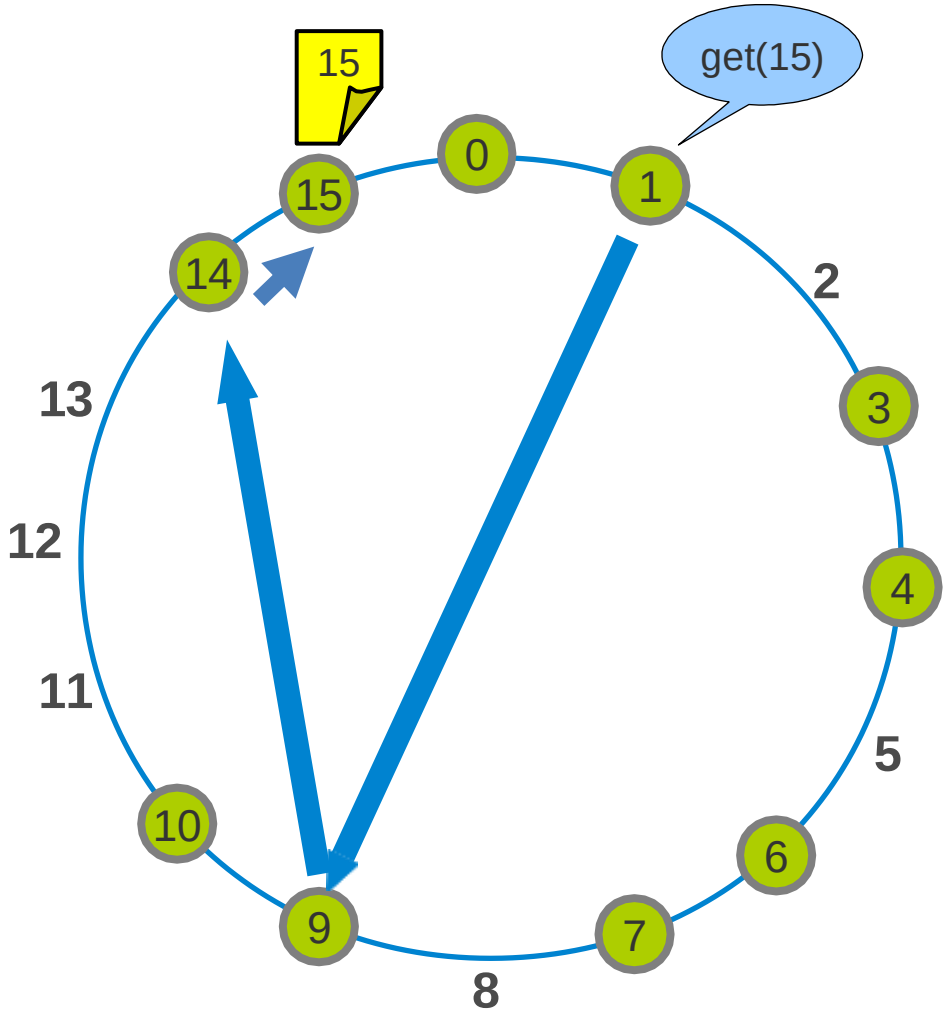
```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```

FingerTable nó 14	
I	Succ (n + 2^{i-1})
1	14 + 2^0 = 15 % 16 → 15
2	14 + 2^1 = 16 % 16 → 0
3	14 + 2^2 = 18 % 16 → 3
4	14 + 2^3 = 22 % 16 → 6



Buscas mais rápidas

```
procedure n.findSuccessor(id, No ini) {
    if (predecessor ≠ nil and id ∈ (predecessor, n]) then ini.response(id, n)
    else if (id ∈(n, successor]) then ini.response(id, sucessor)
    else // encaminha a pergunta pelo anel
        m = closestPrecedingNode(id)
        m.findSucessor(id,ini)
}
```



Conceitos adquiridos

- Busca distribuída em O(log N)
- Fingertable
- Consistent Hashing.
- SPOF – Single Point of Failure.